CS 7295 — Georgia Tech OMSCS

GPU Hardware and Software

講義ノート

kinginu

Spring 2027

目次

1 GPU 入門 **1**
- 1.1 CPU と GPU . . . 1
- 1.2 現代のプロセッサパラダイム . . . 2
- 1.3 並列プログラムの書き方 . . . 4
- 1.4 GPU アーキテクチャ入門 . . . 6
- 1.5 GPU アーキテクチャの歴史的変遷 . . . 8

2 並列プログラミング **10**
- 2.1 並列プログラミングパターン . . . 10
- 2.2 OpenMP による共有メモリプログラミング . . . 12
- 2.3 スケジューリング，データ共有，同期 . . . 14
- 2.4 MPI によるプログラミング . . . 17

3 GPU プログラミング入門 **20**
- 3.1 CUDA プログラミング入門 . . . 20
- 3.2 占有率 . . . 25
- 3.3 ホストコード . . . 28
- 3.4 CUDA によるステンシル演算 . . . 29

4 GPU アーキテクチャ **33**
- 4.1 マルチスレッドアーキテクチャ . . . 33
- 4.2 バンク競合 . . . 36
- 4.3 GPU アーキテクチャのパイプライン . . . 40
- 4.4 グローバルメモリのコアレッシング . . . 42

5 GPU プログラミングの最適化技法 **46**
- 5.1 GPU に向くプログラムと時間の内訳 . . . 46
- 5.2 データ転送の最適化 . . . 47
- 5.3 メモリアクセスの最適化 . . . 51
- 5.4 計算の削減 . . . 52
- 5.5 新しい CUDA の機能 . . . 53
- 5.6 ライブラリとほかのプラットフォーム . . . 56
- 5.7 ソーティングネットワーク . . . 57

6 GPU アーキテクチャの最適化 I **59**
- 6.1 ダイバージェント分岐の扱い . . . 59
- 6.2 レジスタの最適化 . . . 64
- 6.3 レジスタファイルと共有メモリの統合 . . . 67

7 GPU アーキテクチャの最適化 II **70**
- 7.1 GPU の仮想メモリ . . . 70
- 7.2 統合仮想アドレス空間とオンデマンドページング . . . 74
- 7.3 ワープスケジューリング . . . 77

8 GPU シミュレーション **81**
- 8.1 サイクルレベルシミュレーション . . . 81
- 8.2 GPU シミュレータにおけるダイバージェンスとメモリ . . . 84
- 8.3 GPU の解析モデル . . . 87
- 8.4 GPU シミュレーションの高速化 . . . 92

9 マルチ GPU **95**
- 9.1 マルチ GPU のハードウェア . . . 95
- 9.2 GPU 上の並行性 . . . 100

10 コンパイラの基礎 I **105**
- 10.1 GPU コンパイラの流れ . . . 105

10.2 PTX . . . 108
10.3 IR と基本ブロック . . . 110
10.4 データフロー解析入門 . . . 111
10.5 到達定義の例 . . . 114

11 コンパイラの基礎 II **117**
11.1 生存変数解析 . . . 117
11.2 静的単一代入 . . . 120
11.3 コンパイラ最適化の例 . . . 123
11.4 ダイバージェンス解析 . . . 125

12 GPU による機械学習の高速化 **128**
12.1 GPU と機械学習 . . . 128
12.2 浮動小数点形式 . . . 133
12.3 テンソルコア . . . 136

参考文献 **139**

索引 **144**

English index **150**

第 1 章

GPU 入門

講義動画	M1，レッスン 1–5（Instructor and Course Introduction；Modern Processor Paradigms；How to Write Parallel Programs；Introduction to GPU Architecture；History Trend of GPU Architectures）
資料	NVIDIA A100 [1] および H100 [2] ホワイトペーパー（世代ごとの数値）
教科書	K12 [3] ch. 1；A18 [4] ch. 1；CPG [5] §1 Introduction，§2 Programming Model，§4 Hardware Implementation；PMPP [6] ch. 1；PPP [7] ch. 2（任意）；BPG [8] §4.1.3 Understanding Scaling
キーワード	スループット，レイテンシ，仮想 ISA，SIMD，SPMD，SIMT，マルチスレッディング，アムダールの法則，リダクション，フリンの分類，ストリーミング・マルチプロセッサ，ワープ，統合型シェーダ，テンソルコア

本章は本書全体の枠組みを定める．CPU と GPU の設計目標を対比し，CPU コアが並列性を引き出す技法（スーパースカラ発行，アウトオブオーダ実行，マルチスレッディング，マルチプロセッシング）を復習し，アムダールの法則とプログラムを並列な仕事へ分割する 2 つの方法を述べ，続いて GPU ハードウェアの語彙，すなわちストリーミング・マルチプロセッサ，SIMT レーン，ワープを導入する．最後に，固定機能のグラフィックスパイプラインから今日のプログラマブルでテンソルコアを備えたアクセラレータへ至る GPU の進化を概観する．第 2 章は分割の考え方を OpenMP と MPI で発展させ，第 3 章はワープと SPMD スタイルを具体的な CUDA プログラムへ結びつける．

1.1 CPU と GPU

GPU（graphics processing unit）は 3D グラフィックス用のプロセッサとして生まれ，汎用の並列プロセッサへと成長した．今日では機械学習の高速化と高性能計算に最も結びつけられるハードウェアである．**CPU**（central processing unit）は設計空間の反対側の点に向けて作られており，表 1.1 にまとめた両者の対比が本書の残りの部分を貫く軸となる．

▷ GPU

出典 M1，CPU vs GPU；K12 §1.2；CPG §1.1 The Benefits of Using GPUs．

▷ CPU

表 1.1: CPU と GPU の設計点．

	CPU	GPU
対象アプリケーション	レイテンシ重視	スループット重視
精密例外	あり	なし
役割	ホスト	アクセラレータ
ISA	公開（オープン）	公開または非公開；公開の仮想 ISA として PTX
プログラミングモデル	SISD，SIMD	SPMD

対象アプリケーション CPU は**レイテンシ**，すなわち 1 つのタスクを終えるまでの時間に対して最適化されており，単一のスレッドをできるだけ速く走らせる．GPU は**スループット**，すなわち単位時間当たりに終える仕事の量に対して最適化されており，数千のスレッドを同時に走らせることでそれを達成する．個々のスレッドは遅くてもよい．

▷ latency

▷ throughput

レイテンシとスループットは独立した目標である．2 つのスレッドを並べて走らせてスループットを倍にする設計は，各スレッドのレイテンシを変えない．

精密例外 **精密例外** とは，その後のアーキテクチャ状態が，例外を起こした命令の位置における逐次プログラムの状態と正確に一致する例外である．すなわち先行するすべての命令は完了しており，後続の命令はどれも状態を変えていない．CPU はこれを保証する．オペレーティングシステムとデバッガがそれに依存するからである（ページフォールト，ブレークポイント，シグナル）．GPU は保証しない．例外を起こしたカーネルは通常カーネル全体として報告され，カーネル内部の機械状態を命令単位で復元すること

▷ precise exception

はできない．

ホストとアクセラレータ CPU はオペレーティングシステムと I/O スタックを動かす．GPU はどちらも行わないので，仕事を与えてもらうのに CPU に依存する．したがって CPU が**ホスト** であり，GPU は**アクセラレータ**，すなわち I/O リンクを介してホストに接続されたコプロセッサである．

▷ host
▷ accelerator

ISA CPU の**命令セットアーキテクチャ**は公開され安定している．コンパイル済みのソフトウェアが以後のすべての実装で動き続けなければならないからである．GPU はドライバの背後でアクセラレータモードで動作し，ドライバがプログラムを現行のハードウェア世代が実行するものへ変換する．ネイティブ ISA は非公開でもよく，世代ごとに変わってもよい．プログラマに安定した目標を与えるために NVIDIA は PTX，すなわち**仮想 ISA** を公開した．どのハードウェアも直接には実行しないが，ドライバが搭載 GPU のネイティブ ISA へコンパイルする命令セットである．

K12 §1.2（2012 年）は，PCIe リンクがレーン当たり “4 Gbit/s（Gen 2）または 8 Gbit/s（Gen 3）” を提供し，“しばしば最大のボトルネックの 1 つになる” と述べる [3]．ホスト – デバイス間転送は第 5 章で扱う．

▷ instruction set architecture (ISA)
▷ virtual ISA

プログラミングモデル CPU は **SISD**（single instruction, single data：1 つの命令ストリームが一度に 1 つのデータ項目を処理する）としてプログラムされ，**SIMD**（single instruction, multiple data）ベクトル拡張では 1 つの命令が短いベクトルの各要素に同じ演算を適用する．GPU は **SPMD**（single program, multiple data）としてプログラムされる．すべてのスレッドが同じプログラムを実行するが，それぞれ自分のデータに対して実行し，独自の分岐をたどってよい．

PTX そのもの，そのレジスタ，コンパイラフローにおける位置は第 10 章の主題である．

▷ SISD
▷ SIMD
▷ SPMD

SIMD は規則的なデータ並列の仕事（画像処理，ベクトル演算）に対して効率がよい．1 回の命令フェッチが多数の演算を駆動するからである．しかしすべてのレーンがロックステップで実行されるため，データ依存の分岐はレーンを無駄にする．SPMD は各スレッドが独自の制御フローをたどるのでより柔軟だが，多数のスレッドとそれらの間の通信を管理する代償がある．1 つのグループのスレッドがダイバージェンスを起こしたとき，ハードウェアがその柔軟性にどう代価を払うかは第 6 章で示す．

注意 SPMD はプログラムを，SIMT はそれを実行するハードウェアを表す．GPU は SPMD のコードを SIMT のハードウェア上で実行する（1.3 節）．2 つの頭字語は二者択一の関係にはない．

注 1.1. 本コースの目標は，GPU がどう動くかとそのためのプログラムをどう最適化するかを説明すること，GPU アーキテクチャとコンパイラの研究に備えること，そして近年のハードウェアアクセラレータを説明することである．CUDA の最新機能の概観ではなく，CUDA の基礎を築く．基本的な 5 段 CPU パイプラインと C++ プログラミングを前提とし，GPU プログラミングの経験は前提としない．

1.2 現代のプロセッサパラダイム

GPU は CPU コアの機構の大半を再利用するので，本節ではその機構と，CPU 設計者が性能を高めてきた方法を復習する．

出典 M1，CPU pipeline and ILP；PMPP ch. 1；A18 ch. 1．

1.2.1 パイプライン

本書を通じて用いる CPU モデルは図 1.1 の段をもつ．**フロントエンド**が命令を，通常は命令キャッシュからフェッチする．**デコード**段がそれを解釈する（複雑な x86 命令はここで複数のマイクロ命令に分解される）．レジスタはデコードの後に読まれる．**スケジューラ**が実行する命令を選び，**実行**段が計算またはメモリアクセスを行い，**ライトバック**段が結果を格納する．2 つの基本的な CPU 構成が異なるのはスケジューラである．

多くの CPU パイプラインはデコードとスケジューリングの間でレジスタのリネーミングも行い，再利用されたレジスタ名を通じた偽の依存が命令を直列化しないようにする．

インオーダ実行 インオーダ実行 は命令をプログラム順に発行する．命令は，自身のオペランドが揃っていても，先行するすべての命令が発行されるまで待つ．

▷ in-order execution

アウトオブオーダ実行 アウトオブオーダ実行 はプログラム順に関係なくオペランドの揃った命令を発行し，キャッシュミスで停止した命令がその後ろにある独立な命令を妨げないようにする．

▷ out-of-order execution

1.2.2 命令レベル並列性

単一発行のパイプラインは 1 サイクルに高々 1 命令を開始する．**スーパースカラ**プロセッサは 1 サイクルに複数の命令をフェッチ，デコード，実行し（図 1.1b），

▷ superscalar

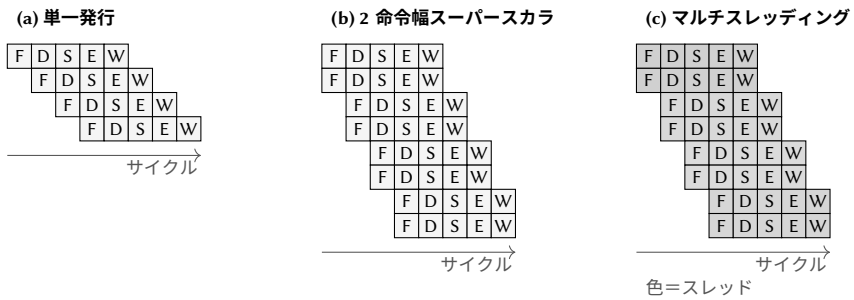


図 1.1: 5 段パイプライン（フェッチ，デコード，スケジュール，実行，ライトバック）に毎サイクル 1 命令を投入した場合 (a)，2 命令幅スーパースカラで毎サイクル 2 命令を投入した場合 (b)，4 つのスレッドから毎サイクル 2 命令を投入した場合 (c)．

その**サイクル当たり命令数**は 1 を超える．実際にいくつの命令を同時に走らせられるかは，プログラムの**命令レベル並列性**，すなわち互いに独立で同時に実行できる命令の数で上から抑えられる．

▷ instructions per cycle (IPC)

実際の幅：Intel の Golden Cove コア（2021 年）は 1 サイクルに最大 6 個の x86 命令をデコードする [9, §2.4.1]．

▷ instruction-level parallelism (ILP)

例 1.2. 実行ユニットを 2 つもつマシンを考える．次の命令対のうち

(a) 独立	(b) 依存
`R1 = R2 * R3`	`R1 = R2 * R3`
`R4 = R5 - R6`	`R4 = R1 - R6`

(a) の 2 命令は互いに素なレジスタを読み書きするので，両方が同じサイクルに発行される．(b) では 2 番目の命令が，1 番目の命令が書く R1 を読むので，1 番目が結果を出すまで開始できない．この対は 2 つの発行スロットを順に使い，2 番目のユニットは遊んだままになる．

アウトオブオーダ実行は，プログラム順で次にある命令以外にも独立な命令を見つけるために存在し，ILP を高めることは CPU 設計の主要なテーマの 1 つであった．さらに 2 つの手段が，並列性を増やさずに単一スレッド性能を高める．パイプラインを深くすればクロック周波数を上げられるが，より優れた**分岐予測**，すなわち分岐の方向と飛び先を実行前に推測して長いパイプラインを満たし続ける機構が必要になる．また，キャッシュを大きくすれば，ミスして DRAM まで到達するメモリアクセスの数が減る．

ヒント 2 つの命令が独立であるとは，どちらも相手が書くレジスタを読みも書きもしないことである．アウトオブオーダのハードウェアは，この条件を満たす命令をストリームのさらに先まで探す．

▷ branch prediction

Pentium 4 は第 1 の手段をパイプライン 20 段（2000 年）[10]，31 段（2004 年）[11] まで推し進めた．

1.2.3 マルチスレッディングとマルチプロセッサ

次の並列性の源は**マルチスレッディング** である．コアが複数のスレッドの状態を同時に保持し，現在のスレッドに発行可能な命令がなければ別のスレッドから発行する（図 1.1c）．独立なスレッドは ILP を探すことなく発行可能な命令を供給し，ハードウェアのコストは主に追加のプログラムカウンタとレジスタであって控えめである．マルチスレッディングは GPU 性能の鍵となる要素の 1 つである．GPU コアは数十のスレッドグループを同時に走らせ，毎サイクルそれらを切り替える．

▷ multithreading

GPU ではこれは毎サイクルコンテキストスイッチを行う細粒度マルチスレッディングである．第 4 章でそれがメモリレイテンシをどう隠すかを示す．

最後の手段はコア全体を複製することである．**マルチプロセッサ**は複数のコアを含み，各コアは自身のパイプラインと通常は自身の 1 次キャッシュをもち，コアに分配できる仕事に対してはコア数に比例してスループットを高める．CPU も GPU もマルチプロセッサであり，異なるのはトランジスタの使い方である（図 1.2）．CPU は，それぞれ ILP と単一スレッドのレイテンシのために作られた少数の大きなコアを，大きな共有キャッシュの上にもつ．GPU は，それぞれマルチスレッド化され SIMD 幅をもつ多数の小さなコアを，比較的小さなキャッシュを挟んでもつ．

▷ multiprocessor

今日の規模：AMD の EPYC 9654 CPU は 96 コア（2022 年）[12]，NVIDIA の H100 は 132 個のストリーミング・マルチプロセッサをもち [2]，各々 128 レーンである．

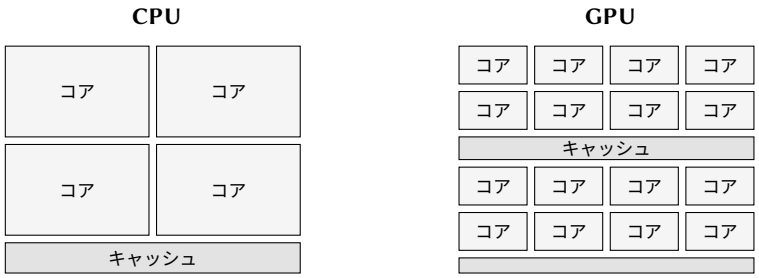


図 1.2: CPU は面積を少数の大きなコアと大きなキャッシュに使い，GPU は多数の小さなコアと小さなキャッシュに使う．

> **要点**
> CPU の性能は ILP（スーパースカラ，アウトオブオーダ），クロック周波数（深いパイプラインと分岐予測），キャッシュ，マルチスレッディング，複数コアから得られる．GPU はマルチスレッディングと複数コアを残し，ILP の機構の大半を捨て，代わりにスレッド数を増やす．

1.3 並列プログラムの書き方

多数のコアをもつハードウェアは，並列な仕事を露わにするプログラムに対してのみ報われる．本節では，利得を上から抑える法則，プログラムを並列な部分へ分ける 2 つの方法，そして GPU が位置づけられる分類を述べる．

出典 M1，Amdahl's law and decomposition；PPP ch. 2 [7]；BPG §4.1.3 Understanding Scaling [8]．

1.3.1 アムダールの法則

定義 1.3 (アムダールの法則). プログラムが逐次実行時間のうち割合 S を並列化できない仕事，すなわち**逐次部分の割合** に費やし，残りの割合 $P = 1 - S$ を N 個のプロセッサが均等に分担する仕事に費やすとする．**アムダールの法則** は逐次実行に対する高速化率を次式で与える．

▷ serial fraction

▷ Amdahl's law

記法：S と P は逐次実行時間に対する割合で $S + P = 1$，N はプロセッサ数．並列部分は完全に分割できると仮定する．

$$\text{speedup} \;=\; \frac{1}{S + P/N}. \tag{1.1}$$

逐次部分の割合が小さく並列部分が大きいときは，高速化率は N にほぼ比例して増える．逐次部分の割合が無視できないときはスケーラビリティが崩れる．$N \to \infty$ で項 P/N は消え，どれだけハードウェアを追加しても高速化率は $1/S$ に近づく．

例 1.4. 逐次部分の割合が $S = 0.1$ のプログラムを考える．$N = 8$ プロセッサでは式 (1.1) より $1/(0.1 + 0.9/8) = 1/0.2125 \approx 4.7$，$N = 64$ では $1/(0.1 + 0.9/64) \approx 8.8$，$N = 1024$ では $1/(0.1 + 0.9/1024) \approx 9.9$ となる．8 から 1024 プロセッサへ，すなわちハードウェアを 128× に増やしても，高速化率はかろうじて 2 倍になるにすぎない．逐次の 10 % だけで，8 プロセッサ上の並列部分全体と同じだけの時間がすでにかかるからである．

ヒント 上限は $1/S$ である．逐次部分の割合が 10 % なら高速化率は 10× で，1 % なら 100× で頭打ちになる．

図 1.3 は式 (1.1) を，いくつかの N について逐次部分の割合に対して描いたものである．曲線は S が約 0.3 を下回るまで寄り集まったままで，逐次部分の割合が小さいときにだけ分かれる．並列プログラマの最初の仕事がプロセッサを増やすことではなく逐次部分を縮めることであるのはこのためである．

注意 アムダールの法則は問題サイズを固定する（強スケーリング）．Gustafson の法則（BPG §4.1.3.2 Weak Scaling and Gustafson's Law [8]）は問題を N とともに大きくし，$N + (1 - N)s$ を与える．ここで s は並列実行における逐次部分の割合であり，逐次部分の重要性は下がる．BPG は N とともに成長するワークロードとしてメッシュ，グリッド，一部のモンテカルロシミュレーションを挙げる．

1.3.2 タスク分割とデータ分割

十分な並列の仕事をもつプログラムが与えられたとき，それを分配する方法は 2 つある．ここでの例は，配列 A に対して合計と最小値・最大値の両方を 2 つのコアで求めるものである（図 1.4）．

タスク分割 **タスク分割** は異なる演算を異なるコアに与える．コア 1 が合計を，コア 2 が最小値と最大値を計算し，A 全体を両方へ送らなければならない．

▷ task decomposition

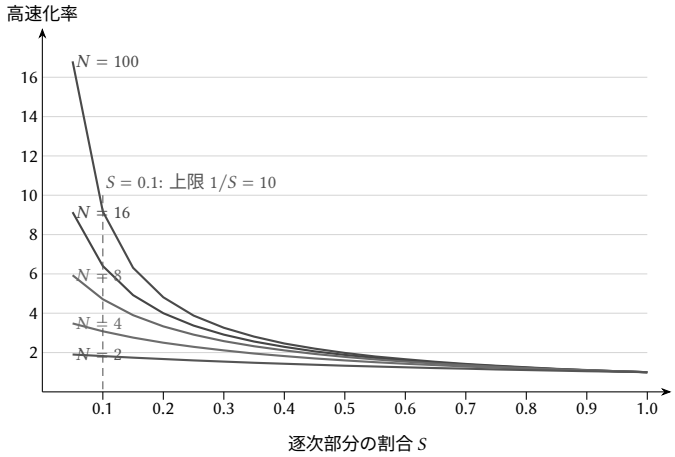


図 1.3: アムダールの法則による高速化率を，$N = 2$，4，8，16，100 プロセッサについて逐次部分の割合 S の関数として示す．$S = 0.1$ では $N = 100$ でも 9.2× にしかならない．

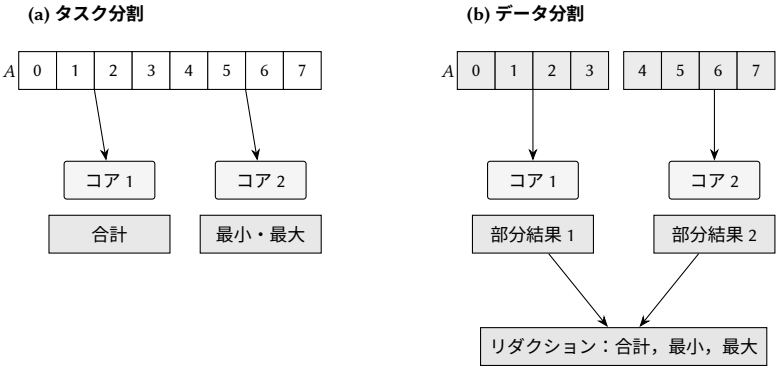


図 1.4: タスク分割は配列全体を各コアへ送り，各コアに異なる演算を与える．データ分割は各コアに配列の一部を与え，部分結果をリダクションで結合する．

データ分割 **データ分割**は異なるデータをコアに与え，各コアは自分の部分に対してすべての演算を行う．コア 1 は A の前半について合計と走査を行い，コア 2 は後半を扱う．その後，部分結果を結合しなければならない．この結合ステップが**リダクション**，すなわち結合的な演算子（合計，最小値，最大値）で多数の値を 1 つに畳み込む演算である．

▷ data decomposition

▷ reduction

第 2 章ではリダクションを OpenMP の節と MPI の集団通信として示す．第 5 章では GPU 上でそれを実装するワープレベルのプリミティブに触れる．

データ分割は自然にスケールする．コアが増えれば各コアが A のより小さな部分を受け持つだけである．一方，タスク分割は異なる演算の数で制限される．データ分割の代価はリダクションであり，それ自体を木として並列化しない限り，アムダールの法則の意味で逐次の仕事である．

1.3.3 フリンの分類

フリンの分類はマシンを，同時に処理する命令ストリームの数とデータストリームの数で分類する（図 1.5）．SISD，SIMD，MISD（multiple instruction, single data：商用例がほとんどない象限），そして **MIMD**（multiple instruction, multiple data：マルチコア CPU のように，独立なプロセッサが独立なデータに対して独立な命令ストリームを走らせる）である．CPU は SISD（1 コアでスカラコード）にも，SIMD（そのベクトルユニット）にも，MIMD（その多数のコア）にもなりうる．GPU は SIMD の象限に **SIMT**（single instruction, multiple threads）の名で置かれる．SIMD と同様に 1 回フェッチした命令が多数の演算を駆動するが，その演算は別々のスレッドに属し，各スレッドが自身のレジスタをもち，ベクトルユニットではなく多数の ALU 上で実行される．グループは 1 つのプログラムカウンタとアクティブマスクを共有する（1.4 節，第 6 章）．この区別は 1.4 節と第 4 章で展開する．

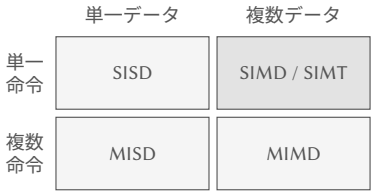


図 1.5: フリンの分類．GPU は単一命令・複数データの象限に SIMT として位置する．

▷ Flynn's taxonomy
▷ MIMD
▷ SIMT

注意 CPG §4.1 SIMT Architecture は境界を次のように引く．SIMD はベクトル幅をソフトウェアに露出するが，SIMT はしない．SIMT のスレッドは通常のスカラスレッドとして書かれる．Volta 以降はハードウェアもスレッドごとにプログラムカウンタを保持する（Independent Thread Scheduling）[5]．第 5 章で再び扱う．

1.3.4 SIMT ハードウェア上の SPMD

SPMD スタイルではすべてのスレッドが同じプログラムを実行し，触れるデータだけが異なる．データ分割がその自然なパターンである．SPMD は GPU をプログラムする典型的な方法であり，SIMT はそれを実行するハードウェアの実行モデルである．プログラマは 1 つのスレッドのための 1 つのスカラプログラムを書き，ハードウェアはその各命令をスレッドのグループ全体に対して 1 回だけフェッチする．

CUDA プログラムはホストコードとカーネル，すなわち GPU 上で走る関数からなる．すべてのスレッドが同じカーネルを実行し，自身のスレッドインデックスを使って自分のデータを選ぶ．下のカーネルはスレッド当たり 1 要素を 2 乗する．インデックス計算と起動構文は第 3 章で説明する．

```
__global__ void square(const float *A, float *B, int n)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) B[i] = A[i] * A[i];
}
```

同じスタイルで配列を合計するのはそれほど単純ではない．すべてのスレッドが 1 つの共有変数に対して `sum += A[i]` を実行するカーネルは競合である．数千のスレッドが同じ場所を同時に読み，加え，書くからである．合計にはリダクションが必要で，スレッドごとの部分和と結合ステップから構成する（OpenMP は第 2 章，GPU のプリミティブは第 5 章）．

注意 カーネルのすべてのスレッドからの `sum += A[i]` は，単に遅いのではなく誤りである．結果はインタリーブに依存する．共有のアキュムレータはすべてリダクションとして扱うこと．

1.4 GPU アーキテクチャ入門

GPU は多数のコアからなるマルチプロセッサであり，各コアは多数のスレッドを走らせる．本節ではそのようなコアの部品を名づけ，そのパイプラインを素描し，スレッドを実行する単位を定義する．

出典 M1，GPU architecture overview and warps；K12 §1.2.1；A18 ch. 1；CPG §4.1 SIMT Architecture．

1.4.1 ストリーミング・マルチプロセッサとレーン

NVIDIA は GPU のコアを**ストリーミング・マルチプロセッサ**，または SIMT マルチプロセッサと呼ぶ．SM の内部（図 1.6）には，命令キャッシュ，デコーダ，共有メモリのブロック，保持するスレッドのためのプログラムカウンタとレジスタファイルの集合，そして多数の実行ユニットがある．各実行ユニットは **SIMT レーン**，すなわち各命令の 1 スレッド分を実行する 1 つの ALU である．NVIDIA はレーンを**ストリーミングプロセッサ（CUDA コア）**として宣伝しており，CPU のコアとの混同を招く．したがって本書では語彙を固定する．**コア**とは SM であり，SIMT レーンをコアと呼ぶことはない．

▷ streaming multiprocessor (SM)

K12 §1.2.1：NVIDIA はコアを SM，AMD は compute unit（CU）と呼ぶ．各々は，ストリーミングプロセッサとも呼ばれる複数の SIMD ユニットを保持する [3]．

▷ SIMT lane

▷ stream processor (CUDA core)

注意 本コースではコア ≠ ストリーミングプロセッサである．コアとは SM であり，ストリーミングプロセッサはその 1 レーンである．“16 896 コア” の H100 は，SM 当たり 128 レーンの 132 SM の GPU である．

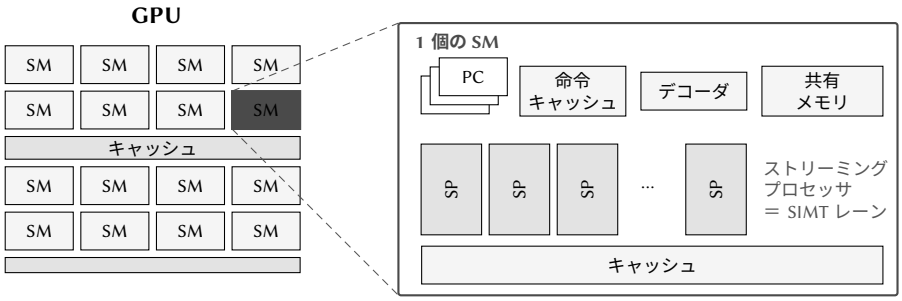


図 1.6: キャッシュを共有するストリーミング・マルチプロセッサの格子としての GPU と，1 個の SM の中身：プログラムカウンタ，命令キャッシュ，デコーダ，共有メモリ，ストリーミングプロセッサ（SIMT レーン），キャッシュ．

1.4.2 GPU パイプライン

SM のパイプライン（図 1.7）は，1.2 節の CPU パイプラインの段を多数のスレッド向けに配置したものである．フェッチ段は，以下で定義するスレッドグループ

であるワープごとに 1 つのプログラムカウンタを保持し，スケジューラが選んだワープのためにフェッチする．選択はラウンドロビン方式か，キャッシュミスと分岐結果を参照する貪欲方式による．デコードとレジスタ読み出しが続く．次にスコアボードをもつ第 2 のスケジューラがソースオペランドの揃ったグループを選び，レーンが命令を実行し，メモリアクセスユニットがロードとストアを処理し，ライトバック段が結果をレジスタへ返す．

ワープの語は織物（“最初の並列スレッド技術”）に由来する：CPG §4.1 SIMT Architecture [5]．

第 4 章ではこのパイプラインをスコアボードとレジスタバンクとともにサイクル単位でたどる．第 7 章ではスケジューリング方式を比較する．

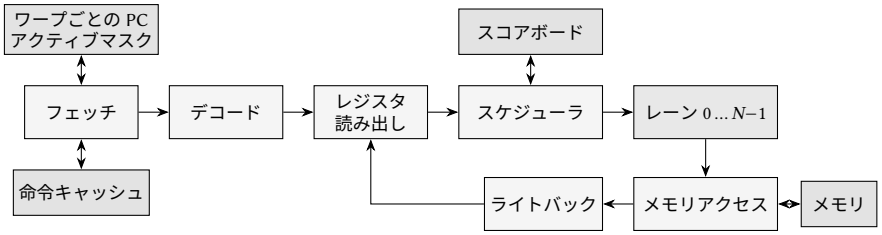


図 1.7: GPU パイプライン：ワープごとに 1 つのプログラムカウンタをもつフェッチ，デコード，レジスタ読み出し，発行可能なワープを選ぶスコアボード付きスケジューラ，SIMT レーン，メモリアクセス，ライトバック．

1.4.3 ワープ

定義 1.5 (ワープ). **ワープ**とは，ハードウェアが 1 つの単位として実行するスレッドのグループである．命令はワープごとに 1 回フェッチされ，そのすべてのスレッドに対して，レーン当たり 1 スレッドで実行される．グループ内のスレッド数が**ワープサイズ** である．AMD は同じ単位を**ウェーブフロント** と呼ぶ．本書では 2 つの語を同じ意味で用いる．

▷ warp

▷ warp size

▷ wavefront

ワープは GPU における実行の基本単位である．そのサイズは NVIDIA のハードウェアでは長く 32 のままだが，アーキテクチャはそれを固定しておらず，将来のハードウェアでは変わるかもしれない．カーネル起動はスレッド数を指定し，スレッドはスレッドブロックにまとめられる（第 3 章）．ハードウェアは各ブロックをワープサイズのワープに切り分ける．ワープのソースオペランドが揃うとワープ全体が命令を実行し，次に別の発行可能なワープが選ばれ，これが繰り返される．ワープは SM と同じ幅である必要はない．8 レーンの SM 上の 32 スレッドのワープは 4 サイクルかけて発行される．

Compute Capability 5.0 から 9.0 までのすべてでワープサイズは 32：CPG Table 21, Technical Specifications per Compute Capability [5]．

例 1.6. 3 命令のカーネルを，ワープサイズが 4 で 4 レーンをもつ SM 上に 12 スレッド（1 スレッドブロック）で起動する（図 1.8）．スレッドはワープ W_0，W_1，W_2 を形成する．ラウンドロビンのスケジューラで，1 サイクルに 1 ワープ命令，ストールなしとすると，実行順序は次のようになる．

G80（2006 年）は SM 当たり 8 レーンだったので，32 スレッドのワープは 1 命令に 4 サイクルを要した：K12 §1.4.1 [3]．現在の SM は 128 個の FP32 レーンを 4 つのパーティションに分けてもつ [2]．

ヒント ブロック当たりスレッド数 ÷ ワープサイズを切り上げたものがブロック当たりのワープ数である．ワープサイズ 32 で 100 スレッドのブロックは 4 つのワープを作り，最後のワープは 4/32 しか埋まらない．

サイクル	1	2	3	4	5	6	7	8	9
ワープ	W_0	W_1	W_2	W_0	W_1	W_2	W_0	W_1	W_2
命令	1	1	1	2	2	2	3	3	3

9 回の命令フェッチで $12 \times 3 = 36$ 個のスレッド命令が実行される．W_0 の命令 2 が命令 1 の積を待たなければならないとすれば（乗算器のレイテンシが両者の間の 3 サイクルより長い場合），スケジューラはサイクル 4 で W_0 を飛ばし，W_1 と W_2 を先に発行する．これが 1.2 節のマルチスレッディングをワープの粒度で行ったものである．

> **要点**
> GPU は SM からなるマルチプロセッサであり，SM は多数のレーンをもつマルチスレッド化された SIMT コアであり，ワープは 1 回の命令フェッチを共有し 1 つの単位としてスケジュールされるスレッドのグループ（NVIDIA の GPU では 32 スレッド）である．占有率からワープダイバージェンス，メモリコアレッシングに至るまで，以後の性能の議論はすべてワープを単位として行う．

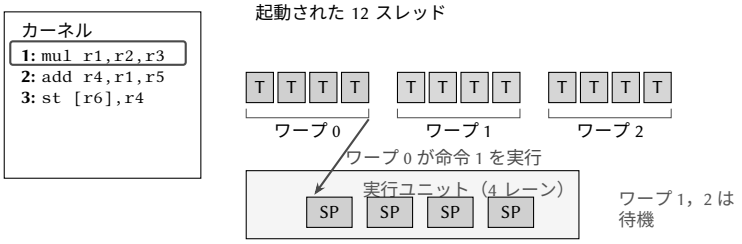


図 1.8: 起動された 12 スレッドが 4 スレッドずつ 3 つのワープを形成する．ワープ 0 が命令 1 のために SIMT 実行ユニットの 4 レーンを占め，その間，他のワープは待つ．

1.5 GPU アーキテクチャの歴史的変遷

GPU アーキテクチャは 4 つの段階を経てきた．

1. **従来のパイプライン型** *GPU*．頂点処理とピクセル処理に別々の固定機能ユニットをもつ．
2. **プログラマブル** *GPU*．頂点ユニットとピクセルユニットがプログラマブルなシェーダになった．
3. シェーダが統合型の，CUDA でプログラム可能なコアへ進み，汎用計算（GPGPU）が開かれた．
4. **現代の** *GPU*．プログラマブルなコアの傍らに，テンソルコアのような機械学習アクセラレータ部品を加える．

出典 M1，graphics pipeline and GPU evolution；K12 §1.1；A18 ch. 1；HWP [2]．

1.5.1 従来の 3D グラフィックスパイプライン

初期の GPU の**グラフィックスパイプライン**（図 1.9）は，3D プリミティブ，典型的には頂点で与えられた三角形を入力に取る．ジオメトリステージが頂点を変換し照明を付け，スクリーン平面上の 2D プリミティブへ投影する．ラスタ化ステージが各 2D プリミティブをそれが覆うピクセルへ変換し，その色を計算する．各頂点に適用されるプログラム（もとは固定機能）が**頂点シェーダ**，各ピクセル（またはフラグメント）に適用されるものが**ピクセルシェーダ**である．

▷ graphics pipeline

▷ vertex shader

▷ pixel shader

どちらのステージも多数の独立な頂点やピクセルに同じ演算を適用する．GPGPU が後に利用したデータ並列性は，グラフィックスのワークロードに最初から組み込まれていた．

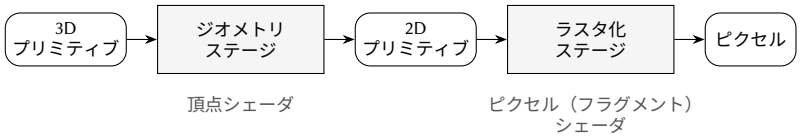


図 1.9: 従来の 3D グラフィックスパイプライン：ジオメトリ処理が 3D プリミティブを 2D プリミティブに変え，ラスタ化がそれをピクセルに変える．

シェーダは最初は固定機能ユニットであり，次にプログラマブルになり，最後に別々の頂点ユニットとピクセルユニットが，どちらの種類のシェーダプログラムも実行できるプロセッサである**統合型シェーダ**の 1 つのプールに置き換えられた．統合型シェーダは NVIDIA の Tesla アーキテクチャの特徴であり，これによって GPU はグラフィックス以外の計算に使えるようになった．**GPGPU**（general-purpose computing on GPUs）の語は，その非グラフィックス用途を表すために作られた．今日ではデータセンタの GPU サイクルの大半が非グラフィックスの仕事に使われるので区別の意味は失われたが，語はタイトルなどに生き残っている．[1]

▷ unified shader

K12 §1.1 は転換点を G80 ファミリ（GeForce 8800，2006 年），すなわちプログラマブルな統合グラフィックス・計算プロセッサと CUDA に置く [3]．

▷ GPGPU

[1] CUDA 以前の GPGPU とは，計算をグラフィックス演算として表現することだった．データはテクスチャに格納し，計算は DirectX や OpenGL を通じてシェーディング言語のピクセルシェーダとして書き，Brook のようなストリーム拡張がその上に薄い層を加えた（K12 §1.1 [3]）．統合型シェーダと CUDA がその間接性を取り除いた．

1.5.2 プログラマブル GPU の進化

プログラマブルになった GPU は，汎用の，次いで機械学習のワークロードが必要とする機能を世代ごとに獲得していった．

- キャッシュ階層（L1 と L2 キャッシュ）．グラフィックスパイプラインはテクスチャへのストリーミングアクセスしか必要としなかった．
- **倍精度**演算を**単精度**に加えて備えること．高性能計算のためである．
- より多様なワークロードのための，アトミック操作と高速な整数演算．

▷ double precision (FP64)

▷ single precision (FP32)

FP32 の半分の速度の FP64 はデータセンタ向けの機能である．A100 は FP32 の 19.5 TFLOP/s に対して FP64 で 9.7 TFLOP/s を出す [1]．

- **広帯域メモリ（HBM）**．インターポーザ上で GPU の隣に積層され，非常に広いインタフェースで接続された DRAM ダイであり，メモリ帯域幅をボードレベルのメモリバスが提供するものをはるかに超えて引き上げた．
- **半精度**．16 ビットの浮動小数点形式で，精度の低下を許容できる機械学習ワークロードに対して演算スループットを倍にし，メモリトラフィックを半減する．
- **テンソルコア**．レーン当たり 1 つのスカラ演算ではなく，小さな行列積和（例えば 4×4×4 の積を 1 ステップで）を実行する実行ユニットで，ニューラルネットワークの密な線形代数のためのものである．
- **Transformer Engine**．FP8 のテンソルコア演算と，層ごとに数値形式を選びスケーリングし直すソフトウェアおよびハードウェアを組み合わせた Hopper の機構であり [2]，トランスフォーマモデルのためのものである．

▷ high bandwidth memory (HBM)

▷ half precision (FP16)

▷ tensor core

▷ Transformer Engine

注意 講義では “transformer cores” と言う．Hopper にトランスフォーマ専用のコアはない．Transformer Engine はテンソルコアの FP8 経路と層ごとの形式管理である [2]．

1.5.3 現代の GPU：NVIDIA H100

H100（Hopper，2022 年）は上のリストの現状を示す．FP64，FP32，FP16，INT8，FP8 の演算をサポートする．テンソルコアと Transformer Engine が機械学習に用いられる．レジスタファイルはより大きい．**テンソルメモリアクセラレータ**がグローバルメモリと共有メモリの間で多次元のデータブロックを非同期に移動し，スレッドをアドレス生成から解放する．SM 数と SM 当たりの浮動小数点ユニット数は再び増えた．そして NVLink Switch System が多数の GPU を 1 つのマシンへ接続する．表 1.2 は，この進化が変えてきた数値を通してデータセンタ向けの 4 世代をたどる．

注意 講義 vs ホワイトペーパー：SM 当たりのレジスタファイルは A100 でも H100 でも 256 KiB であり，SM 数とともに総量だけが 27 MiB から 33 MiB へ増える（HWP Table 3 [2]）．

▷ Tensor Memory Accelerator (TMA)

第 12 章では FP8 とテンソルコアの数値形式を，第 9 章では NVLink 相互接続とマルチ GPU システムを扱う．

表 1.2: NVIDIA のデータセンタ向け 4 世代（A100 ホワイトペーパー [1]；H100 ホワイトペーパー V1.02，SXM5，暫定値 [2]）．

	P100	V100	A100	H100
年	2016	2017	2020	2022
SM 数	56	80	108	132
FP32 レーン数/SM	64	64	64	128
テンソルコア数/SM	—	8	4	4
レジスタファイル/SM	256 KiB	256 KiB	256 KiB	256 KiB
L2 キャッシュ	4 MiB	6 MiB	40 MiB	50 MiB
HBM 帯域	720 GB/s	900 GB/s	1555 GB/s	3000 GB/s
トランジスタ数	15.3×10^9	21.1×10^9	54.2×10^9	80×10^9
プロセス	16 nm	12 nm	7 nm	4N

注意 H100 の列は V1.02 ホワイトペーパーの暫定値である [2]．出荷製品は細部で異なる（PCIe 版 H100 は 114 SM）．

要点 — 章のまとめ

CPU は ILP，深いパイプライン，大きなキャッシュで 1 スレッドのレイテンシを最小化する．GPU は多数の小さなマルチスレッドコアでスループットを最大化し，ホスト CPU の下でアクセラレータとして働き，SIMT ハードウェア上で SPMD スタイルでプログラムされる．アムダールの法則は高速化率を $1/S$ で頭打ちにするので，並列プログラムは逐次部分の割合を縮めなければならない．リダクションを伴うデータ分割がスケールするパターンである．GPU のコアはストリーミング・マルチプロセッサ，そのレーンはストリーミングプロセッサであり，ワープ（NVIDIA の GPU では 32 スレッド）がフェッチとスケジューリングの単位である．GPU は固定機能のグラフィックスパイプラインから，プログラマブルおよび統合型のシェーダを経て，キャッシュ，FP64，HBM，FP16，テンソルコア，Transformer Engine を備えた汎用プロセッサへと成長した．

第 2 章

並列プログラミング

講義動画	M2，レッスン 1–4（Parallel Programming Patterns；OpenMP vs. MPI，parts 1–2；Programming with MPI）
資料	[13]（必修）；[14]；[15]
教科書	PPP [7] ch. 3–5，App. A–B（任意）；PMPP [6] ch. 20
キーワード	並列プログラミングパターン，マスタ／ワーカ，ループ並列性，フォーク／ジョイン，パイプライン，共有メモリ，分散メモリ，OpenMP，スレッド，リダクション，データ競合，クリティカルセクション，スケジューリング，バリア，MPI，ステンシル，ハロー

第 1 章では，GPU をスループット指向の機械として導入し，プログラムを並列な仕事に分割する 2 つの方法，タスク分割とデータ分割を示した．本章では，第 3 章の GPU 固有のモデルに先立ち，その仕事を従来型のマルチプロセッサ上でどのようにコードとして表現するかを示す．まず並列プログラムに繰り返し現れる**パターン**を集め，次に並列プログラムを書く対象となる 2 つのメモリモデルを対比する．共有メモリでは，スレッドが 1 つのアドレス空間を通じて通信し，OpenMP でプログラムする．分散メモリでは，プロセスが MPI でメッセージを交換する．CUDA は両者を組み合わせるので，どちらも CUDA にとって重要である．ブロック内のすべてのスレッドが共有するメモリがある一方で，明示的に移動しなければならないデータもある．本章はステンシル計算で締めくくる．そこでのプロセス間の境界交換は，GPU では第 3 章のハローの読み込みとして再び現れる．

2.1 並列プログラミングパターン

並列アプリケーションは，逐次アプリケーションから 3 つの手順で作られる．仕事を断片に分割し，断片が互いに独立に処理できるようにデータ構造を調整し，断片をプロセッサに配る（図 2.1）．**並列プログラミングパターン** とは，この分割と配布を組織する再利用可能な方法である．以下に挙げる少数のパターンが，後の章の GPU カーネルを含むほとんどの並列プログラムを覆う．

出典 M2，並列プログラミングパターン；PPP ch. 3–5．

▷ parallel programming pattern

2.1.1 4 つのステップ

並列プログラムの設計は 4 つのステップで進む．[1]

1. **並行性を見つける．**計算の**並行性**とは，その各部分が互いに独立に進行できる度合いである．最初のステップでは，そのような独立性がどこに（どのタスクに，どのデータに）存在するかを見つける．
2. **アルゴリズムを構造化し**，並行性を活用できるようにする．並列な仕事の単位は何か，それらがどう通信するかを決める．
3. 対象に適した言語とツール群で**実装する**．OpenMP，MPI，CUDA である．
4. 並列システム上で**実行し最適化する**．性能目標を満たすまで測定と調整を繰り返す．

[1] これらのステップは [7] の 4 つの設計空間，Finding Concurrency，Algorithm Structure，Supporting Structures，Implementation Mechanisms に従う．同書は以下のパターンを異なる空間に置く（Pipeline は Algorithm Structure に，SPMD，Master/Worker，Loop Parallelism，Fork/Join は Supporting Structures に）が，本コースはそれらを並べて挙げる．

▷ concurrency

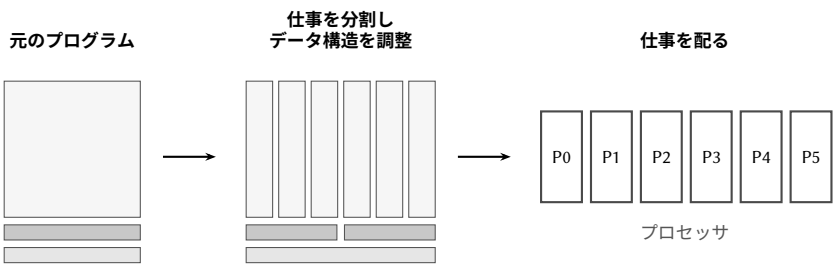


図 2.1: 並列アプリケーションの作成．仕事を分割し，データ構造を調整し，断片を配る．橙と緑の帯は，本体とは異なる分け方をするプログラムの部分である．

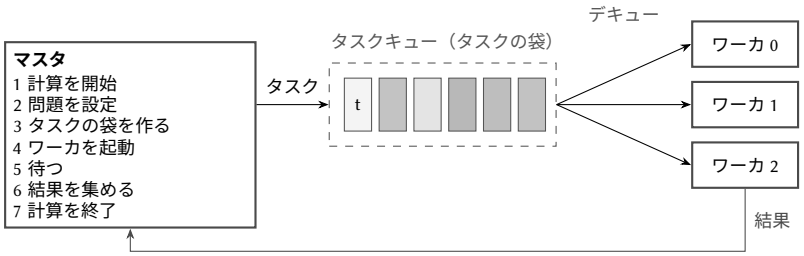
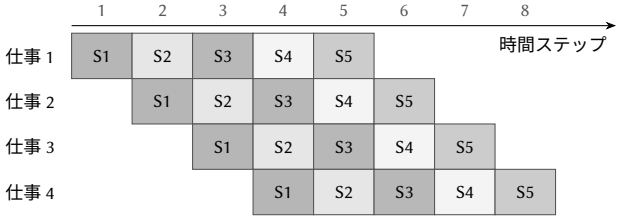


図 2.2: マスタ／ワーカパターン．マスタはタスクキューを満たして待つ．ワーカはキューが空になるまでタスクをデキューし，結果を返す．



S1 – S5: パイプラインの段 (例：伸長，処理，圧縮)

図 2.4: パイプラインパターン．4 つの仕事が 5 つの段を通過する．ステップ 4 と 5 では，5 段のうち 4 段が異なる仕事で埋まる（5 つ以上の仕事が流れていれば全段が埋まる）．s 段と n 個の仕事に対して，パイプラインは sn ステップではなく $s+n-1$ ステップで済む．ここでは 20 ではなく 8 である．

2.1.2 5 つのパターン

マスタ／ワーカ マスタ／ワーカパターン では，1 つのマスタのスレッドまたはプロセスが問題を設定し，仕事の断片を**タスクキュー**（“タスクの袋”）に入れ，ワーカのプールを起動し，待ち，結果を集め，計算を終了する（図 2.2）．各ワーカはタスクをデキューして実行することを繰り返す．ワーカは前のタスクを終えるとすぐ次のタスクを取るので，負荷は自然に均衡する．このパターンは，大きさが不揃いか未知の独立な断片に分かれる仕事に向く．

▷ master/worker pattern

▷ task queue

ループ並列性 ループ並列性 では，ループの反復が互いに独立なとき，それらを並行に実行する．例えば

```
for (int i = 0; i < 16; i++) c[i] = a[i] + b[i];
```

である．マスタ／ワーカのタスクと異なり，反復は通常，異なるデータに対する同一の操作であり，これが**データ分割**（第 1 章）のパターンとなる．

2.3 節 の OpenMP `dynamic` スケジュールは，ループ反復のチャンクをタスクとするタスクキューである．

▷ loop parallelism

SPMD すべての処理要素が，データの自分の担当部分に対して同じプログラムを実行する．**SPMD**（第 1 章）は GPU プログラミングの通常の選択であり，数千のスレッドが 1 つのカーネルを実行し，スレッドインデックスで自分のデータを選ぶ．

同じベクトル加算が第 3 章の最初の CUDA カーネルである．反復 1 つにつきスレッド 1 つ．

フォーク／ジョイン フォーク／ジョインパターン では，親タスクが子タスクをフォークし，処理を続けるか待ち，先へ進む前にそれらをジョインする（完了を待つ）（図 2.3）．単一の入口から逐次フェーズと並列フェーズを混在させる．OpenMP の並列領域はフォーク／ジョインである．

▷ fork/join pattern

パイプライン パイプラインパターン は，多段の計算の各段を 1 つのプロセッサに割り当て，データ項目を段から段へ流す．CPU パイプラインが命令を流すのとまったく同じである（図 2.4）．データストリームに向く．信号処理，第 1 章のグラフィックスパイプライン，圧縮ワークフロー（伸長，処理，圧縮）などである．

▷ pipeline pattern

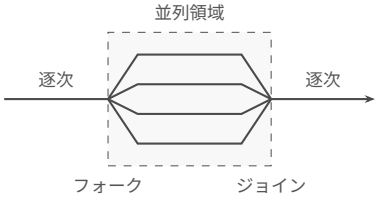


図 2.3: フォーク／ジョインパターン．1 つの制御の流れが 4 つの並列タスクにフォークし，再びジョインする．

要点
パターンは排他的ではない. GPU プログラムは, カーネルのレベルでは SPMD, その内部ではループ並列性（反復 1 つにつきスレッド 1 つ），カーネルを起動して待つホストのレベルではフォーク／ジョインである．

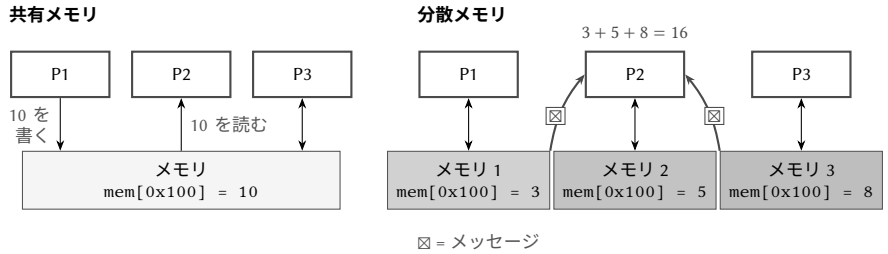


図 2.5: 共有メモリ（左）と分散メモリ（右）．同じアドレス 0x100 が，左では 1 つの場所を，右では無関係な 3 つの場所を指す．

2.2 OpenMP による共有メモリプログラミング

並列プログラムの断片どうしがどう通信するかは，それらがメモリを共有するかどうかで決まる．

出典 M2，OpenMP vs. MPI（第 1 部）；PPP App. A；[13]，メモリアーキテクチャとプログラミングモデル．

2.2.1 共有メモリと分散メモリ

定義 2.1 (共有メモリプログラミング). **共有メモリプログラミング** では，すべてのプロセッサが 1 つのメモリを読み書きする．プロセッサ 1 があるアドレスに書いた値は，プロセッサ 2 がそのアドレスを読むと見える（図 2.5，左）．通信は通常のロードとストアに暗黙に含まれる．

▷ shared-memory programming

定義 2.2 (分散メモリプログラミング). **分散メモリプログラミング** では，各プロセッサが自分のメモリを持ち，他のプロセッサからは見えない．他のプロセッサのメモリにあるデータは，**メッセージ**で要求し送ってもらわなければならない（図 2.5，右）．プロセッサ 2 はプロセッサ 1 と 3 に，それぞれのメモリのあるアドレスの内容を求め，3 つの値を加算し，結果を自分のメモリに書き戻す．

▷ distributed-memory programming

共有メモリの方がプログラムしやすいモデルである．データ共有に明示的な転送が不要だからである．しかしスケールしない．すべてのプロセッサが同じメモリを奪い合い，アクセスの同期はプログラマの責任となる．分散メモリはプロセッサ数とともにスケールし，ローカルアクセスを速く保てるが，その代償としてプログラマがすべての転送を指定しなければならない．

注意 本章の “共有メモリ” は CPU の意味，すなわちすべてのスレッドに対する 1 つのアドレス空間である．第 3 章の CUDA の `__shared__` メモリは，1 ブロックのスレッドだけが共有する小さなオンチップのスクラッチパッドである．

最大級のマシンはハイブリッドである．ノード内は共有メモリ，ノード間はメッセージで，よくある組み合わせは MPI と OpenMP，または MPI と CUDA である．[13]．

2.2.2 OpenMP

定義 2.3 (OpenMP). **OpenMP** は，C，C++，Fortran における共有メモリ並列プログラミングのオープンな標準である．**コンパイラ指示文**（`#pragma omp …`），ライブラリルーチン（`omp_…()`），**環境変数**（`OMP_…`）の集まりであり，逐次プログラムを書き直すのではなく注釈を付けることで並列に実行させる．

▷ OpenMP
▷ compiler directive
▷ environment variable

ツールが何であれ，共有メモリプログラムは 5 つのことを決めなければならない．**並列化**そのもの（どのループや領域を並列に実行するか），**スレッド数**，それらのスレッドへの仕事の**スケジューリング**，**データ共有**の方針（どの変数を共有し，どれをプライベートにするか），そして共有データの一貫性を保つ**同期**である．OpenMP はそれぞれに指示文または指示節を持つ．本節では最初の 1 つを，2.3 節 で残りの 4 つを扱う．

OpenMP 1.0 は 1997 年（Fortran）と 1998 年（C/C++）に登場した．ここで引用する仕様は 5.2（2021 年），最新は 6.0（2024 年）である．`openmp.org`．

2.2.3 スレッド

定義 2.4 (スレッド). **スレッド**とは，命令の列を実行する実体である．自分のレジスタと自分のスタックを持ち，アドレス空間の残り（コード，大域データ，ヒープ）を同じプロセスの他のスレッドと共有する．

▷ thread

スレッドはソフトウェアの概念であり，コアではない．1 つのコアが複数のスレッドを，オペレーティングシステムによるタイムスライスやハードウェアによるインタリーブで実行することがある．したがってこの語は CPU と GPU で異なるものを意味する．CPU のスレッドは独立にスケジュールされる重量級の制御の流れであるのに対し，GPU のスレッドはワープの 1 レーンであり，他の 31

Linux では新しいスレッドのスタックは，既定でプログラム開始時の `ulimit -s` のソフトリミット，多くのディストリビューションでは 8 MiB である．`pthread_attr_setstacksize` でスレッドごとに上書きできる．[16]．

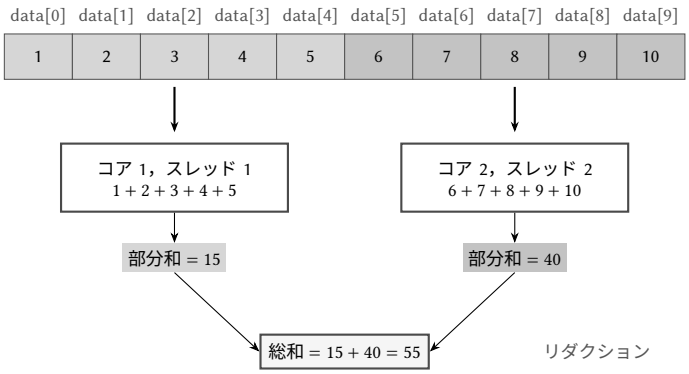


図 2.6: 総和のデータ分割．各スレッドが自分の半分を加算し，2 つの部分和が総和にリダクションされる．

レーンとロックステップで実行される（第 1 章）．本章では簡単のため，コア 1 つにつきスレッド 1 つを仮定する．

マルチスレッディングは CPU コアに 2 つ以上のハードウェアスレッドコンテキストを与える（第 1 章）．GPU の SM は現行のデータセンタ GPU で最大 64 個の常駐ワープを保持し（[5, Table 21]；第 3 章），各ワープはハードウェアがスケジュールするスレッドの群である．

2.2.4 総和を手作業で並列化する

本節を通して用いる例は，10 要素の配列の総和である．逐次版は次のとおり．

```
#include <iostream>
int main() {
  const int size = 10;
  int data[size];
  for (int i = 0; i < size; ++i) data[i] = i + 1;
  int sum = 0;
  for (int i = 0; i < size; ++i) sum += data[i];
  std::cout << "sum = " << sum << '\n';   // 55
  return 0;
}
```

2 コアにわたるデータ分割は配列を半分に分ける．`data[0..4]` はコア 1 のスレッドへ，`data[5..9]` はコア 2 のスレッドへ行く（図 2.6）．手作業で行うと，並列版は 3 つのステップからなる．

1. スレッドを 2 つ作る．
2. 各スレッドに配列の半分を渡し，その上で**部分和**を計算する（値 $1, \ldots, 10$ に対して 15 と 40）．
3. 2 つの部分和を総和 $15 + 40 = 55$ にまとめる．

このまとめは**リダクション**（第 1 章）である．多数の値を結合的な演算で 1 つにまとめる．

数千の GPU スレッドではリダクションは木になる．部分和が $\log_2$ 回のラウンドにわたって対ごとにまとめられ，ワープ内では第 5 章のシャッフル演算で行われる．

2.2.5 共有メモリでのリダクションと競合

共有メモリでは，リダクションは自然に，各スレッドが自分の部分和を初期値 0 の 1 つの共有変数 sum に加算する形で書かれる．文 `sum += partial` は 3 つの機械操作，すなわち sum のロード，加算，sum へのストアである．2 つのスレッドが表 2.1 のように交互に実行されると，両方が初期値 0 をロードし，2 番目のストアが最初のストアを上書きする．結果は 55 ではなく 40（ストアの順序が逆なら 15）になる．

定義 2.5 (データ競合，クリティカルセクション). **データ競合**は，2 つのスレッドが順序付けなしに同じ変数にアクセスし，少なくとも一方のアクセスが書き込みであるときに生じる．結果はインタリーブに依存する．共有データにアクセスするため一度に 1 つのスレッドだけが実行しなければならないコードの部分を**クリティカルセクション** という．

▷ data race

▷ critical section

注意 C と C++ ではデータ競合は単なる誤った数値ではなく未定義動作である．コンパイラはデータ競合が起きないと仮定してよく，例えば sum をループ全体にわたってレジスタに置いたままにできる．

定義 2.6 (ミューテックス). **ミューテックス**（**相互排除**に由来）とは，高々 1 つのスレッドしか保持できない**ロック**である．スレッドはクリティカルセクション

▷ mutex
▷ mutual exclusion
▷ lock

表 2.1: 共有変数 sum への保護されていない 2 つの更新のインタリーブ．一方の部分和が失われる．

ステップ	スレッド 1（部分和 15）	スレッド 2（部分和 40）	sum
1	sum をロード → 0		0
2		sum をロード → 0	0
3	加算：0 + 15 = 15		0
4		加算：0 + 40 = 40	0
5	15 をストア		15
6		40 をストア	40

に入る前にミューテックスを**ロック**（獲得）し，出るときに**アンロック**（解放）する．ミューテックスが既に保持されていれば，スレッドは待つ．sum の更新をミューテックスの中に置けば，2 つの加算は順に行われ，結果は 55 になる．

POSIX システムでこれを低水準に書く方法が **POSIX スレッド**であり，C の呼び出しからなるライブラリである．

- `pthread_create` は与えられた関数でスレッドを開始する．
- `pthread_join` はスレッドの終了を待つ．
- `pthread_mutex_lock` と `pthread_mutex_unlock` はクリティカルセクションを囲む．

スレッドの生成，データの分配，ロック，ジョインはすべてプログラマのコードであり，そのそれぞれがバグの入り込む場所である．

▷ POSIX threads (pthreads)

Pthreads は IEEE POSIX 1003.1c（1995 年）で，コンパイラの支援のないライブラリである．すべてのスレッドとすべてのロックが明示的である．[13]．

2.2.6 OpenMP 版

OpenMP はそのすべてを 1 つの指示文の裏に隠す．

```
#include <iostream>
#include <omp.h>
int main() {
  const int size = 1000;
  int data[size];
  for (int i = 0; i < size; ++i) data[i] = i + 1;
  int sum = 0;
  #pragma omp parallel for reduction(+:sum)
  for (int i = 0; i < size; ++i) sum += data[i];
  std::cout << "sum = " << sum << '\n';   // 500500
  return 0;
}
```

ヒント OpenMP の支援なしに（`-fopenmp` なしで）コンパイルすると，プラグマは無視されプログラムは逐次版になる．動作する逐次プログラムに OpenMP を 1 ループずつ加えていける．

`parallel for` はループ並列性パターンを呼び出す．スレッドのチームを作り，反復をそれらに分配し，ループの終わりでスレッドをジョインする．指示節 `reduction(+:sum)` はコンパイラに，sum が + によるリダクションであることを伝える．各スレッドは単位元 0 で初期化されたプライベートなコピーを得てそこに累積し，ループが終わるとコピーが共有変数 sum に，保護された形でまとめられる（*，max，min，およびビット演算子と論理演算子も使える）．

定義 2.7 (指示文と指示節). OpenMP の指示文は次の形をとる．

```
#pragma omp directive [clause [clause] ...]
```

directive は構文を指名し（`parallel`，`for`，`critical`，...），**指示節**がそれを修飾する（`reduction(+:sum)`，`num_threads(4)`，`schedule(dynamic)`）．コンパイラは各指示文を OpenMP ランタイムライブラリへの呼び出しに置き換え，ランタイムがスレッドを生成し，スケジュールし，ジョインする．

▷ clause

2.3 スケジューリング，データ共有，同期

共有メモリプログラムの残り 4 つの課題，スレッド数，スケジュール，共有の方針，同期には，それぞれ OpenMP の構文がある．

出典 M2，OpenMP vs. MPI（第 2 部）；PPP App. A；[14]．

2.3.1 スレッド数

並列領域のスレッド数は 3 つの場所のいずれかで設定する．
- プログラムの外で，環境変数 OMP_NUM_THREADS により．
- プログラムの中で，ライブラリ呼び出し omp_set_num_threads() により．
- 1 つの領域に対して，指示節 num_threads(*n*) により．

何も設定しなければ，既定はマシンのハードウェア並列度，すなわちオペレーティングシステムが報告する論理プロセッサ（ハードウェアスレッド）の数である．コア数より多いスレッドも許される．オペレーティングシステムがそれらをタイムスライスする．次の例はこの状況から始まる．

既定値は実装定義である [14]．GCC の libgomp はプロセスが使える CPU 1 つにつきスレッド 1 つを起動する [17] ので，コア当たり 2 ハードウェアスレッドの 16 コア CPU では 32 スレッドが起動する．

2.3.2 ループのスケジューリング

parallel for は，どのスレッドがどの反復を実行するかを決めなければならない．この決定が重要になるのは，スレッドの進む速さが同じでないときである．

例 2.8 (進み方の不揃いなスレッド). 等しいコストの 600 000 回の反復からなるループを，4 コアのマシンで 6 スレッドで実行する．オペレーティングシステムはコア 0 と 1 にそれぞれ 2 スレッド，コア 2 と 3 にそれぞれ 1 スレッドを置くので，2 つずつ載ったコア上の 4 スレッドはそれぞれコアの半分しか受け取れない．コア 1 つ丸ごとで 100 000 回の反復に要する時間を t とする．
- **静的分割**，スレッド当たり 100 000 回：コア 2 と 3 のスレッドは時刻 t に，他の 4 つは $2t$ に終わる．ループは $2t$ で終わり，その半分の間 2 つのコアが遊ぶ．
- **理想**：600 000 回を 4 コアに分散すると，コア当たり 150 000 回，すなわち $1.5t$ である．
- **動的，チャンク** 1000：各スレッドは 1 つのチャンクを終えると次のチャンクを取る．コア 1 つ丸ごとを持つスレッドは半分のコアのスレッドの 2 倍の頻度でチャンクを取るので，6 つすべてが $1.5t$ からおよそ 1 チャンク以内に終わる．チャンク要求は 600 回である．

ここでは静的スケジュールは動的スケジュールより 33 % 長くかかり，それは純粋にスレッドの不均等な配置による．

OpenMP の schedule(*kind*, *chunk*) 指示節がスケジュールの種類を選ぶ．実際に使われる 3 つは次のとおりである（図 2.7）．

さらに 2 種類がある．auto は選択をコンパイラまたはランタイムに任せ，runtime は環境変数 OMP_SCHEDULE から取る．[14]．

static **静的スケジューリング** は，ループ開始前に反復をチャンクに分け，固定のラウンドロビン順でスレッドに割り当てる．チャンクサイズを指定しなければ，各スレッドは約 N/スレッド数 回の連続した 1 ブロックを得る．実行時コストはなく，適応もしない．

▷ static scheduling

dynamic **動的スケジューリング** は，与えられた**チャンクサイズ**（既定 1）のチャンクを要求に応じて配る．チャンクを終えたスレッドが次を求める．チャンクサイズ 1 は最もよく適応するが，反復ごとに要求のコスト，すなわち共有カウンタのアトミック更新を払う．1000 ならそのコストを千回の反復で償却する．

▷ dynamic scheduling
▷ chunk size

guided **ガイド付きスケジューリング** は，大きなチャンクから始めてループが進むにつれ縮める（例えば 1000 から 100 まで）．序盤の大きなチャンクが要求コストを低く保ち，終盤の小さなチャンクが終了時刻を揃える．

▷ guided scheduling

guided のチャンクは残り反復数をスレッド数で割った値に比例し，最後のチャンクを除いて *chunk* 引数を下回らない．[14]．

動的スケジューリングとガイド付きスケジューリングは，実行時にスレッドを不均等にするあらゆるものに適応する．例 2.8 のような不均等な配置，異種混在クラスタ内の遅いマシン，コストの異なる反復，他のプログラムからの干渉などである．すべてのスレッドを最後まで忙しくしておくことが**負荷分散**である．並列ループの時間は，その最も遅いスレッドの時間である．

▷ load balancing

ヒント 反復が等しい規則的なループは static．不規則または未知のコストなら，要求のコストがチャンクよりはるかに小さくなる大きさのチャンクで dynamic．反復数が大きければ guided．

2.3.3 データ共有

総和では，各スレッドの部分和はそのスレッド自身のコピーでなければならず，一方で総和は 1 つの場所である．OpenMP は前者を**プライベート変数**（reduction のコピーのように，スレッドごとに 1 つのインスタンス），後者を**共有変数**（全員が見る 1 つのインスタンス）と呼ぶ．プログラマは private(…) と shared(…) 指示節で方針を述べる．既定では，並列領域の外で宣言された変数は共有，ルー

▷ private variable
▷ shared variable

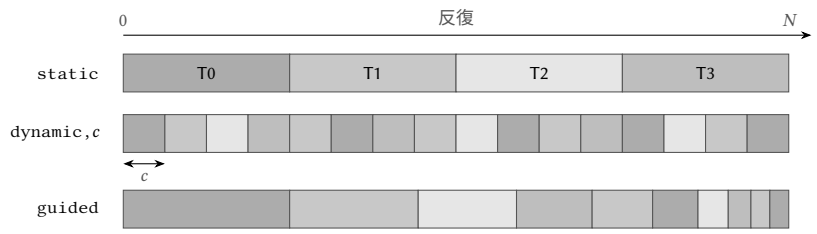


図 2.7: 4 スレッドに対する 3 つのループスケジュール．static は各スレッドに 1 ブロックを事前に割り当てる．dynamic はスレッドが求めた順に固定サイズのチャンクを配る．guided は縮んでいくチャンクを配る．

プインデックスはプライベートである．

2.3.4 同期

同期こそが共有メモリプログラムを正しくする．異なるスレッドの共有データへのアクセスを順序付けるからである．OpenMP は 3 つの構文を提供する．

バリア **バリア**とは，チームのすべてのスレッドが到達するまで，どのスレッドも先へ進めない地点である（図 2.8）．バリアの後の仕事（緑）は，その前のすべての仕事（青）が終わるまで始まらない．`#pragma omp barrier` と書く．`parallel` 領域の終わりは常に暗黙のバリアであり，`for`，`sections`，`single` 構文の終わりも，`nowait` 指示節で取り除かない限り同様である．

クリティカル `#pragma omp critical [(name)]` は続くブロックをクリティカルセクションにする．一度に 1 つのスレッドがそれを実行し，他は入口で待つ（図 2.9）．領域に 4 スレッドあれば，ブロックの 4 回の実行は途切れなく順に直列化される．異なる名前のクリティカルセクションは並行に実行されうる．名前のないものはすべて 1 つのロックを共有する．

```
#pragma omp parallel num_threads(4)
{
  // 並列部分 ...
  #pragma omp critical
  {
    shared_total += local_value;    // 一度に1スレッドだけ
  }
}
```

アトミック `#pragma omp atomic` は，続く 1 つの更新を**アトミック操作**にする．そのロード，加算，ストアは全体として起こるか，まったく起こらないかのどちらかであり，他のスレッドの更新がその間に割り込むことはない．ハードウェアの read-modify-write 命令があればそれで，なければロックで実装され，適用できる場合にはクリティカルセクションより安価である．

```
#pragma omp parallel for
for (int i = 0; i < n; ++i) {
  #pragma omp atomic
  counter++;          // ロード，加算，ストアを一体として
}
```

注意 既定のまま共有で残され，ループ内で書かれる変数はデータ競合である．`default(none)` はすべての変数を明示的に列挙するよう強制し，書き忘れをコンパイルエラーにする．

▷ barrier

CUDA の `__syncthreads()`（第 3 章）は 1 ブロックのスレッドにわたるバリアである．グリッド全体にわたるバリアは，協調起動を伴う cooperative groups によってのみ存在する（第 5 章）．

▷ atomic operation

CUDA の `atomicAdd()` が GPU で同じ役割を果たす．第 5 章では控えめに使うべき操作の 1 つに挙げる．

注意 `atomic` が扱うのは `x++`，`x += e` などの形の 1 つのスカラ更新である．2 つの文や，一貫性を保たなければならない 2 つの変数の更新には `critical` が要る．

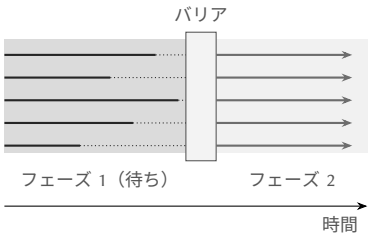


図 2.8: バリア．5 つのスレッドすべてがフェーズ 1 を終えるまで，どのスレッドもフェーズ 2 を始めない．

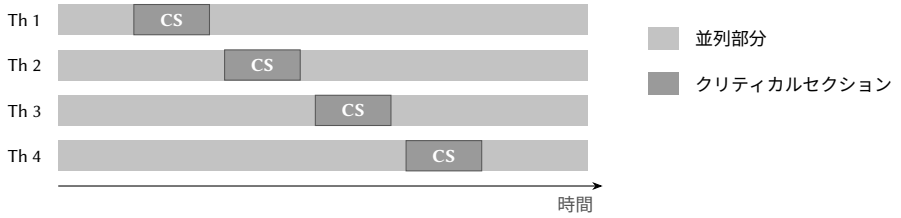


図 2.9: 4 つのスレッドと 1 つのクリティカルセクション．赤の実行は決して重ならず，残りの仕事は重なる．

2.3.5 ループ以外の仕事

並列な仕事のすべてがループとは限らない．**並列セクション**は，独立なコードブロックを並行に，セクション 1 つにつきスレッド 1 つで実行する．

▷ parallel sections
OpenMP 3.0（2008 年）は `#pragma omp task` を加えた．`sections` を動的なタスク集合に一般化したもので，2.1 節 のマスタ／ワーカパターンである．[14].

```
#pragma omp parallel sections
{
  #pragma omp section
  { work_1(); }
  #pragma omp section
  { work_2(); }
}
```

さらに 2 つの構文は，並列性を作るのではなく制限する．

ordered `for ordered` と宣言したループの内側で，`#pragma omp ordered` の付いたブロックは，各反復の残りが並列に実行されても反復順に実行される．5 つのスレッドから “スレッド *k* が反復 *i* を実行中” と印字するループは，`ordered` なしでは行を任意の順（例えば 1，2，4，3，5）で，ありでは 1，2，3，4，5 の順で印字する．

注意 `ordered` ブロックの中身はすべて直列化される．本体の大半が ordered であるループは並列にしても何も得られない．

single `#pragma omp single` は，ブロックをチームのちょうど 1 つのスレッド（どれかは不定）で実行し，他のスレッドはそれを飛ばして直後の暗黙のバリアで待つ．並列領域内での 1 回限りの初期化やメッセージの表示に典型的である．

ヒント 他のスレッドが `single` ブロックを待つ必要がなければ，`single nowait` でそのバリアを外せる．次の `for` や `barrier` がなお全体を順序付ける．

要点
OpenMP は共有メモリプログラムの 5 つの決定を明示化する．`parallel for`（何を），`OMP_NUM_THREADS`（いくつで），`schedule`（誰がどの反復を），`private`/`shared`（誰のデータを），そして `barrier`/`critical`/`atomic`（どの順序で）．同じ 5 つの決定は CUDA では，カーネル，起動構成，ブロックスケジューラ，メモリ空間，`__syncthreads()` として再び現れる．

2.4 MPI によるプログラミング

CUDA プログラミングは 2.2 節の 2 つのモデルを組み合わせる．GPU のすべてのコアで共有されるメモリ領域がある一方，外からは見えず明示的な通信を必要とする領域もある．したがって GPU プログラマには，共有メモリの習慣と同じくらい分散メモリの習慣が必要であり，それがここでメッセージパッシングを導入

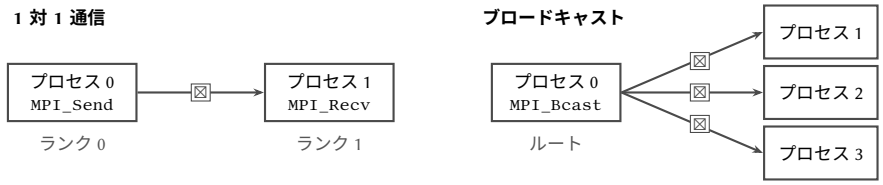


図 2.10: MPI の 1 対 1 通信（左）と集団通信（右）．

する理由である．

2.4.1 プロセスとメッセージ

定義 2.9 (MPI). **MPI** は**メッセージパッシング**の標準ライブラリである．それぞれが自分のアドレス空間を持つ**プロセス**の集合が，同じノード上または異なるノード上で，メッセージの送受信だけによってデータを交換する．各プロセスは**ランク**，すなわち 0 からプロセス数より 1 小さい値までの整数で識別される．

ライブラリの大半は 2 種類の操作に収まる（図 2.10）．

1 対 1 **1 対 1 通信** にはちょうど 2 つのプロセスが関わる．プロセス 0 が整数と宛先ランク 1 を指定して `MPI_Send()` を呼び，プロセス 1 が送信元ランク 0 を指定して `MPI_Recv()` を呼んで値を得る．メッセージは宛先が発行した対応する受信にのみ配達される．すべての転送に両側が関与する．

集団 **集団通信** にはグループのすべてのプロセスが一度に関わる．最も単純なのは**ブロードキャスト**，`MPI_Bcast()` である．ルートプロセスの値が他のすべてのプロセスに配達される．すべてのプロセスが同じ関数を呼ぶ．

最小限のプログラムで両方の呼び出しを示す．同じ実行ファイルがプロセスごとに 1 回起動され（`mpirun -np 4 ./a.out` は 4 つ起動する），どのコピーも先頭から実行し，コピーを区別して各自の仕事を決める唯一のものがランクである（これも SPMD である）．

```
#include <mpi.h>
int main(int argc, char **argv) {
  int rank, value = 0;
  MPI_Init(&argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  if (rank == 0) {
    value = 42;
    MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
  } else if (rank == 1) {
    MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);
  }
  MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);
  MPI_Finalize();
  return 0;
}
```

2.4.2 分散データ上のステンシル

定義 2.10 (ステンシル). **ステンシル**計算は，格子の各要素を，その近傍の固定パターンから更新する．ここでの例は各要素 C を北，西，東，南の 4 近傍の平均で置き換える（図 2.11，左）．

$$C' = \frac{N + W + E + S}{4}. \tag{2.1}$$

ステンシルは高性能計算でよく現れる（差分法ソルバ，画像フィルタ）．

格子を 4 つのプロセスに分散すると，各プロセスは 1 つの四分区画を所有し，それにしかアクセスできない（図 2.11，左）．プロセス 2 の領域の端にある要素 C は，プロセス 0 に属する近傍 1 と，プロセス 3 に属する近傍 3 を必要とする．ロードで取ってくることはできず，メッセージで届かなければならない．

出典 M2，MPI によるプログラミング；PPP App. B；[13]，メッセージパッシングモデル；PMPP ch. 20．

▷ MPI (Message Passing Interface)
▷ message passing
▷ process

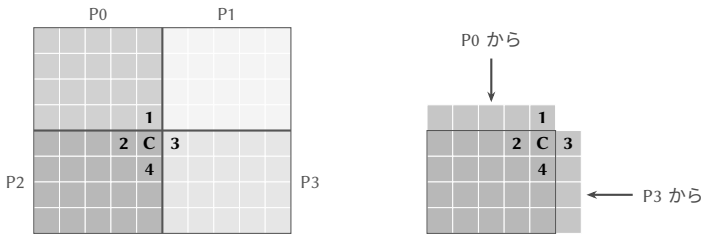
▷ rank
MPI Forum は 1992 年，MPI-1 は 1994 年．ここで引用するのは 4.1 版（2023 年），最新は 5.0（2025 年）である．[13]．

▷ point-to-point communication
メッセージのコストはレイテンシ（0 バイトのメッセージの送信，典型的にはマイクロ秒）に，サイズを帯域幅で割った値を加えたものである．多数の小さなメッセージは 1 つの大きなメッセージよりはるかに悪い．[13]．

▷ collective communication
▷ broadcast
他の集団通信：*scatter*（1 つの配列をランク間で分割），*gather*（その逆），*reduce*（ランクごとの 1 つの値をまとめる，例えば大域総和）．[15]．

▷ stencil

ステンシルは点の数で呼ばれる．これは 5 点ステンシルであり（多くの変種は中心も読む），式 (2.1) は離散ラプラス方程式，すなわち定常熱伝導に対する 1 回の Jacobi 反復である．3 × 3 フィルタによる画像畳み込みは 9 点ステンシルで，第 3 章で扱う．



4 プロセスに分割した格子　　P2 の領域とハロー（灰色）

図 2.11: 4 つのプロセスに分散した格子上のステンシル．左：要素 C とその 4 近傍は 3 つの領域にまたがる．右：更新前にプロセス 0 と 3 から受け取ったハローセル（灰色）を含むプロセス 2 の領域．

定義 2.11 (ハローと境界交換)．プロセスが読むが所有しない隣接領域のセルが，そのプロセスの**ハロー領域**をなす．半径 1 のステンシルでは，共有する各辺に沿った 1 行または 1 列である（図 2.11，右）．更新ステップのたびに，プロセスは**境界交換**を行う．各プロセスは隣が必要とする端のセルを送り，隣から自分のハローを受け取る．その後はステンシルのすべての読み出しがローカルになる．

▷ halo (ghost cells)

▷ boundary exchange

各プロセスはステップごとに，隣接する辺 1 つにつき 1 つのメッセージを送り，1 つ受け取る．2×2 分割では送信 2 回と受信 2 回である．ステンシルの半径を 2 倍にするとハローの幅が 2 倍になるが，メッセージ数は変わらない．

データ依存がプロセス間の境界をまたぐときは必ず，この交換が分散メモリの代価となる．同じ構造が第 3 章で再び現れる．CUDA のスレッドブロックは格子のタイルを共有メモリに所有し，計算の前にタイルの周囲のハローを読み込まなければならない．ただしそこではハローはメッセージで受け取るのではなく，グローバルメモリから読む．

注意 プロセスは，ハローセルを運ぶメッセージが届く前にそのセルを読んではならず，端のセルを更新する前に送ってもならない．分散ステンシルは通信と計算を交互に行い，ハローがまだ転送中の間に更新できるのは内部のセルだけである．

要点 — 章のまとめ

並列プログラムは少数のパターンを再利用する．タスクキューを持つマスタ／ワーカ，ループ並列性，SPMD，フォーク／ジョイン，パイプラインである．共有メモリではスレッドが 1 つのアドレス空間を通じて通信し，OpenMP は逐次プログラムに，並列ループ，スレッド数，スケジューリング（static, dynamic, guided），データ共有（private, shared），同期（barrier, critical, atomic）の指示文で注釈を付ける．共有変数の保護されていない更新はデータ競合である．分散メモリでは，ランクで識別されるプロセスが MPI でメッセージを交換する．1 対 1（`MPI_Send`/`MPI_Recv`）または集団（`MPI_Bcast`）である．分散ステンシルは毎ステップの前にハローセルの境界交換を必要とし，これが第 3 章の CUDA タイルのハロー読み込みのモデルとなる．

第 3 章

GPU プログラミング入門

講義動画	M3，レッスン 1–4（Introduction of CUDA Programming；Occupancy；Host Code；Stencil Operation with CUDA）
資料	[18]（共有メモリ）；NVIDIA Hopper ホワイトペーパー [2]（スレッドブロッククラスタ）
教科書	K12 [3] §1.3；A18 [4] ch. 2；CPG [5] §2 Programming Model，§3.2 CUDA Runtime，§4.2 Hardware Multithreading；BPG [8] §11 Execution Configuration Optimizations；PMPP [6] ch. 2–4，7–8
キーワード	カーネル，ホストコード，グリッド，スレッドブロック，スレッドインデックス，実行構成，共有メモリ，バリア同期，BSP，メモリ空間，占有率，Compute Capability，cudaMalloc，cudaMemcpy，ステンシル，タイル，ハロー

第 1 章では，GPU を，ワープ単位でスレッドを実行するストリーミング・マルチプロセッサから成るスループット機械として導入し，第 2 章では，並列な仕事を共有メモリでは OpenMP で，分散メモリ間では MPI で表現する方法を示した．本章はその両方を具体的な CUDA プログラムに置き換える．まずカーネルと，それが実行されるスレッド，ブロック，グリッドの階層，それらのスレッドから見えるメモリ空間，そしてブロック内のスレッドが同期に用いるバリアを導入する．次に，1 つのマルチプロセッサに同時にいくつのブロックが載るか（占有率）を問い，デバイスとの間でデータを移すホストコードを書き，最後に第 2 章のステンシルを CUDA で書き直す．そこでは MPI の境界交換が，共有メモリへのハローの読み込みに置き換わる．

3.1 CUDA プログラミング入門

CUDA プログラムは 2 つのプロセッサで実行される 2 つの部分に分かれる．CPU で動くコードと，CPU が GPU に渡して多数のスレッドに同時に実行させる関数である．

出典 M3，CUDA プログラミング入門；K12 §1.3；CPG §2.1 Kernels，§2.2 Thread Hierarchy，§2.3 Memory Hierarchy；PMPP ch. 2–3；A18 ch. 2．

3.1.1 カーネルとホストコード

定義 3.1 (カーネル，ホストコード，デバイスコード). **カーネル**とは，GPU 上で動き，多数のスレッドによって並列に実行される関数である．`__global__` 修飾子で宣言する．**ホストコード**は，プログラムのうち CPU（**ホスト**，第 1 章）で動く部分であり，データを準備し，GPU のメモリを管理し，カーネルを起動する．**デバイスコード** は GPU，すなわち**デバイス**で動く部分であり，カーネルとそれが呼び出す関数から成る．

▷ kernel

▷ host code

▷ device code
▷ device
修飾子：`__global__` はカーネル（ホストから呼ばれ，デバイスで動く），`__device__` はデバイスコードから呼ばれる関数，`__host__` は通常の CPU 関数を表す．CPG §7.1 Function Execution Space Specifiers [5]．

最初のカーネルは 2 つのベクトルを加算する．各スレッドは 1 つの要素を計算する．組み込み変数から自分のインデックス `i` を求め，そのインデックスが配列の範囲内にあることを確かめ，加算を 1 回行う．

```
__global__ void vectorAdd(const float *a, const float *b,
                          float *c, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) c[i] = a[i] + b[i];
}
```

要素に対するループはない．逐次プログラムのループ（第 2 章のループ並列性）は，反復 1 つにつき 1 スレッドのスレッドの集合になった．ホストコードは三重山括弧の構文でカーネルを起動し，生成するスレッドの数とそのまとめ方を指定する．

```
int threads = 256;
int blocks = (n + threads - 1) / threads;  // ceil(n / 256)
vectorAdd<<<blocks, threads>>>(d_a, d_b, d_c, n);
```

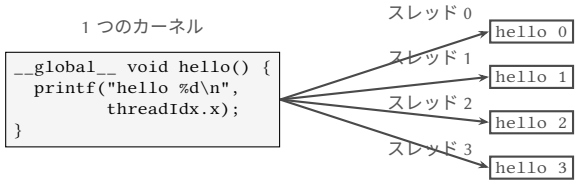


図 3.1: SPMD：1 つのカーネル，4 つのスレッド．すべてのスレッドが同じコードを実行し，`threadIdx.x` だけが互いを区別する．出力の順序は何も定めていない．

定義 3.2 (カーネル起動，実行構成). **カーネル起動**とは，ホストがデバイス上でカーネルを開始する文 `kernel<<<numBlocks, threadsPerBlock>>>(args)` である．山括弧の間の部分が**実行構成**，すなわちスレッドブロックの数とブロック当たりのスレッド数である．生成されるスレッドの総数はその積である．

▷ kernel launch

▷ execution configuration

ヒント ブロック数は n をブロックサイズで割った値の切り上げであり，整数演算では `(n + threads - 1) / threads` と書く．このとき最後のブロックには遊んでいるスレッドができるが，範囲検査 `if (i < n)` がそれらに配列の外への書き込みをさせない．

3.1.2 SPMD とスレッドインデックス

カーネルのスレッドはすべて同じコードを実行する．GPU プログラミングは**SPMD**（第 1 章）である．メッセージを表示するだけのカーネルを 4 スレッドで起動すると，メッセージは 4 回，順序不定で表示される．スレッドに異なる仕事をさせるには，各スレッドが自分がどれであるかを知らなければならない．

```
__global__ void hello() {
  printf("hello %d\n", threadIdx.x);
}
// ホスト側:
hello<<<1, 4>>>();   // hello 0..3 を順不同で表示
```

定義 3.3 (スレッドインデックス). **スレッドインデックス** `threadIdx` は，スレッドごとに異なる組み込み変数であり，そのスレッドのブロック内での位置を与える．3 つの成分 `threadIdx.x`，`.y`，`.z` を持つので，ブロックをスレッドの線，長方形，直方体として，データの形（ベクトル，画像，ボリューム）に合わせて配置できる．

▷ thread index

インデックスを使うと，ベクトル加算の最も単純な形は `c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x]` となる．スレッド 0 が要素 0 を，スレッド 1 が要素 1 を加算し，以下同様である（図 3.1）．この形は 1 つのブロックでしか動かない．上のカーネルがさらに 2 つの組み込み変数を用いるのはそのためである．

3.1.3 スレッド階層

定義 3.4 (スレッドブロック，グリッド). スレッドはブロックに，ブロックはグリッドにまとめられる．**スレッドブロック**とは，1 つのストリーミング・マルチプロセッサ上でまとめて実行され，共有メモリとバリアを通じて協調できるスレッドの集まりである．PTX ではこれを**協調スレッド配列（CTA）**と呼ぶ．**グリッド**は，1 回のカーネル起動の全ブロックの集合である．**ブロックインデックス** `blockIdx` はグリッド内でのブロックの位置を与え，組み込み変数 `blockDim` と `gridDim` は各次元のブロックとグリッドの大きさを保持する．スレッド，ブロック，グリッドというこの 3 段の構造が**スレッド階層**である（図 3.2）．

▷ thread block

▷ cooperative thread array (CTA)

▷ grid

▷ block index

▷ thread hierarchy

上限，CPG Table 21 [5]：ブロック当たり 1024 スレッド；ブロックは $x, y \le 1024$，$z \le 64$；グリッドは $x \le 2^{31}-1$，$y, z \le 65535$（CC 5.0–9.0）．

後の CUDA はワープレベルのプリミティブ（シャッフル，ボート，第 5 章）を通じてワープを露出している．ハードウェアは今でもすべてのブロックを `warpSize` = 32 個の連続したスレッドのワープに切り分ける（第 4 章）．

第 1 章のワープがこの階層にないのは意図的である．ワープはマイクロアーキテクチャ上の単位，すなわちハードウェアがまとめて発行する，ブロック内の連続した 32 スレッドの集まりであり，初期の CUDA のプログラミングモデルはそれを隠していた．プログラマが考える単位はブロックであり，グリッドは起動の単位である．

グローバルインデックス． 複数ブロックのグリッドのすべてのスレッドに別々の要素を与えるには，スレッドインデックスをブロックインデックスと組み合わ

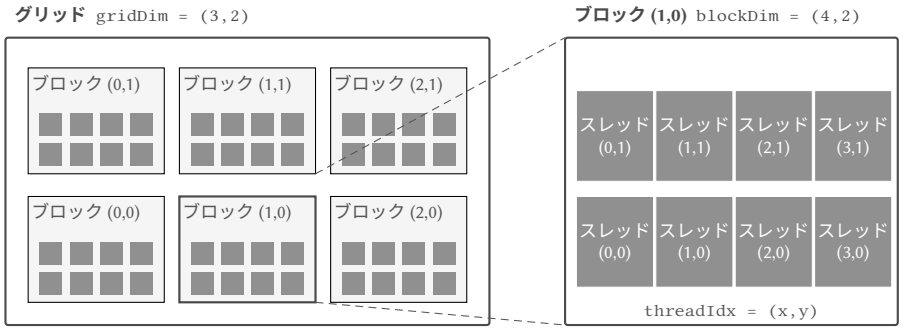


図 3.2: スレッド階層．3×2 ブロックのグリッドで，各ブロックは 4×2 スレッドから成る．スレッドは `blockIdx` と `threadIdx` で指定される．

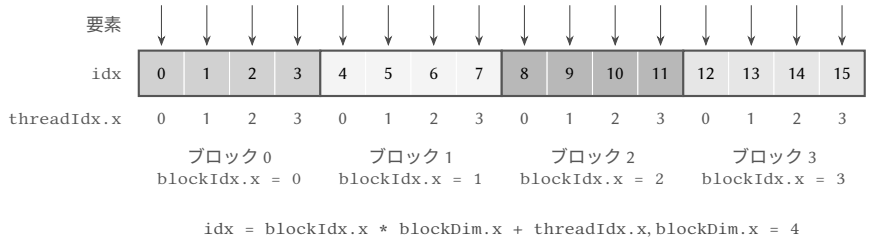


図 3.3: 4 スレッドずつの 4 ブロックにわたる 16 要素のグローバルインデックス．ブロックインデックスが 4 個の組を選び，スレッドインデックスがその中の要素を選ぶ．

せる．1 次元のグリッドとブロックでは

$$\texttt{idx} = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x}. \tag{3.1}$$

図 3.3 は，16 個の要素を 4 スレッドずつの 4 ブロックで処理する様子を示す．ブロック 2 は要素 8–11 を持ち，そのスレッド 1 は要素 9 を受け持つ．

例 3.5 (インデックスの計算). 1000 個の float の配列をブロック当たり 128 スレッドで処理する．

- ブロック数：$\lceil 1000/128 \rceil = 8$ で，$8 \times 128 = 1024$ スレッドが生成される．最後の 24 スレッドは範囲検査に失敗し，何もしない．
- `blockIdx.x = 5, threadIdx.x = 77` のスレッドは要素 $5 \times 128 + 77 = 717$ を処理する．
- 要素 900 はブロック $\lceil 900/128 \rceil = 7$ のスレッド $900 - 7 \times 128 = 4$ が処理する．

幅 W の 2 次元画像では，同じ計算を次元ごとに行う．すなわち

```
x = blockIdx.x * blockDim.x + threadIdx.x,
y = blockIdx.y * blockDim.y + threadIdx.y
```

とし，要素は `img[y * W + x]` である．

3.1.4 実行順序

このモデルには，スレッドやブロックの順序を定めるものは何もない．自分のインデックスを表示する 4 つのスレッドは 0，1，2，3 と表示することも，3，2，1，0 や 0，2，3，1 と表示することもある．図 3.4 の 6 個のブロックは，インデックス順に 1 つずつ実行されることも，他の任意の順に 1 つずつ実行されることも，いくつか同時に実行されることもある．正しいカーネルは，これらのどの順序の下でも同じ結果を出す．

本節で用いる最も単純なモデルは，各ブロックが 1 つのストリーミング・マルチプロセッサで動き，ブロックどうしは互いに独立だというものである．3.2 節はその前半を精密にする．SM は通常，同時に複数のブロックを保持する．

3.1.5 共有メモリ

グリッドのブロックは互いに独立なので，データはブロック間で分割しなければならず，ブロック内のスレッドには自分たちの仕事を結合する場所が必要になる．

ブロック内では，CPG §2.2 は大きさ (D_x, D_y) の 2 次元ブロックのスレッドを $x + yD_x$ と番号付けする．連続する `threadIdx.x` の値は連続するスレッドであり，したがって同じワープに入る．[5].

CPG §2.2：スレッドブロックは “独立に実行されることが求められる．任意の順序で，並列にも逐次にも実行できなければならない”[5]．この独立性があるから，カーネルは SM が 2 個の GPU でも 132 個の GPU でも変更なしに動く．

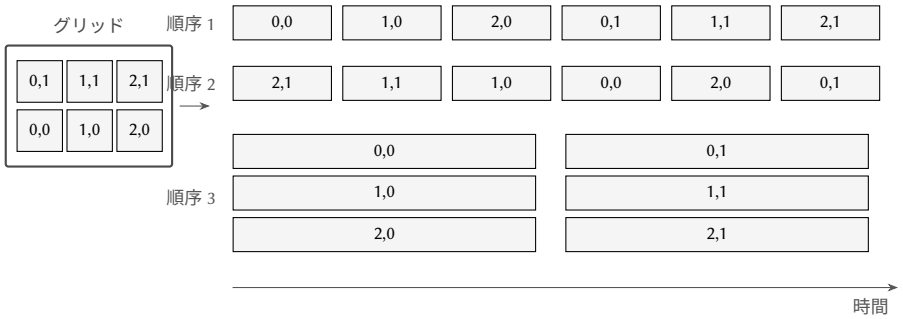


図 3.4: 6 個のブロックから成るグリッドの 3 通りの正当な実行：インデックス順，任意の順，3 個ずつ同時．どれを選ぶかはハードウェアが決め，その選択は実行ごとに異なりうる．

定義 3.6 (共有メモリ). マルチプロセッサの**共有メモリ** とは，チップ上の小さく高速なメモリであり，1 つのブロックのすべてのスレッドから見え，他のブロックからは見えない．これは**スクラッチパッドメモリ** である．つまりソフトウェアが管理し，ハードウェアが満たすキャッシュとは異なり，カーネル自身がデータをそこへコピーし，そこから取り出す．変数は`__shared__` 修飾子で共有メモリに置かれ，ブロックが存在する間だけ存続する．

▷ shared memory

▷ scratchpad memory

次のカーネルでは，各スレッドが 1 つの和を共有メモリに書き，次に別のスレッドが書いた和を読む．

Tesla と Fermi の GPU のレイテンシ，K12 Table 1.4 [3]：共有メモリ 4–32 サイクル，グローバルメモリ数百サイクル．[18] は，共有メモリのレイテンシをキャッシュされないグローバルメモリのおよそ 100× 低いとしている．

```
__global__ void addReverse(const int *a, const int *b,
                           int *c) {
  __shared__ int s[256];          // 256スレッドの1ブロック
  int i = threadIdx.x;
  s[i] = a[i] + b[i];
  __syncthreads();                // 全書き込みの完了
  c[i] = s[255 - i];              // 別のスレッドの和
}
```

読み出し `s[255 - i]` が正しいのは，スレッド $255 - i$ がすでにその値を格納している場合だけである．スレッドの実行順序は決まっていないので，カーネルは全スレッドを待たなければならない．

3.1.6 同期

定義 3.7 (バリア同期). `__syncthreads()` は，1 つのブロックのスレッドにわたる**バリア同期**を行う．ブロックの各スレッドは，全スレッドがその呼び出しに到達するまでそこで待ち，その後全スレッドが先へ進む．また，バリアより前に行われた共有メモリとグローバルメモリへの書き込みが，バリアより後でブロックのスレッドから見えることも保証する．

注意 第 2 章は “共有メモリ” を CPU の意味，すなわち全スレッドに共通の 1 つのアドレス空間の意味で用いた．CUDA の共有メモリはブロックごとでチップ上にあり，あるブロックが書き込んだ値は他のどのブロックからも見えず，ブロックが終わると消える．

▷ barrier synchronization

バリアは 1 つのブロックにしか及ばない．異なるブロックのスレッドは `__syncthreads()` で同期できない．ある段階を全ブロックが終えてからでないと次の段階をどのブロックも始められない計算では，段階を別々のカーネルとして書く．カーネル起動は，その全ブロックが終わってはじめて完了するからである．

注意 ブロックのすべてのスレッドが同じ `__syncthreads()` に到達しなければならない．一部のスレッドだけが入る `if` の中のバリアは未定義であり（実際にはハングする），バリアは条件の外に置く．

CUDA 9 以降，cooperative groups と協調起動によるグリッド全体のバリアがある．ただし全ブロックが同時に常駐している必要がある（第 5 章）．

定義 3.8 (バルク同期並列). 共有メモリを使うカーネルの通常の形は，バリアで区切られた 3 つの段階である（図 3.5）．ブロックのデータをグローバルメモリから共有メモリへ読み込む；`__syncthreads()`；共有のコピーに対して計算する；`__syncthreads()`；結果をグローバルメモリへ書き戻す．並列段階とバリアを交互に繰り返すこの形が**バルク同期並列（BSP）**パターンである．境界交換と計算を交互に行う第 2 章の MPI ステンシルは，プロセス間にわたる同じパターンである．

▷ bulk synchronous parallel (BSP)

2 つ目のバリアが必要になるのは，図 3.5 のように計算段階が結果を共有メモリに書き，格納段階でスレッドが他のスレッドの生成した値をコピーする場合，

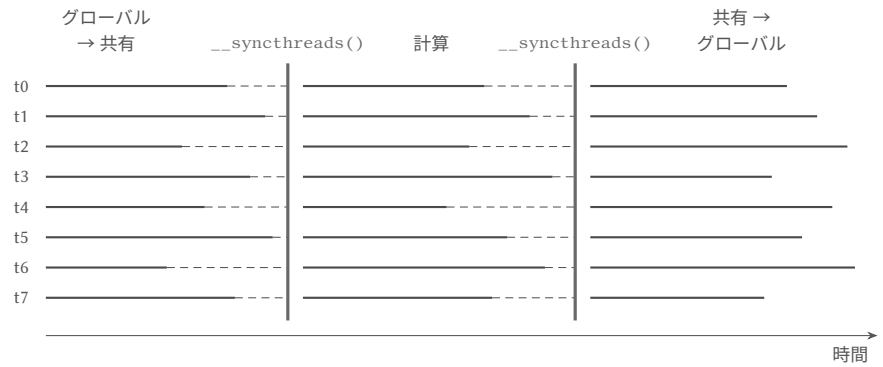


図 3.5: 1 つのブロックの 8 スレッドによる，共有メモリのバルク同期的な使用．スレッドは各段階を異なる時刻に終え（実線），最も遅いスレッドが到着するまでバリアで待つ（破線）．

あるいはバッファに別のタイルを読み込み直す場合である（次の読み込みが，遅いスレッドがまだ読んでいる値を上書きしてしまう）．計算段階が各スレッドの結果を直接グローバルメモリに書くカーネルでは省略できる．

3.1.7 複数のカーネル

互いに異なる複数のタスクを持つプログラムは複数のカーネルを使い，それぞれが独自の実行構成を持つ．逐次的な段階は 1 ブロック 1 スレッドのカーネルであり，データ並列な段階は多数のスレッドのカーネルである．

```
sortKernel<<<1, 1>>>(d_data, n);              // 逐次段階
addKernel<<<blocks, threads>>>(d_data, d_out, n);
storeKernel<<<1, 1>>>(d_out, n);              // 逐次段階
```

同じホストスレッドから起動されたカーネルは，既定では 1 つずつ順に実行されるので，2 つ目は 1 つ目の結果を見る．

カーネルを並行に実行したり，カーネルとコピーを重ねたりするには CUDA ストリームが必要である（第 5 章）．1 スレッドのカーネルは 1 つの SM の 1 レーンしか使わず，GPU の残りを遊ばせる．データがすでにデバイス上にあるのでなければ，逐次的な段階はホストで行うべきである．

3.1.8 メモリ空間

スレッドは 5 つのメモリ空間をアドレス指定できる．それらは，誰から見えるかと，どこにあるかで区別される（表 3.1，図 3.6）．

定義 3.9 (ローカル，グローバル，コンスタント，テクスチャメモリ). スレッドの**ローカルメモリ**はそのスレッド専用であり，レジスタに収まらないスレッドの変数を保持する．共有メモリは 1 つのブロック専用である．**グローバルメモリ** はカーネルの全ブロックの全スレッドから見え，カーネル起動をまたいで存続する．ホストが入力を置き，出力を読む場所である．**コンスタントメモリ**は小さな読み出し専用の空間で，全スレッドから見え，キャッシュされる．物理定数やフィルタなど，カーネルの実行中に変わらない値のためのものである．**テクスチャメモリ**は，2 次元の局所性に最適化されたキャッシュを持つ読み出し専用の空間である．後の 2 つはグラフィックスから受け継いだもので，そこではシーンの定数とテクスチャ画像を保持する．汎用のコードで必要になることはまれである．

▷ local memory

▷ global memory

▷ constant memory

▷ texture memory

サイズ（CPG Table 21 [5]）：コンスタントメモリはデバイス当たり 64 KiB で，SM 当たり 8 KiB のキャッシュを持つ．ローカルメモリはスレッド当たり最大 512 KiB．どちらもデバイスメモリ上にあり，ローカルメモリは L1 キャッシュにヒットしない限りグローバルメモリと同じく遅い．

この階層は情報の共有のしかたを制限する．ブロックの共有メモリ内のデータはその SM にとどまり，2 つのブロックはグローバルメモリを通じてしか通信できず，一般にはカーネルの境界をまたぐ．Hopper 以降，**スレッドブロッククラスタ** が，まとめてスケジュールされ互いの共有メモリを読み書きできる複数のブロックをまとめる．ブロックとグリッドの間の新しい階層である．[1]

▷ thread block cluster

[1] スレッドブロッククラスタ（Compute Capability 9.0）：クラスタのブロックは 1 つの GPU プロセッシングクラスタ上で同時にスケジュールされることが保証され，互いの共有メモリに分散共有メモリとしてアクセスする．クラスタ当たり 8 ブロックまでが移植可能なサイズで，H100 は 16 ブロックまでをサポートする（CPG §2.2.1 [5]；[2]）．第 5 章で再び取り上げる．

表 3.1: CUDA のメモリ空間：誰から見えるか，どれだけ存続するか，どこにあるか．

空間	可視範囲	存続期間	場所，備考
レジスタ	1 スレッド	スレッド	チップ上，コンパイラが割り当てる
ローカル	1 スレッド	スレッド	デバイスメモリ（キャッシュあり）；退避された変数
共有	1 ブロック	ブロック	チップ上，`__shared__`
グローバル	全カーネルの全ブロック	アプリケーション	デバイスメモリ；`cudaMalloc`
コンスタント	全ブロック	アプリケーション	デバイスメモリ，読み出し専用，キャッシュあり；`__constant__`
テクスチャ	全ブロック	アプリケーション	デバイスメモリ，読み出し専用，2 次元キャッシュ

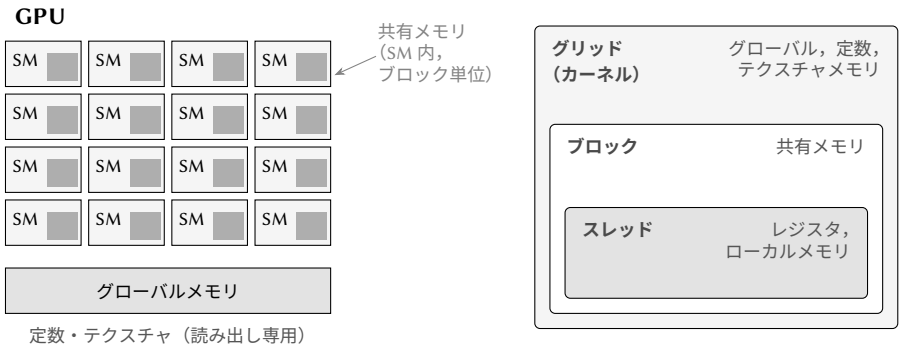


図 3.6: 左：各 SM は自分の共有メモリを持ち，1 つのグローバルメモリがそのすべてに仕える．右：各空間の可視範囲はスレッド階層に従う．

要点
CUDA プログラムはホストプログラムとカーネルから成る．カーネルはスレッドのブロックのグリッドとして実行される．各スレッドは `blockIdx`，`blockDim`，`threadIdx` から自分のデータを見つけ，ブロックは決まった順序なしに実行されるので独立でなければならず，1 つのブロックのスレッドはチップ上の共有メモリを共有し，`__syncthreads()` で同期する．

3.2 占有率

ストリーミング・マルチプロセッサは通常，同時に複数のブロックを実行する．いくつ実行するか，そしてそれがなぜ重要かが占有率の問題である．

出典 M3，占有率；BPG §11.1 Occupancy，§11.3 Thread and Block Heuristics，§11.4 Effects of Shared Memory；CPG §4.2 Hardware Multithreading；PMPP ch. 4．

3.2.1 3 つの資源

定義 3.10 (資源制約). SM に常駐する各ブロックは，その生存期間中ずっと，SM の 3 つの資源の一部を占有する．スレッドスロットの数，レジスタの数，共有メモリの量である．それぞれが**資源制約** であり，同時に常駐できるブロック数は，3 つのどれか（および SM 当たりのブロック数の固定上限）が許す最小の数である．ここでいう**レジスタ**は SM のレジスタファイルの 32 ビットエントリ 1 つであり，スレッドが必要とするレジスタ数はコンパイラが決める．多数のレジスタを必要とするカーネルは**レジスタ圧力** が高いという．

▷ resource constraint

▷ register

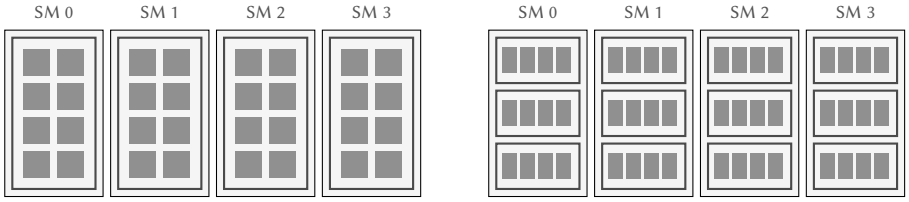
▷ register pressure

CC 7.0，8.0，9.0 の SM：65 536 レジスタ（256 KiB），2048 スレッドスロット（64 ワープ），32 ブロック，96–228 KiB の共有メモリ．他の CC はスロットとブロックが少ない（表 3.3）．

各量を誰が決めるかを表 3.2 にまとめる．ブロック当たりのスレッド数は起動時のプログラマの選択である．スレッド当たりのレジスタ数はコンパイラが決める．ISA がアーキテクチャレジスタの数を固定する CPU（多くの RISC ISA で 32 個）と異なり，GPU ではコンパイラが上限までカーネルの必要なだけレジスタを割り当て，ハードウェアはレジスタファイルが許すだけのスレッドを載せる．

表 3.2: ブロック当たりの資源要求を誰が決め，どこで読み取るか．

量	決める者	確認できる場所
ブロック当たりのスレッド数	ホスト，起動時	実行構成
スレッド当たりのレジスタ数	コンパイラ	`nvcc --ptxas-options=-v`；`__launch_bounds__` または `-maxrregcount` で上限を設定
ブロック当たりの共有メモリ	`__shared__` 宣言（と任意の第 3 起動パラメータ）	同じコンパイラ報告



(a) SM あたり 1 ブロック × 8 スレッド
4 × 1 × 8 = 32 スレッド

(b) SM あたり 3 ブロック × 4 スレッド
4 × 3 × 4 = 48 スレッド

図 3.7: 4 つの SM を満たす．(a) SM 当たり 8 スレッドのブロック 1 つ；(b) SM 当たり 4 スレッドのブロック 3 つ．小さいブロックのほうが，32 スレッドではなく 48 スレッドを常駐させる．

ブロック当たりの共有メモリは宣言から決まる．`__shared__ int s[1000]` は 4 × 1000 = 4000 バイトである．

例 3.11 (SM 当たりのブロック数)．あるマルチプロセッサは 1024 スレッドスロット，32 768 レジスタ，48 KiB の共有メモリを持つ．カーネルはブロック当たり 64 スレッドで起動され，コンパイラはスレッド当たり 40 レジスタと報告し，各ブロックは 6 KiB の共有メモリを宣言する．
スレッド 1024/64 = 16 ブロック；
レジスタ 32 768/(40 × 64) = 12.8，よって 12 ブロック；
共有メモリ 48/6 = 8 ブロック．
SM は min(16, 12, 8) = 8 ブロックを保持でき，共有メモリが制約となる．ブロック当たりの共有メモリを半分にすると上限は 12 に上がり，そこでレジスタが制約となる．レジスタ数が効いてくるのはそのときになってからである．

注意 原典の例はレジスタファイルを “64 キロバイト” としているが，意味しているのは 64 K = 65 536 個のレジスタであり，1 個 4 バイトで 256 KiB になる．占有率の計算におけるレジスタ数は常に 32 ビットレジスタの個数であり，バイト数ではない．

3.2.2 占有率が重要な理由

常駐ブロックが多いほど常駐ワープが多くなり，ワープが多いほど，他のワープがメモリを待つ間に SM が選べる独立な命令が増える．図 3.7 は 4 つの SM を満たす 2 つの方法を比べる．8 スレッドのブロック 1 つずつ（32 スレッド）か，4 スレッドのブロック 3 つずつ（48 スレッド）である．後者は各ブロックが小さいにもかかわらず実行中のスレッドが多く，したがって通常は SM のスループットをより多く引き出す．

ワープが多いとなぜメモリレイテンシを隠せるかは第 4 章（マルチスレッドアーキテクチャ）の主題である．第 7 章は，ワープが多すぎて L1 キャッシュがスラッシングを起こすという例外を示す．

定義 3.12 (占有率，アクティブワープ)．**アクティブワープ**とは，マルチプロセッサに常駐し，レジスタと共有メモリを割り当てられており，したがって次の命令の準備ができればいつでも発行の候補となるワープである．現在のサイクルで準備ができているワープは**発行可能**（eligible）と呼ばれる．マルチプロセッサ上のカーネルの**占有率** とは，アクティブワープの数と，マルチプロセッサが保持できるワープの最大数との比である [8, §11.1]．ブロック当たり T スレッド，スレッド当たり R レジスタ，ブロック当たり S バイトの共有メモリで，表 3.3 の上限を

▷ active warp

▷ occupancy

表 3.3: Compute Capability ごとの SM 当たりの上限（CPG Table 21 [5]；K = 1024，KiB は 2 進）．例の GPU は代表例であり，すべてではない．

Compute Capability （例の GPU）	7.0 V100	7.5 T4	8.0 A100	8.6 RTX 3090	8.9 L40	9.0 H100
SM 当たり最大スレッド数	2048	1024	2048	1536	1536	2048
SM 当たり最大ワープ数	64	32	64	48	48	64
SM 当たり最大ブロック数	32	16	32	16	24	32
SM 当たり 32 ビットレジスタ数	64 K	64 K	64 K	64 K	64 K	64 K
スレッド当たり最大レジスタ数	255	255	255	255	255	255
SM 当たり最大 共有メモリ（KiB）	96	64	164	100	100	228
ブロック当たり最大 共有メモリ（KiB）	96	64	163	99	99	227

持つ SM 上では

$$\text{blocks/SM} = \min\left(\left\lfloor \frac{\text{Threads/SM}}{T} \right\rfloor, \text{Blocks/SM}, \left\lfloor \frac{\text{Regs/SM}}{RT} \right\rfloor, \left\lfloor \frac{\text{Smem/SM}}{S} \right\rfloor\right), \quad (3.2)$$

$$\text{occupancy} = \frac{\text{blocks/SM} \times \lceil T/32 \rceil}{\text{Warps/SM}}. \quad (3.3)$$

式 (3.3) が与えるのは理論占有率である．Nsight Compute が実行時に測定する達成占有率は，ブロックの終了時刻がばらつく場合や，グリッドが小さすぎてすべての SM を満たせない場合に，これより低くなる．

▷ compute capability

定義 3.13 (Compute Capability). **Compute Capability** は，GPU 世代の機能と上限を表す NVIDIA のバージョン番号であり，*major.minor* と書く．7.x は Volta と Turing，8.x は Ampere と Ada，9.0 は Hopper である．表 3.3 の上限は Compute Capability ごとに決まっており，占有率の計算で対象を明示しなければならないのはそのためである．x.0 の製品（V100，A100，H100）は 64 ワープを保持するが，CC 7.5，8.6，8.9（例えば T4，RTX 3090，L40）は 32 または 48 ワープしか保持しない．

CC 8.x と 9.0 では，共有メモリは統合された L1 ／共有メモリの配列から切り出され，その配分はカーネルごとに設定できる（CPG，Compute Capability 8.x [5]）．表は最大値を示す．ワープを消費するワープスケジューラは第 4 章で扱う．

例 3.14 (A100 での占有率). Compute Capability 8.0 の SM 上で，カーネルをブロック当たり $T = 256$ スレッド（8 ワープ）で起動し，スレッド当たり $R = 64$ レジスタ，ブロック当たり $S = 48$ KiB の共有メモリを使う．
スレッド $2048/256 = 8$ ブロック；
ブロック 32；
レジスタ $65\,536/(64 \times 256) = 4$ ブロック；
共有メモリ $\lfloor 164/48 \rfloor = 3$ ブロック．
3 ブロック，すなわち $3 \times 8 = 24$ ワープが常駐し，占有率は $24/64 = 37.5\,\%$ である．共有メモリが制約であり，ブロック当たり 32 KiB に減らせば 4 ブロック（そこでレジスタが制約となる），$32/64 = 50\,\%$ になる．完全な占有率（8 ブロック）には，スレッド当たり最大 32 レジスタ，ブロック当たり最大約 19 KiB の共有メモリ（$164/8 = 20.5$ から 1 KiB の予約分を引く）が必要である．

粒度（式 (3.2) では無視している）：レジスタはワープ単位で 256 個ずつ割り当てられ，CC 8.x と 9.0 では CUDA がブロック当たり 1 KiB の共有メモリを予約する（$\lfloor 164/49 \rfloor$ もやはり 3）．BPG §11.1.1；CPG，Compute Capability 8.x [8, 5]．

実際には，この計算は Nsight Compute の占有率計算機か，ランタイムの占有率 API で行う．

```
cudaOccupancyMaxActiveBlocksPerMultiprocessor(
    &blocks, kernel, T, dynSmem);
```

ここで dynSmem はブロック当たりの動的共有メモリである（静的な`__shared__`の使用量はカーネル自身から取られる）．どちらのツールも，対象デバイスの正確な粒度で式 (3.2) を適用する（BPG §11.1.1）．

ヒント BPG §11.3 による出発点：ブロック当たりのスレッド数はワープサイズの倍数とする；ブロック当たり少なくとも 64，それも SM 当たり複数ブロックがある場合に限る；最初に試すのは 128–256；レイテンシが重要なら，大きな 1 ブロックより小さな複数ブロック．

占有率が高いほど常に速いわけではない．ある点を超えるとワープを増やしても何も加わらず，命令スループットや自身のメモリ帯域幅で制限されるカーネルはそれで何も得ない．しかし占有率が低いと，SM がメモリレイテンシを隠す能力は必ず弱まる [8, §11.1].

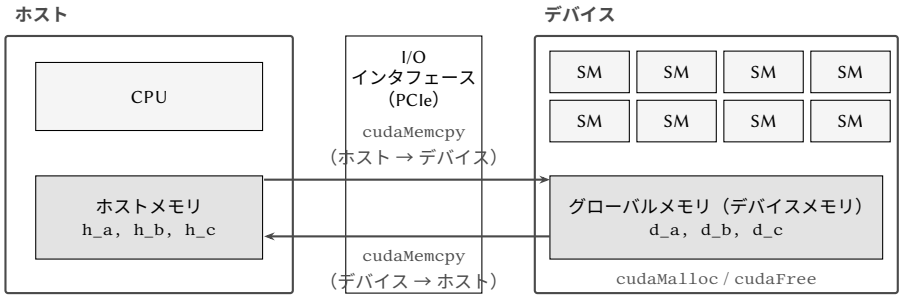


図 3.8: ホストメモリとデバイスメモリ．cudaMalloc はグローバルメモリを確保し，cudaMemcpy は I/O インタフェースを越えてどちらの向きにもデータを移す．

要点
SM 当たりのブロック数 = スレッド，レジスタ，共有メモリ（とブロック数の上限）にわたる最小値；占有率 = 常駐ワープ数を最大数で割ったもの．ブロック当たりのスレッド数は起動時に，スレッド当たりのレジスタ数はコンパイラが，共有メモリは宣言が，上限は Compute Capability が決める．

3.3 ホストコード

共有メモリはブロック内で，グローバルメモリは全ブロックから見えるが，どちらも CPU からは見えない．グローバルメモリは I/O インタフェースの向こうにあるデバイス自身のメモリにあり，ホストは明示的な呼び出しによってそれを管理する．

出典 M3，ホストコード；CPG §3.2.2 Device Memory；K12 §1.3.6．

定義 3.15 (ホストメモリ，デバイスメモリ)．**ホストメモリ**は CPU の主記憶であり，ホストプログラムの変数が置かれる．**デバイスメモリ** は GPU 自身のメモリ（HBM または GDDR）であり，グローバル，コンスタント，テクスチャ，ローカルの各メモリがそこに置かれる．両者は I/O インタフェースで結ばれた別々のアドレス空間である（図 3.8）．一方へのポインタは他方では意味を持たず，データは明示的なコピーによってのみ行き来する．

▷ host memory
▷ device memory

インタフェースがボトルネックである．H100：HBM3 は約 3 TB/s（暫定値）と報告されている [2]．これに対し PCIe Gen 5 x16 は片方向 64 GB/s で，約 47× の差がある．A100 40 GiB：HBM2 は 1555 GB/s [1] で，Gen 4 の片方向約 32 GB/s の約 49×．

デバイスメモリの確保，そこへの転送とそこからの転送，解放は，CUDA ランタイムの 3 つの呼び出しで行う．

```
cudaMalloc((void **)&ptr, size);   // size バイトを確保
cudaMemcpy(dst, src, size, dir);   // size バイトをコピー
cudaFree(ptr);                     // 解放
```

定義 3.16 (メモリ確保と転送)．デバイス上の**メモリ確保**は cudaMalloc が行う．これはグローバルメモリを size バイト確保し，デバイスポインタを ptr に返す．cudaFree はそれを解放する．cudaMemcpy は size バイトを dir の向きにコピーする．向きはホストからデバイス（cudaMemcpyHostToDevice），デバイスからホスト（cudaMemcpyDeviceToHost），またはデバイスからデバイスである．ランタイムの呼び出しはすべて cudaError_t を返し，cudaSuccess は成功を意味する．cudaMemcpy はホストに対して同期的である．デバイスからホストへのコピーはデータが届いた時点で戻り，カーネル起動の後に発行されたコピーはまずカーネルの終了を待つ．

▷ memory allocation

ベクトル加算の完全なプログラム，すなわちカーネルとホストコードは次のとおりである（カーネルは 3.1 節のもの）．確保，コピーイン，コピーアウトという 3 つのメモリ管理の手順が起動を取り囲み，ホストは起動のエラーも検査する．

```
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

__global__ void vectorAdd(const float *a, const float *b,
```

ピン留めされたホストメモリからの非同期コピーと，カーネルとの重ね合わせ：第 5 章．

注意 カーネル起動はすぐにホストへ戻り，それ自身ではエラーを報告しない．cudaGetLastError() は起動時の構成エラー（ブロック当たりのスレッドが多すぎる，SM に対して資源が多すぎる）を捕らえる．カーネル内部のエラーは，結果を読む cudaMemcpy のような次の同期的な呼び出しで表面化する．実際のコードでは両方の戻り値を検査する．

```
                          float *c, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) c[i] = a[i] + b[i];
}

int main() {
  const int n = 1 << 20;                 // 1,048,576
  const size_t size = n * sizeof(float);
  float *h_a = (float *)malloc(size);
  float *h_b = (float *)malloc(size);
  float *h_c = (float *)malloc(size);
  for (int i = 0; i < n; ++i) {
    h_a[i] = i; h_b[i] = 2.0f * i;
  }

  float *d_a, *d_b, *d_c;                // デバイスポインタ
  cudaMalloc((void **)&d_a, size);
  cudaMalloc((void **)&d_b, size);
  cudaMalloc((void **)&d_c, size);
  cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
  cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

  int threads = 256;
  int blocks = (n + threads - 1) / threads;   // 4096
  vectorAdd<<<blocks, threads>>>(d_a, d_b, d_c, n);
  cudaError_t err = cudaGetLastError();
  if (err != cudaSuccess)
    printf("launch: %s\n", cudaGetErrorString(err));

  cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
  printf("%f\n", h_c[1000]);             // 3000.000000

  cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
  free(h_a); free(h_b); free(h_c);
  return 0;
}
```

例 3.17 (コピーのコスト). $n = 2^{20}$ 個の float では各配列が 4 MiB であり，8 MiB がデバイスへ送られ 4 MiB が戻ってくる．2^{20} 回の加算のために合計 12 MiB である．PCIe で持続 25 GB/s とするとコピーには約 0.5 ms かかる．カーネル自体は，デバイスメモリから 8 MiB を読み 4 MiB を書くのに 1.5 TB/s で 10 µs もかからない．転送が約 60 倍支配的である．ベクトル加算だけでは GPU へ移す価値はなく，データはできるだけ多くのカーネルにわたってデバイス上にとどめるべきである．

定義 3.18 (ユニファイドメモリ). **ユニファイドメモリ**（cudaMallocManaged）では，1 つのポインタがホストからもデバイスからも使える．ランタイムと GPU のメモリ管理がページを要求に応じてホストメモリとデバイスメモリの間で移動し，明示的な cudaMemcpy の呼び出しはなくなる．コピーはページフォールトを契機とするページ移動として依然として起こり，そのコストは第 7 章の主題である．

▷ unified memory

第 7 章が数値を示す．GPU のページフォールトは数十マイクロ秒かかり，ドライバはフォールトをバッチで処理する．したがってユニファイドメモリはコードを単純にするが，転送をなくすわけではない．

3.4 CUDA によるステンシル演算

第 2 章のステンシルは格子の各要素 C を 4 つの隣接要素の平均 $C' = (N+W+E+S)/4$ で置き換える．CUDA で書くと，ここまでのすべてが組み合わさる．データを分割するブロックのグリッド，各ブロックが再利用する値のための共有メモリ，読み込みと計算の間のバリア，そしてブロックの端のハローである．

出典 M3，CUDA によるステンシル；PMPP ch. 7（畳み込み）と ch. 8（ステンシル）；BPG §10.2.3 Shared Memory．

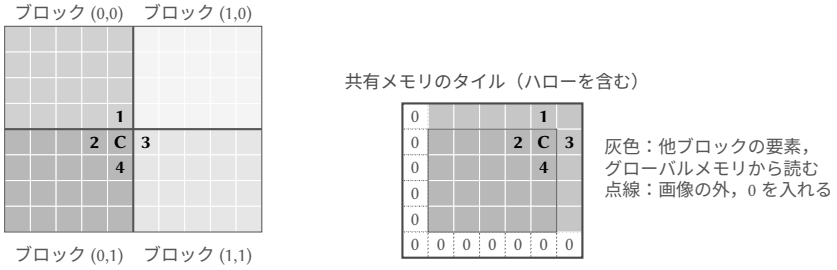


図 3.9: 左：4 つのブロックに分けた 10×8 の画像．C とその隣接要素 1–4 は，そのうち 3 つのブロックにまたがる．右：ブロック (0,1) の共有メモリのタイル．自分の 5×4 の領域と幅 1 セルのハローから成る．画像内のハローセル（灰色）はグローバルメモリから読み込まれ，画像外のもの（点線）には 0 が入る．

3.4.1 分割と再利用

データ分割は図 3.9 の左のとおりである．各ブロックは画像の 1 つの領域を所有し，その各スレッドは 1 つの出力要素を計算するので，すべての出力が並列に計算される．内部の各入力要素は 4 つのスレッド，すなわちその 4 つの隣接要素のスレッドに読まれる（ステンシルが中心を含めば 5 つ）．

定義 3.19 (データ再利用，タイル，タイリング)．計算の**データ再利用**とは，メモリから読み込んだ 1 つの値がその計算の複数の演算に使われる性質である．**タイル**は 1 つのブロックが扱う長方形の部分配列であり，**タイリング**は，タイルをグローバルメモリから共有メモリへ一度読み込み，すべての再利用をそこから賄う技法である．これにより，各グローバル要素は使用 1 回ごとにではなく，ブロック当たり約 1 回読まれるだけになる．ステンシルの再利用パターンは事前にわかっており，ソフトウェア管理のスクラッチパッドが適切なキャッシュとなるのはそのためである．

▷ data reuse
▷ tile
▷ tiling

したがって各スレッドは図 3.5 の BSP の形に従う．グローバルメモリからブロックの共有タイルへ 1 要素を読み込む；`__syncthreads()`；タイルから自分のステンシルを計算する；結果をグローバルメモリへ書く．

同じタイリングは行列積を，乗算 1 回ごとにグローバル読み込み 1 回から，tile size 回の乗算ごとに 1 回へ変える．第 5 章で，タイルの配置が悪いと生じる第 4 章のバンク競合とともに扱う．

3.4.2 境界とハロー

共有メモリはブロックごとだが，ブロックの領域の端にある要素は他のブロックの領域に隣接要素を持つ．図 3.9 では，ブロック (0,1) の角にある C は，ブロック (0,0) の隣接要素 1 とブロック (1,1) の隣接要素 3 を必要とする．それらのブロックの共有メモリには手が届かないので，各ブロックは自分の領域に加えて，その周囲の幅 1 要素の縁，すなわち第 2 章の**ハロー領域**を，グローバルメモリから自分のタイルへ読み込む（図 3.9，右）．これはメッセージのない MPI ステンシルの境界交換である．ハローは全ブロックから見えるグローバルメモリから単に読まれ，ブロック間の同期は必要ない．入力配列はカーネルの実行中読み出し専用であり，出力は別の配列へ書かれるからである．

定義 3.20 (境界条件，ゼロパディング)．**境界条件**は，隣接要素が配列の外にあるときにステンシルが何を読むかを定める．最も単純なのは**ゼロパディング**であり，配列の外の位置を 0 とみなし，読み込み段階ではグローバルメモリを読む代わりにタイルに 0 を格納する．他の選択（端の値へのクランプ，巻き戻し，片側の式）は，読み込みのその分岐を変えるだけである．

▷ boundary condition
▷ zero padding

3.4.3 ステンシルから畳み込みへ

4 回の読み出しと境界ごとの `if` で平均を書く代わりに，次のカーネルは小さな畳み込みとして書かれている．3×3 のフィルタを各要素の周りに適用し，ステンシルをフィルタの係数として表す．これにより特別な場合がなくなり，同じカーネルが任意の 3×3 ステンシルに使える．

定義 3.21 (畳み込み，フィルタ)．2 次元の**畳み込み**は，各出力を同じ位置の周りの入力要素の重み付き和として計算する．重みの配列が**フィルタ**である（カーネ

▷ convolution
▷ filter

ルとも呼ぶが，CUDA での意味と紛らわしいのでここでは避ける）．信号処理の意味での畳み込みはフィルタを反転させるが，対称なフィルタでは両者は一致し，ここで従う機械学習の用法は反転させない．半径 r のフィルタ f では

$$\mathrm{out}[y][x] = \sum_{i=-r}^{r} \sum_{j=-r}^{r} \mathrm{in}[y+i][x+j] \cdot f[i+r][j+r]. \tag{3.4}$$

4 近傍の平均は，4 つの辺の中央に 0.25，それ以外に 0 を持つ 3×3 フィルタである．3×3 フィルタは 9 点ステンシルであり，それが必要とするハローの幅は $r = 1$ である．

3.4.4 カーネル

以下の設計は，$TILE^2$ 個の出力のタイルに対して $(TILE + 2r)^2$ スレッドのブロックを起動する．各スレッドはハローを含むタイルの 1 要素を読み込み，内側のスレッド，すなわち担当要素がハローセルでないスレッドだけが出力を計算して格納する（図 3.10）．フィルタはコンスタントメモリに置く．

```
#define TILE 14                // ブロックの1辺の出力数
#define RADIUS 1               // フィルタ半径 r
#define BLOCK (TILE + 2 * RADIUS) // 1辺16

__constant__ float filter[2 * RADIUS + 1][2 * RADIUS + 1];

__global__ void stencil2D(const float *in, float *out,
                          int width, int height) {
  __shared__ float tile[BLOCK][BLOCK];
  int tx = threadIdx.x, ty = threadIdx.y;
  int x = blockIdx.x * TILE + tx - RADIUS;  // 画像の列
  int y = blockIdx.y * TILE + ty - RADIUS;  // 画像の行

  // 1. 全スレッドが1セルを読み込む．画像外は0
  if (x >= 0 && x < width && y >= 0 && y < height)
    tile[ty][tx] = in[y * width + x];
  else
    tile[ty][tx] = 0.0f;
  __syncthreads();

  // 2. 内側のスレッドが1出力ずつ計算し 3. 格納する
  bool inner = tx >= RADIUS && tx < BLOCK - RADIUS &&
               ty >= RADIUS && ty < BLOCK - RADIUS;
  if (inner && x < width && y < height) {
    float acc = 0.0f;
    for (int i = -RADIUS; i <= RADIUS; ++i)
      for (int j = -RADIUS; j <= RADIUS; ++j)
        acc += tile[ty + i][tx + j]
             * filter[i + RADIUS][j + RADIUS];
    out[y * width + x] = acc;
  }
}
```

ホストはフィルタを cudaMemcpyToSymbol で一度設定し，2 次元の構成で起動する．

```
dim3 block(BLOCK, BLOCK);   // 16 x 16 スレッド
dim3 grid((width + TILE - 1) / TILE,
          (height + TILE - 1) / TILE);
stencil2D<<<grid, block>>>(d_in, d_out, width, height);
```

インデックスの計算は式 (3.1) を r だけずらしたものである．ブロック $(0,0)$ のスレッド $(0,0)$ は画像の位置 $(-1,-1)$ を読み込むが，これは画像外なので 0 になり，スレッド $(1,1)$ は最初の出力である $(0,0)$ を読み込む．1 ブロックのオフセットが BLOCK ではなく TILE なのは，隣り合うタイルがハローの分だけ重なるからである．

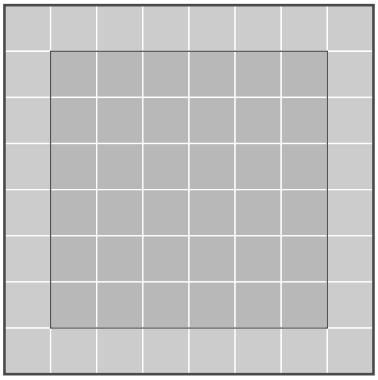


図 3.10: 1 つのブロック内のスレッドの役割（8×8 スレッドで描画）：外周のスレッドはハローセルを読み込むだけで，内側のスレッドは読み込みと計算を行う．

注意 ここではバリアは 1 つで足りる．タイルは一度読み込まれ，その後は読まれるだけだからである．共有バッファを複数のタイルに使い回すカーネルでは，計算段階の後に 2 つ目の __syncthreads() が必要になる．

例 3.22 (64×64 の画像). 64×64 の画像で $TILE = 14$，$r = 1$ とすると：

- グリッドは $\lceil 64/14 \rceil \times \lceil 64/14 \rceil = 5 \times 5 = 25$ ブロックで，各ブロックは $16 \times 16 = 256$ スレッド，4096 個の出力に対して 6400 スレッドである．16 個の内部ブロックのそれぞれで，196 スレッドが計算し，60 スレッドは読み込むだけである；
- ブロック当たりの共有メモリは $16 \times 16 \times 4 = 1$ KiB；
- タイルの読み込みは $25 \times 256 = 6400$ 回で，そのうち $72^2 = 5184$ 回がグローバルメモリを読む（残りは 0 の書き込みである．次元ごとにブロックは $15 + 16 + 16 + 16 + 9 = 72$ 個の位置を読む）．各スレッドが 9 個の入力をグローバルメモリから読む場合の $9 \times 4096 = 36\,864$ 回と比べて約 7 分の 1 である．ハローのオーバーヘッドは，内部ブロックで出力当たり $256/196 = 1.31$ 回の読み込み（画像全体では $5184/4096 \approx 1.27$）である．

各次元の最後のブロックは，幅 64 の画像の列 56–69 を受け持つ．そのスレッドのうち $x \geq 64$ のものは 0 を読み込み，格納を省く．

別の設計として，ブロックがちょうど $TILE^2$ スレッドを持ち，端のスレッドが追加の読み込みでハローを読み込むものがある．遊ぶスレッドは起動しないが，条件分岐のコードが増える．PMPP ch. 8 は両方を示す．

ヒント ハローの割合 $1 - (TILE/BLOCK)^2$：ここではスレッドの 23 %，32×32 のブロックでは 12 %．タイルを大きくすると無駄なスレッドは減るが，共有メモリが多く必要になり，SM 当たりのブロック数が減る（3.2 節）．

ワープの全スレッドは同じフィルタ係数を同時に読む．コンスタントキャッシュは 1 つのアドレスをワープ全体へブロードキャストとして供給する．これがコンスタントメモリの用途である．

要点 — 章のまとめ

CUDA プログラムは，デバイスメモリを確保し（`cudaMalloc`），データをコピーインおよびコピーアウトし（`cudaMemcpy`），実行構成`<<<blocks, threads>>>` でカーネルを起動するホストコードである．カーネルは独立なスレッドブロックのグリッドとして実行される．各スレッドは `blockIdx`，`blockDim`，`threadIdx` から自分のグローバルインデックスを計算し，ブロックは決まった順序なしに実行され，1 つのブロックのスレッドはチップ上の共有メモリを共有し，読み込み，バリア，計算，格納という BSP パターンで `__syncthreads()` により同期する．5 つのメモリ空間，ローカル，共有，グローバル，コンスタント，テクスチャは，その可視範囲がスレッド階層に従う．SM に常駐するブロック数はスレッドスロット，レジスタ，共有メモリが許す最小値であり，占有率はそれらが埋める SM のワープスロットの割合である．占有率が高いほどレイテンシをよく隠せるが，常に速いとは限らない．ステンシルはタイル化されたカーネルになる．各ブロックはタイルとハローを共有メモリに読み込み，画像の端ではゼロパディングを行い，1 回同期し，内側のスレッド 1 つにつき 1 つの出力を計算する．

第 4 章

GPU アーキテクチャ

講義動画	M4，レッスン 1–4（Multi-threaded Architecture；Bank Conflicts；GPU Architecture Pipeline；Global Memory Coalescing）
資料	PTX ISA [19]（特殊レジスタ）；Harris，*Using Shared Memory in CUDA C/C++* [18]；NVIDIA A100 [1] および H100 [2] ホワイトペーパー（HBM の数値）
教科書	A18 [4] ch. 3（指定），ch. 4；K12 [3] §1.4.1 GPU Pipeline，§1.4.3 GPU Memory Systems；CPG [5] §4.2 Hardware Multithreading，§5.3.2 Device Memory Accesses；BPG [8] §10.2.1 Coalesced Access to Global Memory，§10.2.3 Shared Memory；PMPP [6] ch. 4–6
キーワード	レイテンシ隠蔽，細粒度マルチスレッディング，コンテキストスイッチ，ワープごとのプログラムカウンタ，レジスタファイル，ポート，バンク，バンク競合，オペランドコレクタ，スコアボード，アクティブマスク，特殊レジスタ，メモリコアレッシング，メモリトランザクション，SRAM，DRAM，HBM

第 1 章ではストリーミング・マルチプロセッサの構成要素を挙げ，第 3 章ではカーネルのスレッドがブロックとワープにどうまとめられ，SM がそれをいくつ保持できるかを示した．本章は SM の内部を開き，ワープの命令がそこをどう通るかをたどる．まず，そもそも SM がなぜマルチスレッド化されているのかから始める．キャッシュでミスしたロードは，そのままではパイプラインをストールさせる．GPU は，プログラムカウンタとレジスタをすべてチップ上に保持したワープの間で切り替えることで，そのレイテンシを隠す．これを可能にするレジスタファイルは非常に幅が広いので，バンクに分割しなければならず，そこからバンク競合，オペランドコレクタ，そして準備のできたワープを選ぶスコアボードが生じる．続いて小さな例でパイプライン全体をたどり，ワープのフェッチ，デコード，レジスタ読み出し，実行，ライトバックを，レーンを無効にするマスクビットも含めて示す．最後にメモリ側を扱う．1 つの命令からワープがなぜ 32 個のメモリ要求を生成しうるのか，SRAM，DRAM，HBM とは何か，そしてコアレッシングがそれらの要求をどのように少数の幅広いトランザクションに変えるかである．第 5 章ではこれらの機構を最適化の規則に置き換え，第 6 章と第 7 章ではレジスタファイルとワープスケジューラに戻る．

4.1 マルチスレッドアーキテクチャ

GPU は多数の SM から成り，各 SM はマルチスレッディングによってそのワープを実行し，共有メモリとハードウェアキャッシュを持つ（第 1 章の SM の図）．本節では，マルチスレッディングが何をもたらし，どのような状態を必要とし，その状態が CPU のものとどう違うかを説明する．

出典 M4，マルチスレッディング；K12 §1.2.1；A18 ch. 3；CPG §4.2 Hardware Multithreading．

4.1.1 キャッシュミスを乗り切る 3 つの方法

図 4.1 の 4 命令のプログラムを，第 1 章の 5 段パイプライン（フロントエンド，デコードとリネーム，スケジュール，実行，ライトバック）で考える．最初の命令 I1 はメモリから 1 語をロードする．I4 はロードした値を必要とするが，I2 と I3 はそれとは独立である．このロードが**キャッシュミス**を起こしたとする．すなわちその語はキャッシュになく DRAM から取ってこなければならないので，I1 の実行ステージは多くのサイクルの間終わらない．パイプラインは**ストール**する．すなわち，どの命令も完了しないサイクルが過ぎていく．3 つの設計はそれぞれ異なる反応をする．

▷ cache miss

レイテンシの規模：K12 Table 1.4 [3] の Tesla と Fermi の GPU では，グローバルメモリへのアクセスは “数百サイクル” かかる．同じ表でチップ上の共有メモリへのアクセスは 4–32 サイクルである．

インオーダ実行 （図 4.1a）は命令をプログラム順に発行するので，I2，I3，I4 はすべてミスの後ろで待つ．ミスに依存するのは I4 だけであるにもかかわらずである．

アウトオブオーダ実行 （図 4.1b）はミスの間に I2 と I3 を完了させる．待つのは I4 だけである．得られる利益は，ハードウェアが 1 つのスレッドの中

で見つけられる独立な命令の数で制限される．

マルチスレッディング（図 4.1c）は，4 つの異なるスレッドの I1 を連続する 4 サイクルで発行する．異なるスレッドの命令は構成上互いに独立なので，独立性を探す必要はなく，4 つのミスは同時に未処理となる．4 つのメモリ要求は 1 つずつ順に処理されるのではなく重なり合う．

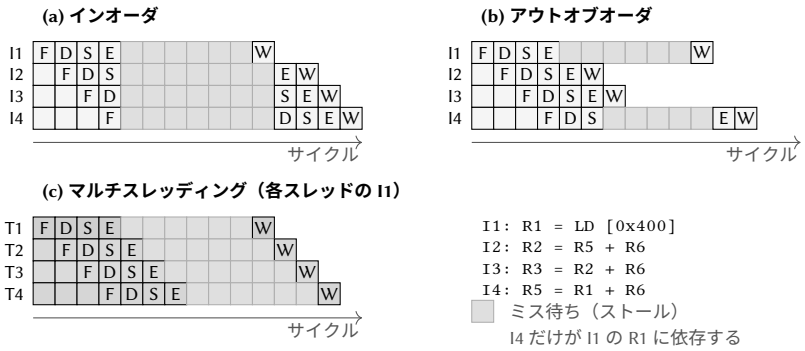


図 4.1: 最初の命令がキャッシュでミスするプログラム：(a) インオーダでは後続のすべての命令が待つ．(b) アウトオブオーダでは依存する I4 だけが待つ．(c) マルチスレッディングでは 4 つのスレッドがそれぞれロードを発行し，4 つのミスが重なる．

定義 4.1 (マルチスレッディングによるレイテンシ隠蔽). 長いレイテンシの操作（キャッシュミス，長い ALU 演算，分岐の解決）が進行している間も，パイプラインを有用な命令で埋め続けることを**レイテンシ隠蔽**という．多数のスレッドの状態をチップ上に保持し，毎サイクルそれらの間で切り替えることで，ストールしたスレッドを単に選ばないようにする方式が**細粒度マルチスレッディング**である．プロセッサが同時に処理中に保てるメモリ要求の数が，その**メモリレベル並列性**（MLP）である．図 4.1c のマルチスレッドパイプラインの MLP は 4 である．

▷ latency hiding

▷ fine-grained multithreading

▷ memory-level parallelism (MLP)

CPU がその面積をアウトオブオーダ論理，大きなキャッシュ，ILP の探索に費やすのに対し，GPU はそれをスレッドに費やす．キャッシュミス，分岐，長いレイテンシの演算によるストールは，SM の他のワープが覆い隠す．その代償は 1 つの規則で表される．メモリレイテンシが長いほど，それを隠すために多くのスレッドが必要になる．

例 4.2. ある SM は 1 サイクルに 1 つのワープ命令を発行する．各ワープはロードを発行した後，ロードした値が必要になるまでにさらに 2 つの命令を発行できる．ロードには 400 サイクルかかる．400 サイクルの間ずっと発行スロットを埋めるには，それぞれ 3 命令ずつを供給できるワープが約 400/3 ≈ 134 個必要である．レイテンシが 800 サイクルなら約 267 個になる．これほど多くのワープを保持する実際の SM はない（上限は 64）．GPU がキャッシュ，ワープ内の独立な命令，そして 4.4 節で示すようにメモリ要求の削減にも頼るのはこのためである．

ヒント 必要なスレッド数 ≈ レイテンシ ÷ 2 つの長レイテンシ操作の間に各スレッドが供給する独立な仕事のサイクル数．レイテンシが 2 倍になればスレッド数も 2 倍になり，独立な仕事が半分になっても同じである．

SM 当たりの最大常駐ワープ数：Compute Capability 7.0，8.0，9.0 で 64，8.6 と 8.9 で 48，7.5 で 32：CPG Table 21 [5]．第 3 章の占有率はこの上限に対して測る．

4.1.2 スレッドごとのプログラムカウンタとレジスタ

マルチスレッディングはパイプラインのフロントエンドを変える（図 4.2）．シングルスレッドのコアは，次にフェッチする命令のアドレスである**プログラムカウンタ**（PC）を 1 つ持つ．SM はワープごとに 1 つの PC を持つ．常駐ワープが 4 つなら PC レジスタも 4 つであり，フェッチステージは毎サイクル選んだワープの PC を読み，そのワープの全スレッドのために 1 つの命令をフェッチする．こうして 1 つの静的な命令がワープ全体に役立つ．これは第 1 章の SPMD モデルをハードウェアで実現したものである．フロントエンドの後ろでは，**レジスタファイル**が，すべての常駐ワープのすべてのスレッドについて専用のレジスタの組を保持する．そのため，あるワープから別のワープへの切り替えにコピーは要らない．次のワープのレジスタはすでにそこにある．

▷ program counter (PC)

▷ register file

SM 当たりのレジスタファイルは 32 ビットレジスタ 65 536 個，256 KiB を保持する．Compute Capability 5.0 から 9.0 まですべて同じである：CPG Table 21 [5]．Fermi（2010 年）は SM 当たり 128 KiB であった：K12 §1.4.1 [3]．

定義 4.3 (コンテキストスイッチ). **コンテキストスイッチ**とは，パイプラインが実行しているスレッドの状態（PC，レジスタ）を，別のスレッドのものに置き換えることである．CPU ではパイプライン，PC，レジスタは一度に 1 つのスレッ

▷ context switch

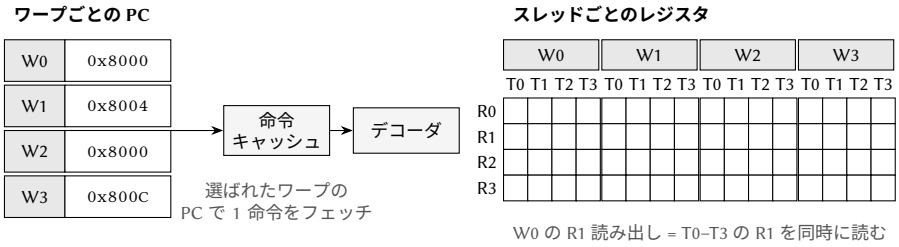


図 4.2: マルチスレッド SM のフロントエンド：ワープごとの 1 つのプログラムカウンタがフェッチする命令を選び，レジスタファイルはワープ単位にまとめてスレッドごとのレジスタの組を保持する．そのためワープ 0 の R1 を読むと，その 4 スレッドの R1 が一度に読まれる．

ドに属するので，切り替えでは実行中のスレッドの状態をメモリに格納し，次のスレッドの状態を読み込む（図 4.3）．これには多くのサイクルがかかる．GPU では常駐するすべてのワープの PC とレジスタがすでにチップ上にある．切り替えはパイプラインが使う PC とレジスタの組を変えるだけであり，コストはかからない．

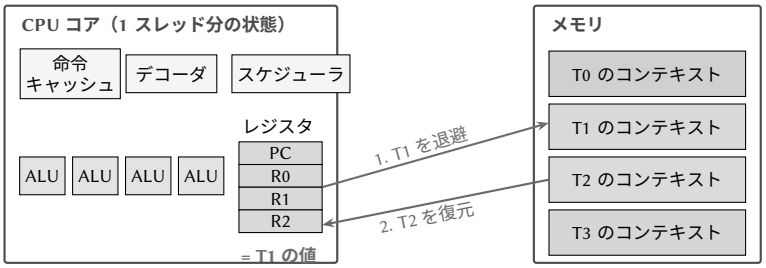


図 4.3: CPU のコンテキストスイッチ：コアは 1 つのスレッドの PC とレジスタを保持するので，T1 から T2 への切り替えでは T1 の状態をメモリに退避し，T2 の状態を復元する．

CPG §4.2 Hardware Multithreading：各ワープの実行コンテキストは “ワープの全生存期間にわたってチップ上に保持される．したがって，ある実行コンテキストから別の実行コンテキストへの切り替えにはコストがかからない”[5]．

注 4.4. **コンテキストスイッチ**という語は 2 つの意味で使われる．オペレーティングシステムのコンテキストスイッチはプロセスやスレッドをメモリに退避するもので，コストが高い．ワープ間の GPU の “コンテキストスイッチ” は，すべてが常駐している状態の中からの選択であり，コストはかからない．同時マルチスレッディングを備えた CPU はコア当たり 2 つのスレッドコンテキストを保持する．SM は数百のスレッドを保持する．

注意 コストのかからない GPU の切り替えが当てはまるのは，SM に常駐しているワープだけである．終了したスレッドブロックを新しいブロックに置き換えるときにはレジスタと共有メモリを割り当てるので，コストがかからないわけではない．ワープのスケジューリングは第 7 章で扱う．

K12 §1.2.1 は，CPU コアの 2 つのハイパースレッドと SM の数百のスレッドを対比している [3]．

Volta 以降，独立スレッドスケジューリングはスレッドごとにプログラムカウンタとコールスタックを持ち，分岐したスレッドをインタリーブできるようにしている：CPG §4.1 SIMT Architecture [5]．第 5 章でこれを論じ，第 6 章はワープごとのモデルを保つ．

4.1.3 必要な資源

マルチスレッディングがチップ上に保持する状態は，2 つの量で数えられる．

- **プログラムカウンタ．**ワープごとに 1 つであり，PC の数は常駐スレッド数をワープサイズで割ったものである．後の GPU はこれより多くのフロントエンド状態を持つが，本書のモデルではワープごとに 1 つの PC を仮定する．
- **レジスタ．**スレッド当たり K 個のレジスタに常駐スレッド数を掛けたものである．K はコンパイラがカーネルごとに選び，アプリケーションによって異なる．K が大きいほど，固定のレジスタファイルに常駐できるスレッドは少なくなる．これが第 3 章の占有率の計算におけるレジスタの制約である．

例 4.5. ある SM は，ワープサイズ 32 で 512 個の常駐スレッドを保持する．したがって $512/32 = 16$ 個のプログラムカウンタを持つ．全スレッドが実行する 12 命令のカーネルは $16 \times 12 = 192$ 回のワープ命令フェッチで済むが，同じ仕事をシングルスレッドのコアで行うと $512 \times 12 = 6144$ 回の命令フェッチが必要である．1 回のフェッチが 32 スレッドに役立つのである．コンパイラがスレッド当たり $K = 40$ 個のレジスタを割り当てると，常駐スレッドは $40 \times 512 = 20\,480$ 個

のレジスタを占め，1 個 4 B で 80 KiB となる．

> **要点**
> GPU はメモリレイテンシを，アウトオブオーダ実行ではなく細粒度マルチスレッディングで隠す．常駐するすべてのワープの PC とレジスタはチップ上にとどまるので，ワープの切り替えにコストはかからず，SM が保持できるワープ数は記憶容量の問題になる（ワープごとに 1 つの PC，スレッドごとに K 個のレジスタ）．

ヒント SM 当たり：PC 数 = スレッド数 ÷ ワープサイズ；レジスタ数 = $K\times$ スレッド数．GPU はその両方をすべての SM で繰り返す．どちらもスレッド数に比例するので，上のレイテンシ隠蔽の規則はチップ上の記憶容量についての命題である．

4.2 バンク競合

前節のレジスタファイルはワープ全体のオペランドを一度に供給しなければならず，そのため容量ではなく帯域幅が設計上の問題になる．本節ではポートとバンク，バンク化がレジスタファイルと共有メモリに生む競合，そしてそれを吸収するオペランドコレクタとスコアボードを導入する．

出典 M4，バンク競合；K12 §1.4.1（レジスタの読み書き，スコアボーディング）；A18 ch. 3（オペランドコレクタ）；BPG §10.2.3 Shared Memory；CPG Compute Capability 5.x，Shared Memory；[18]．

4.2.1 レジスタファイルの帯域幅

典型的な算術命令は 2 つのソースレジスタを読み，1 つのデスティネーションに書く．SM は 1 つの命令を多数のスレッドについて同時に実行する．実行幅が 16 レーンなら 1 つの命令は 1 サイクルに $16\times 3 = 48$ 個のレジスタ値を動かし，より幅の広い SM ではそれに比例して多くなる．それだけ多くの経路から同時に読み書きできるメモリアレイを，ハードウェアは用意しなければならない．

K12 §1.4.1 はこれを拡大している：演算当たり 2 回の読み出しと 1 回の書き込みを行う 1 TFLOP/s の GPU は，64 TB/s 程度のレジスタ読み出し帯域幅を必要とする [3]．これはどの単一のメモリアレイの能力もはるかに超える．

定義 4.6 (ポート). **ポート**とは，記憶アレイから 1 サイクルに 1 つの値を読み出す，あるいは書き込むためのハードウェアのインタフェースである．**読み出しポート**は値を外へ運び，**書き込みポート**は値を中へ運ぶ．各ポートはアレイのすべてのエントリにつながる配線と選択論理の組であるので，そのコストはエントリ数とともに増え，ポートの多いアレイは大きく遅い．

▷ port
▷ read port
▷ write port

図 4.4a は実行幅 4 の場合の力ずくの設計を示す．各レジスタエントリが $2\times 4 = 8$ 個の読み出しポートと 4 個の書き込みポートを持ち，1 サイクルに任意の 8 値を読み，任意の 4 値を書ける．図 4.4b はアレイを分割することで，同じスループットをより安く実現する．

定義 4.7 (バンク). **バンク**とは，レジスタファイルやメモリを分割した複数の独立な記憶アレイの 1 つであり，それぞれが自前の（少数の）ポートを持つ．バンクは同時にアクセスされるので，それぞれ p 個のポートを持つ B 個のバンクは，要求された値が異なるバンクにある限り，1 サイクルに最大 Bp 個の値を供給する．図 4.4b では，それぞれ読み出しポート 2 個と書き込みポート 1 個を持つ 4 つのバンクが，エントリ当たり 4 分の 1 のポート数で，バンク化しない設計と同じ 8 回の読み出しと 4 回の書き込みを実現する．

▷ bank

4.2.2 レジスタバンク競合

この節約には条件がある．1 サイクルに必要な値がバンク全体に散らばっていなければならない．

定義 4.8 (バンク競合). **バンク競合**は，1 回のアクセス（1 つのワープ命令）が 1 つのバンクから，そのバンクのポート数より多くの値を必要とするときに起こる．ハードウェアはアクセスを複数のサイクルに直列化し，バンクが供給できる値のグループごとに 1 サイクルを使う．レジスタファイルにおけるものが**レジスタバンク競合** である．

▷ bank conflict

▷ register bank conflict

図 4.5 は，バンク当たり 2 つの読み出しポートを持つ 4 バンクのファイルで，スレッド k のレジスタをバンク k に割り当てる．

- スレッド T0–T3 の R0 を読むこと，すなわちワープ命令が行うことは，各バンクの 1 エントリに触れる．1 サイクルに 4 値であり，バンク化しないファイルと同じ速さである．

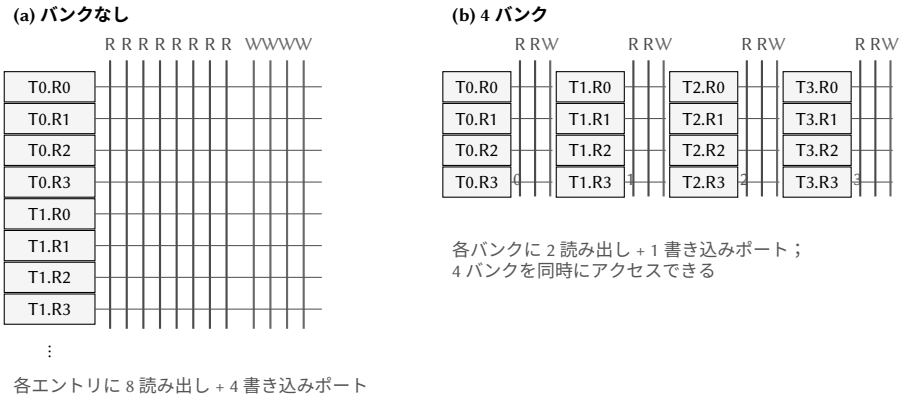


図 4.4: 実行幅 4 におけるポートとバンク．(a) バンクがなければ各エントリに読み出しポート 8 個と書き込みポート 4 個が必要である．(b) それぞれ読み出しポート 2 個と書き込みポート 1 個を持つ 4 つのバンクに同時にアクセスする．

- スレッド T0 の R0–R3 を読むことは，読み出しポートが 2 つしかないバンク 0 の 4 エントリに触れる．2 サイクルかかる．

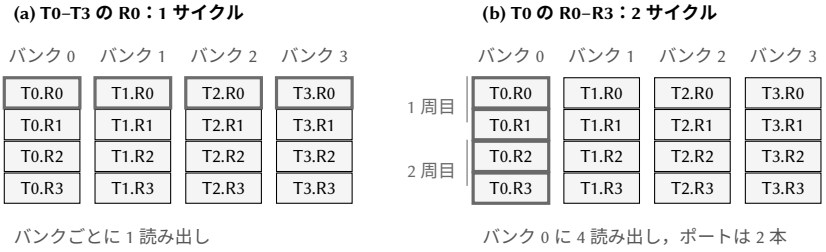


図 4.5: バンク当たり 2 つの読み出しポートを持つ 4 バンクのレジスタファイル．(a) 4 スレッドの R0 は 4 つのバンクにあり，1 サイクルで読まれる．(b) 1 スレッドの 4 つのレジスタは 1 つのバンクにあり，2 サイクルかかる．

ワープはその全スレッドについて常に同じレジスタ番号を読むので，最初の場合は安全に見える．しかしスレッド当たりのレジスタ数が変わると，スレッドからバンクへの配置が変わるので安全ではなくなる（図 4.6）．スレッド当たり 4 レジスタでバンク当たり 1 スレッドの場合（ケース 1），命令 R2 = R0 + R1 は各バンクから 2 値を読み，1 サイクルで完了する．スレッド当たり 2 レジスタの場合（ケース 2）は 2 つのスレッドが 1 つのバンクを共有する．T0–T3 のワープでは同じ命令がバンク 0 に 4 回の読み出し（T0 と T1 の R0 と R1）を，バンク 1 にも 4 回を送り，バンク 2 と 3 が遊んでいる間に 2 サイクルかかる．

実際の設計はスレッドのレジスタをバンク全体に散らす（たとえばスレッドのレジスタ r をバンク $r \bmod B$ に置く）．こうすると競合はスレッド数からではなく，命令が指定するレジスタから生じる．配置の選択は，コンパイラとハードウェアの取り決めの一部である．

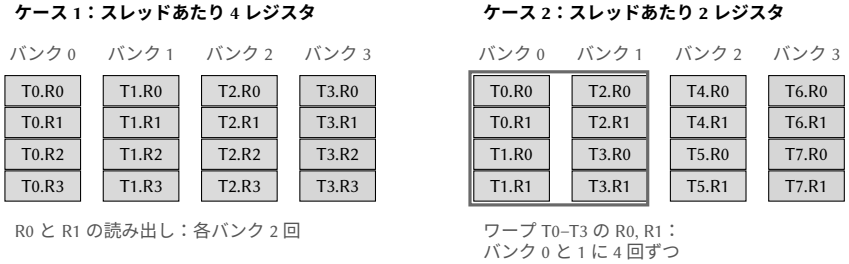


図 4.6: レジスタ数が変わる場合のスレッドとバンクの配置．ケース 1：スレッド当たり 4 レジスタ，バンク当たり 1 スレッド．ケース 2：スレッド当たり 2 レジスタ，バンク当たり 2 スレッド．そのため，ワープ T0–T3 について R0 と R1 を読む命令は，バンク 0 と 1 のそれぞれに 4 回の読み出しを送る．

4.2.3 静的と動的

第一の対策はコンパイラである．レジスタ番号はプログラムテキストの中で固定されているので，コンパイラは，1 つの命令のオペランドが異なるバンクに入るように，命令が使うレジスタと命令を出力する順序を選べる．すべての競合を取り除くことはできない．プログラムの依存関係が順序を制約し，レジスタの数が選択を制約するからである．この対策の背後にある区別は，コンパイラを扱う章を通して用いられる．

レジスタ割り付け，すなわちプログラムの値をレジスタに割り当てるコンパイラのパスは，第 11 章の生存変数解析の上に成り立つ．GPU ではバンク配置がその入力の 1 つになる．

定義 4.9 (静的と動的). プログラムの性質が**静的**であるとは，それがプログラムの実行前に決まり，入力に依存しないことをいう．入力に依存し，実行中にしか分からないとき，その性質は**動的** である．静的解析とはコンパイル時の解析である．**静的命令数**はプログラムテキスト中の命令の数であり，**動的命令数**は 1 回の実行で実行される命令の数である．

▷ static
▷ dynamic
▷ static instruction count
▷ dynamic instruction count

例 4.10. 次のループは 16 語を合計し，5 つの命令から成る．

```
LOOP: LD   R2, [R3]      // load a word
      ADD  R1, R1, R2    // accumulate
      ADD  R3, R3, #4    // next address
      SUB  R4, R4, #1    // count down from 16
      BNEZ R4, LOOP      // repeat while R4 != 0
```

静的命令数は 5 である．R4 を 16 に初期化するとループ本体は 16 回実行されるので，動的命令数は $16 \times 5 = 80$ である．R4 を入力から 1000 に設定すると 5000 になる．レジスタ番号，したがって各命令が読むバンクは静的であり，反復回数は動的である．

4.2.4 オペランドコレクタとスコアボード

第二の対策はパイプラインの中にある．レジスタの読み出しは，バンク競合のために，あるいはバンクに読み出しポートが 1 つしかないために複数サイクルかかることがあるので，SM は 5 段の CPU のようにオペランドの読み出しと実行を連続するステージで行わない．図 4.7 は，レジスタ読み出しと実行の間に 2 つのものを挿入する．

定義 4.11 (オペランドバッファとオペランドコレクタ). **オペランドバッファ**とは，待機中のワープ命令ごと，レーンごとに 1 エントリを持つ記憶であり，レジスタバンクが複数サイクルにわたって供給するソースオペランドがそこに書き込まれる．**オペランドコレクタ**は，これらのバッファを持ち，保持している命令のためにバンクの読み出しを発行し，命令のオペランドがすべて届いたときにその命令を準備完了として報告するユニットである．

▷ operand buffer

▷ operand collector

オペランドコレクタは A18 ch. 3 [4] の用語である．K12 §1.4.1 は同じ構造をオペランドバッファとして説明し，それが結果のキューとしても働くことを述べている [3]．

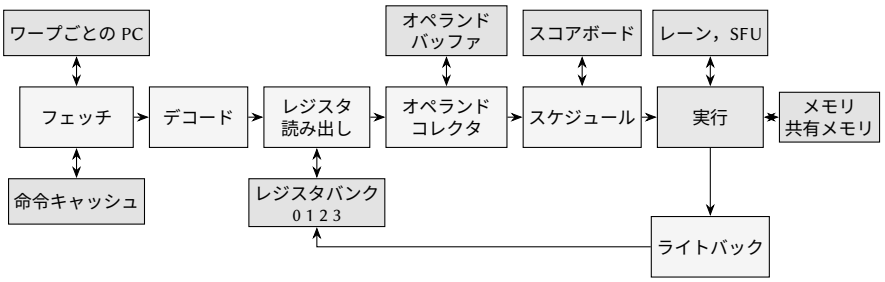


図 4.7: 本章の SM パイプライン：ワープごとに 1 つの PC によるフェッチ，デコード，バンクからのレジスタ読み出し，バッファを持つオペランドコレクタ，スコアボードを持つスケジューラ，レーンと SFU あるいはメモリユニットでの実行，そしてバンクへのライトバック．

定義 4.12 (スコアボード). **スコアボード**とは，保留中の各ワープ命令について，その各ソースオペランドが準備できているかを記録する表である．スケジューラはこれを参照して，オペランドがすべて揃ったワープを選び，そのワープの命令をレーンに発行する．準備のできたワープが複数あるときは，最古優先などのポ

▷ scoreboard

リシを適用する．

CPU では，スコアボードは初期のアウトオブオーダ動的スケジューリングの機構であり，1 つのスレッドの命令間のハザードを追跡した．GPU ではスコアボードはより単純な問い，すなわちワープ命令のオペランドが揃っているかに答え，選択は 1 つのスレッドの命令の間ではなくワープの間で行われる．

例 4.13. 4 つのワープ w0–w3 がそれぞれ 2 オペランドの命令をオペランドコレクタで待たせており，バンクは示した順に 1 サイクルに 1 つのオペランドを供給する．スコアボードの列は，各サイクル後の Src1 と Src2 の準備状態を示す．

サイクル	供給されたオペランド	w0	w1	w2	w3
0	—	··	··	··	··
1	w0.Src1	•·	··	··	··
2	w1.Src2	•·	·•	··	··
3	w2.Src1	•·	·•	•·	··
4	w0.Src2	••	·•	•·	··

サイクル 4 の後，両オペランドが揃っているのは w0 だけなので，スケジューラはそれを選び，そのオペランドはバッファからレーンへ移る．w1 と w2 は 2 つ目のオペランドを待ち続ける．オペランドが届く順序，したがってどのワープが先に発行されるかは動的である．それはプログラムテキストではなく，バンク競合とメモリシステムに依存する．[1]

4.2.5 共有メモリのバンク競合

第 3 章のブロックごとのスクラッチパッドである共有メモリは，レジスタファイルと同じ理由でバンク化されている．ワープのロードは 1 サイクルに 32 語を供給しなければならない．図 4.8 は，連続する語がバンクにインタリーブされ，語 a がバンク $a \bmod 4$ にある 4 バンクの共有メモリを示す．スレッド k が sharedInput[threadIdx.x * 4]，すなわち語 $4k$ を読むと，ワープの全スレッドがバンク 0 を読み，ロードはスレッド数と同じ数のサイクルに直列化される．

定義 4.14 (共有メモリのバンク競合). **共有メモリのバンク競合** は，1 つのワープのスレッドが 1 つの命令で，同じ共有メモリバンクにある異なる語にアクセスするときに起こる．ハードウェアは要求を必要なだけの競合のない要求に分割し，実効帯域幅はその分だけ低下する．同じ語にアクセスするスレッドは競合しない．複数のスレッドによる 1 語の読み出しはブロードキャストで処理され，複数のスレッドによる 1 語への書き込みはそのうちの 1 つが行う（BPG §10.2.3.1 Shared Memory and Memory Banks [8]；CPG，Compute Capability 5.x，Shared Memory [5]）．

Compute Capability 5.0 以降の NVIDIA GPU では，共有メモリは 32 個のバンクを持ち，各バンクは幅 32 bit で 1 クロックに 1 語を供給し，連続する 32 ビット語は連続するバンクに割り当てられる．ワープは 32 スレッドなので，競合はワープの任意の 2 スレッドの間で起こりうる．これらの規則から，バイトアドレス a のバンクは

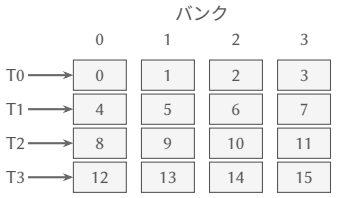
$$\mathrm{bank}(a) = \lfloor a/4 \rfloor \bmod 32, \tag{4.1}$$

となる．32 ビットの要素に一定の**ストライド** s 語，すなわち連続するスレッドが読む要素の間隔（スレッド k が要素 sk を読む）でアクセスするとき，触れるバンクは $32/\gcd(s, 32)$ スレッドごとに繰り返すので，そのアクセスは $\gcd(s, 32)$-way の競合である．

例 4.15. ワープが float の配列から s[threadIdx.x * 8] を読む．語のストライドは $s = 8$ なので，スレッド k はバンク $8k \bmod 32$ を読む．スレッド 0，4，8，…，28 はすべてバンク 0 に，スレッド 1，5，…，29 はバンク 8 に当たり，使われるのはバンク 0，8，16，24 だけである．各バンクを 8 スレッドが共有するので，ロードは 8-way の競合，$\gcd(8, 32) = 8$ であり，1 サイクルではなく共有メモリの 8 サイクルかかる．ストライド 7 ではバンクは $7k \bmod 32$ ですべて 32 個異なり，ロードは 1 サイクルで済む．

注意 講義 vs 教科書：A18 ch. 3 と K12 §1.4.1 では，スコアボードは書き込みが保留中のレジスタを記録し，RAW または WAW ハザードがあるワープの命令を発行前に止める．オペランドの収集は発行の後に行われるので，ハードウェアの順序はデコード，スコアボード検査と発行，オペランド収集，実行である [4, 3]．

[1] スコアボードは CDC 6600（1964 年）で，単一の命令ストリームのために導入された．GPU の形はワープ内ではインオーダである．K12 §1.4.1 は，スケジューラはインオーダだが “準備のできた任意のワープを選べる” ので，SM 全体の実行は “アウトオブオーダ実行のように見えるかもしれない” と述べている [3]．選択ポリシ（ラウンドロビン，greedy-then-oldest）は第 7 章で詳しく比較する．



Tk は語 $4k$ を読む：全員がバンク 0

図 4.8: 語をインタリーブした 4 つの共有メモリバンク．4 語のストライドは全スレッドをバンク 0 に送る．

▷ shared memory bank conflict

共有メモリのレイテンシは “キャッシュされないグローバルメモリのレイテンシのおよそ 100 分の 1” である [18]．n-way の競合は実効帯域幅を n で割る（BPG §10.2.3.1）．

▷ stride

注意 講義 vs ガイド：4 バンクのモデルでは threadIdx.x * 4 は完全な競合（全スレッドが 1 つのバンク）になる．実際の 32 バンクのハードウェアでは，同じアクセスは 8 つのバンクを使う 4-way の競合，$\gcd(4, 32) = 4$ である．

実際によくあるのは，行ごとに格納したタイルを列ごとに読む場合であり，第 5 章のタイル化した行列積がその例である．32 ビット要素の 32 × 32 のタイルでは，1 つの列の要素は 32 語離れており，32-way の競合になる．標準的な解決策は**パディング**である．タイルを 32 × 33 と宣言すると列のストライドは 33 になり，式 (4.1) の剰余演算によって単位ストライドと同じように振る舞い，競合がなくなる．代償は 1 行当たり 1 語の未使用領域である．

▷ padding

```
__shared__ float tileA[32][32];  // column read: 32-way
__shared__ float tileB[32][33];  // column read: no conflict
// (tx, ty) reads tileA[tx][ty] / tileB[tx][ty]:
// stride 32 vs 33
```

BPG §10.2.3.3 Shared Memory in Matrix Multiplication (C = AA^T) が $[w][w+1]$ のパディング規則の出典である [8]．Harris は転置について同じ競合を論じている [18]．

このようにデータ配置やアクセスパターンを組み替えることが，共有メモリの競合に対するソフトウェアの対策である．アドレスは動的なので，コンパイラがこれを自動的に適用することはできない．

ヒント 32 バンクでは奇数の語ストライドはすべて競合しない．$\gcd(s, 32) = 1$ だからである．行を 1 語パディングすると，偶数の行ストライドが奇数になる．

> **要点**
> バンク化は，幅の広いレジスタファイルや共有メモリに，少数のポートのコストで多数のポートの帯域幅を与える．異なるバンクの値は一緒に読まれ，同じバンクの値は直列化される．レジスタバンク競合はオペランドコレクタとスコアボードが吸収し，スケジューラはオペランドの揃ったワープを発行できる．共有メモリの競合はプログラマがストライドとパディングによって取り除く．

4.3 GPU アーキテクチャのパイプライン

本節では，小さな構成でワープ命令を図 4.7 のパイプラインに 1 ステージずつ通し，続いてワープ命令が一部のスレッドを飛ばせるようにするマスクビットを加える．

出典 M4，パイプラインのウォークスルー；K12 §1.4.1；A18 ch. 3；PTX ISA，特殊レジスタ [19]．

4.3.1 ステージ

図 4.7 のステージは，1 つのワープ命令について次のとおりである．

フェッチ．命令フェッチ ステージは，選ばれたワープの PC にある命令を命令キャッシュから読む．ワープ全体で 1 つの命令である．

▷ instruction fetch

デコード．デコードステージは，演算，ソースとデスティネーションのレジスタ，および即値オペランドを識別する．即値はすべてのスレッドで同じであり，全レーンのオペランドバッファに一度にブロードキャストされ，最初から準備完了とされる．

▷ decode

レジスタ読み出し． レジスタバンクが各スレッドのソース値を，1 サイクルまたは複数サイクルかけてオペランドバッファに供給する．

スケジュールと発行． スコアボードがすべてのオペランドの準備完了を示すと，スケジューラはそのワープを選ぶ．**命令発行** は，命令とそのオペランドを実行ユニットへ送る．

▷ issue

実行． 算術レーンが，1 レーンに 1 スレッドで計算する．超越関数（正弦，逆数，平方根）は**特殊関数ユニット**（SFU）へ送られる．SM はこれを少数しか持たないので，ワープの SFU 命令は複数回に分けて処理される．ロードとストアはメモリユニットあるいは共有メモリへ送られる．[2]

▷ special function unit (SFU)

[2] K12 §1.4.1 は G80 の実行ステージの形を示している．8 レーンであり，32 スレッドのワープは 1 命令に 4 サイクルかかった．SM 当たり 2–4 個の SFU が超越関数の近似を計算した．そして CPU パイプラインのようなフォワーディングネットワークはなかった．ワープの結果はワープ全体の幅を持ち（K12 は 1 KiB とする），連続する依存命令の間の隙間は他のワープが埋めることに頼るからである [3]．

ライトバック．ライトバック ステージは，各スレッドの結果をバンク内のそのレジスタに書き込む．

▷ write-back

4.3.2 ウォークスルー

例の構成は意図的に小さくしてある．ワープサイズ 4，8 スレッドのスレッドブロック 2 つ，したがってワープは 4 つで，SM は 4 レーンを持つ．ブロックとスレッドは CUDA と同じく 0 から番号を付けるので，`%ctaid.x` は 0 か 1，`%tid.x` は 0 から 7 である．レジスタは 32 ビットで，初期値は任意である．最初のサイクルの前の状態は表 4.1 の 3 つの表で与えられる．

命令は PTX の 2 つの**特殊レジスタ** を使う．これはハードウェアがスレッド

▷ special register

表 4.1: ウォークスルーの初期状態：各ワープの PC，命令キャッシュ内の命令，ブロック 0 のレジスタファイル．

ワープ	ブロック，スレッド	PC
W0	ブロック 0，T0–T3	0x8000
W1	ブロック 0，T4–T7	0x8004
W2	ブロック 1，T0–T3	0x8000
W3	ブロック 1，T4–T7	0x800C

アドレス	命令
0x8000	`add r1, r1, 2`
0x8004	`sub r2, r2, 1`
0x8008	`mov r1, %tid.x`
0x800C	`mov r2, %ctaid.x`
0x8010	`ld.shared r1, [A]`

ブロック 0	T0	T1	T2	T3	T4	T5	T6	T7
R1	20	21	22	23	30	31	32	33
R2	5	5	5	5	7	7	7	7
R3	1	4	9	16	25	36	49	64
R4	8	6	7	5	3	0	9	2

ごとに値を入れる読み出し専用のレジスタである．`%tid.x` はブロック内でのスレッドのインデックスを，`%ctaid.x` はグリッド内でのそのブロック（CTA）のインデックスを保持する．スケジューラはワープを W0，W1，W3 の順に選ぶものとし，以下の各ステップでは，次のワープを考える前に命令をすべてのステージに通す．

PTX ISA 9.4，特殊レジスタ `%tid`，`%ntid`，`%ctaid`，`%nctaid` [19]．第 3 章の `threadIdx`，`blockDim`，`blockIdx` はこれらにコンパイルされる．PTX 全体は第 10 章で扱う．

1. 0x8000 の W0：`add r1, r1, 2`．フェッチは 4 スレッドのために 1 つの命令を読む．デコードは即値 2 を 4 レーンすべての Src2 にブロードキャストし，準備完了とする．レジスタ読み出しは Src1 を T0–T3 の R1，すなわち 20，21，22，23 で満たす．両オペランドが揃ったので，スコアボードはスケジューラに W0 の発行を許し，4 レーンが加算し，ライトバックは 22，23，24，25 を T0–T3 の R1 に格納する．W0 の PC は 0x8004 に進む．
2. 0x8004 の W1：`sub r2, r2, 1`．スレッド T4–T7 について同じステージを通る．即値 1 がブロードキャストされ，T4–T7 の R2 が 7，7，7，7 と読まれ，$7 - 1 = 6$ がそれぞれに書き戻される．ハードウェアはワープのすべてのスレッドについて同一の仕事を行う．異なるのはレジスタの値だけである．
3. 0x800C の W3：`mov r2, %ctaid.x`．W3 はブロック 1 に属するので，特殊レジスタはすべてのレーンで 1 を返し，ブロック 1 の T4–T7 の R2 は 1，1，1，1 になる．仮に W3 の PC が 0x8008 であったなら，`mov r1, %tid.x` はスレッドインデックス 4，5，6，7 を R1 に書いたはずである．特殊レジスタがスレッドごとに異なるので，同じ命令がレーンごとに異なる値を生む．

表 4.2 は，最初の 2 ステップの後のブロック 0 のレジスタを示す．W2 が，W0 が持っていたのと同じ PC を持つことに注意する．異なるブロックの 2 つのワープは，同じ静的な命令をそれぞれ自分のレジスタに対して実行し，パイプラインの中にそれらを結び付けるものは何もない．

ヒント ワープ命令を手でたどるには：ワープの PC でフェッチする；即値をブロードキャストする；ワープの全スレッドについて指定されたレジスタを読む；レーンごとに計算する；レーンごとに書き戻す；そのワープの PC を進める．

表 4.2: ステップ 1 と 2 の後のブロック 0 のレジスタファイル（変化した値は太字）．

ブロック 0	T0	T1	T2	T3	T4	T5	T6	T7
R1	**22**	**23**	**24**	**25**	30	31	32	33
R2	5	5	5	5	**6**	**6**	**6**	**6**
R3	1	4	9	16	25	36	49	64
R4	8	6	7	5	3	0	9	2

4.3.3 マスクビット

すべての命令がそのワープの全スレッドを必要とするわけではない．ワープサイズ 4 で 6 スレッドのブロックは 2 つ目のワープの 2 レーンを空けたままにし，第 6 章で示すように，一部のスレッドだけが取る分岐は，経路が再収束するまで他のスレッドを遊ばせる．パイプラインはその両方を 1 つの機構で扱う．

第 6 章の SIMT スタックのマスクはまさにこれらのビットであり，分岐によるダイバージェンスと再収束に応じてプッシュとポップが行われる．第 5 章のワープ投票関数とシャッフル関数はアクティブマスクをプログラムに見せる．

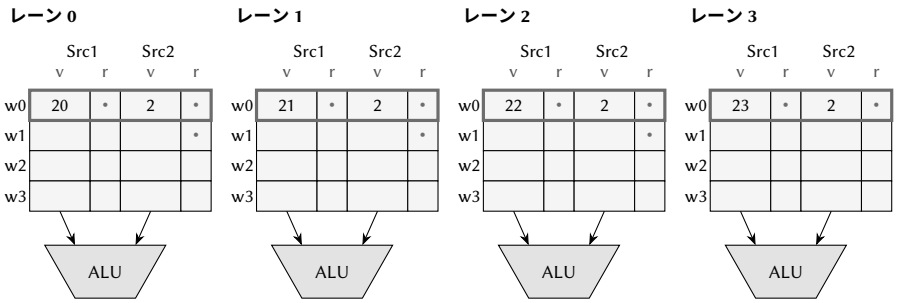
▷ active mask

定義 4.16 (レーンとアクティブマスク). **レーン**とは，パイプラインを通る 1 つの実行経路，すなわちワープ命令の 1 スレッドに仕えるオペランドバッファのエントリ，ALU，ライトバックのスロットである（第 1 章の SIMT レーンをパイプラインの側から見たもの）．ワープの**アクティブマスク**は，レーンごとに 1 ビットを持つビットベクトルである．ビットが 1 のレーンは命令を実行して結果を書き戻し，ビットが 0 のレーンは何の仕事もせず何も書かない．

上のウォークスルーはすべてのレーンをアクティブにして実行した．図 4.9 は，表 4.3 のマスクの下での後の命令についてレーンごとのオペランドバッファを示す．`W0` は 4 レーンすべてで準備ができて選ばれ，`W1` はマスク `1110` の下で動く．その最初の 3 レーンはオペランドを受け取って計算し，4 番目のレーンは非アクティブで，準備完了とされることはない．マスクがすべて 1 のワープが通常の場合であり，マスクは PC とともにワープの状態の一部である．

表 4.3: 後の命令のアクティブマスク（上のウォークスルーはすべてのマスクを `1111` として実行した）．`W1` のレーン 3 と `W2` のレーン 0 は無効になっている．

ワープ	マスク
W0	1111
W1	1110
W2	0111
W3	1111



w0 は全レーンで両オペランドが揃ったので選択される；w1 のレーン 3 はマスク 1110 で非アクティブ

図 4.9: レーンごとのオペランドバッファとスコアボード：`w0` はすべてのレーンで両オペランドが揃っており，4 つの ALU に発行される．`w1` はマスク `1110` の下で動くので，4 番目のレーンは非アクティブのままである．

> **要点**
> ワープ命令は，ワープの PC で 1 回フェッチされ，1 回デコードされ，その後レーンごとに読み出され，実行され，書き戻される．即値と`%ctaid.x` のような特殊レジスタはブロードキャストされ，`%tid.x` と通常のレジスタはレーンごとに異なる．アクティブマスクは，命令ストリームを変えずに個々のレーンを無効にする．

4.4 グローバルメモリのコアレッシング

SM の実行側はワープ命令を 1 つの単位として処理するが，メモリ側はそうはいかない．ロードの各スレッドが自分のアドレスを与えるからである．本節では，ワープが生成しうる要求の数を数え，それを処理しなければならないメモリ技術を概観し，アドレスの近い要求をハードウェアがどのように併合するかを示す．

出典 M4，コアレッシング；K12 §1.4.3；A18 ch. 4；BPG §10.2.1 Coalesced Access to Global Memory；CPG §5.3.2 Device Memory Accesses；PMPP ch. 6．

4.4.1 帯域幅の需要

32 スレッドのワープの 1 つのロード命令は，スレッドごとに 1 つ，最大 32 個の異なるメモリ要求を生成しうる．すべての SM がそのようなロードを毎サイクル発行したときに必要となる**メモリ帯域幅**は，容易に計算できる．

▷ memory bandwidth

例 4.17. ある GPU は 1.5 GHz の SM を 64 個持つ．すべての SM が 1 サイクルに 1 つのワープロードを発行し，各スレッドの要求が 32 B を取ってくるとすると，

SM 群は 1 サイクル当たり 64×32×32 B = 65 536 B を要求し，これは 1.5 GHz で約 98 TB/s になる．（SM が 32 個，要求当たり 64 B，1 GHz の場合，同じ 1 サイクル当たり 64 KiB は約 66 TB/s になり，普通は 64 TB/s と丸められる．）H100 SXM5 のメモリシステムが供給するのは約 3 TB/s（暫定値）であり，およそ 30 分の 1 である．この差を埋めるのは，キャッシュ，すべてのワープが毎サイクルロードするわけではないこと，そして何よりもコアレッシングである．ワープの 32 個の要求は，たいていメモリの少数の隣接するブロックに収まる．

世代ごとの HBM 帯域幅：P100 720 GB/s，V100 900 GB/s，A100（40 GiB）1555 GB/s [1]；H100 SXM5 3000 GB/s（暫定値，HWP V1.02）[2]．

GPU のキャッシュはスレッド数に比べて小さいので，その帯域幅は容易に飽和し，ほとんどのカーネルではオフチップメモリがボトルネックになる．そこにどのメモリ技術があるかが，ボトルネックの幅を決める．

4.4.2 SRAM，DRAM，HBM

図 4.10 は 3 つの技術を対比する．

SRAM． スタティックランダムアクセスメモリ，**SRAM** は，1 ビットを 6 トランジスタのセルに格納する．ビットを保持する 2 つの交差結合したインバータが，ワード線で制御される 2 つのアクセストランジスタを通してビット線につながる．高速でリフレッシュを必要としないが，1 ビット当たり 6 トランジスタなので面積のコストが高く，GPU のダイ上の構造，すなわちレジスタ，共有メモリ，キャッシュに使われる．

▷ SRAM

DRAM． ダイナミックランダムアクセスメモリ，**DRAM** は，1 ビットを 1 つのトランジスタの背後のキャパシタの電荷として格納する（1T1C）．セルはるかに小さいので，DRAM は GPU の主記憶の容量を与えるが，より遅く，定期的なリフレッシュを必要とし，チップは別のパッケージに入っている．出入りするすべてのビットがパッケージのピンを通らなければならず，その数と速度が帯域幅を制限する．SM は複数の**メモリチャネル**を通して L2 と DRAM に到達し，各チャネルは自前の DRAM を持ち，コアレスされた要求はそれらに分散される（K12 §1.4.3 [3]）．

▷ DRAM

HBM． 広帯域メモリ（第 1 章）は DRAM のセルはそのままに，パッケージングを変える．複数の DRAM ダイを垂直に積み重ね，積層を貫いて接続したものが **3 次元積層メモリ** であり，その積層は**シリコンインターポーザ**，すなわち同じパッケージの中で数千本の細い配線を運ぶシリコン基板の上に，GPU ダイと並べて置かれる．GPU へのリンクはもはやパッケージのピンや回路基板を通らないので，数千ビットの幅を持つことができ，パッケージ当たりの帯域幅は GDDR メモリシステムの数倍になる．

▷ 3D-stacked memory
▷ silicon interposer

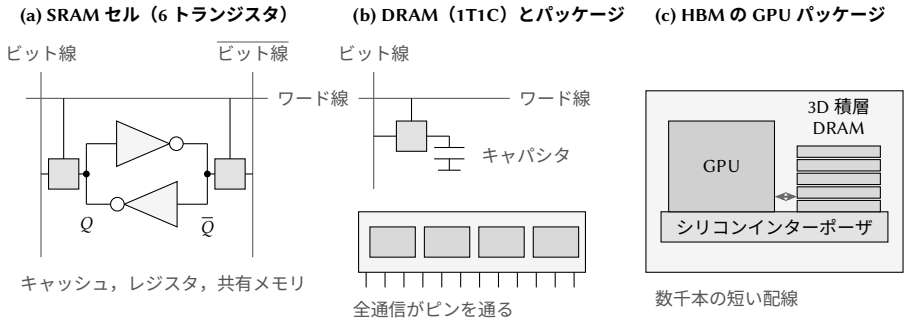


図 4.10: (a) 6 トランジスタの SRAM セル．(b) 1 トランジスタ 1 キャパシタの DRAM セルと，通信がピンを通る DRAM パッケージ．(c) シリコンインターポーザが GPU ダイと DRAM ダイの 3 次元積層を結ぶ GPU パッケージ．

初期の GPU は幅の広いバスで GDDR DRAM を使い，各チャネルに FR-FCFS（first-ready，first-come first-served）スケジューラを置いた：K12 §1.4.3 [3]．インタフェース幅：P100 と V100 は 4096 ビットの HBM2 インタフェース，A100 は 5120 ビットの HBM2 インタフェース [1]，H100 SXM5 は 5120 ビットの HBM3 インタフェースを使う [2]．BPG §9.2.1 は V100 の 898 GB/s を 0.877 GHz × 2 × 4096 bit/8 として導く [8]．

4.4.3 コアレッシング

HBM でさえワープ命令当たり 32 個の別々の要求には応えられないので，SM のメモリユニットは 32 個のアドレスをまとめて調べる．

定義 4.18 (メモリコアレッシング). 1 つのワープ命令の各スレッドのメモリ要求を，ハードウェアがそのアドレスの許す限り少ないメモリトランザクションにま

とめることを**メモリコアレッシング** という．**メモリトランザクション** とは，メモリシステムと SM の間での，固定長で自然にアラインされたバイトのブロックの 1 回の転送である．32 個の要求が少数のトランザクションに収まるアクセスが**コアレスアクセス** であり，ほとんどのスレッドに別々のトランザクションを必要とするものが**非コアレスアクセス**である．

▷ memory coalescing
▷ memory transaction

▷ coalesced access
▷ uncoalesced access

図 4.11 は，8 スレッドが 2 つの PTX ロード

```
ld.global.u32 %r3, [%r1];   // R1: 512, 516, ..., 540
ld.global.u32 %r3, [%r2];   // R2: 0, 128, 256, ..., 896
```

を実行する 2 つの場合を示す．ここで DRAM 要求の長さは 64 B とする．最初のロードでは 8 個のアドレスは 512 から 540 までの連続する 4 バイト語で，合計 32 B なので，`Mem[512--575]` への 1 つの要求にまとまる．このアクセスはコアレスされている．2 つ目ではアドレスが 128 B ずつ離れているので，どの 2 つも要求を共有せず，8 個の要求が発行される．このアクセスは非コアレスであり，同じ 32 B の有用なデータのために 8 倍の帯域幅を消費する．

注意 講義 vs ガイド：例の 64 B の要求は講義が仮定する粒度（“64 to 128 B”）である．Compute Capability 6.0 以降，ハードウェアは 32 B のセクタで数える．BPG §10.2.1 は，ワープのアクセスが “ワープの全スレッドに応えるのに必要な 32 バイトトランザクションの数に等しい数のトランザクション” にコアレスされると述べている [8]．セクタキャッシュには第 8 章で戻る．

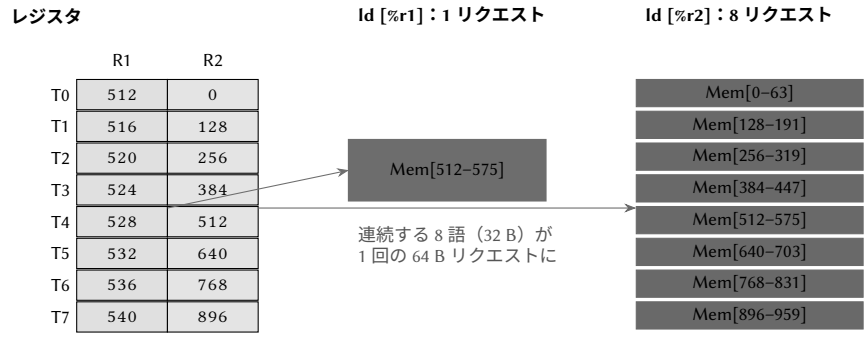


図 4.11: 8 スレッドが `%r1` を通して（連続する語，1 要求），また `%r2` を通して（128 B 離れた語，8 要求）ロードする．

正確な規則には 3 つの要素がある．グローバルメモリには 32，64，128 B のトランザクションでアクセスし，各トランザクションは自然にアラインされていなければならない．すなわち先頭アドレスがそのサイズの倍数でなければならない．これが**アライメント**の要件である．メモリは**キャッシュライン**の単位でキャッシュされる．NVIDIA GPU では 128 B で，4 つの 32 B セクタから成り，ワープのアクセスは触れたセクタの数だけコストがかかる．最後に，アクセスの**ストライド**，すなわち連続するスレッドのアドレスの間隔が，ワープの 32 個のアドレスがいくつのセクタにまたがるかを決める．

▷ alignment

CPG §5.3.2 Device Memory Accesses：“デバイスメモリには 32，64，128 バイトのメモリトランザクションでアクセスする．これらのメモリトランザクションは自然にアラインされていなければならない”[5]．

▷ cache line

例 4.19. 32 スレッドのワープが，スレッドごとに 4 バイト語を 1 つ，合計 128 B の有用なデータを，図 4.12 と表 4.4 の 4 つのパターンでロードする．効率は有用なデータを転送したバイト数で割ったものである．

表 4.4: 4 B の語を 32 個ロードするワープが転送するセクタ（BPG §10.2.1.1–10.2.1.4 による）．

アクセスパターン	セクタ	転送バイト	効率
(a) 連続，128 B 境界にアライン	4	128	100 %
(b) 連続，1 語ずれ	5	160	80 %
(c) ストライド 2 語	8	256	50 %
(d) ストライド 8 語以上	32	1024	12.5 %

パターン (b) は，連続するアクセスでもアライメントが重要であることを示す．1 語のずれで 5 つ目のセクタにはみ出す．パターン (d) は行優先の行列への列方向のアクセスであり，図 4.11 の非コアレスなロードはその極端な形である．連続するスレッドが連続する語に触れるようにカーネルを書くこと，すなわちパターン (a) は，最も重要なただ 1 つのメモリ最適化である．CUDA では，パターンはスレッドインデックスがアドレスにどう入るかで決まる．

K12 §1.4.3：Compute Capability 1.2 より前の GPU は厳格なコアレッシング規則（連続するスレッド，連続しアラインされた語）を課していた．後の世代はそれを緩め，コアレスされたワープのアクセスに対して “1–4 個のメモリトランザクション” を生成する [3]．

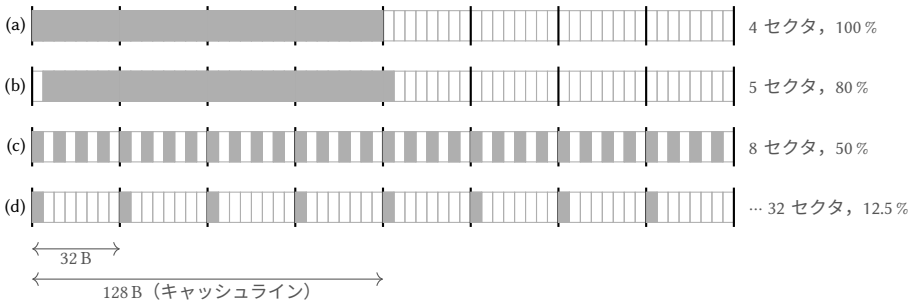


図 4.12: 32 スレッドのワープが触れる語（網掛け）と 32 B のセクタ：(a) アラインされ連続，(b) 1 語ずれ，(c) ストライド 2，(d) ストライド 8（セクタ当たり 1 語）．

```
// coalesced: consecutive threads, consecutive words
out[i] = in[blockIdx.x * blockDim.x + threadIdx.x];
// uncoalesced: stride of one row (n words) per thread
out[i] = in[threadIdx.x * n + blockIdx.x];
```

アルゴリズムがストライドのある見方を必要とするときの対策は，4.2 節の共有メモリである．タイルをコアレスしてロードし，それから計算が望む順序で共有メモリから読む．これは第 3 章のタイリングであり，第 5 章の主題である．

ヒント ロードを確かめるには，ワープのスレッド 0 と 1 がどのアドレスに触れるかを問う．その差が要素のサイズに等しく，スレッド 0 のアドレスがアラインされていれば，アクセスはコアレスされる．

要点 — 章のまとめ

SM はメモリレイテンシを細粒度マルチスレッディングで隠す．ワープごとの 1 つの PC とスレッドごとのレジスタの組がチップ上にとどまるので，ワープの切り替えにはコストがかからず，ワープ数は記憶容量で制限される．レジスタファイルと共有メモリは，ワープのオペランドを毎サイクル供給するためにバンク化されている．1 つのバンクからそのポート数より多くの値を必要とするアクセスがバンク競合であり，レジスタファイルではオペランドコレクタと，準備のできたワープを発行するスコアボードがこれを吸収し，共有メモリ（4 B 幅のバンク 32 個）では奇数のストライドとパディングによってこれを避ける．ワープ命令は 1 回フェッチされ 1 回デコードされ，アクティブマスクの下でレーンごとに実行される．即値と `%ctaid.x` はブロードキャストされ，`%tid.x` はレーンごとに異なる．メモリ側では，1 つのワープロードが 32 個の要求を発行しうる．ハードウェアは同じアラインされた 32 B セクタに収まる要求をコアレスするので，連続しアラインされたアクセスは 128 B のデータに対して 4 セクタを転送し，ストライドのあるアクセスは最大 32 セクタを転送する．インターポーザ上に積層した DRAM という HBM の構成は，残る需要に応えるためにある．

第 5 章

GPU プログラミングの最適化技法

講義動画	M5，レッスン 1（Advanced GPU Programming）
資料	[18]（必読）；[20]（必読，概念のレベル）；[21]，[22]，[23]，[24]（任意）
教科書	A18 [4] ch. 3；BPG [8] §9 Performance Metrics，§10 Memory Optimizations，§11 Execution Configuration Optimizations；K12 [3] ch. 3，§4.1；CPG [5] §5 Performance Guidelines；PMPP [6] ch. 6
キーワード	プロファイリング，ホットスポット，データ転送，ピン留めメモリ，非同期コピー，CUDA ストリーム，ゼロコピー，UVA，実効帯域幅，組み込み関数，プレディケート実行，ワープシャッフル，ワープ投票，Cooperative Groups，独立スレッドスケジューリング，cuBLAS，ソーティングネットワーク

第 3 章では CUDA プログラミングモデルを，第 4 章ではそれを実行するハードウェア，すなわちマルチスレッド SM，そのレジスタと共有メモリのバンク，パイプライン，グローバルメモリアクセスのコアレッシングを導入した．本章はその知識を，CUDA プログラムを高速にするための方法に変える．まず，そもそもどのようなプログラムが GPU に向いているかという問いと，プログラムがどこで時間を費やしているかを示すプロファイラから始める．GPU プログラムの時間は，ホストとデバイスの間のデータ転送，デバイス上のメモリアクセス，計算の 3 つの部分から成り，本章はそれぞれについてチェックリストを与える．転送にはピン留めメモリ，非同期コピー，ストリーム，メモリアクセスにはコアレッシング，アラインメント，バンク競合，計算にはより安価な命令，低い精度，組み込み関数，分岐の削減，アトミック操作の削減を挙げる．続いて，古いコストを取り除く新しい CUDA の機能（ワープシャッフルと投票，Cooperative Groups，独立スレッドスケジューリング，クラスタ，分割バリア），手書きのカーネルより優先すべきライブラリ，ほかの GPU プログラミングプラットフォームを概観し，最後に指定文献のソーティングネットワークを扱う．これは GPU に合わせて形を変えたアルゴリズムの古典的な例である．第 6 章と第 7 章はこの水準より下へ進み，ダイバージェンス，レジスタファイル，ワープスケジューリングの背後にあるハードウェア機構を扱う．

5.1 GPU に向くプログラムと時間の内訳

出典 M5，GPU 向きのプログラムとプロファイリング；BPG §9.1 Timing；K12 §3.5．

すべてのプログラムが GPU の恩恵を受けるわけではない．GPU でよく動くプログラムは 3 つの性質を持つ．

- **大規模なデータ並列性**：同じ操作が多数の独立なデータ要素に適用され，その数は数千のスレッドを埋めるのに十分である（第 1 章と第 2 章）；
- 計算に比べて**ホスト – デバイス間の通信が少ない**：I/O インタフェースを通して移動するデータが全体のコストに占める割合が小さい；
- **コアレスされたグローバルメモリアクセス**：隣り合うスレッドが隣り合うアドレスに触れる（第 4 章）．

第 1 の性質を欠くプログラムにはレーンに与える仕事がなく，第 2 または第 3 の性質を欠くプログラムは，SM がどれほど速くてもバスまたはメモリシステムで制限される．

アムダールの法則（第 1 章）は第 2 の性質を定量的に表したものである．ホスト – デバイス間の転送は，オーバーラップできない限り逐次部分の一部である（5.2 節）．

5.1.1 プロファイリング

最適化は測定から始まる．

定義 5.1 (プロファイリング，ホットスポット). プログラムがどこで時間を費やし，実行中にハードウェアが何をしているかを測定することを**プロファイリング** といい，その測定を収集するツールを**プロファイラ**という．**ホットスポット**とは，プログラムのうち全体の時間の大きな割合を占め，したがって最適化する価値のある部分（ループ，関数，カーネル）である．

▷ profiling
▷ profiler
▷ hotspot

表 5.1: 本章で扱うプロファイラのメトリクス，悪い値が示すこと，本書で対策を扱う箇所．

メトリクス	悪い値の意味	対策
スループット（命令，FLOP/s）	SM が遊んでいる：並列性が不足しているか，メモリでストールしている	5.4 節，第 3 章
ダイバージェント分岐	ワープのレーンが異なる経路をたどり，順番に実行する	5.4 節，第 6 章
コアレスされない（“ダイバージェントな”）メモリアクセス	1 回のワープロードが多数のメモリトランザクションになる	5.3 節，第 4 章
占有率	レイテンシを隠すための常駐ワープが少なすぎる	第 3 章
帯域幅の利用率	達成帯域幅がピークよりはるかに低い：アクセスパターンが悪いか，カーネルがメモリ律速でない	5.3 節

プロファイリングは 2 つの意味で最初に来る．コードを移植する前には *CPU* のプログラムをプロファイルし，GPU が高速化できるホットスポットを見つける．データ並列なホットスポットだけを移し，残りはホストにとどめる．カーネルがデバイス上で動くようになったら，GPU のプロファイラがその内部のボトルネックの位置を示すメトリクスを報告する（表 5.1）．

定義 5.2 (Nsight Systems，Nsight Compute). NVIDIA のプロファイラは，アプリケーション全体のタイムライン（CPU スレッド，CUDA API 呼び出し，メモリコピー，各 GPU 上のカーネル）を記録し，したがって 5.2 節の転送とオーバーラップの問題を示す **Nsight Systems** [23] と，1 つのカーネルを詳細にプロファイルし（命令スループット，メモリトランザクション，占有率，バンク競合，Roofline チャート），5.3，5.4 節の問題を示す **Nsight Compute** [24] である．

ヒント 変更のたびに前後で測定する．カーネル起動と非同期コピーは処理が終わる前にホストに戻るので，CPU のタイマは `cudaDeviceSynchronize()` で挟むか，CUDA イベント（`cudaEventElapsedTime`）でデバイス上の時間を測る；BPG §9.1 Timing．

▷ Nsight Systems

▷ Nsight Compute

2 つの Nsight ツールは `nvprof` と Visual Profiler を置き換えた．これらは Compute Capability 8.0 以降をサポートしない [25]．

5.1.2 実行時間の 3 つの部分

GPU プログラムの実行時間は次のように分解される．

$$T = T_{\text{transfer}} + T_{\text{memory}} + T_{\text{compute}}, \tag{5.1}$$

すなわち，ホストとデバイスの間でデータを移す時間，カーネルがデバイスメモリを待つ時間，計算に費やす時間である．3 つすべてに取り組まなければならず，意外さの順もこの順である．素朴に移植したプログラムは，カーネルにはまったく現れない第 1 項に支配されることが多い．この和は，3 つの部分が直列化された素朴なプログラムを表している．5.2 節の最適化は，第 1 項を短くするか，ほかの 2 項とオーバーラップさせる．以下の節は 3 つの項を順に扱う．ただし，それらすべてに先立つ第 4 の規則がある．計算をカバーするベンダのライブラリ（5.6 節）があれば必ずそれを使う．ライブラリには以下の最適化がすでに含まれているからである．

5.2 データ転送の最適化

ディスクリート GPU は独自のメモリを持ち，ホストは I/O インタフェースを通してしかそこに到達できない（図 5.1，左）．

定義 5.3 (データ転送，PCIe). ホストメモリとデバイスメモリの間でバッファをコピーすること（cudaMemcpy とその変種）を**データ転送**という．それが通るインタフェースは **PCI Express**（PCIe）であり，1–16 レーンのシリアルなポイントツーポイントバスで，その帯域幅はデバイスメモリより 1–2 桁低い．[1]

素朴なプログラムは入力をデバイスにコピーし，カーネルを実行し，結果をコピーして戻す．3 つのステップは順に行われる（図 5.2，0 行目）．4 つの技法が 2 つのコピーを短縮するか隠す．

出典 M5，データ転送の最適化；BPG §10.1 Data Transfer Between Host and Device；[21]；[22]．

▷ data transfer

▷ PCI Express (PCIe)

[1] K12 §1.2 は PCIe Gen 2 で 1 レーン当たり 4 Gbit/s，Gen 3 で 8 Gbit/s とする．すなわち 16 レーンでは 1 方向当たり約 8 GB/s と 16 GB/s である．V100 の HBM2 は 898 GB/s を供給する（BPG §9.2.1）．NVLink 4 は PCIe Gen 5 の 7× とされる [2]；第 9 章．

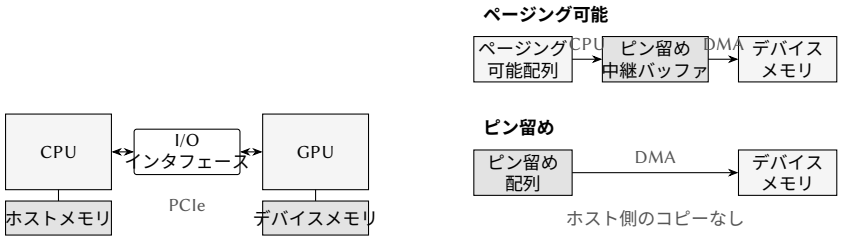


図 5.1: 左：ホストとデバイスのメモリは別々であり，明示的な転送はすべて I/O インタフェースを通る．右：ページング可能なホストメモリからのコピーは，CPU によるピン留めされた中継バッファへのコピーと DMA の 2 回のコピーである．ピン留めメモリからは 1 回の DMA で済む．

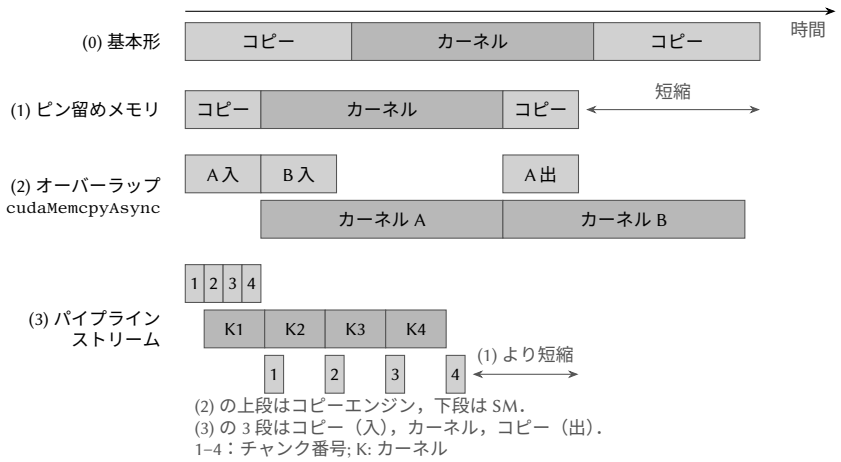


図 5.2: 4 つのデータ転送最適化の下での 1 つのジョブ．(0) 入力コピー，カーネル，出力コピーを順に行う．(1) ピン留めメモリが両方のコピーを短縮する．(2) 次の入力の非同期コピーが，SM が現在のカーネルを実行している間にコピーエンジン上で進む．(3) ストリームが入力，カーネル，出力をチャンクに分けてパイプライン化し，コピーエンジンと SM が同時に働く．

5.2.1 高速な転送：ピン留めメモリ

定義 5.4 (ピン留めメモリ)**.** オペレーティングシステムがページアウトも移動もしないと約束したホストメモリを**ピン留めメモリ**（ページロックメモリ）という．これは GPU の DMA エンジンが使用でき，DMA エンジンは物理アドレスによって直接それを読み書きする．`cudaMallocHost()` または `cudaHostAlloc()` で割り当て，`cudaFreeHost()` で解放する [21].

▷ pinned memory

`malloc` や `new` で得た通常の（**ページング可能な**）ホストメモリは, DMA の転送元にはなれない. オペレーティングシステムがいつでもそのページを移動または追い出しうるからである．したがってページング可能なメモリからの `cudaMemcpy` は，まず CPU 上でデータをドライバが所有するピン留めされた中継バッファにコピーし，それから DMA を開始する（図 5.1，右）．ピン留めメモリからのコピーは中継のステップを省く．ピン留めされた転送が “ホストとデバイスの間で最高の帯域幅を達成する” のはこのためである（BPG §10.1.1 Pinned Memory）.[2]

ピン留めメモリは乏しい資源である．ピン留めされたページはすべてオペレーティングシステムが回収できないページであり，“過度の使用はシステム全体の性能を下げうる”（BPG §10.1.1）．同ガイドは，どれだけが多すぎるかは事前にはわからないと付け加えている．規則は，すべての割り当てではなく転送用のバッファをピン留めすることである．またピン留めは無償でもない．ピン留めメモリの割り当ては `malloc` よりはるかに遅いので，バッファは起動時に 1 回だけ割り当てて再利用する．1 回しか転送しないバッファがピン留めの費用に見合うことはまれである．

[2] 16 MiB の転送で測定した値 [21]：ノート PC（NVS 4200M，PCIe Gen 2）ではピン留めメモリによりホストからデバイスへの帯域幅が 2.31 から 5.77 GB/s へ 2.5× に上がった．デスクトップ（GTX 680）では 5.37 から 6.19 GB/s に上がっただけである．CPU が速く，中継のコピーが安価だったためである．

5.2.2 計算と転送のオーバーラップ：非同期コピー

定義 5.5 (非同期コピー，オーバーラップ). **非同期コピー**とは，データが移動する前，**発行**された時点でホストに制御を返す転送である．cudaMemcpyAsync() は “cudaMemcpy() のノンブロッキング版” である（BPG §10.1.2）．その結果として，コピーエンジンと SM が同時に働けることを**計算と転送のオーバーラップ**という．あるデータセットのカーネルが実行されている間に，次のデータセットのコピーが進行中となる（図 5.2，2 行目）．コピーエンジンは SM とは別の DMA ユニットであり，データセンタ向け GPU は方向ごとに少なくとも 1 つを持つので，ホストからデバイスへのコピーとデバイスからホストへのコピーも互いにオーバーラップできる．

▷ asynchronous copy

▷ overlap of computation and transfer

非同期コピーの間はホストも空いており，次の入力を準備したり前の出力を処理したりできる．オーバーラップにはピン留めメモリが必要である．ページング可能なメモリからの非同期コピーは，戻る前にやはり CPU 上で中継のコピーを行うので，ホストから見るとほぼ同期的になる．

注意 ページング可能なバッファに対する cudaMemcpyAsync はコンパイルも実行もできるが，オーバーラップは得られない．プロファイラのタイムライン（Nsight Systems）を見れば違いはすぐにわかる．非同期転送にはすべてピン留めバッファを使う．

5.2.3 ストリームによるパイプライン化：コピーと実行の並行処理

1 つのジョブをチャンクに分けると，オーバーラップはパイプラインになる．

定義 5.6 (CUDA ストリーム). **CUDA ストリーム**とは，発行された順に実行されるデバイス操作（コピー，カーネル起動，イベント）の列である．**異なる**ストリームの操作の間には順序がなく，並行に，あるいは交互に実行されうる [22]．ストリーム引数をとるランタイム呼び出し（cudaMemcpyAsync，実行構成の第 4 パラメータ）はすべてそのストリームに置かれ，引数のない呼び出しは**デフォルトストリーム**に入る．

▷ CUDA stream

レガシーなデフォルトストリームは，デバイスのほかのすべてのストリームと同期する．そこに発行された操作は，すべてのストリームのそれ以前の仕事を待ち，それ以後の仕事をすべてブロックする（cudaStreamNonBlocking で作成したストリームは除く）[22]．したがってストリーム引数のない cudaMemcpy や <<<grid, block>>> が 1 つ紛れ込むだけで，パイプライン全体が直列化される．

注意 CUDA 7 はスレッドごとのデフォルトストリーム（nvcc --default-stream per-thread）を追加した．これは通常のストリームのように振る舞う．このフラグがなければ，デフォルトストリームはレガシーなものである [22]．

定義 5.7 (コピーと実行の並行処理). ジョブを n 個のストリームにわたってパイプライン化することを**コピーと実行の並行処理**という．入力を n 個のチャンクに分け，チャンク k の入力コピー，カーネル，出力コピーをストリーム k に発行する．ストリーム内では 3 つのステップの順序が保たれるので，各カーネルは自身の入力を待つ．ストリームをまたいでは，チャンク $k+1$ のコピーがチャンク k のカーネルとオーバーラップし，チャンク $k-1$ の出力コピーがその両方とオーバーラップする（図 5.2，3 行目；BPG §10.1.2 Asynchronous and Overlapping Transfers with Computation）．

▷ concurrent copy and execute

```
const int nStreams = 4;
size_t chunk = n / nStreams;  // チャンクのサイズ
float *h_in, *h_out, *d_in, *d_out;
cudaMallocHost(&h_in,  n * sizeof(float)); // ピン留め
cudaMallocHost(&h_out, n * sizeof(float));
cudaMalloc(&d_in,  n * sizeof(float));
cudaMalloc(&d_out, n * sizeof(float));
cudaStream_t s[nStreams];
for (int i = 0; i < nStreams; ++i) cudaStreamCreate(&s[i]);
for (int i = 0; i < nStreams; ++i) {
  size_t off = i * chunk, bytes = chunk * sizeof(float);
  cudaMemcpyAsync(d_in + off, h_in + off, bytes,
                  cudaMemcpyHostToDevice, s[i]);
  scale<<<chunk / 256, 256, 0, s[i]>>>(d_in + off,
                                       d_out + off, chunk);
  cudaMemcpyAsync(h_out + off, d_out + off, bytes,
                  cudaMemcpyDeviceToHost, s[i]);
}
```

表 5.2: 例 5.8 のスケジュール（時間の単位は ms）．各列が 1 つのエンジンであり，ストリーム内では 3 つのステップの順序が保たれる．

チャンク（ストリーム）	入力コピー	カーネル	出力コピー
1	0–2	2–3.5	3.5–5.5
2	2–4	4–5.5	5.5–7.5
3	4–6	6–7.5	7.5–9.5
4	6–8	8–9.5	9.5–11.5

```
cudaDeviceSynchronize();        // すべてを待つ
```

ループは 12 個の操作を発行してすぐに戻る．デバイスはストリームごとの順序だけに従ってそれらを実行する．最後の cudaDeviceSynchronize() は，ホストが h_out を読む前にすべてのストリームを待つ．

例 5.8 (パイプライン化したジョブ). あるジョブは 64 MiB を入力コピーし，カーネルを実行し，64 MiB を出力コピーする．逐次的には入力コピーに 8 ms，カーネルに 6 ms，出力コピーに 8 ms，合計 22 ms かかる．4 つのストリーム上の 4 つのチャンクに分けると，各チャンクは入力に 2 ms，カーネルに 1.5 ms，出力に 2 ms を要する．デバイスは方向ごとに 1 つのコピーエンジンを持ち，カーネルは一度に 1 つずつ実行する．表 5.2 がスケジュールをたどる．カーネルは，その入力が到着し，前のカーネルが終了した時点で始まる．出力コピーは，そのカーネルが終了し，前の出力コピーが完了した時点で始まる．ジョブは 11.5 ms で完了し，高速化率は 1.9× である．ボトルネックは入力側のコピーエンジンであり，0 から 8 ms まで隙間なく働き続ける．それ以降はすべてパイプラインの排出（最後のカーネルと最後の出力コピー，3.5 ms）である．チャンクを増やせば排出は短くなるが，操作ごとのオーバーヘッドが加わる．

ヒント BPG §10.1.2 による見積もり：転送時間を t_T，カーネル時間を t_E とすると，逐次版は $t_E + t_T$ かかり，段階化した版は $t_E > t_T$（カーネルがボトルネック）のとき約 $t_E + t_T/n$，転送がボトルネックのとき $t_T + t_E/n$ かかる．

5.2.4 ホストメモリへの直接アクセス：ゼロコピーと UVA

第 4 の技法は，カーネルにホストメモリを読ませることで，明示的なコピーそのものをなくす．

定義 5.9 (ゼロコピー). デバイスのスレッドがホストメモリに直接アクセスすることを**ゼロコピー**という．ホストのバッファをピン留めで割り当ててデバイスのアドレス空間に**マップ**し（cudaHostAlloc に cudaHostAllocMapped を指定），カーネルは対応するデバイスポインタ（cudaHostGetDevicePointer）を受け取る．カーネル内の各ロードとストアは，デバイスメモリではなく I/O インタフェースを通る（BPG §10.1.3 Zero Copy）．

▷ zero copy

```
float *h_a, *d_a;
cudaHostAlloc(&h_a, bytes, cudaHostAllocMapped);
cudaHostGetDevicePointer(&d_a, h_a, 0);
kernel<<<grid, block>>>(d_a);  // ロードは I/O バスを通る
```

ゼロコピーは**統合型** GPU では明らかに得である．そのメモリは物理的に CPU のメモリなので，置き換えられる “コピー” は同じ DRAM のある領域から別の領域へのコピーだったからである．ディスクリート GPU では各アクセスが I/O インタフェースの帯域幅とレイテンシを払うので，データが 1 回だけ，コアレスされたパターンで読まれ，そのレイテンシをほかのワープで隠せる場合にのみ役立つ．再利用されるデータはデバイスメモリに置くべきである．

定義 5.10 (統合仮想アドレッシング). **統合仮想アドレッシング（UVA）**の下では，ホストメモリとすべてのデバイスのメモリが 1 つの仮想アドレス空間を共有する．ポインタの値はそれだけでそのメモリを特定し，ランタイムはアドレスからポインタがどのメモリを指すかを判別でき（cudaPointerGetAttributes），cudaMemcpy を方向を指定せずに cudaMemcpyDefault で呼べる（BPG §10.1.4 Unified Virtual Addressing）．ドライバとランタイムは，物理的に別々のメモリを 1 つのアドレス方式の背後に隠す．

▷ unified virtual addressing (UVA)

UVA には Compute Capability 2.0 と 64 ビットのホストが必要である．UVA の下ではピン留めバッファのホストポインタとデバイスポインタは同じになり，GPU 間のピアツーピアコピー（第 9 章）が可能になる．BPG §10.1.4．

理想的には，統合されたアドレス空間によってデータの移動は無償になるはず

である．実際にはそうならない．メモリは依然として別々であり，データは依然としてインタフェースを越えなければならず，コピーは明示的に，あるいはランタイムの判断で，そのオーバーヘッドを伴って行われる．UVA が変えるのはメモリの**名前の付け方**であり，その**場所**ではない．

4 つの技法のどれもがすべてのシステムに合うわけではない．ピン留めメモリの効果はホストの CPU に，オーバーラップの効果は転送時間とカーネル時間の比に，ゼロコピーの効果は GPU が統合型かどうかに依存する．規則は，各技法を試して測定することである．

注意 UVA はユニファイドメモリではない．UVA（2011 年，CUDA 4）は別々のメモリの上の 1 つのアドレス空間である．ユニファイドメモリ（`cudaMallocManaged`，CUDA 6）はそれに加えて，要求に応じたメモリ間のページの自動移動を行い，それ自体のページフォールトのコストを伴う．ユニファイドメモリは第 7 章で扱う．

5.3 メモリアクセスの最適化

データがデバイス上に置かれると，式 (5.1) の第 2 項はカーネルがデバイスメモリを待つ時間である．このチェックリストは，第 4 章の機構を規則として繰り返す．

- チップ上のメモリを使う．ブロックが再利用するデータには共有メモリ，局所性のあるデータには L1 と L2 キャッシュ，全スレッドが読む値にはコンスタントメモリを使う；
- グローバルアクセスを**コアレス**し**アライン**して，ワープのロードが触れるメモリトランザクションをできるだけ少なくする．現在の GPU ではトランザクションは 32 B のセクタであり，連続する 4 バイト語を読む 32 スレッドのワープは 4 つのセクタに触れる（BPG §10.2.1 Coalesced Access to Global Memory）；
- **トランザクション数を減らす**：構造体の配列ではなく配列の構造体を使い，データ型を必要なサイズにし，アラインメントが許す場合はスレッド当たり 8 または 16 バイトをロードする；
- **達成帯域幅をピークと比べる**（定義 5.11）：コアレッシングの後も差が残るなら，カーネルはメモリ律速ではなく，取り組むべきは式 (5.1) の次の項である；
- **共有メモリのバンク競合を避ける**：32 ビット語のバンクが 32 個あり，バンクごとに 1 サイクル 1 アクセスなので，レーンが同じバンクの異なるアドレスにアクセスするワープは直列化される（同じ語を読むレーンはブロードキャストを受ける；第 4 章；[18]）．

出典 M5，メモリアクセスの最適化；BPG §9.2 Bandwidth，§10.2 Device Memory Spaces；[18]．

共有メモリのレイテンシは “キャッシュされないグローバルメモリのレイテンシより約 100 倍低い” [18]．Tesla と Fermi の世代について，K12 Table 1.4 は共有メモリに 4–32 サイクル，グローバルメモリに数百サイクルを挙げる．

定義 5.11 (理論帯域幅と実効帯域幅). メモリの**理論帯域幅** は，ハードウェアの仕様から，メモリクロック周波数 × バイト単位のインタフェース幅 × 2（ダブルデータレート）として計算する（BPG §9.2.1 Theoretical Bandwidth Calculation）．カーネルの**実効帯域幅**は，そのカーネルが実際に達成した帯域幅

▷ theoretical bandwidth

▷ effective bandwidth

$$\text{effective bandwidth} = \frac{(B_r + B_w)/10^9}{t}, \tag{5.2}$$

である．B_r と B_w はカーネルが読み書きするバイト数，t は秒単位の実行時間であり，単位は GB/s である（BPG §9.2.2 Effective Bandwidth Calculation）．

BPG 自身の例：V100 の HBM2 は 877 MHz，4096 ビットで，$0.877\times10^9\times512\times2/10^9 = 898$ GB/s となる（BPG §9.2.1）．

両者は同じ除数を使わなければならない．帯域幅は 10 進（10^9）であり，バイト数を代わりに 1024^3 で割ると実効帯域幅を 7 % 過小評価する（BPG §9.2.1）．Visual Profiler は，実効帯域幅に対応する “requested” スループットと，コアレスされないアクセスで無駄になったバイトを含む “actual” スループットを報告した．両者の差がコアレッシングによる損失である（BPG §9.2.3 Throughput Reported by Visual Profiler）．Nsight Compute では，同じ損失がメモリワークロード解析のリクエスト当たりセクタ数として現れる．連続する 32 個の 4 バイトロードでは理想値は 4 である．

例 5.12 (帯域幅の利用率). ある GPU は，メモリクロック 1.2 GHz，5120 ビットのインタフェースの HBM2 を持つ．理論帯域幅は

$$1.2\times10^9\times\frac{5120}{8}\times2\,/\,10^9 = 1536\,\text{GB/s}$$

である．あるカーネルは 256 MiB の配列（$B_r = 268\,435\,456$）を読み，同じサイズの配列を書き，0.5 ms で実行される．式 (5.2) により

$$\frac{(268\,435\,456 + 268\,435\,456)/10^9}{0.5\times10^{-3}} = 1074\,\text{GB/s}$$

となり，ピークの 70 % である．カーネルはメモリ律速であり，アクセスパターンはすでに良好である．残りの 30 % は仕様とストリーミングカーネルの間によくある差であり，コアレスされないアクセスの兆候ではない．同じバイト数で結果が 300 GB/s であったなら，最初に疑うべきはアクセスパターンであり，最初に読むべきメトリクスは Nsight Compute のリクエスト当たりセクタ数である．

5.4 計算の削減

式 (5.1) の第 3 項は計算そのものである．

出典 M5，計算の削減；BPG §12 Instruction Optimization，§13 Control Flow；CPG §5.4 Maximize Instruction Throughput．

定義 5.13 (命令最適化). カーネルが実行する命令を，同じ（または許容できる）結果を与える，より少ないか，より安価な命令で置き換えることを**命令最適化**という．同じ関数に対するより安価な演算，より狭いデータ型，ソフトウェアルーチンに代わるハードウェアの機能ユニット，分岐に代わるプレディケートなどである．

▷ instruction optimization

計算を減らす技法は 6 つある．

より安価な命令． 整数の除算と剰余は，SM 上では複数の命令の列である（BPG §12.1.4）．32 ビット整数の乗算は Volta 以降では 1 命令だが，Maxwell と Pascal ではエミュレートされていた．シフトとマスクは 1 命令である．2 のべき乗の除数 $n = 2^k$ に対しては，非負の `i` について `i / n` は `i >> k`，`i % n` は `i & (n - 1)` である．

強さの低減（第 11 章）の命令レベルの形である．コンパイル時定数の除数ならコンパイラが自分でこれを行う．この書き換えが意味を持つのは，実行時に決まる 2 のべき乗の場合である．

```
int q1 = i / n;         // 除算の命令列
int q2 = i >> k;        // n == 1 << k なら1回のシフト
int r  = i & (n - 1);   // i >= 0 なら i % n
```

低い精度． アプリケーションが許すなら，64 ビットの代わりに 32 ビットの演算を，32 ビットの代わりに 16 ビットや 8 ビットの演算を使う．幅を半分にするたびに演算ユニットのスループットは 2 倍になり，移動するバイト数は半分になる．コンシューマ向け GPU の FP64 スループットは FP32 のごく一部である．FP16 のペア（`half2`）は 1 命令で 2 つの演算を実行する（BPG §12.1）．形式と，テンソルコアがそれらで何をするかは第 12 章で扱う．

注意 低い精度は，性能の判断である前に数値計算上の判断である．多数の FP16 の項の和は加算のたびに桁を失い，100 万要素にわたる FP16 のリダクションは近似ではなく誤りである．累積変数はオペランドより広く保つ．

ハードウェアの特殊関数． SM の特殊関数ユニット（第 4 章）は，逆数，逆平方根，$\log_2$，2^x，正弦，余弦を，精度を落として 1 命令で評価する．

定義 5.14 (組み込み関数). **組み込み関数**とは，コンパイラがライブラリルーチンの呼び出しではなく直接ハードウェア命令に翻訳する関数であり，`__sinf()`，`__expf()`，`__fdividef()` などがある．標準関数 `rsqrtf()` も同様に SFU の逆平方根に対応する．逆平方根は，コンパイラが必ずしも 1 つにまとめられない `1.0f / sqrtf(x)` と書くのではなく，“常に明示的に `rsqrtf()` として呼び出すべきである”（BPG §12.1.6）．

▷ intrinsic function

```
float y1 = 1.0f / sqrtf(x);  // 2演算
float y2 = rsqrtf(x);        // SFU の1命令
```

数学ライブラリ（5.6 節）は組み込み関数を自動的に使う．コンパイラも，`nvcc -use_fast_math` の下ではファイル全体についてそうする．このオプションは非正規化数をゼロにフラッシュし，近似の除算と平方根も選ぶ（BPG §12.1.9）．

注意 `-use_fast_math` は速度だけでなく結果も変える．出力を精密なビルドの出力と照合する．

分岐の削減． 条件がワープのレーン間で異なる分岐は，ワープに両方の経路を実行させる（第 6 章）．対策は 2 つある．`threadIdx.x` ではなく `threadIdx.x / warpSize` に対する条件のように，ワープ全体が同じ経路をとるように条件を書くこと（CPG §5.4.2 Control Flow Instructions），そして短い分岐をコンパイラにプレディケーションで置き換えさせることである．

定義 5.15 (プレディケート実行). **プレディケート実行**では，条件文の両側がすべてのレーンに発行され，各命令は，その結果を書き込むかどうかをレーンごとに決めるプレディケートレジスタを伴う．分岐命令は実行されず，ワープはダイバージェンスを起こさない．代償は，すべてのレーンが両側のサイクルを費やすことである．コンパイラがプレディケーションを

▷ predicated execution

表 5.3: ワープシャッフル関数．mask は参加するレーンを指定する（32 レーンすべてなら 0xffffffff）．省略可能な width はワープを独立なセグメントに分割する．

関数	レーン i が値を受け取るレーン
`__shfl_sync(mask, v, src)`	src（1 つのレーンが全レーンにブロードキャストする）
`__shfl_up_sync(mask, v, d)`	$i-d$（d 未満のレーンは自身の値を保つ）
`__shfl_down_sync(mask, v, d)`	$i+d$（上位 d 個のレーンは自身の値を保つ）
`__shfl_xor_sync(mask, v, m)`	$i \oplus m$（バタフライ交換）

選ぶのは，条件で制御される命令の数がしきい値を下回る場合だけである（BPG §13.2 Branch Predication）．

ハードウェア側は第 6 章が示す．SIMT スタック，アクティブマスク，そして第 10 章の PTX のガードプレディケートである．

アトミック操作を避ける． 多数のスレッドが更新する位置へのアトミックな読み出し – 変更 – 書き込みは，メモリまたは L2 キャッシュでそれらを直列化する．アトミック操作は GPU の世代を経て大幅に安価になったが，数千のスレッドによる 1 つのカウンタへのリダクションは依然として直列化である．まずブロック内でリダクションし（共有メモリ，または 5.5 節のワープシャッフルによる），ブロック当たり 1 回のアトミック操作を発行する．

Kepler は共有メモリのアトミック操作をロック／更新／アンロックの命令列として実装していた．Maxwell は 32 ビット整数に対する共有メモリのアトミック命令をネイティブに追加した [26]．アトミック操作はこの種の変更によって “以前よりはるかに安価” になった．

テンソルコア． テンソルコア（第 1 章）上の行列積和演算は，FP32 レーン上の同じ演算より 1 桁速く実行される．行列積として表せる計算はすべてそう表すべきであり，ライブラリは形状と型が許せばそうする（第 12 章）．

> **要点**
> 式 (5.1) の 3 つの項には 3 つのチェックリストがある．転送：ピン留め，オーバーラップ，パイプライン化，マップ．メモリ：キャッシュ，コアレス，アライン，トランザクションの計数，バンク競合の回避，ピークとの比較．計算：より安価な命令，より狭い型，組み込み関数，ワープ内で一様な条件，アトミック操作の削減，テンソルコア．各ステップの前後でプロファイルする．

5.5 新しい CUDA の機能

上のチェックリストのコストのいくつかは，最初の CUDA モデル以降のハードウェアと API の追加によって取り除かれた．

出典 M5，新しい CUDA の機能；CPG §4.1 SIMT Architecture，§7.19 Warp Vote Functions，§7.22 Warp Shuffle Functions，§7.26 Asynchronous Barrier，§8 Cooperative Groups；[2]．

5.5.1 ワープレベルの操作：シャッフルと投票

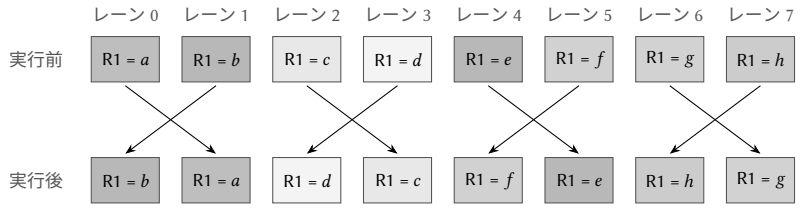
レジスタはスレッドに私有である．最初のモデルでは，同じワープの 2 つのスレッドが値を交換するには共有メモリを通すしかなかった．ストア，`__syncthreads()`，ロードであり，途中でバンク競合の危険もある．ワープレベルの操作は値をレーン間で直接移動する．

定義 5.16 (ワープシャッフル). **ワープシャッフル**とは，ワープの各レーンが同じワープのほかのレーンのレジスタを，共有メモリもバリアも使わずに読む命令である（CPG §7.22 Warp Shuffle Functions）．4 つの変種は読むレーンが異なる（表 5.3）．どの変種も，参加するレーンの**マスク**，値，レーンの選択子をとる．

▷ warp shuffle

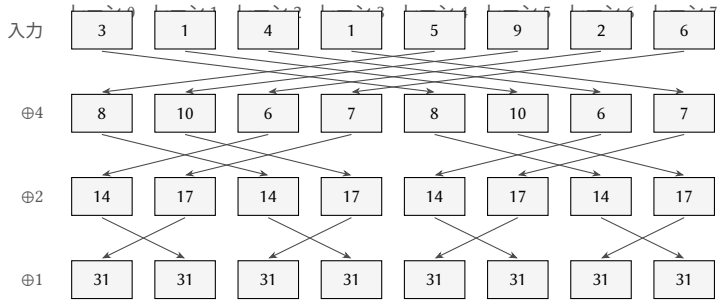
図 5.3 は最も単純な場合である対ごとの交換を示す．重要な用途はリダクションである．ワープの 32 レーンにわたる和は，レーン番号の 1 ビットにつき 1 回，計 5 回のシャッフルで求まり，すべてのレーンが合計を得る（図 5.4，例 5.17）．

```
__device__ float warp_sum(float v) {
  for (int m = 16; m > 0; m >>= 1)
    v += __shfl_xor_sync(0xffffffff, v, m);
  return v;   // 全レーンがワープの合計を持つ
}
```



__shfl_xor_sync(0xffffffff, R1, 1): レーン i はレーン $i \oplus 1$ の値を受け取る

図 5.3: レーンマスク 1 の __shfl_xor_sync：図の 8 レーンは 1 命令で R1 を隣のレーンと交換する．



各段で，レーン i は自分の値にレーン $i \oplus m$ の値を加える

図 5.4: 例 5.17 のバタフライリダクション：マスク 4，2，1 による 3 回の __shfl_xor_sync で，すべてのレーンに和 31 が残る．

ブロックのリダクションは，ワープごとの合計のために 1 回だけ共有メモリを必要とし，グリッドのリダクションはブロック当たり 1 回のアトミック操作を必要とする．

例 5.17 (バタフライリダクション). 8 つのレーンが値 3, 1, 4, 1, 5, 9, 2, 6（和は 31）を持ち，ループはマスク 4, 2, 1 で回る．1 回目の後，レーン i は $v_i + v_{i\oplus 4}$ を持つ．レーン 0–3 は 8, 10, 6, 7 を持ち，レーン 4–7 も同じ値を持つ．マスク 2 の後，レーン 0 と 1 はそれぞれ $8+6=14$ と $10+7=17$ を持ち，これがワープ全体で繰り返される．マスク 1 の後，すべてのレーンが $14+17=31$ を持つ（図 5.4）．8 レーンでは 3 回，完全なワープでは $\log_2 32 = 5$ 回である．

定義 5.18 (ワープ投票関数). **ワープ投票関数**は，参加する各レーンでプレディケート p を評価し，その結果をワープ全体で結合する（CPG §7.19 Warp Vote Functions）．

- __all_sync(mask, p) は，p がすべてのレーンで成り立てば真である；
- __any_sync(mask, p) は，少なくとも 1 つのレーンで成り立てば真である；
- __ballot_sync(mask, p) は，成り立つレーンの 32 ビットマスクを返す．

```
unsigned m = __ballot_sync(0xffffffff, x > 0);
int count = __popc(m);              // x > 0 のレーン数
if (__all_sync(0xffffffff, x > 0))  // ワープ内で一様な分岐
  ...
```

ballot に続く __popc は，共有メモリのヒストグラムならバリアを必要とする計数を，バリアなしに 2 命令で行う．__all_sync や __any_sync の結果に対する条件は構成上ワープ内で一様であり，決してダイバージェンスを起こさない．

5.5.2 Cooperative Groups

定義 5.19 (Cooperative Groups). 固定されたワープとブロック以外のスレッドのグループに名前を付ける CUDA の API を **Cooperative Groups** という（CPG §8 Cooperative Groups）．グループは，ブロック全体（this_thread_block），ワープまたはワープのタイル（tiled_partition），呼び出し時にアクティブなワー

_sync 形は CUDA 9 からである．Volta が独立スレッドスケジューリング（後述）を導入した際に，CUDA 9 はマスクのない古い __shfl() 系を非推奨にした．マスクは，交換の前にどのレーンが到達していなければならないかをハードウェアに伝える．Compute Capability 8.0 ではワープリダクションが単一の関数 __reduce_add_sync とその仲間として加わった（CPG §7.21 Warp Reduce Functions）．

▷ warp vote function

▷ cooperative groups

プのレーンの集合（coalesced_threads），グリッド全体，またはクラスタである．どのグループも sync() とサイズ，ランクを提供する．ワープサイズのグループ（タイルとコアレスされたグループ）はさらに shfl，ballot，any，all，cg::reduce を提供する．グリッドグループには協調起動（cooperative launch）が必要である．

```
#include <cooperative_groups.h>
namespace cg = cooperative_groups;
auto block = cg::this_thread_block();
auto tile  = cg::tiled_partition<8>(block);  // 8レーン単位
v += tile.shfl_xor(v, 4);      // タイル内のシャッフル
tile.sync();                   // 8レーンのバリア
```

この API は，ワープサイズとブロックサイズを，関数に埋め込まれた前提ではなく関数のパラメータにする．タイルをパラメータとしてとるルーチンは 4，8，32 レーンのタイルに対してそのまま動作し，sync()，サイズ，ランクだけを使うルーチンはタイルにも 1024 スレッドのブロックにも同様に動作する．

5.5.3 独立スレッドスケジューリング

定義 5.20 (独立スレッドスケジューリング). Volta で導入された実行モデルを**独立スレッドスケジューリング** という．これはワープの各スレッドに固有のプログラムカウンタとコールスタックを与えるので，1 つのワープのスレッドはワープより細かい粒度で分岐し，交互に実行し，再収束でき，スピンループで待つスレッドが同じワープのほかのスレッドの進行を妨げることはもはやない．

▷ independent thread scheduling

Volta 以前，ワープはプログラムカウンタを 1 つしか持たなかった．ダイバージェント分岐では，ハードウェアは一方の経路を再収束点まで実行してから他方を開始し（第 6 章），他方の経路のスレッドを待つスレッドは永久に待ち続けた．スレッドごとの状態があれば，スケジューラは 2 つの経路を交互に実行できる．これによりワープ内のこのような細粒度の同期が正しく動作するようになるが，その代わりに，ワープのレーンがロックステップで実行されるという保証は失われる．

注意 講義 vs ドキュメント：この機能は，固定されたワープサイズに代わるより小さなワープレベルの操作として説明されている．CPG §4.1 SIMT Architecture はより具体的である：“NVIDIA Volta アーキテクチャ以降，独立スレッドスケジューリングはワープにかかわらずスレッド間の完全な並行性を可能にする … GPU は，プログラムカウンタとコールスタックを含む実行状態をスレッドごとに保持する”．ワープは依然として 32 スレッドである．変わったのは，コードがレーンのロックステップを前提にできなくなったことであり，ワープレベルの関数がマスクをとるのはこのためである．

5.5.4 クラスタ，統合されたオンチップメモリ，分割バリア

さらに 3 つの追加が，オンチップメモリと同期にかかわる．

ブロック間の共有メモリ． Hopper では，第 3 章のスレッドブロッククラスタにより，ブロックはそのクラスタのほかのブロックの共有メモリにアクセスできる．ハードウェアはクラスタのブロックを 1 つの GPU プロセッシングクラスタ上に同時にスケジュールする．NVIDIA はこれを**分散共有メモリ**（distributed shared memory）と呼ぶ [2]．これにより，ブロックはグローバルメモリを往復せずに隣のブロックとデータを交換できる．

L1，共有メモリ，テクスチャの統合ストレージ． Volta 以降，L1 データキャッシュ，共有メモリ，テクスチャキャッシュは SM ごとに 1 つの SRAM アレイであり，カーネルごとにキャッシュとスクラッチパッドに分割される．共有メモリをあまり使わないカーネルは大きな L1 を得る．この分割（カーブアウト）と，それにつながった研究は第 6 章で扱う．

ハードウェアによる到着／待機分割バリア．

> **定義 5.21** (分割型到着／待機バリア). **分割型到着／待機バリア** は，バリアの 2 つの半分を分離する．スレッドはデータの準備ができたときに**到着**（arrive）して独立な仕事を続け，ほかのスレッドのデータが必要になったときにだけ**待機**（wait）する．__syncthreads() は到着の直後に待機を行うものである．CPG §7.26 の非同期バリア（cuda::barrier）は 2 つの半分を公開し，Compute Capability 8.0 はそれらをハードウェアで高速化する．到着と待機の間の時間は，そうでなければスレッドがストールに費やす時間である．CPG §7.27 の非同期コピーを使えば，スレッドは次のタイルの共有メモリへのロードを開始し，到着し，コピーが届くまで現在のタイルで計算できる．

H100：クラスタは最大 16 ブロック（ポータブルでないサイズ）を保持し，SM は L1 と共有メモリを合わせて 256 KiB 持ち，そのうち最大 228 KiB を共有メモリにできる [2, Table 4]．

▷ split arrive/wait barrier

表 5.4: 本章で取り上げる NVIDIA のライブラリ．

ライブラリ	分野
cuBLAS	密な線形代数（BLAS：ベクトル，行列 – ベクトル，行列 – 行列の演算）．GEMM にテンソルコアを使う
cuSPARSE	疎行列と疎 – 密の積
cuTENSOR	テンソルの縮約，リダクション，要素ごとの演算
Thrust	デバイスベクトル上の C++ 並列アルゴリズム（ソート，スキャン，リダクション，変換）．C++ 標準ライブラリのスタイルによる
NCCL	GPU 間の集団通信（all-reduce，broadcast，all-gather）．第 2 章の MPI の集団通信の GPU 版
NVSHMEM	GPU 間の片方向の，分割グローバルアドレス空間型の通信
cuDNN	深層学習のプリミティブ：畳み込み，プーリング，正規化，アテンション

表 5.5: 本章で取り上げるほかの GPU プログラミングプラットフォーム．

名称	内容
OpenCL	異種計算のための Khronos のオープン標準：カーネルは C の方言で書き，GPU，CPU，FPGA の間で移植可能
oneAPI / SYCL	Khronos の単一ソース C++ モデル（SYCL）と，Intel によるその実装とライブラリ（oneAPI）
OpenACC	OpenMP のスタイルによる指示文ベースのオフロード（`#pragma acc`）
HIP	AMD の C++ ランタイム API で，CUDA とソース互換（`hipMalloc`，`hipLaunchKernelGGL`）．CUDA のコードは `hipify` で機械的に移植される
Kokkos	1 つのソースを CUDA，HIP，SYCL，OpenMP のバックエンドにコンパイルする C++ の性能可搬性レイヤ
Python，Julia	GPU 配列ライブラリとカーネルコンパイラを持つ高水準言語（CuPy，Numba；CUDA.jl）

5.6 ライブラリとほかのプラットフォーム

どのチェックリストよりも前の第 1 の規則は，ライブラリを使うことである．

出典 M5，ライブラリとほかのプラットフォーム．

定義 5.22 (数学ライブラリ). **数学ライブラリ**とは，ある分野（密な線形代数，疎行列，テンソル，深層学習の層，通信）のための GPU ルーチンの集まりで，ベンダまたはコミュニティが提供し，GPU の各世代向けにすでにチューニングされている（コアレス，タイル化，テンソルコア対応，パイプライン化）．これを呼ぶプログラムはそれらの最適化を受け継ぎ，再リンクによって世代をまたいでハードウェアに追従する．

▷ math library

表 5.4 は取り上げたライブラリの一覧である．最も広く使われる 3 つは，密な線形代数のライブラリで，その GEMM が機械学習の大半を支えるカーネルである **cuBLAS**（第 12 章），PyTorch などのフレームワークが呼ぶ深層学習プリミティブのライブラリ **cuDNN**，マルチ GPU 学習のための集団通信ライブラリ **NCCL**（第 9 章）である．ライブラリ **Thrust** は異色であり，分野別のライブラリではなく並列アルゴリズムの C++ テンプレートライブラリなので，デバイス上のソートやリダクションがカーネルではなく 1 回の呼び出しで済む．

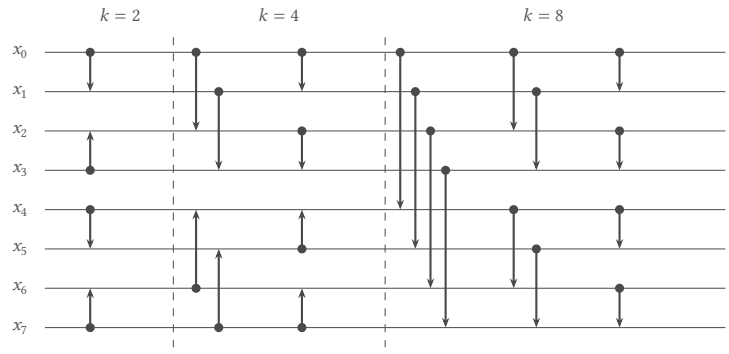
▷ cuBLAS
▷ cuDNN
▷ NCCL
▷ Thrust

CUDA は NVIDIA のプラットフォームであり，同じ考え方はほかにも存在する（表 5.5）．

`device_vector` に対する `thrust::sort` は，プログラムが手にできる最も手早く正しい GPU ソートである．自前で書くことが報われるのは，ライブラリが知りえない構造をデータが持つ場合だけである（5.7 節）．

標準規格 **OpenCL** は，CUDA と同じ実行モデルを別の名前で持つ（スレッド，ブロック，グリッドに対して work-item，work-group，NDRange；K12 §1.3.10）．**SYCL** は CUDA と同様にホストとデバイスのコードを 1 つの C++ ソースに置き，**HIP** は CUDA に十分近いので，本書のプログラムは関数単位で翻訳できる．これらの間で移植できるのは本章の内容のすべてである．式 (5.1) の 3 つの項とそのチェックリストは，言語ではなくハードウェアの性質である．

▷ OpenCL
▷ SYCL
▷ HIP



矢印の先のワイヤが大きい方の値を受け取る．$k = 2$，4 の段は長さ k の昇順・降順の列を交互に作り，$k = 8$ の段が全体を昇順に並べる．

図 5.5: $n = 8$ のバイトニックソーティングネットワーク：6 段，各段 4 個の比較器．フェーズ k は長さ $k/2$ の列の対を長さ k のソート済みの列にマージする．フェーズ内の比較器の距離は $k/2, k/4, \ldots, 1$ である．

5.7 ソーティングネットワーク

指定文献のソーティングネットワーク [20] は，GPU に合わせて形を変えたアルゴリズムの例である．

出典 M5，必読文献；[20]；[27]．

定義 5.23 (マージソート). **マージソート**と呼ばれるアルゴリズムは，列の 2 つの半分を再帰的にソートし，それらを**マージ**することでソートする．2 つのソート済みの列は，先頭のうち小さい方を繰り返し取ることで結合される．計算量は $O(n \log n)$ で，標準的な逐次アルゴリズムである．しかしマージはデータに依存するループであり，次にどちらの先頭を取るかが値に依存するので，その制御フローとメモリアクセスはスレッドごとに異なる．

▷ merge sort

GPU では，データに依存する制御フローはダイバージェンスであり，データに依存するアドレスはコアレスされないアクセスである（表 5.1）．GPU にうまく対応するアルゴリズムは，比較の列があらかじめ固定されているものである．

定義 5.24 (ソーティングネットワーク). **ソーティングネットワーク** とは，与えられたサイズのあらゆる入力をソートする，**比較器**の固定された列である．各比較器は配列の 2 つの位置を比較し，順序が逆なら交換する．各ステップでどの位置を比較するかはステップとサイズだけで決まり，データには決して依存しない．そのため 1 つの段のすべての比較器は並列に実行でき，すべてのスレッドが同じ操作を行い，アドレスはカーネルの実行前にわかる．

▷ sorting network

定義 5.25 (バイトニックソート). **バイトニック列**とは，まず増加し次に減少する列（またはそのような列の巡回シフト）である．ソーティングネットワーク**バイトニックソート**は，方向が交互のソート済みの列をマージして長さ $2, 4, 8, \ldots$ のバイトニック列を作り，各バイトニック列を**ハーフクリーナ**によってソートする．要素 i を要素 $i + k/2$ と比較すると，列は 2 つのバイトニックな半分に分かれ，前半のどの要素も後半のどの要素より小さくなる．それぞれの半分を再帰的にクリーンする（図 5.5）．

Batcher は 1968 年に，ハードウェアのソータのために 2 つの古典的なネットワーク，奇偶マージソートとバイトニックマージソートを導入した [27]．[20] は 2005 年，CUDA 以前に，テクスチャ上のフラグメント（ピクセル）シェーダプログラムとしてそれらを GPU に持ち込んだ（第 1 章）．

▷ bitonic sort

$n = 2^p$ 個の入力に対し，ネットワークは $n/2$ 個の比較器から成る段を $p(p+1)/2$ 段持つ．すなわち比較の回数は $\frac{n}{4}\log_2 n(\log_2 n + 1) = O(n\log^2 n)$ であり，マージソートの $O(n \log n)$ より多い．GPU は規則性のために余分な仕事を受け入れる．各段は 1 回のデータ非依存なパスであり，スレッド i が自分の対について距離 j の比較器を実行する．同じ考え方は，GPU プログラミングのほかのデータ非依存なプリミティブ，すなわち並列プレフィックス和，図 5.4 のバタフライリダクション，FFT の基礎にもなっている．

$n = 2^{20}$ の場合：$p = 20$ で，524 288 個の比較器から成る段が 210 段，比較は約 1.1×10^8 回である．マージソートの $n\log_2 n \approx 2.1 \times 10^7$ に対して 5 倍の仕事だが，そのすべてが規則的である．

要点 — 章のまとめ

プログラムが GPU に向くのは，大規模にデータ並列であり，計算に比べてホストとデバイスの間で移すデータが少なく，グローバルメモリにコアレスされたパターンでアクセスする場合である．プロファイラ（タイムラインには Nsight Systems，カーネルには Nsight Compute）がホットスポットを見つける．移植前は CPU 上で，移植後は GPU 上でである．実行時間は転送，メモリ，計算の和である．転送はピン留めメモリによって短縮され，非同期コピーとチャンクのストリームパイプライン（コピーと実行の並行処理）によって隠される．ゼロコピーと UVA はカーネルにホストメモリをアドレス指定させるが，データは依然としてバスを越える．メモリアクセスはコアレスされ，アラインされ，バンク競合のないものにし，実効帯域幅を理論帯域幅と比較する．計算は，より安価な命令，低い精度，組み込み関数，ワープ内で一様な条件またはプレディケーション，アトミック操作の削減，テンソルコアによって減らす．ワープシャッフルと投票，Cooperative Groups，独立スレッドスケジューリング，分散共有メモリ，分割バリアは，共有メモリを経由する古い回り道と，ロックステップのワープの古い制約を取り除く．ライブラリ（cuBLAS，cuDNN，NCCL，Thrust）はこれらの最適化をすでに備えており，同じ規則は OpenCL，SYCL，HIP でも成り立つ．バイトニックソートのようなソーティングネットワークは，余分な比較と引き換えにデータ非依存なスケジュールを得る．これは GPU のアルゴリズムが求める形である．

第 6 章

GPU アーキテクチャの最適化 I

講義動画	M6，レッスン 1–3（Handling Divergent Branch；Register Optimizations；Unified Register File and Shared Memory）
資料	Fung ら [28]（動的ワープ形成）；Narasiman ら [29]（ラージワープ）；Gebhart ら [30]（レジスタファイルキャッシュ），[31]（コンパイラ管理のレジスタ階層），[32]（統合ローカルメモリ）；Jeon ら [33]（レジスタファイル仮想化）；NVIDIA A100 [1] および H100 [2] ホワイトペーパー；Harris [18]（共有メモリのレイテンシ）
教科書	A18 [4] ch. 3（指定），ch. 4；K12 [3] §1.4.1 GPU Pipeline，§1.4.2 Handling Branch Instructions；CPG [5] §4.1 SIMT Architecture，Compute Capability 7.x–9.0 Shared Memory；PMPP [6] ch. 10
キーワード	ダイバージェント分岐，ダイバージェントワープ，再収束点，直接後支配節点，if 変換，SIMT スタック，ラージワープ，動的ワープ形成，SIMD 効率，生存レジスタと非生存レジスタ，レジスタファイルキャッシュ，レジスタファイル仮想化，ソフトウェア管理キャッシュ，統合ローカルメモリ，共有メモリのカーブアウト

第 4 章では SM のパイプラインを通るワープ命令をたどり，本章の土台となる 2 つの事実を述べた．ワープはレーンを無効にできるアクティブマスクを持つこと，そしてそのレジスタは，バンクに分割しなければならないほど大きなレジスタファイルに置かれることである．本章の前半では，1 つのワープのスレッドが分岐の向きについて食い違ったときに何が起こるかを問う．ハードウェアはそれをどう検出し，どのレーンがどの経路を実行し，マスクとプログラムカウンタのスタックによって経路をどう再び合流させるかである．続いて，ワープの構成を固定しないことでレーンの利用率を高める 2 つの研究上のアイデアを述べる．次にレジスタファイルそのものを扱う．それがなぜ大きく，遅く，電力を消費するのか，そしてレジスタの値に関する 4 つの観察から何が言えるかを調べ，その手段として，前段に置くレジスタファイルキャッシュと，名前の数より小さい仮想化されたファイルを示す．最後にレジスタを共有メモリやキャッシュと比較し，1 つのオンチップのプールで 3 つすべてを受け持てるかを問う．これは NVIDIA GPU の構成可能な L1 と共有メモリを研究として一般化したものである．第 7 章と同様に，本章は研究論文を軸に構成されており，そうした論文を読むことが第 2 の目的である．

6.1 ダイバージェント分岐の扱い

第 1 章の SPMD モデルはすべてのスレッドで 1 つのプログラムを実行するが，スレッドがその中を同じ経路でたどることを強制するものは何もない．本節ではダイバージェンスを定義し，ダイバージェントな分岐を実行する 2 つの方法（プレディケーションと SIMT スタック）を示し，そのコストに対する 2 つのマイクロアーキテクチャ上の対策，すなわちラージワープと動的ワープ形成を紹介する．

出典 M6，ダイバージェント分岐；K12 §1.4.2 Handling Branch Instructions；A18 ch. 3；CPG §4.1 SIMT Architecture；[28]．

6.1.1 ダイバージェント分岐とワープ

次のカーネルでは，各ブロックの最初の 2 スレッドがある処理を行い，残りが別の処理を行う．本章の図のように 4 スレッドのワープを考えると，スレッド 0–3 のワープは半分に分かれる．

本章の図ではマスクを短く保つために 4 スレッドのワープを描く．実際のワープサイズは，Compute Capability 5.0 から 9.0 までのすべての NVIDIA GPU で 32 である（CPG Table 21 [5]）．

```
__global__ void split(float *a) {
  int i = threadIdx.x;       // BB1: 条件を計算する
  if (i < 2)
    a[i] = a[i] * 2.0f;      // BB2: Work-1
  else
    a[i] = a[i] + 1.0f;      // BB3: Work-2
  a[i] = a[i] - 0.5f;        // BB4: 再び両方の経路
}
```

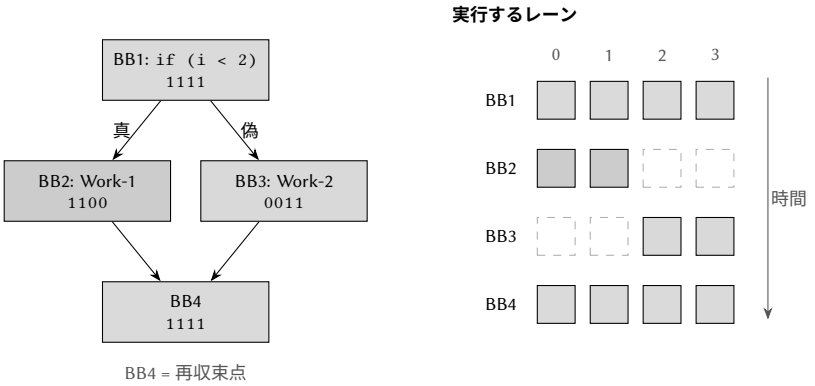


図 6.1: 4 スレッドのワープにおけるダイバージェント分岐．左：制御フローグラフ．各基本ブロックに，それを実行するときのアクティブマスクを添える（左から k 番目のビットがレーン k）．右：各ブロックを実行するレーン．BB2 と BB3 は順に実行され，それぞれレーンの半分が遊ぶ．

コンパイラはこのようなコードを**基本ブロック**に切り分ける．基本ブロックとは，最初の命令からしか入らず，最後の命令の後でしか出ない極大の命令列であり，その命令はすべて実行されるか，どれも実行されないかのどちらかである．ブロックと，それらの間の可能な制御の移動が図 6.1 の制御フローグラフを成す．制御フローグラフは第 10 章で形式的に定義する．

▷ basic block

定義 6.1 (ダイバージェント分岐とダイバージェントワープ). **ダイバージェント分岐** とは，その結果が 1 つのワープのスレッドの間で異なり，その後ワープのスレッドが異なるプログラムカウンタからフェッチすることになる分岐である．このときそのワープは**ダイバージェントワープ** であり，この現象を**ワープダイバージェンス** という．すべてのワープのすべてのスレッドが一致する分岐はワープ一様であり，スカラプロセッサ上と同じコストしかかからない．

▷ divergent branch

▷ divergent warp

▷ warp divergence

第 4 章のアクティブマスクは，各ブロックにどのレーンが参加するかを記録する（図 6.1）．BB1 は 1111，BB2 は 1100，BB3 は 0011，BB4 は再び 1111 で実行される．ワープのプログラムカウンタは 1 つなので，2 つの経路を同時には実行できない．それらは順に実行され，そのたびにもう一方の経路のレーンは無効にされ，2 つの経路の時間は足し合わされる．経路が再び出会う点 BB4 で，ワープは完全なマスクで再開できる．

定義 6.2 (再収束点). ダイバージェント分岐の**再収束点** とは，分岐から出たワープのすべての経路が再び出会うことが保証される最初の命令であり，そこからワープは全体として進むことができる．コンパイラの用語では，ブロック X からプログラムの出口へ至るすべての経路がブロック Y を通るとき，Y は X の**後支配節点**であり，X の**直接後支配節点**（IPDOM）は X 自身を除いて最も近い後支配節点である．分岐ブロックの IPDOM が通常の再収束点であり，図 6.1 ではそれは BB4 である．

注意 ダイバージェント分岐はコードを誤らせるのではなく，遅くするだけである．結果はスカラ機械が計算するものと同じである．異なるワープの間のダイバージェンスにはコストがかからない．各ワープが自分のプログラムカウンタを持つからである．

▷ reconvergence point

▷ post-dominator

▷ immediate post-dominator (IPDOM)

Fung らは，ワープの分かれた部分をカーネルの終わりまで別々に走らせるのではなく直接後支配節点で再収束させると，93.4 % の高速化が得られると報告している [28]．

▷ if-conversion

▷ predicate

できるだけ早く再収束することが重要なのは，分岐の後，再収束点の前に実行されるすべてのブロックが部分的なマスクで実行されるからである．

6.1.2 if 変換とプレディケーション

分岐を取り除く最も古い方法は，ジャンプを取り除くことである．

定義 6.3 (if 変換). **if 変換**と呼ばれる変換は，分岐とその 2 つの経路を直線的なコードで置き換える．1 つの命令が条件を**プレディケート**，すなわちレーンごとの 1 ビットのレジスタに計算し，元の経路の各命令はそのプレディケートまたはその否定によってガードされる（図 6.2）．ガードが偽のレーンは結果を書き込まない．得られたコードは第 5 章のプレディケート実行によって実行される．

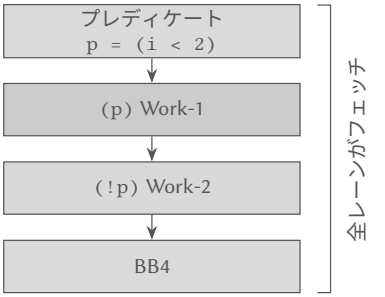


図 6.2: if 変換後の図 6.1 の分岐．ガード付きの命令を全レーンがフェッチする 1 つの基本ブロック．

if 変換はベクトルプロセッサでは標準的であり，CPU も短い分岐本体に用いる．そこでは条件付き移動のほうが，起こりうる分岐予測ミスより安い．GPU で

は if 変換はダイバージェンスを完全に取り除く．どのレーンもジャンプしないので，ワープは 1 つのプログラムカウンタを保つ．その代償は，どちらかの経路をどのレーンも必要としないときでも，両方の経路の命令がフェッチされ発行されることである．そのためコンパイラは if 変換を短い本体にしか適用しない．

PTX ではガードは命令の前に @p または @!p と書く．プレディケートレジスタは第 10 章で示す．

例 6.4. 上のカーネルで，BB1 と BB4 をそれぞれ 2 命令，BB2 と BB3 をそれぞれ 6 命令とする．4 レーンのワープを考え，発行された命令 1 つにつきレーン 1 つあたり 1 つとしてレーンスロットを数える．

- **ダイバージェントなワープ，分岐コード**：発行命令は $2+6+6+2=16$，レーンスロットは 64 で，そのうち $2\cdot4+6\cdot2+6\cdot2+2\cdot4=40$ が仕事をする．
- **ダイバージェントなワープ，*if* 変換後**：同じ 16 命令（プレディケートは BB1 で計算する）と，40 の有用なレーンスロット．
- **一様なワープ（全レーンが *BB2* を取る），分岐コード**：$2+6+2=10$ 命令で，全レーンが働く．
- **一様なワープ，*if* 変換後**：やはり 16 命令である．BB3 の 6 命令はどのレーンでも何もしない．

if 変換はコストをデータに依存しないものにする．この計数では，ワープがダイバージェントなら引き分けで，ワープが一様なら負ける．if 変換が節約するのはこの計数が除外したオーバーヘッド（分岐命令，スタックのプッシュとポップ，再収束の検査，フェッチ先の変更）であり，それが効くのは短い本体の場合だけである．

ヒント if 変換のコスト：常に両方の本体の和．分岐のコスト：ワープが一様なら取った側の本体，ダイバージェントなら両方の和．

6.1.3 SIMT スタック

プレディケーションを使わないなら，ハードウェア自身が両方の経路を実行し，それらを合流させなければならない．この機構は，図 6.1 のワープをたどりながら段階的に組み立てると最も理解しやすい．

1. BB1 の後，4 つのスレッドの次の PC は異なる（レーン 0–1 は BB2，レーン 2–3 は BB3）．分岐はダイバージェントである．1 つの PC と 1 つのマスクでは両方の行き先を表せない．
2. 1 つの経路（BB1，BB2，BB4）だけをたどるのは誤りである．レーン 2–3 のスレッドが BB3 を実行しないからである．
3. 解決策は取らなかった経路を覚えておくことである．すなわち**代替 *PC***（BB3）とそのマスク（0011）を，取った経路の後に実行する．順序は BB1，BB2，BB3，BB4 となる．
4. 分岐が入れ子になると，代替スロット 1 つでは足りない．BB2 自体がダイバージェントになると（図 6.3 左：BB4 は 1000，BB5 は 0100），その代替 PC BB5 が唯一のスロットの BB3 を上書きし，BB3 は失われる．
5. そこで保留中の経路をスタックに積む．

注意 講義 vs 教科書：以下で組み立てる 2 フィールドのエントリでは，再収束点で完全なマスクがどう戻るかが暗黙のままである．K12 §1.4.2 はエントリに 3 つのフィールド，“the PC address of the reconvergence point, active mask bits, and the next PC address” を与えている [3]．

定義 6.5 (SIMT スタック)**.** ワープの **SIMT スタック**は，ダイバージェント分岐の保留中の経路ごとに 1 つのエントリを保持する．エントリは代替 PC とそのアクティブマスク，そして現在の経路が終わる再収束 PC である．ダイバージェント分岐でハードウェアは取らなかった経路のエントリをプッシュし，取った経路を縮小したマスクで続ける．フェッチのたびに，次の PC を先頭エントリの再収束 PC と比較し，等しければそのエントリを**ポップ**して，代わりにエントリのマスクで代替 PC からフェッチする．**入れ子の分岐**，すなわち別の分岐の経路の中にあるダイバージェント分岐は，単にもう 1 つエントリをプッシュする．

▷ SIMT stack

▷ nested branch

表 6.1 は入れ子の例をたどる．BB4 は次の PC BB6 で終わり，これはスタックの先頭の再収束 PC に等しいので，ワープはポップして BB5 を 0100 でフェッチする．BB5 も BB6 で終わるので，ワープは 2 番目のエントリをポップし，BB3 を 0011 でフェッチする．BB3 が終わるとスタックは空になり，BB6 が完全なマスクで実行される．2 フィールドのスタックは 1111 がどこから来るかを示さない．傍注の 3 フィールドのスタックは，分岐前のエントリからそれを復元する．すべてのスレッドがちょうど自分の経路を実行し，6 つのブロックに 6 回のフェッチを要した．[1]

スタックのハードウェアコストは，ワープあたり数個のエントリと，フェッチごとの再収束の比較である．保持すべきエントリ数はダイバージェント分岐の入

[1] K12 §1.4.2 と A18 ch. 3 [3, 4] の 3 フィールドの定式化では，スタックの先頭は常にワープが実行しているものを表す．ダイバージェント分岐では先頭エントリの次 PC が再収束 PC に置き換えられ（分岐前のマスクは保つ），経路ごとに 1 つのエントリが，それぞれ自分の次 PC，マスク，共通の再収束 PC とともにプッシュされる．ワープが先頭エントリの再収束 PC に達するとそのエントリがポップされ，次のエントリが PC とマスクを与える．したがって BB6 では分岐前のエントリが 1111 を復元する．

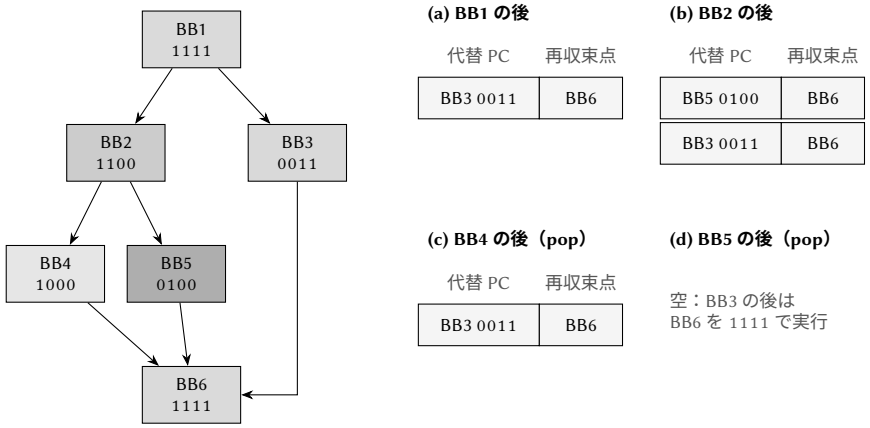


図 6.3: 入れ子の分岐における SIMT スタック．左：BB2 が再び分岐し，すべての経路が BB6 で再収束する．右：BB1 の分岐の後，BB2 の分岐の後，BB4 の終わりのポップの後，BB5 の終わりのポップの後のスタック（先頭エントリが上）．

表 6.1: 図 6.3 の入れ子の分岐を SIMT スタックで実行する様子．スタックのエントリは（代替 PC，マスク；再収束 PC）と書き，先頭を最初に置く．

ブロック	マスク	次の PC	動作	実行後のスタック
BB1	1111	BB2 / BB3	分岐：BB3 をプッシュ，BB2 へ	(BB3, 0011; BB6)
BB2	1100	BB4 / BB5	分岐：BB5 をプッシュ，BB4 へ	(BB5, 0100; BB6), (BB3, 0011; BB6)
BB4	1000	BB6	先頭の再収束点と一致：ポップし BB5 へ	(BB3, 0011; BB6)
BB5	0100	BB6	先頭の再収束点と一致：ポップし BB3 へ	空
BB3	0011	BB6	スタックが空：完全なマスク（暗黙）	空
BB6	1111	—	—	空

れ子の深さとともに増える．

6.1.4 再収束点の求め方

再収束 PC はコンパイル時に計算される．コンパイラは制御フローグラフを構築し，各分岐の直接後支配節点を求め，それをコードに印すか，スタックをプッシュしポップする明示的な命令を挿入する．スタック自体はハードウェアに置かれる．どの分岐が実際にダイバージェントになるかは実行時にしか分からないからである．原理的にはコンパイラは決してダイバージェントにならない分岐についてスタック操作を省けるが，分岐がダイバージェントになるかを静的に判定するのは難しい．そのためコンパイラは，そうでないと証明できない限り分岐をダイバージェントとして扱わねばならず，ダイバージェント分岐を過大に報告する．

第 11 章ではダイバージェンス解析を扱う．これは，どの値が，したがってどの分岐条件がワープのスレッド間で異なりうるかを決めるコンパイラの解析である．

注 6.6. SIMT スタックは分岐の経路を逐次化する．ワープは一方の経路を再収束点まで実行してから，もう一方を始める．もう一方の経路のスレッドが書く値を待ってスピンするスレッドは，自分の経路が先にスケジュールされるとハングする．書き手は，スピンしている経路が再収束点に達した後にしか実行されないが，その経路は決してそこに達しないからである．これが *SIMT デッドロック* である．たとえば，レーン 0 が `then` 経路を取り，メモリ上のフラグがセットされるまでループし，レーン 1 は `else` 経路を取ってそのフラグをセットするとする．スタックが `then` 経路を先に実行すると，レーン 0 は永遠にループし，`else` 経路は決してフェッチされない．1 つのワープの複数のレーンが取るスピンロックも同じように失敗する．A18 ch. 3 はこの問題を，スレッドごとにプログラムカウンタを持ち，スケジューラがもう一方の経路に切り替えられるようにするスタックレスの設計と合わせて論じている．本書の以降の部分では，本節のワープ

Volta 以降の NVIDIA GPU はスレッドごとにプログラムカウンタと呼び出しスタックを持ち（独立スレッドスケジューリング，第 5 章），正しさについて古典的なスタックにもはや依存しない（CPG §4.1 SIMT Architecture [5]）．

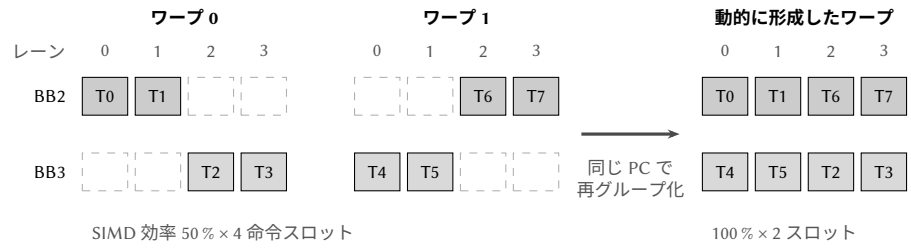


図 6.4: 動的ワープ形成．ワープ 0 と 1 が同じ分岐でダイバージェントになる．BB2 ではワープ 0 のアクティブなスレッドはレーン 0–1 に，ワープ 1 のものはレーン 2–3 にあるので，レーンを離れることなく 1 つの完全なワープを形成する．BB3 も同様にマージされる．

ごとのスタックのモデルを用いる．

6.1.5 ラージワープ

ワープサイズはマイクロアーキテクチャ上の選択であって，法則ではない．1 回のフェッチとデコードで 32，64，128 のレーンを駆動でき，非常に幅の広いワープは SM を，レーンあたりのフロントエンドのコストが小さい幅広いベクトル機械に変える．そのコストはダイバージェンスである．ワープが保持するスレッドが多いほど，そのうちの 1 つが他と一致しない可能性は高く，ダイバージェンスのたびに経路が逐次化され SIMT スタックが占有される．適切な幅は，アプリケーションの制御フローがどれだけ一様かによる．

例 6.7. 各スレッドが独立に確率 $p = 0.1$ で分岐を取るとする．W スレッドのワープが一様になるのは全スレッドが一致するときだけであり，その確率は $0.9^W + 0.1^W$ である．これは $W = 4$ で約 0.66，$W = 32$ で 0.034，$W = 64$ で 0.0012 となる．このモデルでは，32 スレッドのワープのほとんどすべての分岐がダイバージェントになる．

定義 6.8 (ラージワープ). **ラージワープ**とは，従来の 32 より多い（64，128 またはそれ以上の）スレッドを持つワープであり，その命令は全スレッドに対して 1 回だけフェッチ，デコードされるので，フロントエンドのコストがより多くのレーンで分担される．

したがって論文のラージワープは，ダイバージェンスに対抗する方法でもあり，次のアイデアにつながる．

6.1.6 動的ワープ形成

ダイバージェント分岐の後，各経路はレーンの一部だけを働かせて実行される．図 6.1 では BB2 と BB3 でレーンの半分が遊ぶ．働いているレーンの割合がこの損失の尺度である．

定義 6.9 (SIMD 効率). 実行の **SIMD 効率**とは，仕事をするレーンスロットの数を，発行されたレーンスロットの数（ワープサイズ × 発行されたワープ命令数）で割ったものである．ダイバージェンスのないプログラムの SIMD 効率は 100 % であり，例 6.4 のダイバージェントなワープでは $40/64 = 62.5\,\%$ である．

SM の多くのワープはしばしばプログラムの同じ地点にあり，同じダイバージェント分岐の後，異なるワープが相補的なマスクを持って同じ PC で待っている．それらのスレッドは結合できる．

定義 6.10 (動的ワープ形成). **動的ワープ形成**（DWF）と呼ばれる手法は，同じ PC を実行しようとしている異なるワープのスレッドを，実行時に，より満たされたアクティブマスクを持つ新しいワープにまとめ直す（図 6.4）．

制約はレジスタファイルである．第 4 章ではスレッド k のレジスタをバンク k に割り当て，レーン k が自分のバンクから読むようにした．別のレーンに移されたスレッドは，別のレーンに属するバンクからレジスタを読む必要があり，こ

注意 実際の分岐結果には相関がある（`threadIdx.x / 32` に対する条件は構成上ワープごとに一様である）ので，独立な結果は悲観的なモデルである．これは傾向を示すものであって，実際のダイバージェンス率ではない．

▷ large warp

注意 講義 vs 論文：本章は講義に従い，ラージワープを幅の広いワープとして扱う．Narasiman ら [29] はラージワープのアクティブなスレッドを毎サイクル SIMD 幅のサブワープに詰め直すので，ダイバージェンスによって遊ぶレーンが少なくなる．2 レベルのワープスケジューリング（第 7 章）と組み合わせて，従来の GPU コアに対して 19.1 % の向上を報告している．

▷ SIMD efficiency

PMPP ch. 10 [6] は同じ尺度をソフトウェアに適用する．並列リダクションを書き換えて，アクティブなスレッドを少数のワープに詰めたままにし，SIMD 効率を高く保つ．

Fung らは，現実的な DWF の実装で，推定 4.7 % の面積増加に対して平均 20.7 % の高速化を報告している [28]．

▷ dynamic warp formation (DWF)

のレイアウトが避けるように設計されたレジスタバンク競合を生む．そのため DWF は各スレッドをホームレーンに置いたまま，アクティブなレーンが重ならないワープだけをマージする（図 6.4）．どちらもレーン 0–1 を使う 2 つの半ワープはマージできない．

例 6.11. 例 6.4 のブロックサイズを持つ分岐を実行する 2 つのワープを考え，条件がデータに依存して，図 6.4 のようにワープ 0 はレーン 0–1 で，ワープ 1 はレーン 2–3 で BB2 を取るとする．DWF がなければ BB2 と BB3 は SIMD 効率 50 % で $2 \times (6 + 6) = 24$ 命令スロットかかる．DWF があれば，マージされたワープは 100 % で $6 + 6 = 12$ 命令スロットで済む．カーネル全体では $2 \times 16 = 32$ から $2 \cdot 2 + 12 + 2 \cdot 2 = 20$ ワープ命令に減り，SIMD 効率は 62.5 % から $80/80 = 100$ % に上がる．

ヒント DWF が同じ PC の 2 つのワープをマージできるのは，それらのマスクのビットごとの AND が 0 のときだけである．新しいマスクはそれらの OR である．

> **要点**
> ダイバージェント分岐は，ワープに両方の経路を部分的なマスクで実行させる．if 変換はそれを常に両方の経路をフェッチすることと引き換えにし，SIMT スタックは経路を順に実行して直接後支配節点で再収束させる．幅の広いワープはフロントエンドを分担するがダイバージェンスが増え（Narasiman らのラージワープのマイクロアーキテクチャはアクティブなスレッドをサブワープに詰め直す），動的ワープ形成は各スレッドをホームレーンに置いたまま遊んでいるレーンを取り戻す．

6.2 レジスタの最適化

SM のレジスタファイルは常駐するすべてのスレッドに専用のレジスタの組を保持する（第 4 章）ので，その大きさはスレッド数とスレッドあたりのレジスタ数の積であり，世代を追って SM あたり 64 KiB，128 KiB，256 KiB となってきた．大きな SRAM は遅く電力を消費するが，GPU は遅いものを受け入れられない．レジスタファイルはレーンが消費する速さでオペランドを供給しなければならず，その読み出し帯域は実行帯域に等しくなければならない．CPU の設計から 2 つの対策が得られる．よくあるアクセスを速くするための階層と，必要な記憶容量を減らすための共有である．GPU プログラムがレジスタをどう使うかについての 4 つの観察が，それらがどこに当てはまるかを示す．

出典 M6，レジスタの最適化；K12 §1.4.1 GPU Pipeline；[30, 31, 33]．

SM あたり 256 KiB（V100，A100，H100）．GPU あたりでは A100 で 27 648 KiB，H100 SXM5 で 33 792 KiB（暫定値）[2, Table 3]．

6.2.1 4 つの観察

すべてのスレッドがアクティブなわけではない． 同じカーネルのスレッドは同じレジスタ名を使うが，同時に終わるわけではない．実行は非同期であり，終了したスレッドも，そのワープまたはブロックが解放されるまでレジスタを占有し続ける．ダイバージェント分岐では，2 つの経路が異なるレジスタを使うこともある．

すべてのレジスタが生存しているわけではない． `add r1, r2, #1` の後に `mov r2, #10` が続くとき，`r2` の値は add の後二度と読まれない．`mov` が上書きするまで，それは非生存である．

生産者と消費者は近い． `mov r2, #10` の後に `add r3, r2, #20` が続くとき，値はすぐ次の命令によって消費され，スケジューラは 2 つを近くに保てる．そのような値は大きなレジスタファイルに届く必要すらないかもしれない．

ほとんどの値は一度しか読まれない． `mov r2, #10` の後に `add r2, r2, #20` が続くとき，`r2` の最初の値は 1 つの命令に読まれた後に上書きされる．多くの値がこのパターンに従い，それらを主レジスタファイルに格納することには疑問がある．

注意 講義 vs 論文：講義はレジスタの値のほぼ 70 % が一度しか読まれないとする．Gebhart らは自分たちのワークロード全体で “up to 70 %” と測定しており，HPC のコードはより多く再利用する（値の 40 % が 2 回以上読まれる）．また，一度だけかつ生成から 3 命令以内に読まれるのは “up to 40 %” である [30]．

定義 6.12 (生存レジスタと非生存レジスタ)**.** プログラムのある地点でレジスタが**生存レジスタ**であるとは，それが保持する値が上書きされる前に後で読まれることであり，そうでなければ**非生存レジスタ**である．値の**値の生存期間**とは，それを生成する命令から最後に読まれるまでの区間である．

▷ live register
▷ dead register
▷ value lifetime

2 番目の観察は CPU より GPU で重い（図 6.5）．CPU では add と mov は連続するサイクルで実行され，`r2` が非生存なのは 1 サイクル以下である．GPU ではス

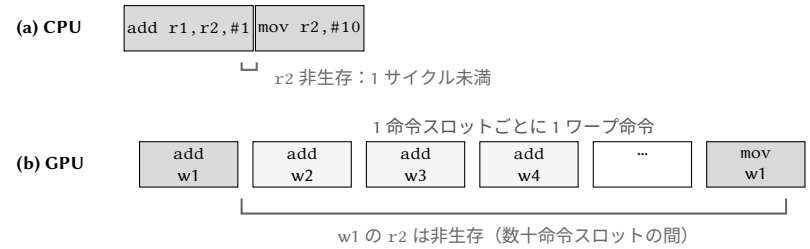


図 6.5: add r1,r2,#1 の後，レジスタ r2 は mov r2,#10 まで非生存である．(a) CPU では 2 つの命令は隣接する．(b) GPU では間の命令スロットは他のワープのものなので，ワープ w1 の r2 の非生存区間は数十命令にわたる．

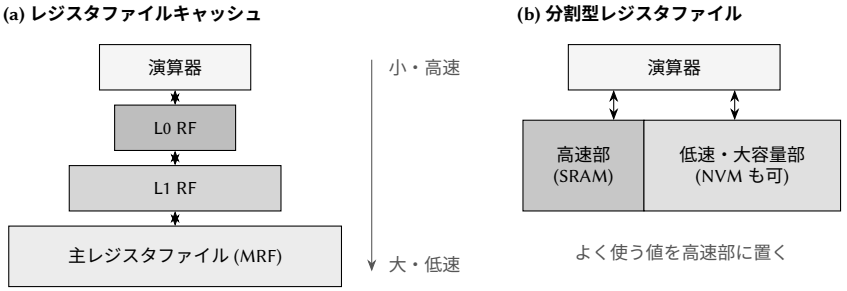


図 6.6: 2 つの階層型レジスタファイル．(a) レジスタファイルキャッシュ：演算器と主レジスタファイルの間にある小さな L0 と L1 のレジスタファイル．(b) 分割型レジスタファイル：高速部と，不揮発性メモリを使いうる低速で大きい部分．

ケジューラがワープをインタリーブするので，同じワープの次の命令は数十ワープ命令後に発行されることがある．その間ずっと，r2 の非生存の値がワープの全スレッドのレジスタを 1 つずつ占有する．

このインタリーブを決めるワープスケジューラは第 7 章で論じる．貪欲なスケジューラは間隔を縮め，ラウンドロビンのスケジューラは間隔を準備のできたワープの数まで引き延ばす．

例 6.13. SM が 24 ワープをラウンドロビンでインタリーブし，1 巡ごとに各ワープ 1 命令を発行するので，1 つのワープの連続する命令は 24 サイクル離れる．命令 k が生成し，命令 $k+1$ が最後に読み，命令 $k+5$ が上書きする値は，24 サイクル生存し $4 \times 24 = 96$ サイクル非生存である．そのレジスタが有用なデータを保持するのは時間の $24/120 = 20\,\%$ である．1 サイクルに 1 命令を発行する CPU では，同じ値は 1 サイクル生存し 4 サイクル非生存であり，同じく 20 % であるが，非生存レジスタのコストは小さなファイルの 1 エントリである．GPU では，数百キビバイトのファイルでスレッドごとに 1 エントリを 96 サイクル占める．

6.2.2 階層型レジスタファイル

3 番目と 4 番目の観察は階層を示唆する．生成後すぐに，しかも一度だけ読まれる値は，演算器の近くにある小さな構造から供給できる．

ヒント 生産者の直後に発行される命令によって一度だけ読まれる値は，理想的なキャッシュ候補である．それは小さなファイルで読まれて死に，主レジスタファイルへの書き込みを一度も生じない．

定義 6.14 (階層型レジスタファイル). **階層型レジスタファイル** は，より小さく速いレジスタ記憶を**主レジスタファイル**（MRF）の前に置く．図 6.6 の 2 つの形態は，最近生成された値を保持して MRF への読み書きをふるい分ける 1 段または 2 段の小さな階層（L0 と L1 RF）である**レジスタファイルキャッシュ**（RFC）と，高速部と低速部を持ち，低速で大きい部分をより高密度の技術で作れる**分割型**レジスタファイルである．

▷ hierarchical register file
▷ main register file (MRF)
▷ register file cache (RFC)

RFC があれば，ほとんどの読み出しは小さなファイルでヒットし，追い出される前に死ぬ値は MRF に書き込む必要がない．そのため MRF を小さくするか，同じ大きさのままアクセス頻度を下げることができる．小さな SRAM は大きなものよりアクセスあたりのコストが小さいので，これはエネルギーを節約する．GPU ではこれが CPU より容易である．K12 §1.4.1 が述べるように GPU は精密例外をサポートしないので，“register files do not have to maintain architectural states” [3]，値はキャッシュの中だけに存在してよい．

Gebhart ら：スレッドあたり 6 エントリの RFC で MRF の読み出しは 50 %，書き込みは 59 % 減る．スレッドの 4 分の 1 をアクティブに保つ 2 レベルスケジューラと組み合わせると，レジスタファイルのエネルギーは 36 % 下がる [30]．

分割型ファイルの低速部の候補は**不揮発性メモリ**（NVM），すなわち電力なしで内容を保持するメモリである．NVM で作った低速で大きい部分はエネルギー

▷ non-volatile memory (NVM)

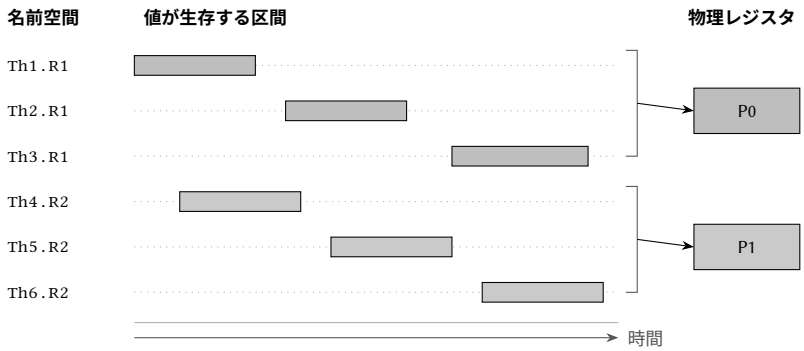


図 6.7: レジスタファイル仮想化．スレッド 1–3 の R1 の値は互いに重ならない区間で生存し，物理レジスタ P0 を共有する．スレッド 4–6 の R2 の値は P1 を共有する．6 つの名前に 2 つの物理レジスタで足りる．

の節約を狙ったものであり，頻繁に使われる値は高速部にとどまる．

キャッシュの判断は，値の生存期間を知っているコンパイラに移すこともできる．Gebhart，Keckler と Dally [31] はハードウェアの RFC を，コンパイラが内容を割り当てるスレッドごとの小さなファイルである**オペランドレジスタファイル**（ORF）と，最新の結果を即座の再利用のために保持するスレッドあたり 1 エントリの**ラストリザルトファイル**（LRF）で置き換える．コンパイラがすべての値を明示的に配置するので，ORF にはタグもハードウェアによる置換も要らない．これはスクラッチパッドとキャッシュを分けるのと同じ性質である（6.3 節）．

レジスタファイルの NVM は研究の方向であって，出荷された設計ではない．現在の GPU はレジスタファイル全体を SRAM で作っている．

▷ operand register file (ORF)

▷ last result file (LRF)

コンパイラ管理の階層はレジスタファイルのエネルギーを最大 54 % 削減すると報告されている．ハードウェアのみのキャッシュでは 34 % である．彼らのベースラインでは，レジスタファイルはプロセッサの動的エネルギーの 15–20 % を消費する [31]．

6.2.3 レジスタファイル仮想化

最初の 2 つの観察は共有を示唆する．どの瞬間にも多くのレジスタが非生存であるか，終了したスレッドに属しているなら，名前の数より少ない物理レジスタで足りる [33]．

定義 6.15 (アーキテクチャレジスタと物理レジスタ). **アーキテクチャレジスタ** とは，プログラムから見たレジスタ名（スレッド 3 の R1）であり，**物理レジスタ** とは，レジスタファイルの記憶エントリである．実行時のアーキテクチャレジスタから物理レジスタへの対応付けが**レジスタリネーミング** である．アウトオブオーダの CPU は，偽の依存を取り除くために，少数のアーキテクチャレジスタをより多くの物理レジスタにリネームする．

▷ architectural register

▷ physical register

▷ register renaming

定義 6.16 (レジスタファイル仮想化). **レジスタファイル仮想化** と呼ばれる手法は，全スレッドのアーキテクチャレジスタの名前空間を保ったまま，それをより小さな物理レジスタファイルに対応付ける．物理レジスタは値が生成されるときに割り当てられ，値が死ぬときに解放されるので，値が異なる時刻に生存する名前どうしが記憶を共有する（図 6.7）．プログラムの意味は変わらず，ハードウェアだけが小さくなる．

▷ register file virtualization

注意 方向は CPU のリネーミングと逆である．CPU はアーキテクチャレジスタより多くの物理レジスタを持つが，GPU のレジスタファイル仮想化では少ない．仮想化されたファイルは資源の共有であり，維持すべきなのは名前空間だけである．

対応付けにはオペランドごとに表の参照がかかり，設計は値が死んだちょうどそのときにレジスタを解放しなければならない．これにはコンパイラからの生存期間の情報（各値の最後の読み出し）が必要である．また物理レジスタが尽きたときの進行も保証しなければならない．すべてが使われているときに新しいレジスタを必要とするワープは，他のワープが解放するのを待たねばならず，すべてのワープが待てばどれも進まない．

例 6.17. あるカーネルがスレッドあたり 32 レジスタを使い，SM が 1024 スレッドを常駐させるとすると，名前空間は $32 \times 1024 = 32\,768$ レジスタ，1 つ 4 B として 128 KiB を持つ．プロファイリングの結果，非生存レジスタのためと終了したスレッドのために，どの瞬間にも生存する値を持つのは多くとも 60 % だと分かったとする．すると $0.6 \times 32\,768 \approx 19\,661$ 個の物理レジスタ，約 77 KiB の仮想化されたファイルですべての生存する値を保持でき，残りの 51 KiB は取り除くか，常駐スレッド数を増やすために残しておける．

計算のために選んだ数値であり，測定された生存割合を意味するものではない．

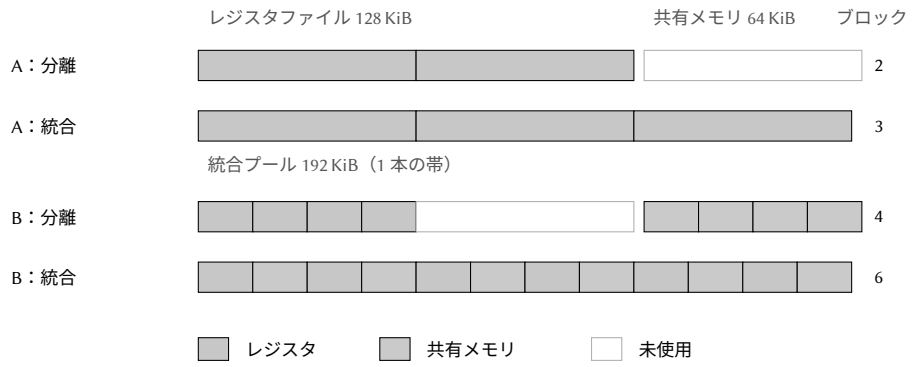


図 6.8: ブロックあたり 256 スレッドの 2 つのカーネルにおける分離型と統合型の記憶．カーネル A はレジスタファイルを埋め，共有メモリを遊ばせる．カーネル B は共有メモリを埋め，レジスタファイルの半分を遊ばせる．1 つの 192 KiB のプールなら A は 3 ブロック，B は 6 ブロックを受け入れる．

要点
GPU のレジスタファイルは，常駐するすべてのスレッドのレジスタを保持するので大きく，すべてのレーンにオペランドを供給するので速くなければならない．ほとんどの値は長い区間にわたって非生存であるか，生成後すぐに一度だけ読まれる．これが，レイテンシとエネルギーのための階層（レジスタファイルキャッシュ，ORF/LRF，高速部と低速部への分割）と，容量のための仮想化（名前より少ない物理レジスタ）を正当化する．

6.3 レジスタファイルと共有メモリの統合

第 3 章では，占有率はカーネルが最初に使い果たす資源，すなわちスレッドスロット，レジスタ，共有メモリのいずれかによって制限されることを示した．カーネルによって後の 2 つの使い方は大きく異なるので，それらの間の固定された分割は記憶を無駄にする．本節では 3 種類のオンチップ記憶を比較し，統合された設計を紹介する．

出典 M6，レジスタファイルと共有メモリの統合；[32]；A18 ch. 4；CPG Compute Capability 7.x–9.0，Shared Memory．

6.3.1 占有率再訪

128 KiB のレジスタファイル（32 768 レジスタ）と 64 KiB の共有メモリを持つ SM と，ブロックあたり 256 スレッドで起動される 2 つのカーネルを考える（図 6.8）．

- カーネル A はスレッドあたり 64 レジスタを使い，共有メモリは使わない．ブロックあたりのレジスタは $64 \times 256 \times 4\,\mathrm{B} = 64\,\mathrm{KiB}$ なので，2 ブロックでレジスタファイルが埋まり，共有メモリ全体が遊ぶ．
- カーネル B はスレッドあたり 16 レジスタと，ブロックあたり 16 KiB の共有メモリを使う．ブロックあたりのレジスタは 16 KiB である．共有メモリは 4 ブロックを受け入れ，それらはレジスタファイルの半分しか使わない．

もし 192 KiB が 1 つのプールであり，各ブロックがそこからレジスタと共有メモリを取るなら，カーネル A は $192/64 = 3$ ブロック，カーネル B は $192/32 = 6$ ブロックが収まる．どちらも常駐スレッドが 50 % 増える（表 6.2）．統合設計では，占有率はレジスタと共有メモリによって合わせて制限される．

構成可能な分割は統合設計より前からある．Fermi の SM（2009 年）は 64 KiB を 48 KiB の共有メモリと 16 KiB の L1，またはその逆に分割する [34]．

常駐スレッド数の上限（Compute Capability 7.0，8.0，9.0 では SM あたり 2048，CPG Table 21 [5]）は依然として適用される．カーネル B の 6 ブロックは 1536 スレッドである．

表 6.2: 図 6.8 のカーネルの SM あたり常駐ブロック数（スレッド数）．

カーネル	分離	統合
A	2 (512)	3 (768)
B	4 (1024)	6 (1536)

6.3.2 キャッシュ，スクラッチパッド，レジスタファイル

レジスタと共有メモリが記憶を共有できるかどうかは，それぞれがどうアドレス指定されるかによる．キャッシュとスクラッチパッドは，何を保持するかを誰が決めるかで異なる．

定義 6.18 (ハードウェア管理キャッシュとソフトウェア管理キャッシュ). **ハードウェア管理キャッシュ** は，どのメモリブロックを保持するかを自ら決める．したがって各エントリは**タグ**，すなわち保持するブロックのアドレスの上位ビット

▷ hardware-managed cache
▷ tag

表 6.3: 3 種類のオンチップ記憶．

	ハードウェアキャッシュ（L1）	スクラッチパッド（共有メモリ）	レジスタファイル
内容を決めるもの	ハードウェア	プログラム	コンパイラ
タグ	あり	なし	なし
位置の特定	アドレスとタグ	位置	位置（レジスタ名）
命令あたりのアクセス	1 アドレス	1 アドレス	2–3 読み出し，1 書き込み
パターンが分かる時点	実行時	実行時	コンパイル時

を持ち，要求されたアドレスがあるかどうかを知るためにアクセスのたびに比較される．満杯のキャッシュは**置換ポリシ**が選んだブロックを追い出す．たとえば **LRU** は，最後のアクセスが最も過去にあるブロックを追い出す．**ソフトウェア管理キャッシュ**（第 3 章のスクラッチパッドメモリ，CUDA の共有メモリ）はプログラムが置いたものを保持する．プログラムはすべてのデータがどこにあるかを知っているので，タグもミスも置換もない．

▷ replacement policy
▷ LRU (least recently used)
▷ software-managed cache

レジスタファイルはスクラッチパッドに似ている．どちらもデータだけを格納し，タグを持たず，メモリアドレスではなく位置によってエントリを選ぶ．両者は 2 つの点で異なる（表 6.3）．算術命令は 2 つのソースレジスタ（融合積和演算 $d = a \cdot b + c$ では 3 つ）を読み，1 つに書き込むので，1 回のアクセスで 1 つのアドレスを処理するメモリよりも多くのポートが必要である．また，命令が使うレジスタは命令の中で名指しされるので，そのアクセスパターンは実行前に静的に分かる．一方スクラッチパッドのアドレスは実行時に計算される．この静的な知識こそが，帯域が高いにもかかわらず，第 4 章のバンク配置と上述のコンパイラ管理の ORF を可能にする．

NVIDIA GPU はグローバルメモリを，32 B のセクタ 4 つから成る 128 B のラインでキャッシュし（第 4 章），ワープのアクセスは 32 B のトランザクションで処理される（BPG §10.2.1 Coalesced Access to Global Memory [8]）．

共有メモリのレイテンシは，キャッシュされないグローバルメモリのおよそ 100 分の 1 である [18]．

6.3.3 統合された構造

3 つの構造はすべて同じ SM 上の SRAM アレイなので，1 つのアレイでそれらすべてを受け持ち，カーネルごとに分割できる．

定義 6.19 (統合ローカルメモリ). **統合ローカルメモリ** と呼ばれる設計は，SM ごとの単一のオンチップ SRAM プールであり，その容量はカーネルごとにレジスタファイル，共有メモリ，そして完全な提案では **L1 データキャッシュ**，すなわちグローバルメモリとローカルメモリへのアクセスのためのハードウェア管理の 1 次キャッシュに分けられる．

Gebhart らは，“reduce accesses to the main register file by 60 %” という以前のレジスタ階層と 2 レベルスケジューラの上に構築しており，そのため専用のレジスタファイルよりバイトあたりのポートが少ないプールでもオペランドを供給できる [32]．

▷ unified local memory
▷ L1 data cache

統合プールは各利用者の最も厳しい要求を満たさなければならない．すなわちレジスタファイルの帯域，したがって十分な読み出しポートと書き込みポート，そしてどちらの構造単独よりも多くのバンクである．レジスタアクセス（全レーンで同じレジスタ）と共有メモリアクセス（計算されたアドレスによる）はパターンが異なり，どちらも空いたバンクを見つけなければならないからである．その代わりに，各カーネルは必要な分割を得て，占有率の制限が緩和される．

出荷されている GPU が 1 つのアレイを共有させているのは L1 データキャッシュと共有メモリの間だけであり（Fermi がそうであり，Volta 以降の GPU が再びそうしている），レジスタファイルは別のままである．

注意 講義 vs 論文：本章は講義に従い，レジスタファイルと共有メモリを統合する．Gebhart らは *3* つの構造，“the register file, scratchpad (shared memory in NVIDIA terminology), and cache” を統合し，“a performance benefit as high as 71 % along with an energy reduction up to 33 %” を報告している [32]．

定義 6.20 (共有メモリのカーブアウト). **共有メモリのカーブアウト** とは，SM の L1 データキャッシュと共有メモリを合わせたアレイのうち，共有メモリとして構成される部分である．残りは L1 キャッシュとして働く．CUDA ランタイムはそれを，カーネルの共有メモリの使用量と任意の優先指定に基づいて，固定されたサイズの集合からカーネルごとに選ぶ．

▷ shared memory carve-out

優先指定は 1 回のランタイム呼び出しでカーネルごとに設定する．

ヒント ブロックあたり 48 KiB を超える動的共有メモリを必要とするカーネルは，`cudaFuncAttributeMaxDynamicSharedMemorySize` で明示的に有効にしなければならない（CPG，Compute Capability 7.x，Shared Memory [5]）．

```
// カーネル k に共有メモリの最大容量の約半分を
// 希望する（ヒントであり，ランタイムは対応する
// カーブアウトに切り上げる）．
cudaFuncSetAttribute(k,
    cudaFuncAttributePreferredSharedMemoryCarveout, 50);
```

SM あたりの L1 と共有メモリの合計：V100 で 128 KiB，A100 で 192 KiB，H100 で 256 KiB．カーブアウト：V100 は 0，8，16，32，64，96 KiB，A100 は最大 164 KiB，H100 は 0，8，16，32，64，100，132，164，196，228 KiB（CPG，Compute Capability 7.x，8.x，9.0 [5]）．

こうして共有メモリをほとんど使わないカーネルは大きな L1 を得，データをタイル化するカーネルはアレイの大部分をスクラッチパッドとして得る．これは図 6.8 の柔軟性を 2 つの構造について実現したものである．

要点 — 章のまとめ
ダイバージェント分岐では，ワープは経路を部分的なマスクで順に実行し，直接後支配節点で再収束する．if 変換は分岐を，両方の経路をフェッチするプレディケート付きのコードに変える．SIMT スタックは，入れ子のものも含めダイバージェント分岐で（代替 PC，マスク，再収束 PC）のエントリをプッシュし，再収束 PC でポップする．幅の広いワープはフェッチを節約するがダイバージェンスが増える．動的ワープ形成は 1 つの PC にある異なるワープのスレッドを，それぞれホームレーンに置いたままマージする．非生存の値，短命の値，一度だけ読まれる値が，レジスタファイルキャッシュ，ORF/LRF の階層，高速部と低速部への分割を正当化する．仮想化は名前をより少ない物理レジスタに対応付ける．統合ローカルメモリは 1 つのプールをカーネルごとにレジスタ，共有メモリ，L1 に分割する．現在の GPU は L1 と共有メモリについて，カーブアウトによってそれを行っている．

第 7 章

GPU アーキテクチャの最適化 II

講義動画	M7，レッスン 1–3（GPU Virtual Memory；GPU Virtual Memory - UVA；GPU Warp Scheduling）
資料	Power ら [35]（GPU MMU）；Ausavarungnirun ら [36]（複数のページサイズ）；Li ら [37]（マルチ GPU の TLB）；Ganguly ら [38]（ユニファイドメモリにおけるプリフェッチと追い出し）；Kim ら [39]（フォールトのバッチ処理）；Rogers ら [40]（キャッシュ考慮型ウェーブフロントスケジューリング）；Narasiman ら [29] および Gebhart ら [30]（2 レベルスケジューリング）；Jog ら [41]（プリフェッチを考慮したスケジューリング），[42]（OWL）
教科書	A18 [4] ch. 3，ch. 4；K12 [3] §1.4.1 GPU Pipeline；CPG [5] §4.2 Hardware Multithreading，§19 Unified Memory Programming；BPG [8] §10.1.4 Unified Virtual Addressing
キーワード	仮想メモリ，アドレス変換，TLB，多段ページテーブル，ページテーブルウォーカ，ラージページ，セグメントマッピング，コピー後実行，共有仮想メモリ，オンデマンドページング，ファーフォールト，ツリーベース近傍プリフェッチ，オーバーサブスクリプション，ワープスケジューラ，ラウンドロビン，GTO，2 レベルスケジューリング，ワーキングセット，キャッシュ考慮型ウェーブフロントスケジューリング

第 6 章では SM の内部を変えた．ワープがダイバージェント分岐をどう乗り越えるか，そしてそのレジスタをどう格納するかである．本章では SM の仕組みのさらに 2 つの部分を扱う．最初の 2 つの節はメモリアドレスに関するものである．CPU とデータ構造を共有する GPU には仮想メモリが必要であり，ワープ全体のアドレスを一度に変換することは，CPU コアでは決して起こらない形で変換ハードウェアに負荷をかける．CPU と GPU が 1 つのアドレス空間を共有すると，ページの割り当てと移動も，カーネルの実行中に PCIe を越えて，要求に応じて行わなければならない．第 3 の節は第 4 章のパイプラインのスケジューラ段に戻り，複数のワープが準備完了のときにどのワープを発行すべきかを問う．この選択は，メモリレイテンシがどれだけうまく隠蔽されるか，そして各ワープが L1 キャッシュをどれだけ保持できるかを決める．第 6 章と同様に，本章は研究論文を軸に構成されており，各節はその拠り所となる論文を挙げる．

7.1 GPU の仮想メモリ

出典 M7，GPU 仮想メモリ；A18 ch. 4；[35]；CPG §19.2.2.1 Memory Paging and Page Sizes．

仮想メモリには 2 つの側面がある．プログラムが発行するすべてのアドレスの変換と，その背後にある物理ページの割り当てである．CPU ではオペレーティングシステムがページを割り当てる．GPU では，割り当てはもともと CPU が GPU ドライバとともに，`cudaMalloc`，`cudaMallocManaged`，`cudaMemcpy` を通じて行っていた．これには 7.2 節で立ち返る．本節ではアドレス変換を復習し，スレッドではなくワープがアドレスを発行するときに何が異なるかを示し，変換を安価にする 3 つの方法を紹介する．

7.1.1 アドレス変換

定義 7.1 (仮想メモリ). **仮想メモリ**のもとでは，プログラムは私的なアドレス空間の中で**仮想アドレス**を発行し，ハードウェアがアクセスのたびにそれをメモリ上の**物理アドレス**に対応づける．この対応づけを**アドレス変換**という．対応は**ページ**の粒度で保持される．ページとはアドレス空間の固定サイズで整列したブロックであり，典型的には 4 KiB または 2 MiB である．

▷ virtual memory
▷ virtual address
▷ physical address
▷ address translation
▷ page

仮想アドレスは 2 つのフィールドに分かれる（図 7.1）．上位ビットが**仮想ページ番号**（VPN），下位ビットがページ内の位置を表す**ページオフセット**である．変換は VPN を**物理フレーム番号**（PFN）で置き換え，オフセットは変えない．PFN

▷ virtual page number (VPN)
▷ page offset
▷ physical frame number (PFN)

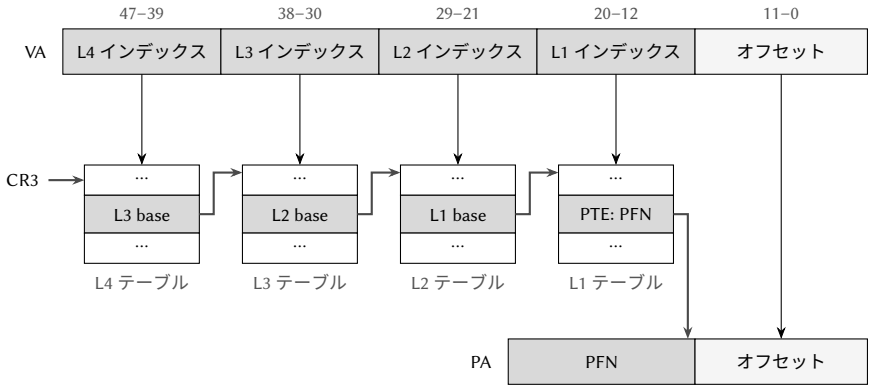


図 7.2: x86-64 のレイアウトによる 4 段のページテーブルウォーク．各インデックスは次のテーブルのベースを保持するエントリを選ぶ．第 1 段のエントリが PTE であり，その PFN をオフセットと結合したものが物理アドレスである．

は物理ページ番号（PPN）とも呼ばれる．2^k バイトのページは k ビットのオフセットを持つ．

定義 7.2 (ページテーブル，TLB，ページフォールト). **ページテーブル**は，対応を保持するメモリ上のデータ構造である．各**ページテーブルエントリ**（PTE）は 1 つの VPN を PFN に対応づけ，有効ビットや権限ビットなどのメタデータを持つ．**TLB**（translation lookaside buffer）は最近使われた変換の小さなキャッシュである．**TLB ミス**は TLB が変換を保持していないときに起こり，ページテーブルを読まなければならない．**ページフォールト**は，物理ページがまだ割り当てられていないためにページテーブルが有効な対応を保持していないときに起こり，オペレーティングシステムがページを割り当てなければならない．

TLB が存在するのは PTE そのものがメモリ上にあるからである．TLB がなければ，すべてのロードはまず変換を見つけるために 1 回以上のメモリアクセスを必要とする．

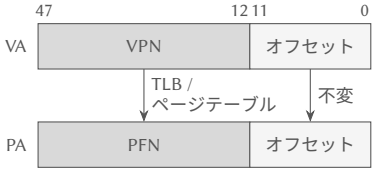


図 7.1: 変換はページオフセットを保ち，VPN を PFN で置き換える（4 KiB ページと 48 ビットの仮想アドレスの場合のビット位置）．

▷ page table
▷ page table entry (PTE)
▷ TLB (translation lookaside buffer)
▷ TLB miss
▷ page fault

注意 TLB ミスとページフォールトは別の事象である．TLB ミスはページテーブルを読むハードウェア（または短いハンドラ）が解決する．ページフォールトにはオペレーティングシステムが必要であり，GPU ではホストが必要である（7.2 節）．

▷ multi-level page table

7.1.2 多段ページテーブル

VPN で索引づける単一のテーブルは，v ビットの VPN に対して，プログラムがそのページを使うかどうかにかかわらず 2^v 個のエントリを必要とする．

定義 7.3 (多段ページテーブルとページテーブルウォーク). **多段ページテーブル**は VPN を段ごとに 1 つずつ，複数のインデックスに分割する．最上段のインデックスは最上段のテーブルのエントリを選び，そのエントリは次の段のテーブルのベースアドレスを保持する．そのテーブルのエントリは次のインデックスで選ばれ，以下同様に続き，最下段のエントリが PTE である．この連鎖をたどることが**ページテーブルウォーク** である．これはハードウェアの状態機械である**ページテーブルウォーカ**（PTW）か，ソフトウェアのミスハンドラが行う．使われるどのアドレスからも到達しないテーブルは割り当てられない．

▷ page table walk
▷ page table walker (PTW)

図 7.2 は x86-64，ARM，GPU が用いる 4 段の形を示す．ウォークはルートレジスタから始まり，第 4 段のインデックスを使って第 3 段のテーブルのベースを読み，以下同様に第 1 段のエントリ，すなわち PTE まで下る．PFN とオフセットを合わせたものが物理アドレスである．節約した空間の代価は時間である．TLB でミスした 1 回の変換には，少なくとも 4 回の依存したメモリアクセスがかかる．

4 段の x86-64：48 ビットの仮想アドレス，9 ビットのインデックス 4 つと 12 ビットのオフセット．各テーブルは 8 B のエントリ 512 個で，ちょうど 4 KiB ページ 1 枚である．ルートレジスタは CR3 である．

例 7.4. x86-64 のレイアウト，すなわち 36 ビットの VPN と 8 B のエントリを考える．

- *1 段*：$2^{36} \approx 6.9\times10^{10}$ 個のエントリ，プロセスあたり 512 GiB のページテーブルであり，そのほぼすべてが一度も使われないページのためのものである．
- *4 段，最良の場合*：すべてのページが 1 つの第 1 段テーブルの下にあるプログラムは各段に 1 つのテーブルを必要とし，$2^9 + 2^9 + 2^9 + 2^9 = 2048$ 個のエントリが 16 KiB に収まり，最大 512 ページ（2 MiB）を対応づける．
- *疎なプログラム*：アドレス空間内で遠く離れた 2 つの配列は第 4 段テーブ

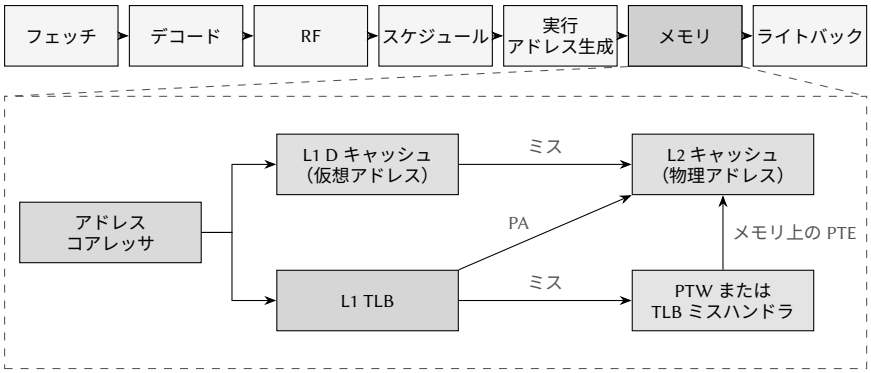


図 7.3: アドレス変換を含む GPU パイプラインのメモリ段．コアレッサがワープの要求を併合する．TLB が変換する間，L1 データキャッシュは仮想アドレスでアクセスされる．TLB ミスは PTW またはミスハンドラへ送られる．L2 キャッシュは物理アドレスでアクセスされる．

ルの下に 2 本の連鎖を必要とし，$1+2\cdot3=7$ 個のテーブル，それでも 28 KiB にすぎない．

数千のエントリが数百億のエントリに取って代わる．

注意 単一段のテーブルの大きさは $2^{\text{VPN bits}}$ であって $2^{\text{address bits}}$ ではない．オフセットのビットはテーブルの索引に使われない．

7.1.3 GPU におけるアドレス変換

初期の GPU には仮想メモリがないか，固定の 1 対 1 の対応しかなかった．GPU がプログラマブルになり，CPU とデータを共有し始めると，仮想メモリが必要になった．最も目に見える例が第 5 章の統合仮想アドレッシングである．変換は CPU と同じように働くが，1 つの違いがある．それは**アドレスコアレッシング**の後に行われることである．

Power らは，400 レーンを持つ AMD A10 APU は “can require up to 400 unique translations in a single cycle” であると指摘している [35]．

第 4 章のパイプラインでは，メモリ段は今や図 7.3 のステップを含む．コアレッサがワープのスレッドの要求を併合し（第 4 章），その結果の各要求は TLB で引かれる変換を必要とする．ミスは PTW か TLB ミスハンドラが解決する．この構成では，L1 データキャッシュは仮想アドレスで，すべての SM が共有する L2 キャッシュは物理アドレスでアクセスされる．

両方の配置が存在する．Power らの概念実証の MMU はコアレッサの後，L1 の前で変換するので，その L1 は物理アドレスでアクセスされる [35]．

ワープ命令がいくつの変換を必要とするかはコアレッシングが決める．同じ命令の要求を含む 2 つのページには，要求どうしがどれだけ近くても 2 回の TLB 参照が必要であり，1 つのページ内の 2 つの要求には 1 回で足りる．

例 7.5. 8 スレッドのワープ，64 B のメモリ要求，256 B のおもちゃのページサイズを考える．VPN はアドレスを 8 ビット右シフトしたものである．R1 を介したロードは連続する語 64, 68, ..., 92 を読む．1 つの要求，1 つのページ，1 回の TLB 参照である．R2 を介したロードは 160 B のストライドで読む．

256 B のおもちゃのページは数を小さく保つためのものである．4 KiB ページでは，4 KiB の行を持つ行優先行列の列のように，キロバイト単位のストライドで同じ効果が現れる．

スレッド	0	1	2	3	4	5	6	7
アドレス	0	160	320	480	640	800	960	1120
16 進	000	0A0	140	1E0	280	320	3C0	460
VPN	0	0	1	1	2	3	3	4

どの 2 つのアドレスも 1 つの 64 B 要求に収まらないので，このロードは 8 つの要求を発行する．それらは 5 つの異なるページに触れるので，同じページへの要求を併合した後でも，1 つの命令に 5 回の TLB 参照が必要である．256 B 以上のストライドでは，すべてのスレッドがそれぞれ自分のページに触れる．

最悪の場合，コアレスされないロードはワープのスレッド数と同じだけの変換を必要とする．したがってコアレスされないアクセスは 2 度コストを払う．1 度はメモリ帯域幅で（第 4 章），もう 1 度は変換の帯域幅である．

ヒント ワープ命令あたりの TLB 参照の回数は，そのアドレスが触れる異なるページの数であり，最大でワープのスレッド数である．コアレスされたアクセスは 1 回か 2 回で済む．

7.1.4 変換のスケーラビリティ

GPU のアドレス変換の課題はスケーラビリティである．1 つのワープ命令が一度に多くの TLB ミスを生むことがあり，SM あたり数十のワープと数十の SM が

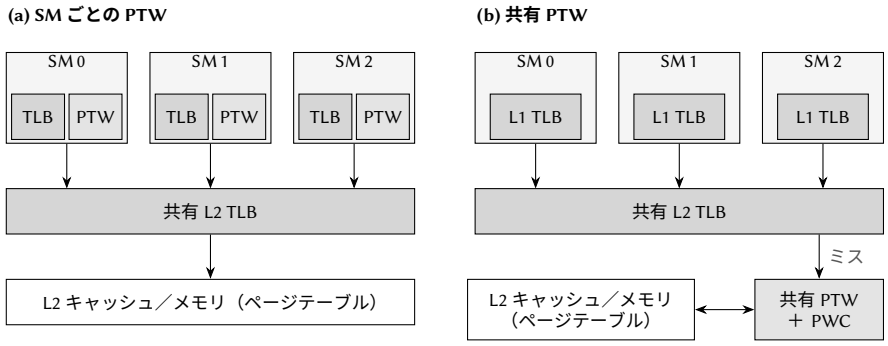


図 7.4: (a) 各 SM の専用 PTW と共有 L2 TLB．(b) L1 TLB だけを持つ SM と，共有 L2 TLB の隣にあるページウォークキャッシュ（PWC）付きの 1 つの共有高帯域幅 PTW．Power らの設計は L2 TLB のない (b) である．

それらを同時に発行する．したがってページテーブルウォーカは高いウォークの帯域幅を維持しなければならない．図 7.4 は 2 つの構成を示す．SM ごとの専用 PTW では，各 SM が自分のミスをウォークするが，すべての SM がほとんどの時間遊んでいるウォーカの代価を払う．共有 PTW では，SM は L1 TLB だけを持ち，そのミスを共有 L2 TLB の隣にある 1 つの高帯域幅ウォーカに送る．

Power，Hill と Wood は共有の構成を既知の CPU の部品から組み立てる [35]．コアレッサの後に置いた計算ユニットごとの TLB，多数の未完了のウォークの状態を一組のバッファに保持する高度にマルチスレッド化された PTW，そして**ページウォークキャッシュ** である．ページウォークキャッシュは MMU 内の小さなキャッシュで，ページテーブルの上位（非リーフ）段のエントリを保持し，ほとんどのウォークが 4 回未満のメモリアクセスで済むようにする．この設計には L2 TLB がない．CU ごとの TLB が共有ウォーカとそのページウォークキャッシュに直接つながる．ページウォークキャッシュの代わりに共有 L2 TLB を置くと平均で 2× 以上遅くなり，その横に追加しても性能は約 0.1 % しか変わらなかった [35, §6]．

Rodinia ベンチマークとデータベースのソートで測定した値：コアレッシングとスクラッチパッドにより変換の頻度は 6.8× 減り，TLB ミスは平均 60 個の同時ミスのバーストとして生じ，コアレッサ後の TLB は 29 % の確率でミスする [35]．

▷ page walk cache

ページウォークキャッシュは平均ウォークレイテンシを 95 % 以上削減する．PTE がキャッシュされる GPU の L2 は 300 サイクル近くかかるので，ウォーカにはこれが必要である．完全な設計は，4 KiB ページと無修正の x86-64 ページテーブルで，平均して理想的な MMU の 2 % 以内に入る [35]．

7.1.5 変換のコストを減らす

3 つの技法が変換の回数またはコストを減らす．

セグメントマッピング．セグメントマッピング では，各プログラムの仮想空間が物理メモリの 1 つの連続した領域に対応づけられるので，変換はベースの加算になる（図 7.5）．図では，プログラム 2 の仮想アドレス 0x1234 は物理アドレス 0x011234 である．テーブルもウォークも不要だが，各プログラムの空間は物理メモリ上で連続していなければならない．

ラージページ． 少数の大きなページは少ない変換で済む（後述）．

多段 TLB． SM ごとの小さな L1 TLB を，より大きな共有 L2 TLB で支えると（図 7.4 のように），ウォークが始まる前に，別の SM がすでに持ち込んだ変換で L1 のミスに応えることができる（ページウォークキャッシュがある場合，Power らはそこから利得を得られなかった．前述）．

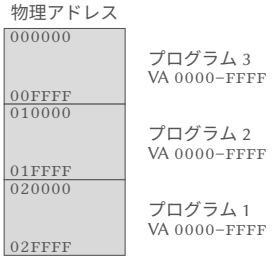


図 7.5: セグメントマッピング：同じ仮想範囲を持つ 3 つのプログラムが 3 つの 64 KiB の物理セグメントに置かれる．

▷ segment mapping

▷ large page

定義 7.6 (ラージページ). **ラージページ**は基本ページよりはるかに大きなページであり，典型的には 4 KiB の代わりに 2 MiB である．オフセットは 12 ビットではなく 21 ビットであるため VPN は 9 ビット短くなり，x86-64 ではウォークが 1 段早く終わる．このとき 1 つの TLB エントリは 2 MiB/4 KiB = 512 倍のメモリをカバーする．

TLB が一度に対応づけられるメモリの量，すなわちエントリ数とページサイズの積を，そのリーチという．ラージページのコストは**内部断片化**である．ページ単位で割り当てられたが使われないメモリであり，割り当ての最後のページの中で失われる．

CPG §19.2.2.1 Memory Paging and Page Sizes：x86-64 の CPU は 4 KiB の物理ページを使うが，“NVIDIA GPUs support multiple physical page sizes, but prefer 2MiB physical pages or larger” であり，TLB ミスは “are generally significantly more expensive on the GPU compared to the CPU” [5]．

▷ internal fragmentation

例 7.7. 32 エントリの L1 TLB のリーチは，基本ページでは 32 × 4 KiB = 128 KiB，ラージページでは 32 × 2 MiB = 64 MiB である．10 KiB の割り当ては 4 KiB ページを 3 枚占めて 2 KiB を無駄にするが，2 MiB ページ 1 枚では 2038 KiB，ページの

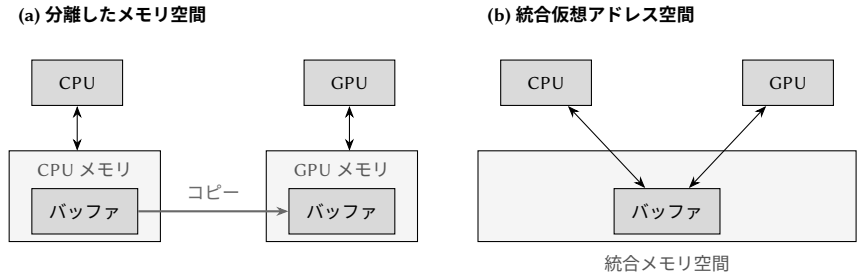


図 7.6: (a) 分離したメモリ空間：各プロセッサは自分のバッファを読み書きし，明示的なコピーがデータを動かす．(b) 統合仮想アドレス空間：両方のプロセッサが読み書きする 1 つの論理バッファ．

99 % 以上を無駄にする（計算のために選んだ本書の数値）．

ラージページとスモールページを混在させ，メモリの各領域にそれに合ったページサイズを与えることもできる．[1]

> **要点**
> GPU はワープ全体のアドレスを変換する．命令が必要とする変換の数はコアレッシングが決め，コアレスされないアクセスはスレッドごとに 1 回の変換を必要としうるし，ウォーカは多数のミスを同時に処理しなければならない．ページウォークキャッシュを持つ共有の高帯域幅ウォーカ，多段 TLB，セグメント，ラージページがコストを減らす．ラージページは TLB のリーチと内部断片化を交換する．

7.2 統合仮想アドレス空間とオンデマンドページング

仮想メモリの第 2 の側面はページの割り当てであり，GPU が CPU と最も異なるのはここである．本節では明示的なコピーと共有アドレス空間を比較し，GPU のページフォールトをホストまでたどって戻り，オンデマンドページングを実用的なコストにするプリフェッチと追い出しのポリシを紹介する．

7.2.1 コピー後実行と共有アドレス空間

定義 7.8 (コピー後実行モデル). **コピー後実行モデル** では，ホストはカーネルを起動する前にデバイスメモリを割り当て（`cudaMalloc`），入力をそこへコピーし（`cudaMemcpy`），終了後に結果をコピーして戻す．カーネルが触れるすべてのデータは，カーネルの開始前にデバイスメモリにある．

データ転送は明示的であり，カーネル起動前の準備の中で行われる．これには 1 つの長所がある．プログラマがスケジュールできる一括転送は，第 5 章のピン留めバッファとストリームが示したように，PCIe の帯域幅を使い切る．短所は，プログラマがすべてのバッファとすべてのコピーを手作業で管理することであり，ポインタを含むデータ構造はまったく共有できないことである．

定義 7.9 (共有仮想メモリ). OpenCL/CUDA のプログラミングモデルで用いられる名前である **共有仮想メモリ**（SVM）では，CPU と GPU は 1 つの仮想アドレス空間を使うので，1 つのポインタは両方で同じデータを意味する（図 7.6）．

第 5 章の **統合仮想アドレッシング（UVA）** は分離したメモリの上の 1 つのアドレス空間である．ページも自動的に移動するようになると，共有アドレス空間はプログラマビリティをさらに高める．高性能計算の GPU のほとんどはディスクリートである．すなわち自分の物理メモリを持つので，共有アドレス空間でもホストメモリとデバイスメモリの間のコピーは依然として必要であり，それが今度は要求に応じて行われる．これには **動的なページ割り当て** が必要である．ページをどこに置くかの決定が，カーネルの実行中に下される．

定義 7.10 (オンデマンドページング). **オンデマンドページング** のもとでは，マ

[1] 研究はこの方向で続いている．Mosaic [36] はアプリケーションを変更せずに複数のページサイズを同時にサポートし，複数アプリケーションのワークロードで平均 55.5 %（同種）と 29.7 %（異種）の向上を報告しており，理想的な TLB との差はそれぞれ 6.8 % と 15.4 % 以内である．複数の GPU については，Li ら [37] が変換をピアの GPU 間で共有し，TLB エントリを IOMMU の TLB にあふれさせることで，単一および複数アプリケーションのワークロードで平均 23.5 % と 16.3 % の向上を得ている．

出典 M7，UVA とオンデマンドページング；CPG §19 Unified Memory Programming；BPG §10.1.4 Unified Virtual Addressing；[38, 39]．

▷ copy-then-execute model

注意 “SVM” は OpenCL 2.0 の用語である．CUDA はこれを使わず，共有アドレス空間を統合仮想アドレッシング，移動する変種を統合（マネージド）メモリと呼ぶ．
▷ shared virtual memory (SVM)

第 3 章は `cudaMallocManaged` のプログラミング面を紹介した．第 5 章は UVA（1 つのアドレス空間）とユニファイドメモリ（自動的なページ移動）を区別した．

▷ on-demand paging

ネージドメモリ（cudaMallocManaged で割り当てたメモリ）は前もってデバイスメモリに置かれない．常駐していないページへの GPU の最初のアクセスがフォールトを起こし，そのときページがデバイスメモリに割り当てられ，そのデータがホストから移される．

n 個の float をその場で何倍かするカーネル scale について，2 つのモデルを並べる．

```
// コピー後実行
float *d;
cudaMalloc(&d, n * sizeof(float));
cudaMemcpy(d, h, n * sizeof(float),
           cudaMemcpyHostToDevice);
scale<<<(n + 255) / 256, 256>>>(d, n);
cudaMemcpy(h, d, n * sizeof(float),
           cudaMemcpyDeviceToHost);

// オンデマンドページング
float *m;
cudaMallocManaged(&m, n * sizeof(float));
for (int i = 0; i < n; ++i) m[i] = i;  // CPU が触れる
scale<<<(n + 255) / 256, 256>>>(m, n);  // GPU がフォールト
cudaDeviceSynchronize();       // m は CPU で有効
```

2 つ目の版にはプログラム中のコピーがないが，コピーはやはり起こる．各ページはカーネルが初めて触れたときに PCIe を渡る．

CPG §19.3.2.1 Data Migration and Coherency：Compute Capability 6.x より前は，カーネル起動のたびにすべてのマネージドメモリが GPU へ移された．Compute Capability 6.x（Pascal）は GPU のページフォールトを導入し，ページを要求に応じて移すようになった [5]．

7.2.2 ページフォールトの経路

定義 7.11 (ファーフォールトと GMMU). ユニファイドメモリを持つ GPU では，**GPU メモリ管理ユニット**（GMMU）が SM の TLB ミスを受け取り，GPU のページテーブルをウォークする．**ファーフォールト**とは，デバイスメモリに存在しないページに対する GPU のページフォールトであり，ホストのドライバがページを割り当ててデータを移すことで解決しなければならない．これに対し**ニアフォールト**は，デバイスメモリが応える L2 ミスである．

▷ GPU memory management unit (GMMU)
▷ far fault

図 7.7 はファーフォールトをたどる．

1. GMMU のウォークは有効な PTE を見つけられない．フォールトは GPU のページフォールト用 MSHR（miss status holding register）に記録され，フォールトしたワープは待つ．
2. フォールトは PCIe を越えて CPU へ送られる．
3. オペレーティングシステムと GPU ドライバがそれを処理する．デバイスメモリにページを割り当て，データを移す．
4. 新しい対応が送り返され，PTE が有効にされ，アクセスが再実行される．

この往復には数十マイクロ秒かかる．

両方のプロセッサは，デバイス用の変換ユニットを通じてメモリに到達する．

定義 7.12 (IOMMU と DMA). **DMA**（direct memory access）とは，CPU がコピー

注意 講義 vs 論文：出典の一節はこのレイテンシを “tens of milliseconds” としているが，正しくは数十マイクロ秒である．先行研究はフォールトあたり 20–50 µs を報告しており，Kim らはフォールトのバッチ全体について 223–553 µs（中央値 313 µs）を測定している [39]．

▷ DMA (direct memory access)

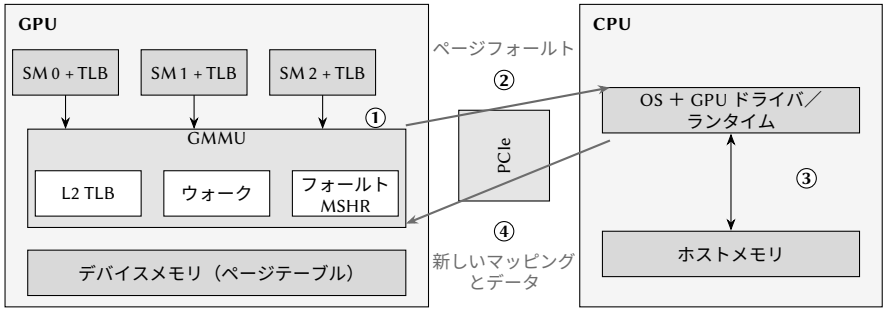


図 7.7: ディスクリート GPU でのファーフォールト：(1) GMMU が有効な PTE を見つけられずフォールトを記録する．(2) フォールトが PCIe を渡る．(3) OS と GPU ドライバがページを割り当てて移す．(4) 対応が戻り，アクセスが再実行される．

を実行することなく，デバイスとメモリの間でデータを転送することである．**IOMMU**（input–output memory management unit）は，デバイスが DMA に用いるアドレスを変換し，デバイスが物理アドレスではなく仮想アドレスで動作できるようにする（I/O 仮想化）．

▷ IOMMU

GPU のメモリはかつて，周辺デバイスのメモリとして IOMMU を通じてのみ管理されていた．共有アドレス空間では CPU があらゆる対応の変更に関与し，これがオンデマンドの GPU メモリのコストである．ページ管理はホストが行い，CPU から対応を取ってくるには PCIe の往復に加えて CPU の割り込み処理ルーチンがかかり，しばしば数十マイクロ秒になる．[2]3 つの対策が考えられる．

- **ページをプリフェッチする**：必要になるページを予測し，フォールトする前に移す．
- **ページ対応の管理を *GPU* 側に移す**：フォールトがホストを必要としないようにする．
- ***CPU* と *GPU* を協調させる**：CPU はメモリの大きなチャンクを管理し，GPU は与えられたチャンク内の小さなページを管理する．

NVIDIA のドライバが実装している対策はプリフェッチである [38]．

Ganguly らは，Pascal GPU でのファーフォールトの処理レイテンシを，データの PCIe 転送の前で典型的に 45 µs としている [38]．

[2] ドライバはフォールトを 1 つずつ処理するのではない．フォールトをバッファに集めてバッチで処理し，バッチの処理中に起きたフォールトは次のバッチを待つ．Kim ら [39] はこの直列化がスループットを損なうことを示し，より大きなバッチ（スレッドブロックのコンテキストを切り替えて，より多くのフォールトがまとめて起こるようにする）と，移動と重ね合わせた追い出しを提案している．最先端のプリフェッチに対して平均 2× の高速化，バッチは 2.27× 大きく 51 % 少なく，バッチ処理時間は 27 % 短い．

7.2.3 ページのプリフェッチ

定義 7.13 (プリフェッチ)**.** 将来のアドレスの予測に基づいて，要求される前にデータを取ってくることを**プリフェッチ**という．これを統合アドレス空間のページに適用したものが**ページプリフェッチ**であり，GPU がフォールトする前にページをデバイスメモリへ移す．

▷ prefetching
▷ page prefetching

アドレス 0x40，0x80，0xC0 を見た逐次予測器は，次に 0x100 と 0x140 を予測する．連続するインデックスで `a[idx]` と `b[idx]` を読むベクトル加算はまさにそのような系列を生み，そのような配列のページアドレスも同じく予測可能である．逐次ページ予測器はページ p がフォールトしたときにページ $p+1$ を取ってくる．

GPU ではこの予測は破綻する．スレッドブロックは順序どおりに実行されないからである（第 3 章）．図 7.8 では各ブロックが連続する 3 ページを読むが，ブロックは 2, 0, 1 の順に実行され，フォールトは 6, 7, 8, 0, 1, 2, 3, 4, 5 の順に届く．ページ 8 の後，“次のページ” 予測器はどのブロックも読まないページ 9 を取ってきてしまい，ページ 0 への後戻りは予見できないフォールトになる．空間的局所性はフォールトの近傍には存在するが，フォールトの順序には存在しない．



図 7.8: 2, 0, 1 の順に実行される 3 つのスレッドブロックが読む 9 ページ：フォールトは 6, 7, 8, 0, 1, ...の順に届く．

定義 7.14 (ツリーベース近傍プリフェッチ)**.** NVIDIA のユニファイドメモリドライバのポリシは**ツリーベース近傍プリフェッチ**（TBNp）と呼ばれ，マネージドな割り当ての各 2 MiB 領域を 2 つの 1 MiB の半分に，それらをさらに 64 KiB のノードに分割する．4 KiB のフォールトは 64 KiB のノード全体，16 ページを移す．1 つの親の下のいくつかのノードがフォールトすると，最大 1 MiB の親ノードが取ってこられる．

▷ tree-based neighborhood prefetching

このツリーは，フォールトの順序にかかわらず各フォールトの周囲の空間的局所性をとらえ，1 回のプリフェッチの大きさを抑える．

例 7.15. 2 MiB 領域の葉 0，1，2，4 にこの順でファーフォールトが起きたとして，Ganguly らの規則を適用する（図 7.9）．各フォールトの後，GMMU はすべての祖先の有効サイズを合計する．半分を超えて有効なノードは，空いているほうの子を満ちているほうの子のサイズまで満たす．

フォールト	50 % を超える祖先	移動量
葉 0	なし（128-KiB ノード：128 のうち 64）	64 KiB
葉 1	128-KiB ノードは満杯で満たすものなし（256-KiB ノード：256 のうち 128）	64 KiB
葉 2	256-KiB ノード：256 のうち 192．葉 3 を満たす	128 KiB
葉 4	512-KiB ノード：512 のうち 320．葉 5–7 を満たす	256 KiB

4 回のフォールトで，4 回のオンデマンドな 4 KiB の移動による 16 KiB ではなく，

注意 講義 vs 論文：Ganguly ら [38] は正確な規則を GTX 1080 Ti で測定した．ツリーは 64 KiB の葉を持つ完全二分木であり，その間に 128，256，512 KiB の段がある．ノードは “to-be valid size” が容量の “strictly greater than 50 %” のときに満たされるので，1 回のプリフェッチは 64 KiB から 1 MiB までの範囲になる．

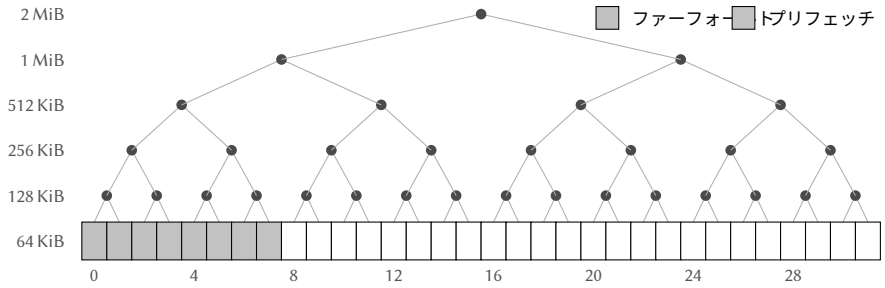


図 7.9: 2 MiB 領域のプリフェッチツリー：64 KiB（各 16 ページ）の 32 枚の葉が，ルートまでの二分の段の下にある．網掛けは例 7.15 の最終状態であり，葉 0，1，2，4 のファーフォールトとプリフェッチされた葉を示す．

512 KiB，8 枚の葉が持ち込まれる．葉 0 だけでは自分のノードを越えるプリフェッチは起こらなかった．ちょうど 50 % ではノードは満たされない．

7.2.4 オーバーサブスクリプションと追い出し

定義 7.16 (オーバーサブスクリプションと追い出し)．GPU の物理メモリが保持できるより多くのマネージドメモリを割り当てることを**メモリのオーバーサブスクリプション** という．このとき場所を空けるためにページがデバイスメモリから出ていかなければならず，これが**ページ追い出し**である．どのページが出ていくかはポリシが決める．

▷ memory oversubscription

▷ page eviction

CPG §19.1.2.7 GPU Memory Oversubscription：ユニファイドメモリでは，プログラムが “allocate and share arrays larger than the memory capacity of any individual processor in the system” ことができる [5]．

プリフェッチと追い出しは相互に作用する．積極的なプリフェッチはまだ必要なページを追い出し，プリフェッチャを無視した追い出しポリシは散在する 4 KiB ページを追い出して，プリフェッチャが頼る連続した領域を壊す．この問題はカーネルのメモリ需要とともに大きくなる．1 つの答えは**事前追い出し**である．LRU リストが追い出しに選んだページとともに，その周囲の連続したページを，プリフェッチャがそれらを持ち込んだ 64 KiB ブロックの倍数単位で追い出し，プリフェッチャが再び使える単位で空き領域が生まれるようにする．

▷ pre-eviction

Ganguly ら：ツリーベースの事前追い出しをハードウェアプリフェッチャと組み合わせると，4 KiB および 2 MiB ページの LRU 置換に対して平均 93 % と 18.5 % の高速化が得られる [38]．

> **要点**
> 共有アドレス空間は明示的なコピーを取り除くが，データの移動は取り除かない．ディスクリート GPU では，ページへの最初のアクセスはすべてファーフォールトとなり，PCIe を越えてホストへ行って戻り，数十マイクロ秒かかる．各フォールトの周囲のプリフェッチ（ツリーベースの方式）は，スレッドブロックが順序どおりに実行されなくても往復のコストを償却する．オーバーサブスクリプションのもとでは，追い出しポリシはプリフェッチャが扱う単位を尊重しなければならない．

7.3 ワープスケジューリング

第 4 章のパイプラインの説明は，一度に準備完了のワープが 1 つであると仮定していた．通常は複数のワープが同時にスコアボード上ですべてのオペランドをそろえており，その中から 1 つを選ばなければならない．本節では基本的なポリシを定義し，その選択がレイテンシ隠蔽と L1 キャッシュにどう影響するかを示し，これらの観察の上に築かれた研究上のスケジューラを紹介する．

出典 M7，GTO とラウンドロビン；A18 ch. 3；K12 §1.4.1 GPU Pipeline；CPG §4.2 Hardware Multithreading；[40, 29, 30]．

定義 7.17 (ワープスケジューラ)．**ワープスケジューラ**は，毎サイクル，次の命令が準備完了のワープの中から 1 つを選び，その命令を発行するパイプライン段である．

▷ warp scheduler

CPG §16.6.1（Compute Capability 7.x，Architecture）：SM は 4 つのワープスケジューラを持ち，ワープをそれらに静的に分配する．各スケジューラは準備完了のワープの 1 つについて毎サイクル 1 命令を発行する [5]．CC 7.0 の常駐ワープ 64 個（CPG Table 21；CC 7.5 では 32 個）では，スケジューラあたり最大 16 ワープになる（本書の計算）．

7.3.1 命令スケジューリングからワープスケジューリングへ

アウトオブオーダの CPU にも似たユニットがある．その命令スケジューラはスケジューリングキューの準備完了の命令の中から選ぶ．既定のポリシとなるのは

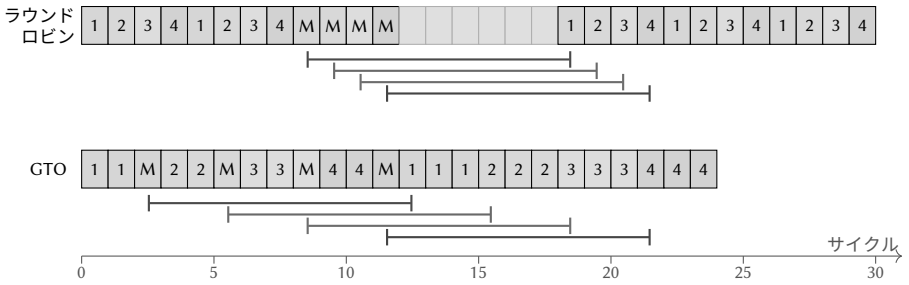


図 7.10: 例 7.19 の発行スロット：数字はワープ，M はそのロード，灰色はストールサイクルである．各行の下のバーは 10 サイクルのロードレイテンシである．ラウンドロビンは 4 つのロードを続けて発行してストールし，GTO はそれらをずらす．

最古優先であり，準備完了の命令のうち最初にキューに入ったものが発行される．別の方式は**クリティカル**な命令を優先する．ロードを先にしてアドレスがわかり次第メモリ要求を始めるか，依存する命令の多い命令を先にしてより多くの命令が早く準備完了になるようにする．

▷ oldest-first

GPU のワープスケジューラは各ワープの中ではインオーダである．ワープの命令はプログラム順に発行される．ワープをまたいでは任意の準備完了のワープを選べるので，SM 全体の命令の流れはアウトオブオーダに見える．それぞれ命令 I1，I2，I3 を実行する 4 つのワープは，たとえば次のように発行されうる．

I1 w1, I1 w2, I2 w1, I1 w3, I2 w2, I3 w1, I1 w4, I2 w3, I2 w4, I3 w2, I3 w4, I3 w3

これはどのワープの中でも I1，I2，I3 の順序を保っている．どの準備完了のワープを先にするかがポリシである．

K12 §1.4.1：スケジューラはインオーダだが “can select any warps that are ready” ので，SM の実行は “might look like an out-of-order execution” [3]．

7.3.2 ラウンドロビンと GTO

定義 7.18 (ラウンドロビンと GTO). **ラウンドロビン**（RR）のスケジューラは，固定の巡回順でワープに順番を与え，1 命令ずつ発行させる．**ルーズラウンドロビン**（LRR）の形では，自分の番に発行できないワープは飛ばされ，順番で次のワープに機会が回る．**GTO スケジューリング**（greedy-then-oldest）のスケジューラは，1 つのワープが長レイテンシの事象（キャッシュミス，分岐，長いテンソルコア演算）でストールするまでそのワープから発行し続け，それから最も古い準備完了のワープに切り替える．ワープの年齢とは，そのワープが SM に割り当てられた時刻である．

▷ round-robin (RR)

▷ loose round-robin (LRR)

▷ greedy-then-oldest (GTO)

2 つのポリシは，ワープがメモリ命令に到達する時期が異なる．ラウンドロビンではすべてのワープがそろって進み，ほぼ同時にロードに到達する．GTO では 1 つのワープが先行して早くロードを始め，他のワープが 1 つずつ後に続く．

Rogers らは両方の基準方式を定義している．GTO は “runs a single wavefront until it stalls then picks the oldest ready wavefront” であり，年齢は “the time it is assigned to the core” である [40]．

例 7.19. 4 つのワープが同じプログラムを実行する．2 つの算術命令 c1，c2，キャッシュでミスして 10 サイクル後に戻るロード m3，そして 3 つの算術命令 c4–c6 であり，c4 はロードした値を使う．図 7.10 は発行スロットを示す．

記法：毎サイクル 1 命令が発行される．サイクル t に発行されたミスにより，依存する命令はサイクル $t + 10$ に発行できる．本書の数値．

- **ラウンドロビン**：サイクル 0–7 がすべてのワープの c1 と c2 を，サイクル 8–11 が 4 つのロードを発行する．すべてのワープが自分のロードを待つことになり，最初のロードはサイクル 18 に戻るので，サイクル 12–17 はストールである．残りの 12 命令はサイクル 18–29 を要する．30 サイクル，そのうち 6 サイクルがストールである．
- *GTO*：ワープ 1 はサイクル 0–2 に c1，c2，m3 を発行してストールする．ワープ 2 がサイクル 3–5，ワープ 3 が 6–8，ワープ 4 が 9–11 に続く．ワープ 1 のロードはワープ 4 のロードが発行された 1 サイクル後のサイクル 12 に戻り，各ワープはデータが届くにつれて残りの 3 命令を終える．24 サイクル，ストールなしである．

GTO ではロードが分散し，他のワープの算術命令がそれぞれのロードを隠蔽する．

GTO が常に優れているわけではない．利得はプログラムに依存する．例では，利得はロードの後の算術命令から来ている．すべてのワープがロードの直後に同じバリアでストールするプログラムは何も得ない．ロードレイテンシが他のすべてのワープの算術命令の合計より長ければ GTO もストールする．ワープがデー

タを共有する場合は，ラウンドロビンでそろって発行するほうが，1 つのワープのミスで他のワープの要求にも応えられることがある．

7.3.3 2 レベルスケジューリング

毎サイクルすべての準備完了のワープを探索するスケジューラは，1 つの発行スロットのために数十の候補を比較し，この探索は電力を消費する．

定義 7.20 (2 レベルワープスケジューリング)．**2 レベルワープスケジューリング**では，ワープは**アクティブワープ集合／待機ワープ集合**の組に分けられる．探索されるのは少数のアクティブなワープだけである．長レイテンシの演算に出会ったワープは待機集合に移り，演算が完了した待機中のワープは，空きがあればアクティブ集合に戻る（図 7.11）．

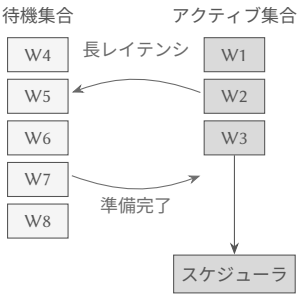


図 7.11: 2 レベルスケジューリング：スケジューラはアクティブ集合だけを探索する．

▷ two-level warp scheduling

▷ active warp set / pending warp set

この考えには 2 つの論文が引かれ，その動機は異なる．Narasiman らはワープを**フェッチグループ**に分ける．たとえば 32 ワープを 8 ワープずつの 4 グループにする．1 つのグループは，そのすべてのワープがストールするまで優先権を持ち，グループ内でもグループ間でもラウンドロビンである．こうしてグループはそれぞれ異なる時期に長レイテンシの演算に到達する．これは例 7.19 と同じずらしであり，あるグループがメモリを待つ間，常にどこかのグループに仕事がある．

注意 講義 vs 論文：Gebhart ら [30] はアクティブ／待機の分割をエネルギーのために導入している（レジスタファイルキャッシュと合わせて，レジスタファイルのエネルギーを 36 % 削減）．Narasiman ら [29] は性能のためにワープをフェッチグループに分ける．ラージワープと合わせて 19.1 % である．

▷ fetch group

7.3.4 スケジューリングとキャッシュの振る舞い

スケジューラは，どのデータが同時に L1 キャッシュにあるべきかも決める．

定義 7.21 (ワーキングセット，スラッシング，初期参照ミス)．ある区間におけるプログラムの**ワーキングセット**とは，その区間にプログラムが使うメモリブロックの集合である．ワーキングセットがキャッシュを超え，ブロックが再利用される前に追い出されてほとんどのアクセスがミスするとき，キャッシュは**キャッシュスラッシング**を起こしている．**初期参照ミス**はブロックへの最初のアクセスのミスであり，どのキャッシュも避けられない．

▷ working set

▷ cache thrashing

▷ compulsory miss

同じワープがキャッシュに持ち込んだブロックへのヒットは**ワープ内局所性**を，別のワープが持ち込んだブロックへのヒットは**ワープ間局所性**を利用している．次の例のように各ワープが自分のデータを再利用するとき，ラウンドロビンは最悪の順序である．

▷ intra-warp locality

▷ inter-warp locality

Rogers らはベンチマークのヒットをこのように分類している．キャッシュに強く依存するものでは，再利用のほとんどがワープ内である [40]．

例 7.22. 3 つのワープがそれぞれ自分のブロックを 4 回読む．ワープ k はブロック Bk を読む．L1 は LRU 置換（第 6 章）で 2 ブロックを保持し，各アクセスは次のアクセスが発行される前にヒットかミスかが決まる．

順序	アクセス（太字＝ミス）
RR	**B1 B2 B3 B1 B2 B3 B1 B2 B3 B1 B2 B3**
GTO	**B1** B1 B1 B1 **B2** B2 B2 B2 **B3** B3 B3 B3

ラウンドロビンでは，各ブロックはそのワープが戻ってくる前に他の 2 つに追い出される．12 回のアクセスで 12 回のミスである．GTO では各ワープが次のワープの開始前に自分のブロックを終えるので，3 回の初期参照ミスだけである．

注意 GTO の行は，ワープが最初のミスの後も発行を続けると仮定している．そのミスの時点で他のワープが準備完了なら GTO は切り替え，そのワーキングセットは複数のワープのデータを含む．そのため下の式で GTO の $SW \geq 1$ となる．

一般に，各ワープのワーキングセットの大きさを SS_W とし，SM に N 個のワープが常駐するとする．ラウンドロビンではすべての N ワープが同時に進行中であり，L1 は $SS_W \times N$ を保持する必要がある．スケジュール可能なワープ数を SW に制限すれば $SS_W \times SW$ で済む．GTO は長レイテンシの事象のたびにワープを切り替えるので，SW は必ずしも 1 ではない．例 7.22 では $SS_W = 1$ ブロック，$N = 3$ で，2 ブロックのキャッシュを超える．$SW = 2$ ならワーキングセットは収まる．

ヒント 必要な L1 $\approx SS_W$ と発行しうるワープ数の積：ラウンドロビンでは N，制限下では SW．大きさ C のキャッシュに収めるには $SW \leq C/SS_W$ に制限する．

定義 7.23 (キャッシュ考慮型ウェーブフロントスケジューリング)．Rogers らのスケジューラは**キャッシュ考慮型ウェーブフロントスケジューリング**（CCWS）と呼ばれ [40]，発行できるワープの数を制限して，それらのワーキングセットが L1 に収まるようにする．ストールの際には少数のワープ（たとえば W1–W3）の中から選び，それらが完了したときにだけ別のワープ（W4）を開始する．

▷ cache-conscious wavefront scheduling (CCWS)

注意 講義 vs 論文：ここで述べた固定の制限は，論文の基準方式 “static wavefront limiting” である．CCWS そのものは適応的である．検出器が，追い出しによってワープの局所性が壊されたことに気づくと，スケジューラは他のワープを絞ってそのワープに L1D へのより排他的なアクセスを与える．

▷ lost intra-wavefront locality detector (LLD)

論文の仕組みは**ウェーブフロント内局所性損失検出器**（LLD）であり，ワープ

ごとに，そのワープが L1 に持ち込み追い出しによって失ったラインのタグ（データを持たない犠牲タグ配列）を保持する．ワープ自身の犠牲タグにヒットする L1 ミスは失われたワープ内局所性であり，そのワープの局所性損失スコアを引き上げる．他のワープの高いスコアによってスコアスタックのカットオフを越えて押し出されたワープ（論文ではカットオフより上）は，スコアが減衰するまで L1D にロードを発行できない．報告された結果は，従来のスケジューリングポリシに対し，L1 データキャッシュミスが平均 25 % 減，調和平均 24 % の高速化である．

CCWS のハードウェアはチップ全体の面積に 0.17 % を加える．単純な貪欲スケジューラに対しては，キャッシュに強く依存するベンチマークでの調和平均の利得は 63 % である [40]．

ワープを制限すると 2 つの問題が生じる．

- **同期.** バリア（第 3 章）は，ブロックのワープがすべて到着したときにだけそれらを解放する．他のワープが `__syncthreads()` で待っているブロックのワープをスケジューラが留め置くと，そのブロックのどのワープも進めない．デッドロックである．したがって制限は，他のワープが待っているワープを決して締め出してはならない．
- **制限の大きさ.** ワープが多いほどレイテンシを隠蔽するスレッドレベル並列性が増え，ワープが少ないほどワーキングセットが小さくなる．最良の値はキャッシュとプログラムで決まり，CCWS は実行時に適応させる．

次の形のカーネルでは，どのワープもフェーズ 2 を始める前に，ブロックのすべてのワープがバリアに到達しなければならない．

```
__global__ void twoPhase(float *a) {
  // フェーズ1：ブロックのすべてのワープが働く
  __syncthreads();   // すべてのワープを待つ
  // フェーズ2
}
```

7.3.5 その他のスケジューリングポリシ

スケジューラは L1 より先のメモリシステムにも役立てられる．第 4 章のメモリチャネルと同様に，各 DRAM 内のバンクは並列に要求に応えられる．同時に稼働しているバンクの数がその**バンクレベル並列性** である．

GPUMech（第 8 章）は RR と GTO の両方の性能を解析的にモデル化し，平均誤差は 13.2 % と 14.0 % である [43]．

▷ bank-level parallelism

- **プリフェッチを考慮したスケジューリング** [41] は 2 レベルスケジューラを基礎とするが，そのグループを**連続しない**ワープから作る．連続するワープは隣接するデータにアクセスするので，1 つのワープが起動した単純なプリフェッチャが次のワープのデータを取ってこられる．離してスケジュールすれば，次のワープはプリフェッチが間に合うだけ遅れて実行され，同時に進行中の要求はより多くの DRAM バンクに分散する．
- **CTA 考慮スケジューリング**（OWL [42]）は，どの CTA（スレッドブロック，第 3 章）を一緒にスケジュールするかを決め，少数の CTA のワープをまとめて，そのデータがキャッシュに残るようにする．これを局所性とバンクレベル並列性を考慮したワープ選択と組み合わせる．

Jog らは，シミュレートした 30 コアの GPU で，それぞれプリフェッチ付きのラウンドロビンと 2 レベルスケジューリングに対して平均 25 % と 7 % の向上を報告している [41]．

▷ CTA-aware scheduling

OWL：シミュレートした 28 コアの GPU で，ラウンドロビンのワープスケジューリングに対して平均 33 % の向上．その動機として，38 のアプリケーションのうち 19 が平均してサイクルの 62 % をどのワープも実行していない状態で費やす [42]．

要点 — 章のまとめ

GPU の仮想メモリはスレッドではなくワープを変換する．変換の数はコアレッシングが決め，コアレスされないロードはスレッドごとに 1 回の変換を必要としうる．ページウォークキャッシュを持つ共有の高帯域幅ページテーブルウォーカ，多段 TLB，セグメントマッピング，ラージページがコストを抑える．ラージページは TLB のリーチを内部断片化と交換する．共有仮想アドレス空間はコピー後実行をオンデマンドページングに置き換え，そのファーフォールトは PCIe 越しに数十マイクロ秒かかる．ツリーベース近傍プリフェッチがそれを償却し，オーバーサブスクリプション下の追い出しはプリフェッチャの単位を尊重しなければならない．ワープスケジューラはワープの中ではインオーダに発行するが，準備完了のワープの中から選ぶ．ラウンドロビンはすべてのワープをそろって進めてミスを集中させ，GTO はそれらをずらす．2 レベルスケジューラはエネルギーと性能のために小さなアクティブ集合を探索する．そしてスケジュール可能なワープを制限すること（CCWS）は，バリアを尊重する限り，L1 のワーキングセットを縮める．

第 8 章

GPU シミュレーション

講義動画	M8，レッスン 1–4（GPU Cycle Level Simulation，Parts 1 and 2；Analytical Models of GPUs；Accelerating GPU Simulation）
資料	Liu ら [44]（Photon，指定）；Williams ら [45]（Roofline，指定）；Hong と Kim [46]（MWP/CWP モデル）；Huang ら [43]（GPUMech）；Karkhanis と Smith [47]（一次近似モデル）；シミュレータ [48, 49, 50, 51, 52]；Sherwood ら [53]（SimPoint）；NVIDIA GEMM ガイド [54]
教科書	Eeckhout [55]（任意）；K12 [3] ch. 2 Performance Principles，§4.1.1 Execution Time Modeling，§4.2.4 Roofline Model，§4.2.6 Other GPGPU Performance Modeling Techniques；BPG [8] §10.2.1 Coalesced Access to Global Memory
キーワード	性能モデリング，サイクルレベルシミュレーション，実行駆動シミュレーションとトレース駆動シミュレーション，キューベースモデル，セクタキャッシュ，解析モデル，ルーフラインモデル，演算強度，リッジポイント，CPI スタック，インターバル解析，MWP と CWP，サブコア，イベント駆動シミュレーション，サンプリングシミュレーション，SimPoint，データ駆動モデル

これまでの章で扱った機構（第 6 章の SIMT スタックと動的ワープ形成，レジスタファイルキャッシュ，第 7 章のワープスケジューラ）は，いずれもシリコン上に存在する前に提案され，評価されたものである．本章は，そうした評価を可能にする道具を扱う．まず，キューとクロックから GPU のサイクルレベルシミュレータを組み立て，次に GPU に固有のもの，すなわちワープ，ダイバージェンス，コアレッシング，セクタキャッシュを加える．続いて，何もシミュレーションせずに少数のパラメータから性能を予測する解析モデルに移る．GPU の一次近似設計，ルーフラインモデル，CPI スタック，インターバル解析である．最後に，詳細なシミュレーションがなぜこれほど遅いのか，それに対して何ができるのかを問う．並列シミュレータとイベント駆動シミュレータ，より単純なモデル，サンプリング，より小さなワークロード，データから学習したモデルである．

8.1 サイクルレベルシミュレーション

プロセッサを作る前に，設計者はそれが実際のプログラムをどれだけ速く実行するか，そして設計上の各選択がその速度をどう変えるかを予測しなければならない．本節では，そのための手法を列挙し，サイクルレベルシミュレーションとその 2 つの方式を定義し，パイプラインのシミュレータがクロックで進められるキューとしてどう構成されるかを示す．

出典 M8，サイクルレベルシミュレーション（パート 1）；Eeckhout [55]．

8.1.1 6 つの手法

定義 8.1 (性能モデリング)．プロセッサの設計の性能，あるいはある設計上でのプログラムの性能を，完成したハードウェアで測定せずに予測することを**性能モデリング**という．そのような予測を出せるだけの詳しさでハードウェアの振る舞いを模倣するプログラムが**シミュレータ**である．

▷ performance modeling
▷ simulator

一般に使われる手法は 6 つある．

1. **サイクルレベルシミュレーション**：マシンの各部分が各サイクルに何をするかをモデル化する（本節と次節）．
2. **イベント駆動シミュレーション**：何かが変化する時点だけをモデル化する（8.4 節）．
3. **解析モデル**：少数のパラメータによる方程式（8.3 節）．
4. **サンプリングに基づく手法**：プログラムの選んだ部分だけを詳細にシミュレーションする（8.4 節）．
5. **データ駆動モデル，統計モデル，機械学習モデル**：測定したデータやシミュレーションで得たデータにモデルを当てはめる（8.4 節）．

6. **FPGA エミュレーション**：設計またはそのモデルを FPGA（field-programmable gate array）上に実装する．ソフトウェアシミュレータより桁違いに速く動くが，設計の手間がはるかに大きい．

▷ FPGA emulation

FPGA エミュレーションはサイクルレベルシミュレーションと同じだけ詳細で，速いだけである．FPGA ベースのシミュレータは “fast and accurate, but they take significant effort to develop and use” とされる [56]．本章ではこれ以上扱わない．

8.1.2 サイクルレベルの手法

定義 8.2 (サイクルレベルシミュレーション). **サイクルレベルシミュレーション**では，シミュレータはグローバルなクロックを持ち，シミュレーションする各サイクルで，各ユニットにおけるそのサイクルの事象をモデル化する．どの命令がフェッチされ，どれがデコードされ，どれがスケジューラを出て，どの要求がキャッシュに届くか，である．

▷ cycle-level simulation

サイクルレベルシミュレータはアーキテクチャ研究の標準的な道具であり，特に測定すべきハードウェアがまだない設計の初期に使われる．クロックが 1 つというのは単純化である．実際のチップは，コア，メモリ，オンチップネットワークを異なる周波数で動かす．設計のうち 1 つのクロックで駆動される部分を**クロックドメイン**と呼び，シミュレータはドメイン（コア，メモリ，ネットワークオンチップ）ごとに 1 つのクロックを持ち，それぞれを固有の速さで進めることができる．

▷ clock domain

NVIDIA GPU では SM のクロックとメモリのクロックは独立している．`nvidia-smi -q -d CLOCK` は “Graphics”，“SM”，“Memory” の各クロックを別々に報告する．

8.1.3 実行駆動シミュレーションとトレース駆動シミュレーション

サイクルレベルシミュレータはタイミングをモデル化する．それに加えて，どの命令が，どのアドレスとどの分岐結果で実行されるかも知る必要があり，それを得る方法は 2 つある．

定義 8.3 (実行駆動シミュレーションとトレース駆動シミュレーション). **実行駆動シミュレーション** では，シミュレータ自身がプログラムの命令を実行し，シミュレーションの進行に合わせてその結果を計算する．**トレース駆動シミュレーション** では，実際の実行または機能的な実行から事前に収集した命令（およびそのアドレスと分岐結果）の記録である**トレース** を再生する．こうして実行とタイミングが切り離される．

▷ execution-driven simulation

▷ trace-driven simulation

▷ trace

実行駆動シミュレータでも，命令を**いつ**機能的に実行するかは選ぶ必要がある．命令のフェッチ時に実行すれば（*execute-at-fetch*）機能モデルは単純に保たれ，タイミングモデルが実行ステージに達したときに実行すれば（*execute-at-execute*），値がシミュレーション上の事象の順序に依存できる．2 つの方式の得失は次のとおりである．

実行駆動． トレースを保存する必要がなく，実行そのものに依存する振る舞いを忠実にモデル化できる．どのスレッドが先にロックを獲得するか，スレッドがバリアでどれだけ待つか，データ依存の分岐が異なるスケジュールのもとでどの経路をとるか，である．

トレース駆動． シミュレータは機能モデルを必要としないので，より単純で軽く，開発しやすい．キャッシュだけを調べるなら，メモリアドレス（メモリトレース）があれば足りる．しかしトレースは，記録時の事象の順序を固定する．ロックの獲得，バリア，そのほか実行時のタイミングに依存するものは，シミュレーション上のマシンで変化できない．

注意 2 つの選択は通常同じ結果を与える．違いが出るのはシミュレーション上のタイミングが値を変える場合で，たとえば 2 つのスレッドがロックを奪い合うときである．execute-at-fetch では勝者はフェッチ順で決まり，execute-at-execute ではシミュレーション上の実行順で決まる．

ヒント 経験則：問いが “この設計はどれだけ速いか” であればトレースで足りる．設計がプログラムの振る舞いそのもの（同期やスレッドのたどる経路）を変えうるなら，シミュレーションは実行駆動でなければならない．

8.1.4 キューベースのモデル化

第 4 章のパイプラインは，キューで結ばれたステージとしてモデル化するのが最も簡単である（図 8.1）．

定義 8.4 (キューベースモデル). **キューベースモデル**は，パイプラインをキューで結ばれたステージの集合として表す．各キューは **FIFO**（first in, first out）であり，命令は入った順に出ていく．例外はスケジューラで，**準備のできた**任意の命令を選ぶので，FIFO ではなくプールである．各接続は 2 つのパラメータで記述される．キューの**深さ**は命令がそのステージで過ごすサイクル数であり，**パイプライン幅**は 1 サイクルにあるキューから次のキューへ移動できる命令数（フェッチ，発行，実行の帯域）である．

▷ queue-based model

▷ FIFO

▷ pipeline width

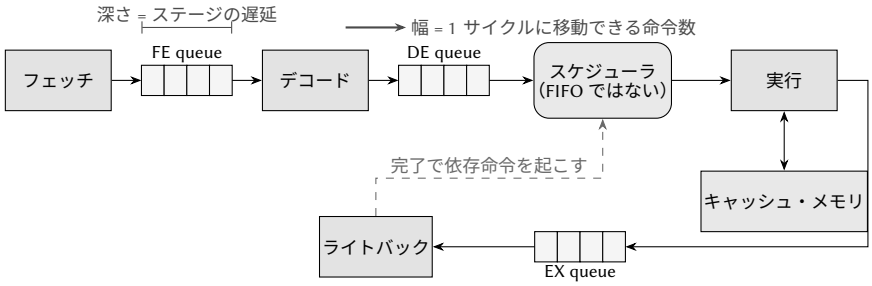


図 8.1: パイプラインのキューベースモデル．フェッチ，デコード，スケジュール，実行，ライトバックは FIFO キューで結ばれ，スケジューラは準備のできた命令を選び出すプールである．実行ステージはメモリのレイテンシをキャッシュとメモリのモデルから得て，完了した命令はそれに依存する命令を起こす．

モデルにタイミングを与えるものは 3 つある．第 1 は依存関係である．命令が完了すると，その結果はそれを待つ命令に通知されなければならず，待っていた命令は準備完了になる．これは第 4 章のスコアボードがハードウェアで行うことと同じである．第 2 に，メモリ命令のレイテンシはキャッシュとメモリのモデルから得られ，そのモデルが各レベルでのヒットかミスかを決める．第 3 に，その他の命令のレイテンシはパラメータであり，たとえば整数加算に 1 サイクル，乗算に 3 サイクルといった値を，文献，CACTI のような回路モデル，あるいは設計のレジスタ転送レベル（RTL）記述から得る．シミュレータ自身は回路をシミュレーションしない．したがって，実行に依存しない実行レイテンシは参照表である．

CACTI は，キャッシュやレジスタファイルのような SRAM アレイのアクセス時間，面積，エネルギーを，その容量と構成から求める広く使われたモデルである．

8.1.5 シミュレータの構造

サイクルレベルシミュレータは，1 回の反復でクロックを 1 サイクル進め，すべての構成要素に 1 サイクル分の仕事をさせるループである．構成要素は階層をなす．シミュレータはコア，メモリシステム，ネットワークオンチップを進め，各コアはそのパイプラインステージを進める．

Georgia Tech の CPU・GPU シミュレータである MacSim [51] はこのように構成されている．リストは我々自身のスケッチであり，そのコードではない．

```
// 最上位：1回の反復 = シミュレーション上の1サイクル
while (!sim.all_done()) {
  ++sim.cycle;
  for (Core &c : sim.cores) c.tick();
  sim.memory.tick();
  sim.noc.tick();
}

// 1つのコア：ステージをパイプラインの逆順に呼ぶ
void Core::tick() {
  writeback(); execute(); schedule();
  decode(); fetch();
}
```

ステージは最後のものから最初のものへ向かって呼ばれる．`execute()` が動くとき，それは `schedule()` が**前の**サイクルに出力した命令を，`schedule()` がそれをこのサイクルの出力で上書きする前に消費する．パイプラインの順に呼ぶと，1 つの命令が 1 サイクルのうちに 5 つのステージすべてを通過してしまう．

ヒント 逆順は，ステージ間のパイプラインレジスタのソフトウェア上の等価物である．各ステージは，前のステージが 1 サイクル前に書いたものを読む．

シミュレーションされる各命令は小さなレコード（*op*）で表され，それがタイミングの状態とともに命令をステージ間で運ぶ．鍵となるフィールドは命令が完了するサイクルである．

$$done_cycle = schedule_cycle + latency. \tag{8.1}$$

レイテンシが固定の命令であれば，スケジューラは発行の時点でこれを計算できる．ロードのレイテンシは固定ではない．アクセスが L1 キャッシュでヒットするか，L2 キャッシュでヒットするか，どちらでもミスするかに依存するからである．それはキャッシュ階層でアクセスをシミュレーションした後に初めて判明し，そのとき発行サイクルに加えられる．

表 8.1: シミュレータが 4 つの命令について計算する完了時刻（我々自身のレイテンシ）．ロードのレイテンシは，キャッシュモデルがどこでヒットするかを決めた後に初めて判明する．

命令	スケジュール	レイテンシ	完了	レイテンシの出所
`mul r1, r2, r3`	10	3	13	表
`ld r4, [r5]`	11	30	41	キャッシュモデル：L1 ミス，L2 ヒット
`mul r6, r1, r1`	13	3	16	表；r1 を待つ
`add r7, r4, 1`	41	1	42	表；r4 を待つ

例 8.5. add に 1 サイクル，mul に 3，L1 でヒット，L2 でヒット，両方でミスするロードにそれぞれ 4，30，200 サイクル（我々自身の数値）というレイテンシのもとで，シミュレータは 4 つの命令を表 8.1 のようにスケジュールする．add はロードに依存するので，ロードの *done_cycle* になって初めて準備完了になる．

スケジューラはモデルの中で最も詳細な部分である．各サイクルに，それは準備のできた命令を，その命令が必要とする資源の制約のもとで選ぶ．空いた浮動小数点ユニット，ロードキューまたはストアキューのエントリ，空いた実行ポートやキャッシュポートである．あるユニットを数サイクル占有する命令はその間ずっと資源を保持し，スケジューラはその間に 2 つ目の命令をそこへ発行してはならない．

8.1.6 GPU のモデル化

サイクルレベルの GPU シミュレータも同じ構造をとるが，単位が 1 つ変わる．スレッドではなく**ワープ**をモデル化する．SIMT ハードウェアは 1 つのワープ命令をそのすべてのスレッドについてまとめてフェッチ，デコード，スケジュール，実行するからである（第 4 章）．スケジューラは各ワープの先頭にある命令を検討する．1 つのワープの命令は順に発行され，異なるワープの命令は任意の順に発行される．CPU に対応物のないものが 3 つある．ダイバージェントなワープ（8.2 節），ワープ，スレッドブロック，カーネルの階層，そして第 7 章のワープスケジューリングポリシである．

この階層は，シミュレーションがいつ終わるかも決める．スレッドブロック全体が 1 つの SM に割り当てられ（第 3 章），シミュレータはすべてのスレッドを最後の命令を実行するまで追跡する．ブロックはそのすべてのスレッドが終了したときに終了し，その SM の資源を次のブロックのために解放する．カーネルはそのすべてのブロックが終了したときに終了し，アプリケーションはそのすべてのカーネルが終了したときに終了する．

GPU シミュレータが 1 サイクルにモデル化すべきものは CPU シミュレータよりはるかに多い．各 SM は最大 64 の常駐ワープを保持でき（CPG Table 21 [5]），GPU には数十の SM がある．

SM が一度に保持するブロック数もモデル化される制限である．SM あたりの常駐ブロックは Compute Capability 7.0，8.0，9.0 で最大 32，7.5 と 8.6 で 16 である（CPG Table 21 [5]）．

> **要点**
> サイクルレベルシミュレータはグローバルなクロック（クロックドメインごとに 1 つ）を進め，各サイクルに各ユニットをモデル化する．命令の流れは，プログラムを実行するか，トレースを再生して得る．パイプラインは，深さがステージのレイテンシ，転送速度がパイプライン幅である FIFO キューの集合に，資源制約のもとで準備のできた命令を選ぶスケジューラを加えたものである．GPU シミュレータはスレッドではなくワープをモデル化する．

8.2 GPU シミュレータにおけるダイバージェンスとメモリ

GPU シミュレータのモデル化の単位はワープである．命令キャッシュへのアクセスとプログラムカウンタは，スレッドではなくワープに属する．ワープのすべてのスレッドがすべての命令に参加するわけではないので，シミュレータは第 4 章のアクティブマスクも保持し，結果の計算と，命令が実際に使う資源の計数の両方に用いる．本節では，これまでの章で扱った GPU 固有の 2 つの効果，ダイ

出典 M8，サイクルレベルシミュレーション（パート 2）；BPG §10.2.1 Coalesced Access to Global Memory [8]．

バージェンスとコアレッシングをシミュレータがどう扱うかを示し，続いてオープンソースの GPU シミュレータを列挙する．

8.2.1 ダイバージェント分岐

シミュレータがダイバージェント分岐（第 6 章）をどう扱うかは，命令をどう得るかによる．

実行駆動． シミュレータは各スレッドの分岐結果を計算し，ハードウェアとまったく同じように SIMT スタックを自ら管理する．スタックがモデルの一部なので，それを置き換えることで別の機構（動的ワープ形成，別の再収束規則）をモデル化できる．

トレース駆動． トレースは，ワープが実行したすべての経路を，各基本ブロックのアクティブマスクとともに記録しなければならない．たとえば第 6 章の分岐なら BB1 1111，BB2 1100，BB3 0011，BB4 1111 である．シミュレータはこれらのブロックを記録された順に再生する．

したがってトレース駆動シミュレータは，ダイバージェンスの処理そのものを調べるのには向かない．ダイバージェンスがタイミングを形づくる効果の 1 つにすぎない場合には十分である．

注意 トレースは，それを収集したマシンのダイバージェンスの振る舞いを記録する．そのようなトレースから別のダイバージェンス処理機構をシミュレーションするのは難しい．トレースには分岐条件も，新しい機構が選ぶはずの順序も含まれていないからである．

8.2.2 コアレッシング

下流のすべて，すなわちキャッシュアクセス，ミス，DRAM トラフィック，帯域を左右するのは，スレッドの数ではなく，ワープ命令が生成するメモリトランザクションの数である．したがってメモリコアレッシング（第 4 章）のモデル化は GPU シミュレータの最も重要な部分の 1 つである．要求は，それが最初に到達するキャッシュの粒度，通常は 1 キャッシュラインの単位で併合される．本節の小さな構成，すなわち 64 B のラインとワープあたり 8 スレッドでは，スレッドが連続する 4 B のワードを読むロード R1 は 1 つの 64 B の要求に併合されるが，スレッドが 128 B ずつ離れたワードを読むロード R2 は，スレッドごとに 1 つ，それぞれ 1 ライン全体に対する 8 つの要求を生む．

トレースはワープ命令のメモリアクセスを 2 通りの方法で表せる．すべてのスレッドのアドレス（0x00，0x04，0x08，...）を列挙してコアレッシングをシミュレータに任せるか，コアレッシングの結果，すなわちトランザクションごとに 1 つのベースアドレスとサイズを格納するかである．後者ははるかに小さいが，トレースを収集したマシンのトランザクションサイズを固定してしまう．

注意 コアレッシング済みのトレースは 1 つのマイクロアーキテクチャに縛られる．別のラインサイズやトランザクションサイズのキャッシュを調べるには，新しいトレースを収集しなければならない．

8.2.3 キャッシュ階層とセクタキャッシュ

コアレッシングの後，要求はキャッシュ階層に入り，それは CPU の場合とほぼ同じようにモデル化される．アドレスは TLB で変換され（ミス時にはページテーブルウォーク，第 7 章），続いてモデル化するマイクロアーキテクチャに応じて L1 キャッシュと L2 キャッシュにアクセスする．異なるのは，その前段にあるコアレッシングの重要性と，キャッシュ自体の 1 つの特徴である．

定義 8.6 (セクタキャッシュ). **セクタキャッシュ**は，各キャッシュブロックを複数のセクタに分割する．各セクタは固有の有効ビットを持ち，すべてがブロックの 1 つのタグを共有する．**セクタ**はミス時にメモリから埋められる単位であり，キャッシュはブロック全体ではなく，要求されたデータを含むセクタだけを取り込む（図 8.2）．

▷ sectored cache

▷ sector

コアレッシングされたロード R1（連続する 8 つの 4 B ワード，アドレス 0 から 31 の 32 B）では，従来型のキャッシュは 64 B のブロック全体を埋めるが，セクタキャッシュはデータを含む 32 B，すなわちセクタ 0 と 1 だけを埋める．節約は 2 倍で，取り込んだバイトはすべて使われる．コアレッシングされないロード R2 では，従来型のキャッシュは $8 \times 4 = 32$ B の有用なデータを届けるために $8 \times 64 = 512$ B を取り込み，セクタキャッシュは $8 \times 16 = 128$ B を取り込む．したがってセクタ化の利点は散在したアクセスに対するメモリ帯域幅の削減であり，複数のセクタで 1 つのタグを共有することで，大きなブロックのタグ記憶量が保たれる．代償は暗黙のプリフェッチが減ることである．セクタ単位で埋められる

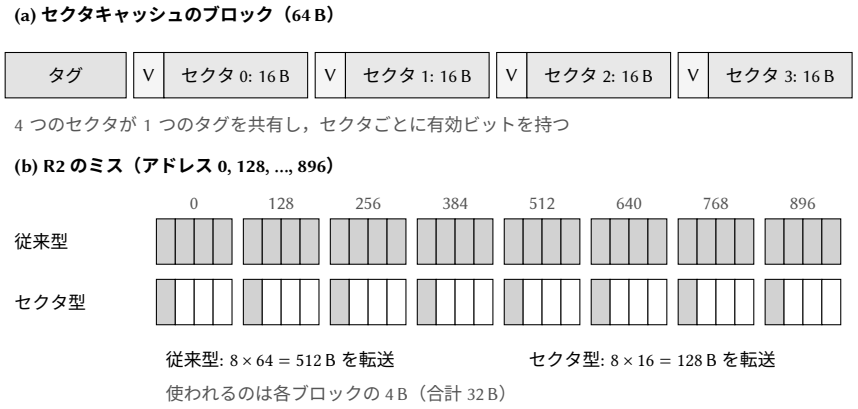


図 8.2: セクタキャッシュ（小さな例：64 B のブロック，16 B のセクタ）．(a) セクタは 1 つのタグを共有する．(b) コアレッシングされないロード R2 は 8 つのブロックでミスする．従来型のキャッシュは 8 つのブロック全体を埋め，セクタキャッシュはそれぞれ 1 セクタだけを埋める．

表 8.2: オープンソースの GPU シミュレータ（本章の分類による）．

シミュレータ	ISA	種類	対象
GPGPU-Sim [48]	NVIDIA PTX, SASS	実行駆動	汎用コードを実行する GPU
Accel-Sim [49]	NVIDIA PTX, SASS	トレース駆動	GPU とアクセラレータ
MGPUSim [50]	AMD GPU	実行駆動	複数の GPU
MacSim [51]	NVIDIA と Intel の GPU	トレース駆動	CPU と GPU のヘテロジニアスシステム
gem5-gpu [52]	AMD GPU または NVIDIA PTX	実行駆動	CPU と GPU のヘテロジニアスシステム

ブロックは，空間的局所性によってまもなく使われたはずの隣接データを持ち込まなくなる．

注 8.7. セクタキャッシュをシミュレーションすると，シミュレータが数えるものが変わる．ブロックが一部のセクタだけ有効な状態で存在しうるので，アクセスがタグにヒットしながらセクタではミスすることがある．タグのヒットだけを数えるシミュレータは，ヒット率を過大に，DRAM トラフィックを過小に見積もる．

注意 講義 vs ガイド：64 B のブロック中の 16 B のセクタは小さな例のサイズである．NVIDIA GPU は 128 B のラインに 32 B のセクタを用い，Compute Capability 6.0 以降，ワープのアクセスのコストは “the number of 32-byte transactions necessary to service all of the threads of the warp” である（BPG §10.2.1 [8]）．第 4 章はこれらのセクタを数えている．

8.2.4 オープンソースの GPU シミュレータ

表 8.2 は，研究で広く使われているオープンソースの GPU シミュレータを示す．それらは，モデル化する ISA，命令の得方，1 つの GPU を超えて何をモデル化するかで異なる．

GPGPU-Sim はこれらの中で最も広く使われている．gem5-gpu はそれを最も広く使われている CPU シミュレータ gem5 と組み合わせ，CPU と GPU がともに実行してメモリシステムを共有できるようにする．MGPUSim は AMD の GPU をモデル化し，8.4 節の Photon のサンプリング手法はその上に構築された．[1]

[1] GPGPU-Sim は ISPASS 2009 での CUDA ワークロードの研究とともに発表された [48]．表は本章の出典の分類に従っている．詳しく言えば，Accel-Sim はトレース駆動の SASS モードと並んで GPGPU-Sim の実行駆動の PTX モードを残しており [49]，gem5-gpu（“Gem5-GPGPU-Sim” とも記される組み合わせ）は gem5 を GPGPU-Sim の NVIDIA モデルと結合する．gem5 の AMD GPU モデルは別の開発である．MacSim の文書は，それを x86，ARM64，PTX，Intel GEN の命令に対して “either a trace driven or execution-driven cycle level simulator” として構成できるものと説明している [51]．

要点
GPU シミュレータはワープをそのアクティブマスクとともにモデル化する．SIMT スタックを自ら管理する（実行駆動）か，記録されたブロックとマスクを再生する（トレース駆動．トレースを取ったマシンのダイバージェンス処理に縛られる）．コアレッシングはトランザクション数を決め，モデル化すべき最も重要なものの 1 つである．コアレッシング済みのトレースはトランザクションサイズを固定する．セクタキャッシュはミスごとに 1 セクタを埋める．帯域は減るが，暗黙のプリフェッチも減る．

8.3 GPU の解析モデル

サイクルレベルシミュレータは，1 つの設計と 1 つのプログラムについての問いに，そのプログラムを実行して答える．解析モデルはそれに式で答える．

出典 M8，解析モデル；K12 ch. 2 Performance Principles，§4.1.1 Execution Time Modeling，§4.2.4 Roofline Model，§4.2.6 Other GPGPU Performance Modeling Techniques；[45, 46, 43, 47]．

▷ analytical model
▷ first-order model

定義 8.8 (解析モデル). **解析モデル**は，設計とプログラムの少数のパラメータから，プログラムを実行するのではなく方程式によって性能を予測する．**一次近似モデル**は，性能を支配する効果だけを残し，二次的な相互作用を無視する．

解析モデルは完全な実行を必要とせず，推論できるほど単純で，洞察を与える．設計が**なぜ**速いのか遅いのか，どのパラメータを変えるべきかを教えてくれる．その精度はシミュレータより低いので，設計空間を探索してその最良の点をシミュレーションする前段として，またシミュレーション結果を説明するために使われる．

8.3.1 GPU の一次近似設計

1 つの目標に対して GPU の規模を決める．1 秒あたり 10^{12} 回の内積ステップであり，各ステップは`sum += x[i] * y[i]` である．

定義 8.9 (融合積和演算). **融合積和演算（FMA）**は，$a \cdot b + c$ を 1 回の丸めで 1 つの命令として計算する．これは 2 回の浮動小数点演算（FLOP）として数える．

▷ fused multiply-add (FMA)

演算． 各ステップは 1 回の乗算と 1 回の加算なので，目標は 2 TFLOP/s，すなわち 1 秒あたり 10^{12} 回の FMA である．1 GHz ではこれに 1000 個の FMA ユニットが必要であり，2 GHz では 500 個である．

メモリ． 各ステップは x の要素を 1 つ，y の要素を 1 つ読む．この設計演習のように要素あたり 1 バイトと数えると，GPU には 2 TB/s のメモリ帯域幅が必要である．

ワープ． 500 個の FMA ユニットは，各レーンが 1 サイクルに 1 つの FMA を発行する幅 32 のワープ約 16 個（$16 \times 32 = 512$）に相当する．GPU は，それぞれが 1 サイクルに 1 ワープを発行する 16 個の SM としても，それぞれが 2 ワープを発行する 8 個の SM としても構成できる．どちらも 2 GHz である．

注意 講義 vs 実際：“ステップあたり 2 バイト” は 1 バイトの要素を前提としている．FP32 の要素ではステップは 2 FLOP に対して 8 B を読み，同じ設計には 8 TB/s が必要になる．これは H100 SXM5 の 3 TB/s（暫定値，[2]）の約 2.7 倍である．

8.3.2 マルチスレッディングの幅と深さ

SM が保持しなければならないワープの数は，2 つのパラメータから決まる．

定義 8.10 (W_{width} と W_{depth}). W_{width} は 1 サイクルに実行するスレッドの数である（たとえば 1 ワープの 32）．W_{depth} は，1 つのワープがストールしている間に SM がスケジュールできるワープの数であり，これによってストールが隠される．SM は $W_{\text{width}} \times W_{\text{depth}}$ 個の常駐スレッドを保持する．

図 8.3 は，ワープが演算命令とメモリ命令を 1 つずつ交互に実行するプログラムについてこの考え方を示す．1 ワープでは，SM は 2 つの命令を発行した後，メモリ命令が完了するまでアイドルになる．2 ワープでは，2 つ目のワープがアイドルなスロットのうち 2 つを埋める．4 ワープではアイドルなスロットがなくなる．一般に，ストールは他のワープの仕事で覆われなければならない．各ワープが n 個の命令を発行した後，その最初の命令から L サイクル後まで再び発行できないならば，およそ

$$W_{\text{depth}} \approx \frac{L}{n} \tag{8.2}$$

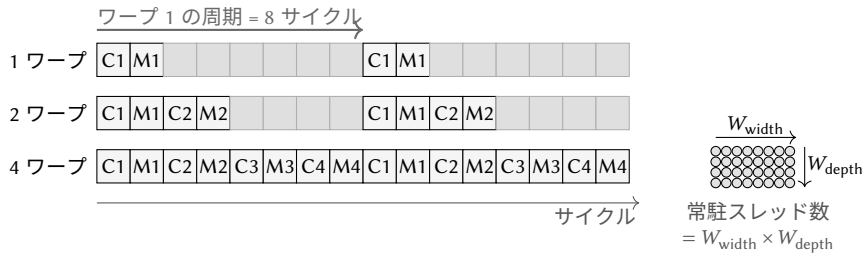


図 8.3: メモリのストールの隠蔽．各ワープは演算命令（C）とメモリ命令（M）を 1 つずつ発行し，演算命令の 8 サイクル後になって初めて再び発行できる．1 ワープでは発行スロットは 8 サイクル中 2 サイクル，2 ワープでは 8 サイクル中 4 サイクル埋まり，4 ワープでは常に埋まる．$W_{\text{depth}} = 4$ である．

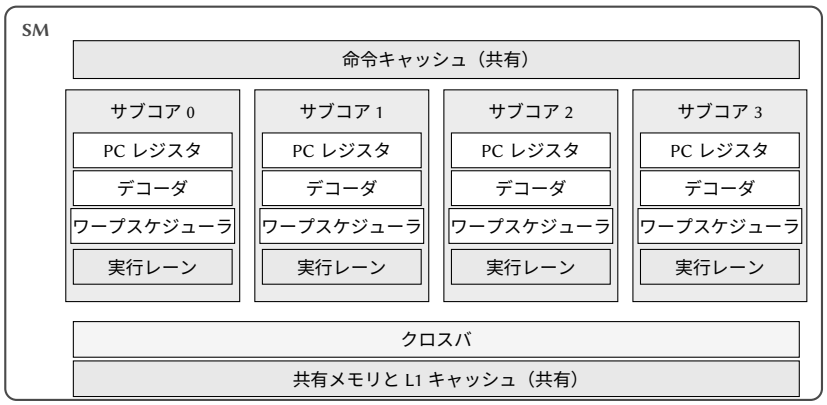


図 8.4: 4 つのサブコアに分割された SM．それぞれが固有のフロントエンドとレーンを持ち，命令キャッシュとデータキャッシュは共有される．

個のワープが必要である．メモリレイテンシが 200 サイクルで，ワープあたり演算命令 1 つとメモリ命令 1 つなら $W_{\text{depth}} = 200/(1+1) = 100$ であり，演算命令 4 つあたりメモリ命令 1 つなら $200/(1+4) = 40$ である．メモリアクセスあたりの演算が多いほど，レイテンシを隠すのに必要なワープは少ない．

どちらのパラメータもハードウェアによって決まる．W_{width} は ALU の数（とスケジューラの幅）で制限される．W_{depth} はワープの状態を格納する記憶容量で制限される．W_{depth} が 20 なら，スレッドあたり r 個のレジスタについて $20 \times 32 \times r$ 個のレジスタと，ワープごとに 1 つ，少なくとも 20 個のプログラムカウンタが必要である．レイテンシが数百サイクルのときの式 (8.2) は，GPU が SM あたり数十のワープを保持する理由と，そのレジスタファイルがこれほど大きい理由（第 6 章）を説明する．

ヒント ワープが 2 回の待ちの間に発行スロットを占めるサイクル数を数え，待ち時間をそれで割る．

実際の制限：SM あたりの常駐ワープは最大 64（Compute Capability 7.0，8.0，9.0），SM あたりの 32 ビットレジスタは 65 536 個なので，32 スレッドのワープ 64 個はそれぞれ最大 32 個のレジスタを得る（CPG Table 21 [5]）．

8.3.3 SM の数とサブコア

上の 2 つの設計，すなわち 1 サイクルに 1 ワープを発行する 16 個の SM と 2 ワープを発行する 8 個の SM は，実行帯域が同じであり，同じチップ面積なら合計のキャッシュ容量も同じである．違うのはその容量の分け方である．少数の大きな SM はアクセス時間の長い大きなキャッシュを持つが，1 つの SM 上のスレッドブロックはより多くのデータを共有し，それがヒット率を上げうる．多数の小さな SM は小さく速いキャッシュを持つ．どちらがよいかはブロックがどれだけデータを共有するかに依存し，このトレードオフは一次近似モデルでは決着がつかない．より細かいモデルかシミュレーションが必要である．

定義 8.11 (サブコア). **サブコア**は SM をいくつかに分割した区画の 1 つであり，それぞれが固有のプログラムカウンタ，デコーダ，ワープスケジューラ，実行レーンを持ち，SM の命令キャッシュと，クロスバを介してその大きなキャッシュと共有メモリを共有する（図 8.4）．

▷ sub-core

サブコアは 2 つの設計の中間の方法である．各サブコアは自分のレーン上で少数のワープをスケジュールし，大きな構造は共有されたままである．モデルに

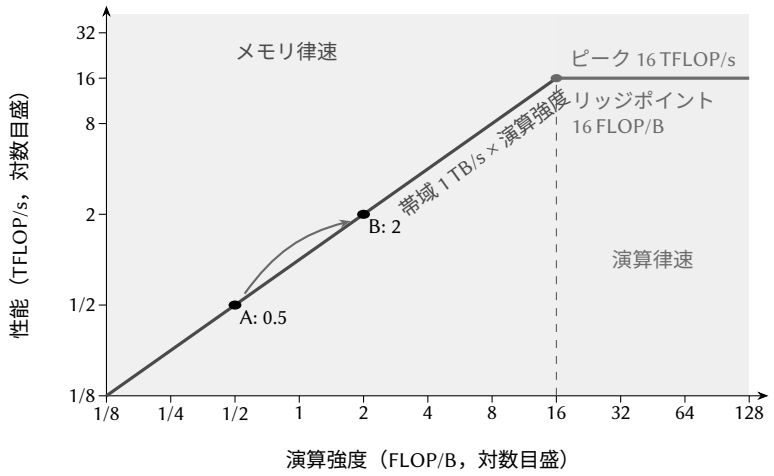


図 8.5: 16 TFLOP/s と 1 TB/s のマシンのルーフラインモデル（リッジポイントは 16 FLOP/B）．カーネル A（0.5 FLOP/B）は最大でも 0.5 TFLOP/s しか達成できない．演算強度を 2 FLOP/B に上げると（カーネル B），上界は 4 倍になる．

とっては資源管理が加わる．異なるサブコア上の 2 つのワープは，共有キャッシュとクロスバを奪い合うが，発行スロットは奪い合わない．

現在の NVIDIA の SM は 128 個の FP32 レーンを 4 つの区画に分割しており [2]，各サブコアは 32 レーンに対して発行する．

> **要点**
> 一次近似設計は目標スループットから出発する．FLOP 数が（与えられたクロックでの）FMA ユニットの数を決め，FLOP あたりのバイト数がメモリ帯域幅を決め，隠すべきレイテンシが $W_{\text{depth}} \approx L/n$，したがって SM あたりのレジスタとプログラムカウンタを決める．

8.3.4 ルーフラインモデル

一次近似設計は，プログラムがどれだけの演算とどれだけの帯域を必要とするかを問うた．ルーフラインモデルは逆の問いを立てる．マシンが与えられたとき，その 2 つのうちどちらがプログラムを制限するか，である．

定義 8.12 (演算強度). プログラムまたはカーネルの**演算強度** は，それがアクセスするメモリ 1 バイトあたりに行う浮動小数点演算の数であり，単位は FLOP/B である．

▷ arithmetic intensity

定義 8.13 (ルーフラインモデル). **ルーフラインモデル**は，ピーク演算性能 P_{peak} とピークメモリ帯域幅 B_{peak} を持つマシン上で，演算強度 I のカーネルが達成できる性能を

$$P(I) = \min\bigl(P_{\text{peak}},\ B_{\text{peak}} \cdot I\bigr) \tag{8.3}$$

で抑える．両対数軸では，この上界は傾き 1 の斜めの線とそれに続く平らな屋根になる（図 8.5）．2 本の線が交わる演算強度 $I^* = P_{\text{peak}}/B_{\text{peak}}$ を**リッジポイント**と呼び，その左のカーネルは**メモリ律速**であって，性能は帯域で制限される．その右のカーネルは**演算律速**であり，ピークの FLOP レートで制限される．

▷ Roofline model

▷ ridge point

▷ memory-bound

▷ compute-bound

このモデルを導入した Williams らは，演算強度をキャッシュの後で測る．彼らの**演算強度（DRAM バイト当たり）**は DRAM トラフィック 1 バイトあたりの演算数を数えるので，データがキャッシュにとどまるカーネルでは，operational intensity が arithmetic intensity より高くなる [45].

▷ operational intensity

一次近似設計の単位，すなわち要素あたり 1 バイトでの 2 つの例が，カーネルがチャート上をどう動くかを示す．ベクトル和 `z[i] = x[i] + y[i]` は，2 つのロード（それぞれ 1 バイト）だけを数えると 2 バイトあたり 1 FLOP を行う．その演算強度は 0.5 FLOP/B で，カーネルはメモリ律速である．内積の変形 `sum += x[i]*x[i]*y[i]*y[i]` は同じ 2 バイトを読むが，4 FLOP（3 回の乗算と 1 回の加算）を行い，演算強度は 2 FLOP/B になる．チャート上で右へ，演算律速の領域に向かって動く．このモデルの実用上の用途は，カーネルに改善の余地

注意 講義 vs 論文：本章は講義と同様に，プログラムが要求するバイト数を数える．論文は “the term operational intensity to mean operations per byte of DRAM traffic” を用い，上界を “Attainable GFlops/sec = Min(Peak Floating Point Performance, Peak Memory Bandwidth x Operational Intensity)” と述べている [45].

があるかを判断することである．屋根のはるか下にあるカーネルは別の何か（レイテンシ，ダイバージェンス，バンク競合）に制限されており，屋根の上にあるカーネルは演算強度を上げるか，別のマシンに移ることでしか改善できない．屋根はマシンごとに 1 回描かれ，“not once per kernel” である [45]．各カーネルは演算強度軸上の 1 点になる．

実際の GPU のリッジポイント（FP16 テンソルのピークを HBM 帯域で割ったもの）：V100 138.9 FLOP/B [54]；A100 312/1.555 ≈ 201；H100 SXM5 1000/3 ≈ 333（暫定値 [1, 2]）．

例 8.14. 図 8.5 の我々のマシンは $P_{\text{peak}} = 16\,\text{TFLOP/s}$，$B_{\text{peak}} = 1\,\text{TB/s}$ なので，$I^* = 16/1 = 16\,\text{FLOP/B}$ である．FP32 の内積は FMA（2 FLOP）あたり 8 B を読むので $I = 2/8 = 0.25\,\text{FLOP/B}$ であり，達成できるのは最大でも $1 \times 0.25 = 0.25\,\text{TFLOP/s}$，ピークの約 1.6 % である．上の 4 FLOP の変形は FP32 で 0.5 FLOP/B に達する．大きなタイル化行列積のように $I = 32\,\text{FLOP/B}$ のカーネルは $\min(16, 32) = 16\,\text{TFLOP/s}$，すなわちピーク全体を達成する．

8.3.5 CPI モデル

解析モデルの第 2 の系統は，命令あたりの時間から出発する．

定義 8.15 (CPI)．プロセッサ上のプログラムの**命令当たりサイクル数（CPI）**は，それにかかるサイクル数を実行する命令数で割ったものであり，第 1 章の IPC の逆数である．

▷ cycles per instruction (CPI)

最も単純な CPI モデルは，理想的なパイプラインの CPI に，あらゆる種類の乱れの事象のコストを，その発生頻度で重み付けして加える．

$$\text{CPI} = \text{CPI}_{\text{steady}} + \sum_i f_i \cdot p_i, \tag{8.4}$$

ここで f_i は命令あたりの種類 i の事象の数，p_i はその事象 1 回のペナルティ（サイクル）である．このモデルはペナルティが重ならない，すなわち直列化されると仮定する．

注意 直列化された和は，ペナルティを互いに，また有用な仕事と重ね合わせるアウトオブオーダプロセッサやマルチスレッドプロセッサに対しては悲観的である．

例 8.16. 5 段のインオーダーパイプラインでは $\text{CPI}_{\text{steady}} = 1$ である．分岐予測ミスのコストは 3 サイクルで命令の 2 %で発生し，キャッシュミスのコストは 5 サイクルで 5 %で発生するとする．

$$\text{CPI} = 1 + 0.02 \cdot 3 + 0.05 \cdot 5 = 1 + 0.06 + 0.25 = 1.31.$$

和の各項は別々に示す価値がある．時間がどこに使われているかを示すからである．

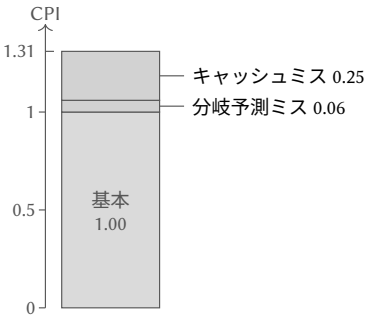


図 8.6: 例 8.16 の CPI スタック．

定義 8.17 (CPI スタック)．**CPI スタック**は，CPI の成分を積み上げた棒グラフである．基本（定常状態）の成分と，事象の種類ごとに 1 つの成分からなる（図 8.6）．

▷ CPI stack

ヒント IPC は CPI の逆数である．CPI 1.31 は IPC $1/1.31 \approx 0.76$ である．

この例では，キャッシュミスが時間の $0.25/1.31 \approx 19\,\%$ を，予測ミスが 5 % 未満を占める．よりよい予測器よりよいキャッシュの方が効果が大きい．

マルチスレッドプロセッサは，あるスレッドのストールを他のスレッドの仕事と重ね合わせる．理想的には，W_{depth} 個のスレッドでストールはそれらの間で分割される．

$$\text{CPI}_{\text{MT}} \approx \frac{\text{CPI}_{\text{single}}}{W_{\text{depth}}} + \text{contention terms}. \tag{8.5}$$

競合項（contention terms）は，多くのスレッドが動いているときに足りなくなる共有資源を表す．ビジーな実行ユニット，DRAM 帯域，そして未処理のミスを追跡するレジスタである．

定義 8.18 (MSHR)．**MSHR**（miss status holding register）は，未処理のキャッシュミス 1 つ（ミスしたブロックと，それを待つ要求）を記録するエントリであり，これによってキャッシュはミスの処理中にも他のアクセスに応じ続けられる．MSHR の数が，同時に処理中のミスの数を制限する．

▷ MSHR (miss status holding register)

したがって MSHR は第 4 章のメモリレベル並列性に上限を課す．たとえば 32 個の MSHR（説明のための数値）では，準備のできたワープがいくつあっても，33 個目の未処理ミスは 1 つが戻るまでストールする．

注 8.19. (8.5) は，すべてのスレッドの命令を数えた，プロセッサ全体の命令あたりサイクル数として読む．理想項は発行の限界（1 サイクルに 1 命令を発行するパイプラインでは 1）を下回れない．例 8.16 の CPI 1.31 では，2 スレッドですで

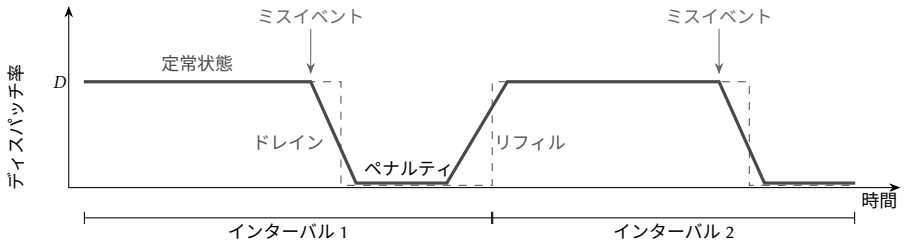


図 8.7: インターバル解析．各インターバルは実効ディスパッチ率 D の定常状態と，それに続く 1 つのミスイベントからなる．破線：単純なモデルの急停止．実線：ペナルティの前のパイプラインのドレインと後のリフィルを含む一次近似モデル．

にその限界に達する．GPU は多数のワープを持つマルチスレッドプロセッサなので CPI モデルが当てはまるが，競合項，特に MSHR と DRAM 帯域が支配的になる．

8.3.6 インターバル解析

CPI モデルは，各ペナルティを，それ以外は完璧なパイプラインに加わる固定サイクル数として扱う．インターバル解析は，実行の時間的な形に目を向ける．

定義 8.20 (インターバル解析). **インターバル解析**の手法は，実行をインターバルに分割する．各インターバルは，命令が**実効ディスパッチ率**でディスパッチされる定常状態と，それに続く 1 つの**ミスイベント**からなる．ミスイベントは長いレイテンシの事象（分岐予測ミス，キャッシュミスや TLB ミス）であり，そのペナルティの間は命令がディスパッチされない（図 8.7）．実行時間は定常状態の時間とペナルティの和である．

▷ interval analysis

▷ miss event

パイプラインは急に止まって急に再開するわけではない．ミスイベントが起こっても，すでに処理中の命令はパイプラインから流れ出ていく．それが解決した後も，ディスパッチが定常の速さに達するまでにはパイプラインが再び満たされる時間が必要である．Karkhanis と Smith の一次近似スーパースカラモデルはこの**ドレイン**と**リフィル**の段階を加える．その形は事象の種類による．予測ミスは分岐が実行されて初めて検出され，長いキャッシュミスはリオーダバッファの先頭を塞ぎ，バッファが満杯になるとディスパッチを止める [47].

Eeckhout [55] はインターバル解析と一次近似モデルを，ミスイベントの種類ごとのペナルティも含めて詳しく述べている．

GPU に適用するとき，素朴な方法は GPU をもう 1 つのマルチスレッドプロセッサとして扱う．それでは GPU を最もよく特徴づける 2 つの効果を取り逃がす．

分岐ダイバージェンス． ワープのすべてのレーンがアクティブとは限らず，ダイバージェント分岐の経路は直列化される（第 6 章）ので，ワープ命令は一定量の仕事を表さない．

メモリダイバージェンス． ワープのメモリ命令のレイテンシは，そのアクセスがコアレッシングされるかどうかに依存する．コアレッシングされないアクセスは多数のトランザクションを生成し，その最後のものを待つ（第 4 章）．

より新しい GPU のインターバルモデルは，サブコア，セクタキャッシュ，その他の共有資源の競合を加えている．

GPUMech はインターバル解析を GPU に適用し，第 7 章のラウンドロビンと greedy-then-oldest のスケジューラを，シミュレーションの 97 倍の速さで 13.2 % と 14.0 % の誤差でモデル化し，CPI スタックを生成する [43].

注 8.21 (メモリと計算の並列性による GPU モデル). 本ユニットの任意の読み物の 1 つである Hong と Kim の GPU の解析モデルは，2 つの量に基づく [46]．**メモリワープ並列度**（MWP）は，1 つのワープのメモリ待ち期間中に SM あたり同時にメモリにアクセスできるワープの最大数である．それはレイテンシ，連続する要求の間の発行間隔（departure delay），帯域，アクティブなワープ数 N で制限される．

▷ memory warp parallelism (MWP)

$$\mathrm{MWP} = \min(\mathrm{MWP}_{\text{without BW}}, \mathrm{MWP}_{\text{peak BW}}, N)$$

であり，ここで $\mathrm{MWP}_{\text{without BW}} = \mathit{Mem_L}/\mathit{Departure_delay}$，$\mathrm{MWP}_{\text{peak BW}}$ はピーク帯域を，1 ワープが必要とする帯域とアクティブな SM の数の積で割ったものである．**計算ワープ並列度**（CWP）は，1 回のメモリ待ち期間中に SM が実行できるワープの数に 1 を加えたものである．$\mathrm{CWP} = \min\big((\mathit{Mem_cycles} + \mathit{Comp_cycles})/\mathit{Comp_cycles},\ N\big)$．$\mathrm{CWP} \geq \mathrm{MWP}$ ならば，他のワープの計算はメモリシステムが重ね合わせられる以上を覆う．カーネルはメモリ律速であり，そ

▷ computation warp parallelism (CWP)

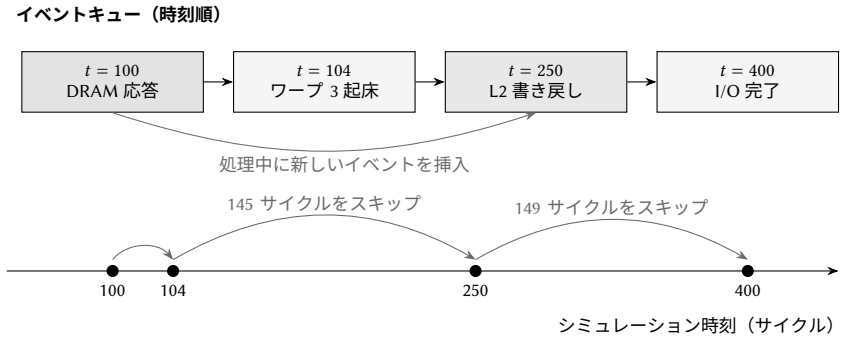


図 8.8: イベント駆動シミュレーション．イベントキューはタイムスタンプ順に並び，イベントの処理は新しいイベントを挿入しうる．シミュレーション時刻はイベントからイベントへ跳び，その間のサイクルを飛ばす．

の時間は N/MWP 回のメモリ期間に比例して増える．MWP > CWP ならメモリレイテンシは隠され，時間はおよそメモリレイテンシ 1 回分と $N \times Comp_cycles$ の和である．

Hong と Kim は，マイクロベンチマークで 5.4 %，アプリケーションで 13.3 % の幾何平均誤差を報告している [46]．

> **要点**
> 解析モデルは精度と引き換えに洞察を得る．ルーフラインはカーネルの演算強度をマシンのリッジポイントと比べ，演算律速かメモリ律速かを判定する．CPI モデルは定常状態の CPI に頻度で重み付けしたペナルティを加え，CPI スタックは時間がどこに使われるかを示す．インターバル解析はこれをドレインとリフィルで精密にし，GPU 版は分岐ダイバージェンスとメモリダイバージェンスを加えなければならない．

8.4 GPU シミュレーションの高速化

詳細なシミュレーションは遅い．各サイクルに各ユニットをモデル化するサイクルレベルシミュレータは，1 秒あたりシミュレーション上の命令千個程度（1 KIPS）の速さで動く．その速さでは，プロセッサが約 1 秒で実行する 10^9 個の命令のシミュレーションに約 28 時間かかり，実ハードウェアで数時間動く機械学習のワークロードには 1 世紀程度かかる．出口は 2 つある．シミュレータを速くするか，シミュレーションする量を減らすかである．

出典 M8，シミュレーションの高速化；Eeckhout [55]（サンプリング）；[44, 53]．

注意 講義 vs 計算：$10^9/10^3 = 10^6$ s は約 11.6 日であり，28 時間ではない．28 時間は 10 KIPS（10^5 s）に相当する．1 KIPS では，ネイティブ実行 1 時間分に約 3600×11.6 日 $\approx$ 114 年かかり，これが 1 世紀である．

8.4.1 より速いシミュレータ

並列シミュレーション． シミュレータは複数のホストスレッド，たとえばシミュレーションするコアごとに 1 スレッドで動かせる．コアはほぼ独立しているが，メモリシステムとネットワークオンチップは共有されている．スレッドは相互作用のたびに通信するか，共有モデルへのアクセスを直列化しなければならない．ほとんどのワークロードはメモリ律速なので，こうした相互作用は頻繁で，メモリモデルが並列シミュレータのボトルネックになる．

CPU シミュレータの速さ：gem5，Flexus，MARSS は詳細なモデルで約 200 KIPS に達する．zsim は詳細なアウトオブオーダコアを 20 MIPS で，1024 コアのチップを 16 コアのホスト上で最大 300 MIPS でシミュレーションする [56]．

イベント駆動シミュレーション．

定義 8.22 (イベント駆動シミュレーション). **イベント駆動シミュレーション** では，シミュレータはタイムスタンプ付きの将来のイベントのキューを時刻順に保持する．最も早いイベントを取り出し，シミュレーション時刻をそのタイムスタンプまで直接進めて処理することを繰り返す．処理は新しいイベントを挿入しうる（図 8.8）．

▷ event-driven simulation

イベント間のサイクルにはコストがかからないので，イベント駆動シミュレーションはイベントがまばらなとき，たとえばレイテンシが長いときに効果を発揮する．数千サイクルかかる I/O 操作や不揮発性メモリへの書き込みは，数千回のティックではなく 1 つのイベントになる．

ヒント サイクルレベルシミュレーションのコストはサイクルごとに，イベント駆動シミュレーションのコストはイベントごとにかかる．忙しいパイプラインはほぼ毎サイクルにイベントがあるのでコアの得るものは小さく，レイテンシの長いデバイスが最も得をする．

より単純なモデル． すべてのイベントが同じように重要なわけではない．シミュレータは重要でないイベントには平均のレイテンシやスループットを使い，問いが必要とするところだけ詳細を保つことができる．メモリ階層を調べるなら，コアを発行幅に等しい IPC で命令を発行するだけのものに縮め，キャッシュとメモリだけをモデル化すればよい．多コア CPU の高速シミュレータである zsim は，詳細なアウトオブオーダモデルと並んで，そのような単純なコア（1 サイクルに 1 命令）を提供する [56]．コアを調べるなら，メモリに固定のレイテンシを与えればよい．

残る 2 つの高速化，サンプリングと統計モデルが，本節の残りの主題である．

注意 どの単純化も相互作用を 1 つ取り除く．たとえば固定のメモリレイテンシは，帯域を大量に消費する設計が生むキューイング遅延，まさに (8.5) の競合項を隠してしまう．

8.4.2 サンプリング

定義 8.23 (サンプリングシミュレーション). **サンプリングシミュレーション** では，プログラムの実行のうち選んだ部分（サンプル）だけを詳細にシミュレーションし，それらから全体の性能を推定する．サンプルはランダムに選んでも，プログラムを解析して選んでもよい．

▷ sampled simulation

サンプルの間の部分も何らかの方法で飛ばさなければならない．実行駆動シミュレータは，タイミングなしで機能的に実行する（はるかに速い）ことでそこを早送りするか，各サンプルを**チェックポイント**から開始する．チェックポイントは保存されたアーキテクチャ状態（レジスタとメモリ）であり，そこからそれ以前の部分を実行せずにシミュレーションを再開できる．トレース駆動シミュレータは，単にサンプルのトレースだけを収集する．

▷ checkpoint

サンプルは現実的なマイクロアーキテクチャ状態から始まるわけではない．そのキャッシュと分岐予測器は空であるか，前のサンプルが残した内容を保持している．サンプルの**ウォームアップ**は，その前にシミュレーションされ，タイミングは捨てられる実行区間であり，その間にこれらの構造が満たされる．あるいは，コールドスタートが無視できるほどサンプルを長くする．

▷ warm-up

ランダムなサンプルで正確な推定を得るには多くの抽出が必要である．しかしプログラムはランダムではなく，同じ振る舞いを繰り返す．

定義 8.24 (プログラムフェーズと SimPoint). **プログラムフェーズ**は，振る舞い（IPC，ミス率）が均質な実行区間であり，何度も繰り返されうる．**SimPoint** は，プログラムの実行のインターバルをそれが実行するコードによってクラスタリングすることでプログラムのフェーズを識別し，フェーズごとに 1 つの代表インターバルを，そのフェーズが実行に占める割合で重み付けして選ぶ（図 8.9）[53]．各インターバルは**基本ブロックベクトル**，すなわちその中で各基本ブロックが何回実行されたかで記述され，ベクトルは *k*-means クラスタリングでまとめられる．

▷ program phase
▷ SimPoint

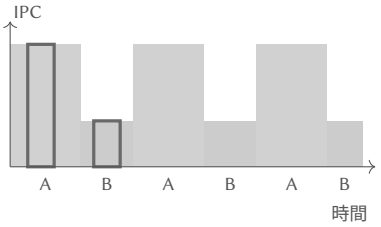


図 8.9: IPC の高いフェーズ A と低いフェーズ B を交互に繰り返すプログラムの IPC の時間変化．フェーズに基づくサンプリングは，それぞれの代表 1 つ（枠で囲んだもの）をシミュレーションする．

少数のフェーズが数百回繰り返されるなら，各フェーズの 1 インターバルをシミュレーションして重み付けすることで，完全なシミュレーションの精度の大部分を，そのコストのわずかな割合で得られる．

8.4.3 GPU でのサンプリング

GPU の 1 スレッドは短すぎてその内部でサンプリングすることはできないが，GPU のプログラムは同じコードを実行する非常に多くのスレッドからなり（SPMD，第 1 章），別の粒度が使える．

ブロック単位． グリッド全体の数千のスレッドブロックの代わりに，100 個程度のスレッドブロックをシミュレーションする．

カーネル単位． 数十のカーネルの代わりに 1 つか 2 つのカーネルをシミュレーションする．

ワープ単位． カーネルのより少ないワープを詳細にシミュレーションする．

定義 8.25 (カーネル単位サンプリング). **カーネル単位サンプリング** は，異なるカーネルごとに，あるいは類似した起動のグループごとに 1 回の起動を詳細にシミュレーションし，その時間を他の起動に用いる．

▷ kernel-level sampling

カーネル単位サンプリングは，層と反復ごとに 1 回ずつ同じカーネルを数百回起動する機械学習のワークロードで特に効果的である．本ユニットの指定の読み

物の 1 つである Photon は，カーネル，ワープ，基本ブロックの 3 つのレベルのサンプリングを組み合わせ，事前にワークロードをプロファイルする代わりに軽量なオンライン解析によって反復的な振る舞いを見つける [44].

Photon は ResNet-152 の 1 回の推論（バッチサイズ 1）のシミュレーションを 7.05 日から 1.7 時間に短縮し，サンプリング誤差は 10.7 % である．MGPUSim の上に構築されている [44].

8.4.4 ワークロードの縮小

もう 1 つの出口は，シミュレーションする仕事を減らすことである．

- **反復を減らす**：機械学習の訓練の反復は似た振る舞いをするので，1000 回ではなく 1 回の訓練反復をシミュレーションする．
- **入力を小さくする**：10^9 ノードではなく 10^6 ノードのグラフ．
- **ホットなコードだけ**：数百の関数のうち 1 つか 2 つが時間の 90 % を占めるなら，それらだけをシミュレーションする．

定義 8.26 (マイクロベンチマークと代表ワークロード). **マイクロベンチマーク**は，マシンの 1 つの機能（あるキャッシュレベル，あるメモリアクセスパターン，ある命令のレイテンシ）を単独で動かすために書かれた小さなプログラムである．**代表ワークロード** は，調べる機能について振る舞いが完全なワークロードと一致するように作った縮小版のプログラムまたは入力である．

注意 小さな入力は振る舞いを短くするだけでなく変えることがある．縮小したサイズでは L2 キャッシュに収まるワーキングセットが完全なサイズでは収まらないことがあり，縮小した実行は実際の実行のメモリ律速な振る舞いを隠しうる．

▷ microbenchmark
▷ representative workload

8.4.5 データ駆動モデル

最後の方法はマシンをまったくモデル化しない．

定義 8.27 (データ駆動モデル). **データ駆動モデル**は，ハードウェアの動作を組み込むことなく，データに当てはめた統計モデルや機械学習モデルによって，プログラムの特徴量（命令数，ロード数，ダイバージェント分岐の数，...）から性能を予測する．

▷ data-driven model

手順は 3 段階である．一連のプログラムをハードウェア（または詳細なシミュレータ）で実行してその性能，たとえば IPC を記録する．ハードウェア性能カウンタまたはシミュレータから特徴量を抽出する．そしてモデルを当てはめる．そのようなモデルの単純な形は

$$cycles = C_1 N_{\text{inst}}^{e_1} + C_2 N_{\text{FP}}^{e_2} + C_3 N_{\text{mem}}^{e_3} \tag{8.6}$$

である．ここで N_{inst}，N_{FP}，N_{mem} は全命令，浮動小数点命令，メモリ命令の数であり，係数 C_k と指数 e_k はデータに当てはめて決める．

ハードウェアカウンタは Nsight Compute（第 5 章）が読むものである．種類ごとの実行命令数，メモリトランザクション，分岐ダイバージェンスなど．

注意 解析モデルはマシンの理解から作られ，データ駆動モデルはデータだけから作られる．それは当てはめに使ったものと似たプログラムに対しては正確だが，その外では何の保証も与えず，洞察もほとんど与えない．

> **要点 — 章のまとめ**
> サイクルレベルシミュレータはクロックを進めて各サイクルに各ユニットをモデル化し，実行またはトレースによって駆動される．パイプラインは深さと幅を持つ FIFO キューの集合とスケジューラであり，GPU シミュレータはワープ，そのマスクまたは SIMT スタック，コアレッシング，セクタキャッシュをモデル化する．解析モデルは一次近似の答えを与える．設計の FMA ユニット，帯域，$W_{\text{depth}} \approx L/n$，リッジポイントを持つルーフラインの上界 $\min(P_{\text{peak}}, B_{\text{peak}} \cdot I)$，CPI スタック，分岐ダイバージェンスとメモリダイバージェンスを含むインターバル解析である．約 1 KIPS のシミュレーションは実際のワークロードには遅すぎる．並列シミュレーションとイベント駆動シミュレーション，単純化したモデル，サンプリング（フェーズ，SimPoint，ブロック単位，カーネル単位，ワープ単位），縮小したワークロード，データ駆動モデルによって，それを扱える規模にする．

第 9 章

マルチ GPU

講義動画	M9，レッスン 1–2（Multi-GPU Hardware；Concurrency on GPUs）
資料	Arunkumar ら [57]（指定）；NVIDIA NVLink ページ [58]，GPUDirect RDMA ドキュメント [59]，MPS 概要 [60]，Yu ら [61]，Ye ら [62]（任意）；NVIDIA A100 [1] および H100 [2] ホワイトペーパー
教科書	A18 [4] ch. 5；PMPP [6] ch. 20；CPG [5] §3.2.8 Asynchronous Concurrent Execution，§3.2.9 Multi-Device System
キーワード	マルチ GPU，モノリシック GPU，マルチチップモジュール，GPU モジュール，NUMA，近接メモリと遠隔メモリ，ファーストタッチ配置，ページマイグレーション，スレッドブロックスケジューリング，NVLink，NVSwitch，RDMA，GPUDirect RDMA，利用率，マルチテナンシ，MPS，MIG，カーネルの並行実行，CUDA グラフ

これまでの章では 1 つの GPU を扱ってきた．そのプログラミングモデル（第 3 章と第 5 章），SM とメモリシステム（第 4 章と第 6 章），そして仮想メモリとワープスケジューリング（第 7 章）である．多くのワークロード，とりわけデータセンタでの大規模モデルの学習は，1 つのチップが提供するより多くの計算能力を必要とする．本章は GPU をどのように組み合わせるかを問う．前半では，1 つのダイからパッケージ内の複数のダイへ，ノード内の複数の GPU へ，そしてネットワーク上の複数のノードへとハードウェアをたどる．メモリは不均一になり，スレッドブロックがどこで実行され，そのページがどこに置かれるかが問題になり始める．また，PCIe では運びきれない通信を新しいインターコネクト（NVLink，NVSwitch，RDMA）が担う．後半では逆の問いを立てる．1 つの GPU が 1 つのジョブにとって大きすぎるとき，複数のジョブでそれをどう共有できるか．ストリーム，Multi-Process Service，Multi-Instance GPU は 3 つの答えであり，GPU のどこまでを分割するか，どれだけの分離を与えるかが異なる．第 2 章の分散メモリプログラミングは，最大の規模で再び現れる．

9.1 マルチ GPU のハードウェア

出典 M9，マルチ GPU のハードウェア；A18 ch. 5；[57]．

計算能力への需要は 1 つの GPU の成長より速く伸びており，GPU はチップの数を増やすことでそれに応えてきた．本節ではこのスケーリングの段階，それが生むメモリ構成，そこから生じる 2 つの配置の判断，そしてノード内とノード間で GPU を結ぶインターコネクトをたどる．

9.1.1 1 つのチップを超えるスケーリング

より速い GPU への第 1 の道は，1 つのダイにより多くのコアを載せることである．

定義 9.1 (モノリシック GPU，マルチ GPU). **モノリシック GPU** とは，1 つのダイとして作られた GPU である．すべての SM，キャッシュ，メモリコントローラが 1 枚のシリコン上にある．**マルチ GPU** システムは，1 つのパッケージ内，1 枚のボード上，あるいは複数のボードにまたがる複数の GPU チップを組み合わせ，1 つのワークロードに取り組ませる．

▷ monolithic GPU

データセンタ向け GPU 当たりの SM 数：56（P100），80（V100），108（A100），132（H100 SXM5，暫定値）．トランジスタ数は同じ 4 世代で 153 億から 800 億へ増えた（AWP Table 4 [1]；HWP Table 3 [2]）．

▷ multi-GPU

第 1 章の各 GPU 世代はモノリシックなダイに SM を追加してきた．図 9.1 は，その道がどこで終わるかを示す．ダイが大きいほどコストは上がり，その**歩留まり**，すなわち製造されたダイのうち動作するものの割合は下がる．どのダイも致命的な欠陥を含む可能性が高くなるからである．さらにリソグラフィがダイサイズの絶対的な上限を定める．そこで次の段階ではダイを小さく保ち，それらを複数つなぐ．複数のソケットに置いた別々の GPU チップとするか，1 つのパッケージ内のシリコンインターポーザ（第 4 章）上に複数のダイを載せるかであり，いずれも高帯域のスイッチで結ぶ．最後の段階では，GPU ボード全体を PCIe でつなぐ．

データセンタ向けのダイはすでに大きい：815 mm^2（V100），826 mm^2（A100），814 mm^2（H100）（AWP Table 4 [1]；HWP Table 3 [2]）．Arunkumar らは，128 SM を超える GPU は 1 つのダイとしては作れないと仮定している [57]．

定義 9.2 (マルチチップモジュール，GPU モジュール，チップレット). **マルチチッ**

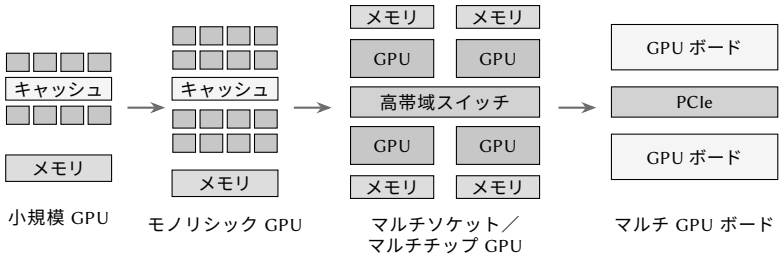


図 9.1: GPU スケーリングの 4 段階．小規模な GPU；1 つのダイにより多くのコアを載せたモノリシック GPU；それぞれ独自のメモリを持ち，高帯域スイッチで結ばれた複数の GPU チップ（マルチソケットまたはマルチチップ）；PCIe で結ばれた複数の GPU ボード．

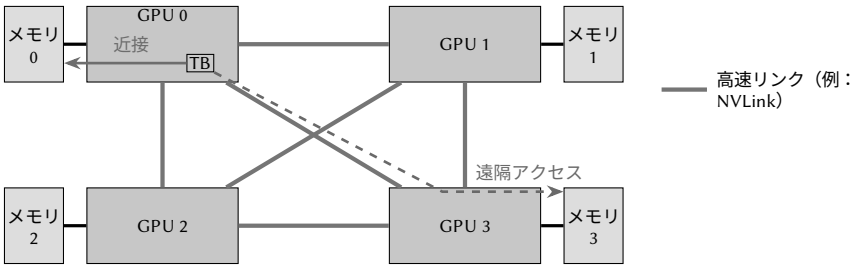


図 9.2: 4 つの GPU からなる NUMA システム．各 GPU は独自のメモリを持ち，6 本のリンクがすべての組を結ぶ．GPU 0 上のスレッドブロックは自身のメモリには直接到達し（近接アクセス），GPU 3 のメモリにはリンクを介して到達する（遠隔アクセス）．後者は帯域幅が低く，レイテンシが高い．

プモジュール（MCM）とは，パッケージ上の短いリンクで結ばれた複数のダイを収めるパッケージである．Arunkumar らの MCM-GPU では，各ダイは **GPU モジュール**（GPM），すなわち独自の SM，L2 スライス，DRAM パーティションを持つより小さな GPU であり，GPM のパッケージはソフトウェアからは 1 つのより大きな論理 GPU に見える [57]．このように他のダイと組み合わせることを前提に設計されたダイを**チップレット**という．

▷ multi-chip module (MCM)

▷ GPU module (GPM)

▷ chiplet

論文はこれを “より大きな論理 GPU を構築するための，複数の GPU モジュールのパッケージレベルの統合” と呼び，各 GPM が良い歩留まりで製造できるほど小さく保たれるので，“ムーアの法則を超えた継続的な性能スケーリング” が可能になると論じている [57]. [1]

[1] 論文で調べられた MCM-GPU は，64 SM の GPM を 4 つ（256 SM）持ち，DRAM 帯域幅は合計 3 TB/s，ローカルパーティション当たり 768 GB/s である．その最適化版は “実装可能な最大のモノリシック GPU より 45.5 % 高速” と報告され，作ることのできない同じ規模のモノリシック GPU との差は 10 % 以内，同じ SM 数と DRAM 帯域幅を持つマルチ GPU システムより 26.8 % 高速である [57]．

9.1.2 不均一メモリを持つ 1 つの論理 GPU

プログラマから見ると，高速リンクで結ばれた複数の GPU のパッケージやノード（図 9.2）も依然として多数の SM であり，すべてのメモリにグローバルメモリとして，すなわち 1 つの共有アドレス空間として到達できる．各 GPU は独自の DRAM を持ち，それは通常，SM が要求する帯域幅のための HBM スタック（第 1 章）である．一方，システムの残りへの I/O 帯域幅はすべての GPU で共有される．変わるのはアクセスのコストである．

定義 9.3 (NUMA，近接メモリと遠隔メモリ). 1 つのアドレス空間が複数のメモリに分散しており，アクセスのレイテンシと帯域幅が，そのアドレスを保持するメモリとアクセスを発行するプロセッサの位置関係によって決まるとき，そのメモリシステムは**不均一メモリアクセス（NUMA）**を持つという．2 つの場合をそれぞれ**近接メモリ／遠隔メモリ** と呼ぶ．近接メモリはアクセスを発行する GPU に接続されたメモリ，遠隔メモリは別の GPU に接続されたメモリであり，後者へのアクセスは GPU 間リンクを渡らなければならない．

▷ non-uniform memory access (NUMA)

▷ near memory / far memory

例 9.4 (近接と遠隔を数値で). H100 上のカーネルが 1 GiB（1.074 GB）のバッファを 1 回読む．暫定値 3 TB/s の自身の HBM3 からなら，転送には少なくとも 0.36 ms かかる．バッファがピア GPU 上にあれば，NVLink 4 を介して 1 方向当たり 450 GB/s，すなわち合計 900 GB/s の半分で届き，少なくとも 2.4 ms かかる．1 方向当たり

64 GB/s の PCIe Gen 5 では少なくとも 16.8 ms かかる [2]．遠隔アクセスは，レイテンシの差を数える前から，NVLink では近接アクセスの 6.7×，PCIe では 47× 遅い．

例 9.4 の数値はピーク帯域幅である．実際の転送はそれより低い値にしか達せず，小さな遠隔アクセスが続くパターンではリンクのレイテンシも効いてくる．

プログラムにとって NUMA が意味するのは絶対値ではなく比である．データはそれを使う SM の近くにあるべきである．ここから 2 つの問いが生じ，本章ではそれをスレッドブロックとページについて問う．

9.1.3 ブロックをどこで実行し，ページをどこに置くか

定義 9.5 (スレッドブロックスケジューリング，ファーストタッチ配置)．カーネルの各スレッドブロックをどの GPU（または GPM）の，どの SM で実行するかの決定を**スレッドブロックスケジューリング** という．**ファーストタッチ配置** では，ページは，それに最初にアクセスしたスレッドブロックの GPU の近くのメモリに割り当てられる．

▷ thread block scheduling
▷ first-touch page placement

ファーストタッチはプログラムについての知識を必要としない．ページは最初に求められた場所に置かれる．各ページが同じ GPU 上のブロックによって使われる限りはうまく働くが，別の場所で実行される後のブロックが同じページを必要とすると破綻する．ブロックがラウンドロビンで GPU ごとに 1 つずつ配られ，各ブロックがファーストタッチによって自分のページを自分の GPU のメモリに持ち込むとする．後で GPU 0 にスケジュールされたブロックが GPU 3 上のブロックとページを共有していると，そのページはすでに GPU 3 のメモリに置かれており，それへのアクセスはすべて遠隔アクセスになる（図 9.2）．対策は 2 つある．データがある GPU にブロックをスケジュールするか，データを移すかである．

定義 9.6 (ページマイグレーション)．プログラムの実行中にページをある GPU のメモリから別の GPU のメモリへ移すことを**ページマイグレーション** という．通常は，そのページがもう一方の GPU から，コピーのコストを償うのに十分な数の遠隔アクセスを受けたときに行う．

▷ page migration

マイグレーションは第 7 章のページ移動のマルチ GPU 版である．第 7 章ではユニファイドメモリがページフォールトを契機にホストとデバイスの間でページを移す．Li らはそのようなマルチ GPU システムのアドレス変換を調べている [37]．

マイグレーションはリンクを介したページ全体のコピーとページテーブルの更新からなる．そのため，最初の遠隔アクセスでマイグレーションするポリシは，2 つの GPU が交互に同じページを使う場合，益より害をもたらしうる．

例 9.7 (ラウンドロビンとグループ化したブロックスケジューリング)．8 つのスレッドブロック TB0–TB7 からなるカーネルが 4 つの GPU 上で，GPU 当たり 2 ブロックずつ，2 つのウェーブで実行される．ブロック TBi はページ Pi を読み，次にページ P(i+1) を読む．隣り合うブロックが境界のページを共有する 1 次元ステンシルのようなものである．ページはファーストタッチで配置され，ウェーブ内ではブロックが番号順に自分のページに触れる．表 9.1 は 2 つのスケジュールをたどる．ラウンドロビンは TBi を GPU i mod 4 に置くので，隣り合うブロックは常に異なる GPU 上にあり，16 回のページアクセスのうち 7 回が遠隔になる．連続するブロックをグループ化し，TB0 と TB1 を GPU 0，TB2 と TB3 を GPU 1，というように置くと，共有ページが遠隔になるのは 2 つのグループの境界だけであり，16 回のうち 3 回になる．

例 9.7 のグループ化したスケジュールは，論文の分散スケジューラの考え方である．連続するブロックはデータを共有しやすいので，同じ GPU を共有すべきである．これはカーネルの起動をまたいでも効く．ループの中で再び起動されたカーネルは同じブロック番号を同じ GPU に送り，それらの GPU のメモリにはファーストタッチで置かれたページがすでにある [57]．

注意 講義 vs 論文：本章は講義と同様に，問題をファーストタッチ配置と局所性を考慮したブロックスケジューリングに帰着させている．MCM-GPU の論文は 3 つの最適化を組み合わせて提案している．遠隔データだけを保持する GPM 側の “L1.5” キャッシュ，連続する CTA のグループを 1 つの GPM に送る “分散かつバッチ化した” CTA スケジューリング，そしてファーストタッチによるページ割り当てである．組み合わせると，基本の MCM-GPU に対して 22.8 % の高速化と GPM 間通信の 5× の削減が得られる [57]．

9.1.4 NVLink と NVSwitch

遠隔アクセスのコストは GPU 間のリンクで決まる．

定義 9.8 (インターコネクト，NVLink，NVSwitch)．**インターコネクト** とは，プロセッサとメモリがデータをやり取りするリンク（とスイッチ）のネットワークである．NVIDIA の **NVLink** は GPU 間のポイントツーポイントのインターコネクトとプロトコルであり，GPU 間の通信において PCIe に代わるものである．

▷ interconnect
▷ NVLink

世代ごとの GPU 当たりの NVLink：900 GB/s，18 リンク（第 4 世代，Hopper）；1800 GB/s，18 リンク（第 5 世代，Blackwell）；3000 GB/s，36 リンク（第 6 世代，Vera Rubin，暫定値）．NVLink Switch のドメインは 8 GPU，NVL72 ラックでは 72 GPU に及ぶ [58]．

表 9.1: 例 9.7 のトレース．各項目はブロック，その GPU，2 回のページアクセスである．“遠隔” は，それより前のブロックがすでに別の GPU に置いたページを示す．

	ラウンドロビン		グループ化	
ウェーブ	ブロック，GPU	ページ	ブロック，GPU	ページ
1	TB0，G0	P0 新規，P1 新規	TB0，G0	P0 新規，P1 新規
	TB1，G1	P1 **遠隔**，P2 新規	TB2，G1	P2 新規，P3 新規
	TB2，G2	P2 **遠隔**，P3 新規	TB4，G2	P4 新規，P5 新規
	TB3，G3	P3 **遠隔**，P4 新規	TB6，G3	P6 新規，P7 新規
2	TB4，G0	P4 **遠隔**，P5 新規	TB1，G0	P1 近接，P2 **遠隔**
	TB5，G1	P5 **遠隔**，P6 新規	TB3，G1	P3 近接，P4 **遠隔**
	TB6，G2	P6 **遠隔**，P7 新規	TB5，G2	P5 近接，P6 **遠隔**
	TB7，G3	P7 **遠隔**，P8 新規	TB7，G3	P7 近接，P8 新規
遠隔アクセス：16 回中 7 回			16 回中 3 回	

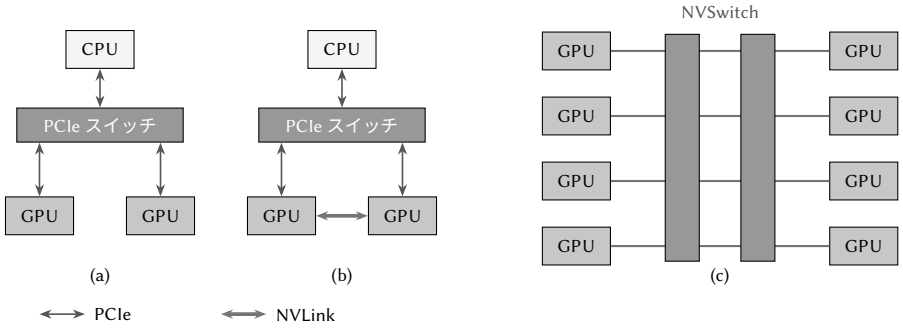


図 9.3: (a) CPU と共有する PCIe スイッチを介してしか通信できない 2 つの GPU．(b) 同じ GPU を直接の NVLink で結んだもの．(c) 8 つの GPU がそれぞれすべての NVSwitch チップにつながり，GPU のどの組もスイッチを介して結ばれる．

NVSwitch は多数の NVLink ポートを持つスイッチチップである．GPU の間に NVSwitch を置くと，ノード内のすべての GPU が他のすべての GPU にリンクの全帯域幅で到達できる．

▷ NVSwitch

NVLink がなければ，2 つの GPU の間のデータは CPU と共有する PCIe スイッチを渡る（図 9.3a）．直接の NVLink はこれを迂回する（b）．H100 の第 4 世代 NVLink は 18 リンクで GPU 当たり 900 GB/s を提供し，ホワイトペーパーはこれを PCIe Gen 5 の帯域幅の 7× としている [2]．H100 の HBM3 帯域幅 3 TB/s と比べると，NVLink はローカルメモリ帯域幅の 30 % を提供する（両方向を数えた場合）．しかしポイントツーポイントのリンクはスケールしない．GPU が持つリンクの数は決まっており，n 個の GPU のすべての組を直接つなぐには $n(n-1)/2$ 本のリンクが要る．NVSwitch チップがこれを解決する（図 9.3c）．各 GPU は自分のリンクをスイッチに分散させ，スイッチは GPU のすべての組を，直接リンクの連鎖よりも高い帯域幅と低いレイテンシで結ぶ．

このように，ノード内の GPU は 1 つの NUMA マシンを成す．アドレス空間は 1 つで，近接メモリと遠隔メモリがある．

注意 講義 vs 実際：講義はこれを約 4 分の 1 としている．1 方向当たりでは NVLink 4 は 450 GB/s（25 GB/s のリンク 18 本）を運び，HBM の 15 % である [2]．

第 5 章のユニファイド仮想アドレッシングにより，`cudaDeviceEnablePeerAccess` で PCIe または NVLink 上のピアツーピアアクセスを有効にすれば，ある GPU 上のカーネルがピア GPU のメモリを指すポインタを参照外しできる（CPG §3.2.9.4 Peer-to-Peer Memory Access）．

9.1.5 ボードをまたぐ：RDMA

異なるボードや異なるノード上の GPU は PCIe を介してそれぞれのホストに接続されており（図 9.4），CPU を経由する PCIe には，それらの間でメモリを共有するのに十分な帯域幅がない．それらのメモリは別々の空間であり，システムは第 2 章と同じ分散メモリシステムである．データは明示的なメッセージ，ここでは RDMA によって移動する．

従来のネットワーキングでは，バッファを TCP/IP のソフトウェアスタックを通して送る（図 9.5a）．送信側の CPU はアプリケーションのバッファからオペレーティングシステムを経てネットワークへデータをコピーし，受信側の CPU はそれをさらに宛先のバッファへコピーする．両方の CPU がメッセージごとに

プログラムがこれらの転送を自ら発行することはまれである．NCCL（第 5 章）のような集団通信ライブラリが，GPU の組ごとに NVLink，NVSwitch，GPUDirect RDMA のいずれかを選ぶ．

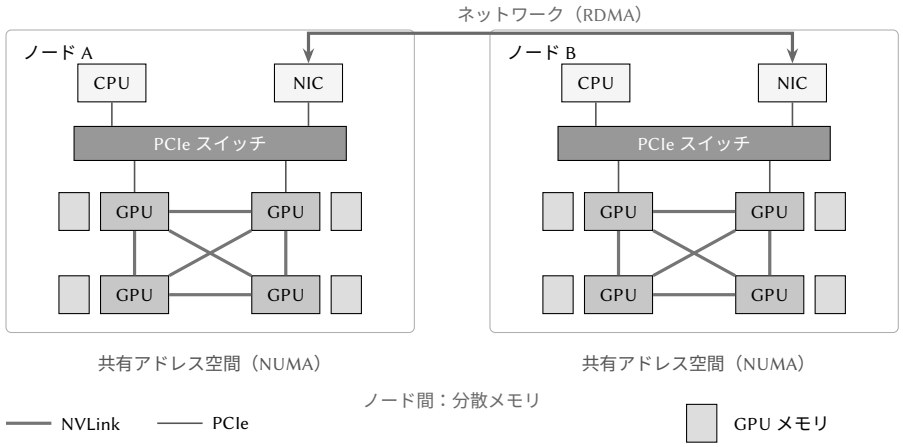


図 9.4: 2 つのマルチ GPU ノード．ノード内では NVLink が 4 つの GPU を 1 つの NUMA アドレス空間にまとめる．CPU とネットワークインタフェースカードは PCIe スイッチ上にある（GPU との接続は 2 本だけ描いた）．ノード間ではメモリは別々であり，データは RDMA によってネットワークを渡る．

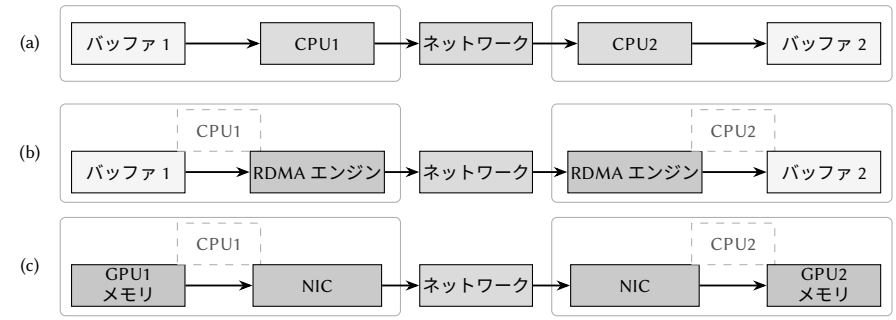


図 9.5: (a) 従来のネットワーキング：データは両方の CPU を通る．(b) RDMA：RDMA エンジンがデータをメモリからメモリへ移し，CPU（破線）は経路に加わらない．(c) GPUDirect RDMA：NIC が GPU メモリを直接読み書きするので，GPU のバッファはホストメモリを経由するコピーなしにリモートの GPU に届く．

サイクルを費やし，CPU が忙しければ転送は遅れる．

定義 9.9 (RDMA，ホストバイパス，NIC). **ネットワークインタフェースカード**とは，ホストをネットワークに接続するデバイスである．**リモートダイレクトメモリアクセス（RDMA）**と呼ばれる技術は，第 7 章の DMA をネットワークをまたいで拡張する．一方の NIC の RDMA エンジンが送信元のバッファをメモリから読み，もう一方の NIC の RDMA エンジンがそれを宛先のメモリに書き込み，どちらの CPU もデータをコピーしない（図 9.5b）．このような転送は**ホストバイパス**の技術である．ホストプロセッサとそのソフトウェアスタックはデータの経路から外される．

▷ network interface card (NIC)

▷ remote direct memory access (RDMA)

▷ host bypass

定義 9.10 (GPUDirect RDMA). NVIDIA の技術である **GPUDirect RDMA** は，GPU メモリと，NIC やストレージアダプタのようなサードパーティの PCIe デバイスとの間に，“PCI Express の標準機能を用いて” 直接のデータ経路を提供する [59]．これにより GPU のバッファは，ホストメモリに中継されることなく，ネットワークを通してリモートの GPU や別のデバイスへ送られる（図 9.5c）．

▷ GPUDirect RDMA

2 種類のリンクは協調して働く．ノード内では NVLink と NVSwitch が GPU 間の通信を運び，ノード間では GPUDirect RDMA がそれをネットワーク越しに移す．PCIe だけでは，これほど多くの GPU がデータを交換する需要を満たせない．

注意 GPUDirect RDMA では，GPU とサードパーティのデバイスが “同じ上流の PCI Express ルートコンプレックスを共有する” 必要がある．これは Kepler 世代の GPU と CUDA 5.0 で導入された [59]．

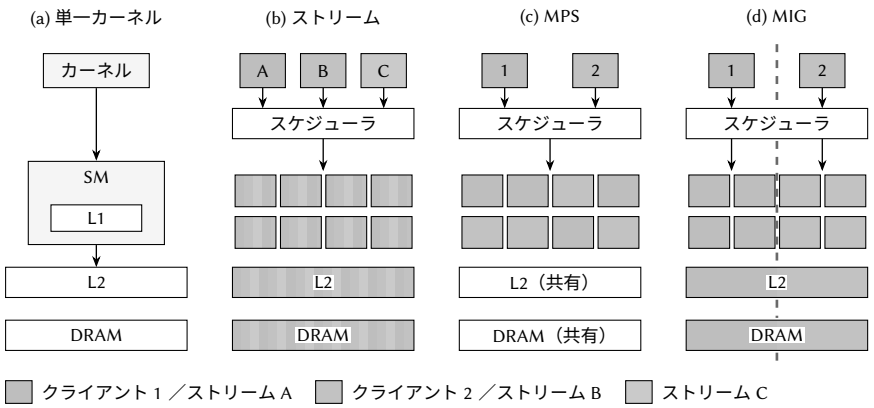


図 9.6: GPU の並行性の仕組み．(a) 1 つのカーネルが SM，L2，DRAM を使う．(b) 1 つのクライアントのストリーム：それらの仕事はすべての SM と L2，DRAM で混ざり合い，何も分割されない．(c) MPS：クライアントのブロックは SM 全体に分散する（ここではクライアントごとの SM の取り分として描いた）．L2 と DRAM は共有される．(d) MIG：SM，L2，DRAM がクライアントの間で物理的に分割される．

要点
1 つのダイを超えるスケーリングは，マルチチップモジュール，マルチ GPU ノード，クラスタへと進む．パッケージやノードの中ではメモリは 1 つの NUMA 空間であり，ブロックスケジューリングとページ配置が遠隔アクセスの数を決める．NVLink と NVSwitch は遠隔アクセスを安くする．ノード間ではメモリは分散しており，RDMA（GPU メモリには GPUDirect RDMA）が CPU を介さずにデータを移す．

9.2 GPU 上の並行性

大きな GPU が常によく使われているとは限らない．本節では複数のジョブが 1 つの GPU をどう共有できるかを問い，それを可能にする 3 つの仕組みを比較し，プログラムが複数のストリームと複数の GPU をどう駆動するかを示す．

出典 M9，GPU 上の並行性；CPG §3.2.8 Asynchronous Concurrent Execution，§3.2.9 Multi-Device System；[1]；[60]．

9.2.1 利用率とマルチテナンシ

定義 9.11 (利用率，データセンタ，マルチテナンシ). GPU の**利用率**とは，その資源（SM，メモリ帯域幅，時間）のうち有用な仕事をしている割合である．**データセンタ**とは，多くの利用者が共有する多数のサーバからなる施設であり，**マルチテナンシ**とは，1 台の機械，ここでは 1 つの GPU を，複数の独立した利用者（テナント）のジョブで共有することである．

▷ utilization

▷ datacenter

▷ multi-tenancy

すべてのジョブが大きな GPU の全体を使えるわけではない．小さなバッチの推論サービス，開発セッション，小規模な学習ジョブは，SM の一部しか忙しくしないことがある．専用の GPU では残りは無駄になり，これは GPU が高価な資源であるデータセンタで最も問題になる．従来のデータセンタは，この規模のジョブを持つ多数のテナントにサービスを提供する．一部の AI ワークロードは振る舞いが異なる．大規模言語モデルの学習は，1 つのテナントのために GPU 全体を多数使い，そのようなジョブは最も遅いタスクと同じ速さでしか進まない．各ステップがそのすべての GPU を待つからである．どちらのパターンも，1 つの GPU に複数の仕事の流れを載せる方法を必要とする．

Ye らは GPU データセンタにおける深層学習ジョブのスケジューリングを概観し，“ビッグデータや HPC のワークロード向けに設計された従来の手法では，DL ワークロードが GPU 資源を使い切れるようにはできない” と述べている [62]．Yu らはマルチテナントの推論を概観している [61]．

9.2.2 3 つの仕組み

図 9.6 は 3 つの仕組みを，何を分割するかによって比較する．

ストリーム． 1 つのクライアント（1 つのプロセス）が複数の CUDA ストリーム（第 5 章）に仕事を発行する．並行に実行されるのは独立したストリームだけであり，それらのカーネルは SM，L2，DRAM のすべての資源を共有

する．

MPS．別々のプロセスである複数のクライアントが，異なる SM 上で同時に実行される．L2 キャッシュと DRAM 帯域幅は共有されたままである．

MIG．GPU は物理的に分割される．各クライアントは独自の SM，L2 スライス，DRAM 帯域幅を受け取る．

9.2.3 Multi-Instance GPU

定義 9.12 (Multi-Instance GPU，GPU インスタンス，QoS)．**マルチインスタンス GPU（MIG）**と呼ばれる技術は，1 つの GPU を複数の分離されたパーティションに分割する．それぞれが **GPU インスタンス**であり，独自の SM，L2 キャッシュスライス，メモリコントローラ，DRAM を持つより小さな GPU として振る舞う [1]．MIG はメモリ，L2 キャッシュのポート，DRAM 帯域幅，オンチップのクロスバのポートを分割するので，各インスタンスには**サービス品質（QoS）**が保証される．すなわち，他のインスタンスが何をしていても予測可能なスループットとレイテンシである．

▷ Multi-Instance GPU (MIG)

▷ GPU instance

▷ quality of service (QoS)



図 9.7: 7 つの MIG スライスの GPU 上の 4 つのアプリケーション（GPU インスタンスは 3，2，1，1 スライス）．メモリは簡略化した．A100 のメモリスライスは 8 つで，3 スライスのインスタンスは 4 つを得る [63, §6.2]．

こうして MIG は 1 つの大きな GPU を，メモリとコアを複数の利用者の間で分離した，複数のジョブの効率的な受け皿にする．A100 は “最大 7 つの別々の GPU インスタンスに安全に分割できる”[1]．これは，クラウド事業者が 1 つの GPU の一部を異なる顧客に貸し出すのに必要なことである（図 9.7）．分離の代償は硬直性である．インスタンスの大きさは固定されており，仕事が遊んでいる隣のインスタンスにあふれ出すことはできない．

A100 の GPU インスタンスは，14 SM の GPC 1 つと L2 スライス 10 個の 1 グループからなる “GPU スライス” で組み立てられ，GPU はこれを 7 つ持つ [1]．プロファイルは A100 40 GB で 1g.5gb から 7g.40gb まで [63, §6.2]．H100 の第 2 世代 MIG は 7 インスタンスを保ち，コンフィデンシャルコンピューティングを加えた [2]．

9.2.4 Multi-Process Service

定義 9.13 (タイムスライシング，空間分割，MPS)．**タイムスライシング**では，GPU は一度に 1 つのプロセス（1 つの CUDA コンテキスト）の仕事を実行し，プロセスの間で切り替える．**空間分割**では，複数のプロセスの仕事が，異なる SM のような GPU の互いに素な部分で同時に実行される．**マルチプロセスサービス（MPS）**は，複数のプロセスの CUDA の仕事を 1 つの GPU 上で並行に実行させる NVIDIA のランタイムサービスであり，クライアントごとに使用できる SM の上限を任意に設けられる．

▷ time slicing

異なる CUDA コンテキストのカーネルは，MPS なしでは並行に実行できない．“GPU は各コンテキストの進行を保証するためにタイムスライスを行うことがある”（CPG §3.2.8.2 Concurrent Kernel Execution）．V100 以降で空間分割をサポートする MPS を使うと，クライアントのカーネルは GPU を同時に共有するが，資源は厳密には分割されない．L2 と DRAM 帯域幅は共有されるので，あるクライアントの性能は他のクライアントの振る舞いに依存し，予測できないことがある．MPS は MPI プログラムを CUDA に移すことも容易にする．1 つのノード上の MPI ランクは別々のプロセスであり，MPS を使えばそれらのカーネルが一緒になって 1 つの GPU を埋められる．QoS が重要でない場合，MPS は MIG より高い利用率を達成できる．クライアントの取り分がスライス単位に切り上げられず，遊んでいる SM を仕事のある誰にでも与えられるからである．

注意 講義 vs ドキュメント：MPS は各クライアントに専用の SM を与えない．プロビジョニングの上限は取り分を制限するだけで何も予約せず，“異なる MPS クライアントコンテキストから起動されたカーネルは同じ SM 上で実行されうる”．クライアントが互いに素な SM に限られるのは，不均一な分割（たとえば 30 % と 70 %）か Ampere 以降の静的な SM パーティションの場合だけである [60, §1.1.2.2.5, §1.1.2.2.11]．

▷ spatial partitioning

▷ Multi-Process Service (MPS)

例 9.14 (3 つのジョブを収める)．3 つの推論ジョブが GPU の SM のそれぞれ 30 %，25 %，10 % を必要とする．7 つの等しいスライスを持つ MIG では，1 スライスは 1/7 ≈ 14.3 % なので，ジョブは 3，2，1 スライス（42.9 %，28.6 %，14.3 %）を必要とする．有用な仕事をしている SM は 65 % だけなのに 7 スライスのうち 6 つが占有され，20 % を必要とする 4 つ目のジョブはもう収まらない．MPS では各ジョブをその取り分に制限できる（CUDA_MPS_ACTIVE_THREAD_PERCENTAGE を 30，25，10 とし，サポートされるスレッド数に切り下げる）．何も予約されないので，残りの能力は他のクライアントのために使える状態のままである．その代償として，L2 と DRAM 帯域幅に加えて SM も共有し，どのジョブにも性能は保証されない [60, §1.1.2.2.5]．

注意 Volta より前にも，MPS はクライアントの仕事を 1 つのコンテキストにまとめるサーバを通して，クライアントを並行に実行させていた．Volta はハードウェアサポートを加え，“MPS クライアントプロセスは完全に分離された GPU アドレス空間を持つ” ことと “実行資源の制限付きプロビジョニング”，すなわちクライアントごとの SM の取り分（環境変数 CUDA_MPS_ACTIVE_THREAD_PERCENTAGE）を可能にした [60]．

注意 MPS の分離は第 2 の点でも限られる．Volta の MPS クライアントの 1 つで致命的な GPU フォールトが起きると，それが発生した GPU を共有するすべてのクライアントに “どのクライアントがエラーを起こしたかを示さずに” 報告され，それらのクライアントは終了しなければならない [60]．MIG ではフォールトはそのインスタンスの中にとどまる [1]．

9.2.5 ストリームに基づくプログラミング

1 つのプロセスの中では，ストリームが並行性の手段である．

定義 9.15 (カーネルの並行実行). 1 つの CUDA コンテキストの複数のカーネルを GPU 上で同時に実行することを**カーネルの並行実行** という．異なるストリームに発行されたカーネルは，資源が許せば並行に実行されうる．同じストリーム内のカーネルは順に実行される（CPG §3.2.8.2 Concurrent Kernel Execution）．

▷ concurrent kernel execution

V100，A100，H100 では最大 128 個のカーネルが同時に常駐できる（“maximum number of resident grids per device”，CPG Table 21 [5]）．

こうしてストリームは，カーネルの並行実行，ストリームをまたいだ通信と計算のオーバーラップ（第 5 章），そして各自のストリームを持つ複数の CPU スレッドによる仕事の投入を可能にする．次のループは，ジョブをストリームの数より多いチャンクに分け，チャンクを順番にストリームに配る．各反復は 1 つのチャンクの入力コピー，カーネル，出力コピーを発行してから次のチャンクへ進む．これは**深さ優先**の起動順序である．

```
// h_x, h_y pinned; d_x, d_y device buffers of n floats
const int nStreams = 4, nChunks = 16;
size_t chunk = n / nChunks;
size_t bytes = chunk * sizeof(float);
cudaStream_t st[nStreams];
for (int k = 0; k < nStreams; ++k)
  cudaStreamCreate(&st[k]);
for (int i = 0; i < nChunks; ++i) {    // depth-first
  cudaStream_t s = st[i % nStreams];
  size_t off = i * chunk;
  cudaMemcpyAsync(d_x + off, h_x + off, bytes,
                  cudaMemcpyHostToDevice, s);
  square<<<(chunk + 255) / 256, 256, 0, s>>>(
      d_x + off, d_y + off, chunk);
  cudaMemcpyAsync(h_y + off, d_y + off, bytes,
                  cudaMemcpyDeviceToHost, s);
}
cudaDeviceSynchronize();               // all streams
```

チャンク 0，4，8，12 はストリーム 0 を共有するので順に実行され，4 つのストリームどうしはオーバーラップする．このスケジューリングにはコストがある．操作ごとにドライバが起動を準備し，ハードウェアは同じストリームの操作間の依存関係を追跡しなければならない．短いカーネルでは，このオーバーヘッドが実行時間のかなりの割合を占めうる．

ヒント OpenMP プログラムはこのスタイルに自然に移植できる．反復が独立な並列ループは，各反復の仕事をそれぞれのストリームに発行するループになり，各 OpenMP スレッドが自分のストリームを駆動することもできる．

定義 9.16 (CUDA グラフ). **CUDA グラフ**とは，依存関係で結ばれた操作（カーネル起動，コピーなど）の集合であり，その実行とは別に定義される．グラフは一度構築され，事前に検証と準備が済んだ実行可能グラフにインスタンス化され，その後は 1 回の呼び出しで必要なだけ何度でも起動される（CPG §3.2.8.7 CUDA Graphs）．

▷ CUDA Graph

```
cudaGraph_t g;  cudaGraphExec_t ge;
cudaStreamBeginCapture(s, cudaStreamCaptureModeGlobal);
for (int k = 0; k < 20; ++k)           // 20 short kernels
  step<<<blocks, 256, 0, s>>>(d_u, n);
cudaStreamEndCapture(s, &g);           // record, not run
cudaGraphInstantiate(&ge, g, 0);       // set up once
for (int it = 0; it < 1000; ++it)
  cudaGraphLaunch(ge, s);              // one call each
cudaStreamSynchronize(s);
```

ストリームキャプチャは既存のストリームのコードをグラフに変える．`cudaStreamBeginCapture` と `cudaStreamEndCapture` の間では，ストリームに発行された仕事は実行されずに記録される．最初の起動の前に依存関係の連鎖全体がわかっているので，準備のコストはカーネルごとではなく一度だけ払えばよく，ランタイムはワークフローを全体として最適化できる（CPG §3.2.8.7）．

注意 ストリームキャプチャはレガシーのデフォルトストリームでは動作しない（CPG §3.2.8.7.3 Creating a Graph Using Stream Capture）．`s` はレガシーのデフォルトストリームであってはならず，作成したストリームか `cudaStreamPerThread` を使う．

9.2.6 複数の GPU のプログラミング

ホストプログラムはカレントデバイスを切り替えることで複数の GPU を駆動する．`cudaSetDevice(d)` の呼び出しは，呼び出したホストスレッドにとって

表 9.2: ストリーム，MPS，MIG の比較．

	ストリーム	MPS	MIG
分割の種類	なし	論理的	物理的
最大パーティション数	無制限	48（注意を参照）	7
SM の分離	なし	割合による（上限）	あり
メモリ帯域幅の分離	なし	なし	あり
性能の QoS	なし	部分的	あり
再構成	動的	プロセス起動時	アイドル時

GPU d をカレントにする．割り当てとカーネル起動はカレントデバイスに対して行われ，ストリームとイベントは，作成されたときにカレントだったデバイスに属する（CPG §3.2.9.2 Device Selection）．次のプログラムは配列をすべての GPU に分割する．これは第 2 章のデータ分割であり，GPU ごとに別々の割り当て，コピー，カーネル起動を行う．

h_x がページング可能なメモリなら，各 cudaMemcpyAsync はまず CPU 上の中継バッファにコピーし（第 5 章），GPU は 1 つずつ順に動き始める．ピン留めされたホストメモリならすべてのコピーがオーバーラップできる．

```
int nDev;
cudaGetDeviceCount(&nDev);
size_t part = n / nDev;                // n divisible
size_t bytes = part * sizeof(float);
float *d_x[16];                         // nDev <= 16
for (int d = 0; d < nDev; ++d) {
  cudaSetDevice(d);                     // select GPU d
  cudaMalloc(&d_x[d], bytes);           // on GPU d
  cudaMemcpyAsync(d_x[d], h_x + d * part, bytes,
                  cudaMemcpyHostToDevice);
  square<<<(part + 255) / 256, 256>>>(d_x[d], d_x[d],
                                      part);
}
for (int d = 0; d < nDev; ++d) {       // wait for all
  cudaSetDevice(d);
  cudaDeviceSynchronize();
}
```

カーネル起動と非同期コピーはすぐに戻るので，最初のループはどの仕事も終わらないうちにすべての GPU で仕事を開始する．その後 GPU は，それぞれ自分の部分について並列に実行される．各デバイスは独自のデフォルトストリームを持ち（CPG §3.2.9.3 Stream and Event Behavior），ストリームは複数の GPU と組み合わせられる．各 GPU は，それがカレントである間に作成された自分のストリームを受け取る．

DGX H100 ノードは，4 つの NVSwitch で結ばれた 8 つの H100 GPU と，8 つの 400 Gbit/s ConnectX-7 ネットワークアダプタを持つ（HWP App. A [2]）．

注意 cudaDeviceSynchronize が待つのはカレントデバイスだけである．起動ループの後に 1 回呼んでも最後の GPU しか待たない．ホストは上のように各デバイスを順に選択して同期しなければならない．

9.2.7 比較

表 9.2 は 3 つの仕組みをまとめたものである．これらは排他的ではない．1 つの MIG インスタンス内のプロセスもストリームを使え，MPS は MIG インスタンスの中でも動作する．その場合クライアント数の上限はインスタンスの大きさに比例して下がる [63, §7.2].

注意 講義 vs ドキュメント：MPS クライアント 48 は以前のドキュメントにあった Volta（CC 7.0）の上限であり，Volta より前は 16 だった [64]．現在の概要では，CUDA_DEVICE_MAX_CONNECTIONS が既定値の 2 のとき，デバイスおよびサーバ当たり最大 60 まで許され，これを引き上げると少なくなる [60, §1.1.2.2.6]．

ストリームは準備のコストがなく実行時に適応できるが，分離は与えない．MIG は最も強い分離を与えるが，パーティションを変更できるのは GPU がアイドルのときだけである．MPS はその中間にあり，SM の上限は通常クライアントプロセスの開始時に設定される（コンテキストごとの上限を後から設定することもできる [60, §1.1.2.2.5]）．

ヒント QoS が重要なら MIG，利用率が重要なら MPS，1 つのプロセス内の並行性にはストリームを使う．

要点 — 章のまとめ

モノリシック GPU はダイサイズと歩留まりによって制限されるので，GPU は GPU モジュールからなるマルチチップモジュール，マルチ GPU ノード，クラスタとしてスケールする．パッケージやノードの中ではメモリは近接メモリと遠隔メモリからなる 1 つの NUMA 空間であり，ファーストタッチ配置，局所性を考慮したスレッドブロックスケジューリング，ページマイグレーションがアクセスを近接に保ち，NVLink と NVSwitch は遠隔アクセスを PCIe より安くする．ノード間ではメモリは分散しており，RDMA（GPU のバッファには GPUDirect RDMA）が CPU を介さずにデータを移す．1 つの GPU の利用率を上げるには，ストリームが 1 つのプロセスの独立な仕事を分割なしに実行し，MPS が複数のプロセスを同時に実行してクライアントごとの SM の上限を任意に設けられるが L2 と DRAM は共有し，MIG が SM，L2，DRAM を QoS 付きの最大 7 つの分離されたインスタンスに分割する．CUDA グラフはストリームの起動ごとのオーバーヘッドを取り除き，`cudaSetDevice` は割り当て，コピー，起動のそれぞれについて GPU を選択する．

第 10 章

コンパイラの基礎 I

講義動画	M10，レッスン 1–5（GPU Compiler Flow；PTX；IR and Basic Block；Introduction to Data Flow Analysis；Example of Reaching Definitions）
資料	NVCC ドキュメント [65]；PTX ISA 9.4 [19]；Compiling CUDA with clang [66]；NVVM IR 仕様 [67]；Lattner と Adve [68]（LLVM）；LLVM Language Reference [69]
教科書	K12 [3] §1.3.7；CPG [5] §3.1 Compilation with NVCC；Aho ら [70] §8.4，§9.2
キーワード	コンパイラ，フロントエンド，バックエンド，中間表現，抽象構文木，nvcc，LLVM，NVPTX，PTX，SASS，仮想アーキテクチャ，ファットバイナリ，JIT コンパイル，ガードプレディケート，3 番地コード，リーダ，制御フローグラフ，データフロー解析，伝達関数，到達定義，gen 集合と kill 集合，不動点

これまでの章では，CUDA カーネルを GPU がそのまま実行するものとして扱ってきた．しかしソースファイルと SM の間にはコンパイラがあり，前の章で見たいくつかの仕組みは実はコンパイラの仕事であった．第 6 章のプレディケート付きコード，第 3 章で各スレッドが受け取るレジスタ，第 4 章の PTX の断片がそうである．本章と次章は，そうしたコードについて考えるのに必要なコンパイラの基礎を与える．前半では CUDA プログラムをコンパイルの流れに沿って追い，ソースコードから中間表現と PTX を経て GPU が実行する機械語に至るまでをたどり，PTX そのものを見る．後半では最適化が対象とする構造，すなわち 3 番地コード，基本ブロックと制御フローグラフ，そしてデータフロー解析を導入し，到達定義と呼ばれる解析で具体的に計算する．第 11 章は生存変数解析，SSA 形式，古典的な最適化，そしてこれらすべてをワープに結びつけるダイバージェンス解析へと続く．

10.1 GPU コンパイラの流れ

コンパイラは，ある言語で書かれたプログラム，すなわちソースプログラムを，別の言語，通常は機械の命令セットで書かれた等価なプログラムに翻訳する．コンパイラは 3 つの部分からなるパイプラインとして構成される（図 10.1）．

▷ compiler

出典 M10，GPU コンパイラの流れ；CPG §3.1 Compilation with NVCC；K12 §1.3.7；[65]．

定義 10.1 (フロントエンド，IR，バックエンド). **フロントエンド**はソースプログラムを読み，その構文と意味を検査し，**中間表現**に翻訳する．中間表現とは，ソース言語にもターゲットマシンにも属さないプログラムの形式である．**オプティマイザ**は，IR を同じ結果をより速く，あるいはより少ない命令で計算する IR に変換する．**バックエンド**は**コード生成**を行う．すなわち IR をターゲットマシンの命令に翻訳し，そのマシンに依存する最適化を施す．

▷ front end

▷ intermediate representation (IR)

オプティマイザはミドルエンドとも呼ばれる．LLVM ではそれはパスの列であり，各パスは IR から IR への変換である [68]．

▷ back end

▷ code generation

この分割を有用にしているのは IR である．言語ごとに 1 つのフロントエンドとターゲットごとに 1 つのバックエンドが単一のオプティマイザを共有するので，言語や機械を追加しても，組ごとに完全なコンパイラを 1 つずつ作るのでは

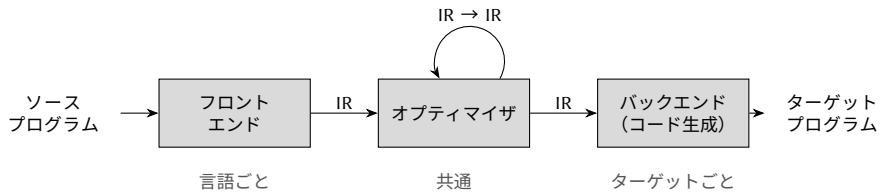


図 10.1: 古典的なコンパイラのパイプライン．フロントエンドはソースプログラムを IR に変換し，オプティマイザは IR をよりよい IR に書き換え，バックエンドはターゲットマシン向けのコードを生成する．

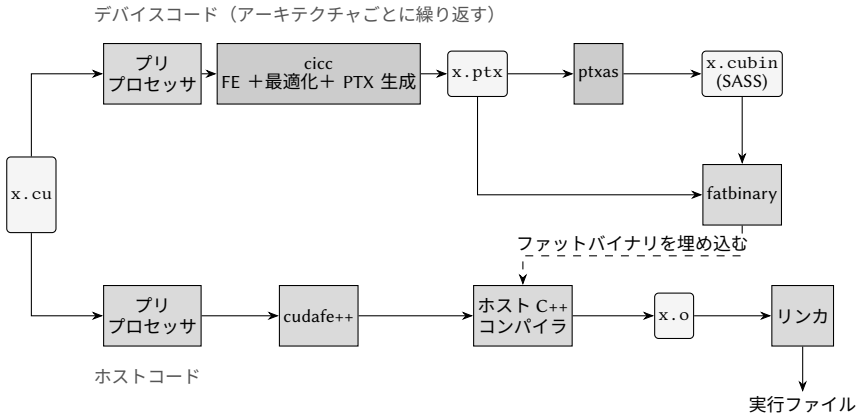


図 10.2: nvcc のコンパイル軌跡（簡略化）．デバイスコードはアーキテクチャごとに 1 回ずつ cicc と ptxas を通る．fatbinary は PTX と機械語（cubin）をファットバイナリにまとめ，ホストのオブジェクトがそれを埋め込む．デバイスリンカ nvlink は省略している．

なく，部品を 1 つ追加するだけで済む．IR は最適化にも適した水準である．ソース言語より単純でありながらどの機械にも依存せず，型やソース上の位置といったメタデータをフロントエンドから後段へ運ぶことができる．

L 個の言語と T 個のターゲットがあるとき，IR による分割では $L \cdot T$ 個の完全なコンパイラの代わりに $L + T$ 個のフロントエンドとバックエンドで足りる．

10.1.1 GPU プログラムの流れ

CUDA のソースファイルにはホストコードとデバイスコードが混在している（第 3 章）．そのため GPU コンパイラは，まず両者を分離し，それぞれを独自のパイプラインに通す．デバイスコードは GPU 向けにコンパイルされ，ホストコードは通常の C++ コンパイラでコンパイルされる．GPU 向けの結果はホストプログラムに格納され，プログラムの実行時にそれがロードされる．

NVIDIA のコンパイラドライバ **nvcc** は，この流れを図 10.2 の *CUDA* **コンパイル軌跡**（CUDA compilation trajectory）として実装している [65, §3]．入力ファイルは 2 回前処理される．デバイス側では，フロントエンド兼オプティマイザである `cicc` が `.ptx` ファイルを生成し，アセンブラ `ptxas` がそれを機械語の `.cubin` ファイルに翻訳する．この 2 段階は要求されたアーキテクチャごとに繰り返される．ツール `fatbinary` は PTX と `.cubin` の結果を 1 つのイメージにまとめる．ホスト側では `cudafe++` がホストコードを分離して CUDA 拡張を標準 C++ に書き換え，ホストの C++ コンパイラがデバイスイメージを埋め込んだそのコードをコンパイルする．最後にリンカが実行ファイルを生成する．デバイスコードを分割コンパイルする場合は，まずデバイスリンカ `nvlink` がすべての翻訳単位の `.cubin` ファイルをリンクする．

CPG §3.1.1.1 Offline Compilation [5]：nvcc はデバイスコードをホストコードから分離し，デバイスコードを PTX またはバイナリ形式にコンパイルし，カーネル起動の構文を CUDA ランタイムの呼び出しに置き換える．
▷ nvcc

定義 10.2 (仮想アーキテクチャ). **仮想アーキテクチャ**とは, PTX コードが対象とする GPU である. プログラムが使ってよい機能の集合であり, Compute Capability X.Y にちなんで `compute_XY` と名付けられる. `ptxas` のターゲットである実際の GPU は**実アーキテクチャ**（real architecture）であり，`sm_XY` と名付けられる．

ツール名は NVCC ドキュメントの軌跡の図による [65, §3]．`nvcc --dryrun` は実行するコマンドを一覧表示し，`--keep` は中間の `.ptx` ファイルと `.cubin` ファイルを残す（§4.2.5.9，§4.2.5.11）．
▷ virtual architecture

したがって nvcc のコマンドはどれも 2 つのアーキテクチャを指定する．第 1 段階のための仮想アーキテクチャと第 2 段階のための実アーキテクチャであり，実アーキテクチャは仮想アーキテクチャを実装していなければならない [65, §5.4]．仮想アーキテクチャはプログラムが必要とする機能を述べるものであり，プログラムが実行される GPU を固定するものではない．

ヒント 仮想アーキテクチャはできるだけ低く選び，PTX ができるだけ多くの GPU で動くようにする．実アーキテクチャはターゲットの GPU が許す限り高く選ぶ（NVCC §5.4 [65]）．

10.1.2 オープンソースのツールチェーン：Clang と LLVM

オープンソースの CUDA コンパイラにも同じ構造が現れる．

定義 10.3 (LLVM，Clang，NVPTX). **LLVM** は，型付きの 1 つの IR である LLVM IR を中心に構築されたコンパイラ基盤であり，その上の最適化のライブラリと多数のターゲット向けのバックエンドを備える [68]．**Clang** は，CUDA を含む

▷ LLVM

▷ Clang

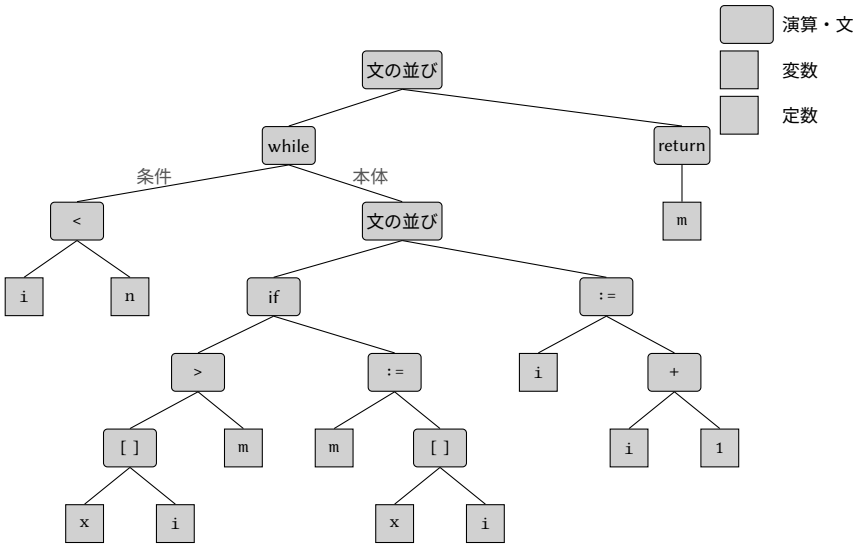


図 10.3: 例 10.5 のループの抽象構文木．演算と文は青，変数は緑，定数は赤で示す．

C 系言語のための LLVM のフロントエンドである．**NVPTX** は PTX を生成する LLVM のバックエンドである．

▷ NVPTX

スプリッタがホストコードとデバイスコードを分離する．デバイスコードは Clang, IR の最適化, NVPTX のコード生成器を通って PTX アセンブリとなり，ホスト側のコンパイルがそのデバイスコードをファットバイナリに埋め込む．[1]NVIDIA 自身のデバイスコンパイラも同じ技術の上に作られている．そのフロントエンドは *NVVM IR*，すなわち “LLVM IR に基づく” IR を出力し，それ自体 LLVM に基づく NVVM コンパイラがそこから PTX を生成する [67]．

フロントエンドはソースを構文解析して木にする．

[1] Clang は構文解析の前にファイルを分割しない．“merged parsing” を用い，すべてのホストコードとデバイスコードが両方のコンパイル段階に現れ，そのどちらでも意味的に正しくなければならない．nvcc と異なり，Clang は常に `ptxas` も実行し，要求された GPU ごとに機械語を埋め込む（`--offload-arch=sm_80`）．既定では PTX を埋め込まない．前方互換性のために `--cuda-include-ptx=sm_80`（または `all`）で追加できる [66]．

定義 10.4 (抽象構文木). **抽象構文木**（AST）は，プログラムの構造を木として表す．各内部ノードは演算または文（文の並び，ループ，分岐，代入，算術演算子）であり，その子はオペランドまたは部分文である．葉は変数と定数である．括弧やセミコロンのような区切り記号は現れない．木の形がすでにそれを表しているからである．

▷ abstract syntax tree (AST)

例 10.5 (ループの AST). 次のループは配列の現時点での最大値を保持する．

```
while (i < n) {
  if (x[i] > m) m = x[i];
  i = i + 1;
}
return m;
```

その AST（図 10.3）は，根に文の並びがあり，その子は `while` ループと `return` の 2 つである．ループのノードは条件 `i < n` と本体を持ち，本体はそれ自体 `if` 文とインクリメントからなる文の並びである．配列アクセス `x[i]` はそれぞれ，配列と添字を子に持つ添字ノードである．

ヒント `clang -fsyntax-only -Xclang -ast-dump file.c` は，Clang がファイルに対して構築する AST を表示する．

構文解析器が動く前に，C と C++ の**プリプロセッサ**が純粋なテキスト置換を行う．`#include` ファイルとマクロを展開し，条件付きコンパイルを評価する．次の定義があるとき，

▷ preprocessor

プリプロセッサはテキストに対して働くので，`a + b` というマクロ本体を `2 * M` で使うと `2 * a + b` になる．そのためマクロ本体は括弧で囲んで書く．

```
#define BLOCK_SIZE 256
int threads = BLOCK_SIZE;
```

コンパイラ本体が目にするのは `int threads = 256;` だけである．

10.1.3 PTX，SASS，ファットバイナリ

デバイス側のバックエンドの出力は，まだ機械語ではない．

定義 10.6 (PTX と SASS). **PTX**（Parallel Thread Execution）は NVIDIA の仮想 ISA である（第 1 章）．アーキテクチャに依存しない命令セットであり，そのプログラムは後で機械語に翻訳される．レジスタは仮想的であるため，PTX コードにはレジスタ割り付けがない．**SASS**（シェーダアセンブリ）は 1 つの GPU アーキテクチャの機械語であり，物理レジスタが割り付けられている．SASS は GPU の世代ごとに変わるが，PTX は互換性を保つ．古い PTX バージョン向けに書かれたコードは新しい GPU 向けにもコンパイルでき，新しい PTX バージョンは命令を追加していく．

▷ PTX (Parallel Thread Execution)

▷ SASS

PTX ISA は目的を直接述べている．“PTX は汎用の並列スレッド実行のための仮想マシンと ISA を定義する”．その目標には “複数の GPU 世代にわたる安定した ISA” が含まれる [19, §1.1–1.2].

例 10.12 の `.version` である PTX ISA 8.0 は CUDA 12.0 とともに登場した．PTX ISA 9.4 のリリース履歴は CUDA 1.0 の PTX ISA 1.0 から始まる [19, §13].

定義 10.7 (ファットバイナリ). **ファットバイナリ**とは，ホストコードに加えて，同じデバイスコードの複数の翻訳，すなわちいくつかの実アーキテクチャ向けの機械語といくつかの仮想アーキテクチャ向けの PTX を持つ実行ファイルである．実行時にデバイス部分はドライバに渡され，ドライバは存在する GPU に最も適した翻訳を選ぶ [65, §5.6.2].

PTX のレジスタはある型の変数であり，カーネルはいくつでも宣言できる．`ptxas` はそれらをスレッド当たり最大 255 個の物理レジスタに対応付け（CPG Table 21 [5]），残りをローカルメモリにスピルする [19, §5.1.1]．レジスタ割り付けは第 11 章で扱う．

▷ fat binary

ファットバイナリが存在する GPU 向けの機械語を含まない場合，ドライバは実行時に PTX をコンパイルする．

定義 10.8 (JIT コンパイル). **JIT コンパイル**（ジャストインタイムコンパイル）では，アプリケーションが PTX をロードするときにデバイスドライバがそれを機械語にコンパイルする．CPG §3.1.1.2 Just-in-Time Compilation は，これが “アプリケーションのコンパイル時に存在しなかったデバイスでアプリケーションを実行する唯一の方法” であると述べている [5].

▷ JIT compilation

JIT コンパイルは起動時間を要する．そのためドライバは各結果のコピーを**コンピュートキャッシュ**（compute cache）に保持する．このキャッシュはドライバの更新時に無効化されるので，新しい JIT コンパイラが古いアプリケーションを改善できる（CPG §3.1.1.2）．想定する GPU 向けの機械語も持つファットバイナリはこの遅延をまったく生じず，その PTX によってプログラムは後の GPU でも動き続ける．

コンピュートキャッシュは Linux では `~/.nv/ComputeCache` に置かれ，既定の容量は 1 GiB（組み込みプラットフォームでは 256 MiB），最大 4 GiB であり，`CUDA_CACHE_MAXSIZE` で設定する（CPG §18 CUDA Environment Variables [5]）．

```
nvcc x.cu --gpu-architecture=compute_80 \
          --gpu-code=compute_80,sm_86,sm_89
```

は Compute Capability 8.6 と 8.9 向けの機械語を生成し，Compute Capability 8.0 向けの PTX を残す．後の GPU はこの PTX を JIT コンパイルできる [65, §5.6.2]．これに対して PTX は常に同じかそれより高い Compute Capability にコンパイルできるが，その結果は仮想アーキテクチャより後に追加された機能を使えない．Compute Capability 6.0 向けの PTX を 7.0 の GPU で JIT コンパイルしても，テンソルコアは使われない（CPG §3.1.3 PTX Compatibility）．

注意 機械語はメジャーな Compute Capability をまたいで移植できない．Compute Capability X.y 向けの cubin は，$z \geq y$ である X.z でのみ動作する（CPG §3.1.2 Binary Compatibility [5]）．

10.2 PTX

PTX は CUDA コードを読む通常の最下層であり，その命令形式は詳しく知っておく価値がある．

出典 M10，PTX；PTX ISA §4.3，§9.2–9.3 [19]；K12 §1.3.7．

10.2.1 命令形式

PTX の命令は，演算の名前である**オペコード**と，**オペランド**の並びからなる．PTX ISA はこれを次のように述べている．“PTX の命令は一般に 0 個から 4 個のオペランドを持ち，加えてオペコードの左側の `@` 記号の後に置く省略可能なガードプレディケートを持つ” [19, §9.2].

▷ opcode

▷ operand

```
@p  opcode;
@p  opcode a;
@p  opcode d, a;
@p  opcode d, a, b;
@p  opcode d, a, b, c;
```

値を生成する命令では，d がデスティネーション，a，b，c がソースであり，デスティネーションが先に来る．算術命令とメモリ命令はすべて，add.s32 や ld.global.f32 のように操作の対象とする型を指定する．

オペコードには接尾辞が付く．型（.s32 符号付き，.u32 符号なし，.f32 浮動小数点，.b32 型なしのビット列，.pred）と，メモリ命令では状態空間（.global，.shared，.local，.param）である．

定義 10.9 (ガードプレディケート，プレディケートレジスタ). **プレディケートレジスタ**は .pred 型の 1 ビットの PTX レジスタであり，他のレジスタと同様に .reg .pred p, q; のように宣言する．命令の前に置いた**ガードプレディケート** @p はその命令を条件付きにする．命令は p が真であるスレッドでのみ効果を持つ．@!p はその否定を使う．ガードのない命令は無条件に実行される．

▷ predicate register

▷ guard predicate

プレディケートレジスタはすべての PTX レジスタと同様に仮想的である．.pred 型は，1 ビットで足りることを ptxas に伝える [19, §9.3]．プレディケートを設定するのは最も一般には setp であり，これは “2 つの数値を関係演算子で比較する” [19, §9.7.7.2]．setp は | で区切った 2 つのデスティネーションに書き込むことができ（shfl.sync などいくつかの命令も同じ記法を使う），2 つ目には比較結果の否定が入る．

プレディケートは，第 5 章と第 6 章の if 変換とプレディケート実行の背後にある仕組みである．

例 10.10 (プレディケートによる分岐の除去). 文 `if (a < b) c = c + 1; else c = c - 1;` は，a，b，c がそれぞれ %r1，%r2，%r3 にあるとき，分岐を必要としない．

```
     setp.lt.s32  %p1|%p2, %r1, %r2; // p1 = a<b, p2 = !p1
@%p1 add.s32      %r3, %r3, 1;       // then: c = c + 1
@%p2 sub.s32      %r3, %r3, 1;       // else: c = c - 1
```

ワープのどのスレッドも 3 つの命令すべてを発行し，各スレッドは 2 つの更新のちょうど一方をコミットする．分岐命令に同じガードを付けると，本当の条件ジャンプになる．

```
     setp.lt.s32  %p1, %r1, %r2;
@!%p1 bra         ELSE;              // a >= b ならジャンプ
```

ヒント 比較演算子は，符号付き整数と浮動小数点では eq，ne，lt，le，gt，ge である．符号なし整数では eq，ne，lo，ls，hi，hs（lower，lower or same，higher，higher or same）を使う [19, §9.3.1]．

プレディケート付きの形は，ダイバージェンス（第 6 章）の危険と引き換えに両側を発行する．このような短い本体ではコンパイラはこの形を選ぶ．

10.2.2 文

PTX プログラムは文の並びである．PTX ISA の言葉では，“PTX の文はディレクティブか命令のいずれかである．文は省略可能なラベルで始まり，セミコロンで終わる” [19, §4.3].

定義 10.11 (ディレクティブ，ラベル). **ディレクティブ**は，キーワードがドットで始まり，計算ではなく宣言を行う文である．PTX のバージョンとターゲット（.version，.target，.address_size），関数とカーネル（.func，.entry），状態空間内の変数（.reg，.global，.shared）を宣言する．**ラベル**は，文の前に置かれ，コロンが続く名前である．分岐命令はそれを分岐先として使う．

▷ directive

▷ label

例 10.12 (完全な PTX カーネル). 次のカーネルは，グローバル配列の要素をスレッドごとに 1 つずつ 2 倍にする．

```
.version 8.0
.target sm_80
.address_size 64
.global .f32 buf[1024];

.visible .entry twice()
{
    .reg .b32  %r<2>;
    .reg .b64  %rd<4>;
    .reg .f32  %f<3>;
start:
    mov.u32        %r1, %tid.x;
    mov.u64        %rd1, buf;       // buf のアドレス
    mul.wide.u32   %rd2, %r1, 4;    // バイトオフセット 4*tid
```

```
    add.u64         %rd3, %rd1, %rd2;
    ld.global.f32   %f1, [%rd3];
    add.f32         %f2, %f1, %f1;  // 2 * buf[tid]
    st.global.f32   [%rd3], %f2;
    ret;
}
```

最初の 4 行はディレクティブである．`.reg .b32 %r<2>` はレジスタ `%r0` と `%r1` を宣言する．ラベル `start` は最初の命令を示し，算術命令とメモリ命令はすべて型を指定している．

`%r<2>` は番号付きのレジスタの範囲を表す PTX の略記である．`mul.wide.u32` は 2 つの 32 ビット値を掛けて 64 ビットの結果を得る．64 ビットのアドレス空間へのバイトオフセットにはこれが必要である．

10.2.3 その他の命令

算術とメモリアクセスのほかに，PTX には通常の命令群がそろっている．制御フローには `bra`（ラベルへの分岐），`call`（関数呼び出し），`ret`（復帰）を使う．同期には，`__syncthreads()` がコンパイルされる先のバリア `bar.sync` と，**メモリフェンス** 命令 `membar` と `fence` がある．後者は“メモリ操作の順序を強制する”[19, §9.7.15.4]．すなわち，スレッドのフェンス前のメモリアクセスは，フェンス後のアクセスより先に他のスレッドから見えるようになる．アトミックな読み出し・変更・書き込み操作には `atom` を使い，`atom.global.add.u32` のように常に演算と型を指定する．

特殊レジスタ（第 4 章）は各スレッドに座標を与える．`%tid`（CTA 内のスレッドインデックス），`%ntid`（CTA の各次元のスレッド数），`%ctaid` と `%nctaid`（ブロックインデックスとグリッドの大きさ）である [19, §10]．命令とその変種の完全な一覧は PTX ISA 自体にある．

注意 PTX ISA にはスタックポインタの特殊レジスタはない．ただし，そのようなレジスタが`%tid` や `%ntid` と並べて挙げられることがある．固定サイズのローカル記憶域は `.local` 配列であり，コンパイラが生成する PTX は，`%SP` や `%SPL` と名付けた通常のレジスタを通してそれをアドレス指定する．実行時にしか大きさのわからないスタック記憶域は `alloca` で確保し，`stacksave` と `stackrestore` がスタックポインタを読み出し，元に戻す [19, §7.3]．

10.3 IR と基本ブロック

オプティマイザには，命令が単純で制御フローが明示的な形のプログラムが必要である．第 1 の要件は 3 番地コードに，第 2 の要件は基本ブロックとフローグラフにつながる．

▷ memory fence

すべての組み合わせがあるわけではない．半精度のアトミック加算は `atom.add.noftz.f16`（ゼロへのフラッシュなし）と書き，PTX ISA 6.3 と `sm_70` を必要とする [19, §9.7.15.5]．

出典 M10，IR と基本ブロック；Aho ら §6.2，§8.4 [70]．

10.3.1 3 番地コード

定義 10.13 (3 番地コード). **3 番地コード**では，すべての命令が高々 1 つの演算子と高々 3 つのアドレス（変数，一時変数，定数の名前）を持ち，`A = B op C` という形をとる．複雑な式は，コンパイラが生成する一時変数を使って，このような命令の列に分解される．

▷ three-address code

LLVM IR と PTX はこの形に近い．その命令の大半は 1 つの演算，1 つのデスティネーション，1 つか 2 つのソースを持つ（PTX には `mad` のような 3 ソースの命令もある）．LLVM IR の

```
%sum = add i32 %x, %y
```

では，`%sum` がデスティネーション，`add` が演算，`i32` が結果の型（32 ビット整数），`%x` と `%y` がソースである [69]．同じ演算の PTX の形は `add.s32 %r3, %r1, %r2` である．式 `y = a * b + c` は `t = a * b` と `y = t + c` の 2 つの命令になる．

LLVM IR では符号付き整数と符号なし整数に 1 つの `add` を使う．どちらも同じビット列を生むからである．PTX は解釈をオペコードに書き込む（`add.s32`，`add.u32`）．

10.3.2 基本ブロック

第 6 章では**基本ブロック**を非形式的に導入した．その形式的な定義は，ほとんどの最適化が対象とする単位である．

定義 10.14 (基本ブロック). 基本ブロックとは，入口と出口を 1 つずつ持つ，連続した命令の極大の列である．制御はその最初の命令からしか入れず，いったん入るとブロックのすべての命令が順に実行され，最後の命令の後でのみブロックを離れる．最後の命令は通常，制御フロー命令（分岐，ジャンプ，復帰）である．次の命令が分岐先である場所でもブロックは終わる．

定義 10.15 (制御フローグラフ，先行ブロック，後続ブロック). **制御フローグラフ**（CFG）は，関数の基本ブロックをノードとし，ジャンプ，成立した条件分岐，次のブロックへのフォールスルーのいずれかによって B の終わりから C の始まりへ制御が移りうるとき，辺 $B \to C$ を持つ．このとき B は C の**先行ブロック**，C は B の**後続ブロック**である．入口（entry）ノードは最初の命令のブロックに先行し，出口（exit）ノードは関数を抜けるすべてのブロックの後に続く．

▷ control flow graph (CFG)

▷ predecessor

▷ successor

`if/else` は第 6 章のダイヤモンドを生む（図 10.4）．条件を持つブロック B1 には 2 つの後続ブロック，すなわち 2 つの分岐先 B2 と B3 があり，それらは 1 つの後続ブロックである合流ブロック B4 を共有する．辺は B1→B2，B1→B3，B2→B4，B3→B4 である．1 つのブロックの中での最適化は単純である．ブロックを通る経路は 1 つしかないからである．ブロックが `a + b` を 2 回計算し，その間にどちらのオペランドも再定義されなければ，2 回目の計算は 1 回目の結果を再利用できる．

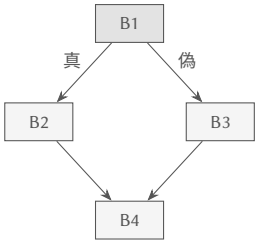


図 10.4: `if/else` のダイヤモンド型 CFG．

定義 10.16 (局所最適化). **局所最適化**は，一度に 1 つの基本ブロックを対象とし，そのブロックに入る経路やそこから出る経路についての情報は使わない．

▷ local optimization

10.3.3 基本ブロックの求め方

命令の列のブロックは，そのリーダから決まる．

定義 10.17 (リーダ). **リーダ（先頭命令）**は，基本ブロックを開始する命令である．3 つの規則ですべてのリーダが求まる．(1) 最初の命令はリーダである．(2) 条件ジャンプまたは無条件ジャンプの分岐先はすべてリーダである．(3) ジャンプの直後の命令はすべてリーダである．各基本ブロックは，1 つのリーダから次のリーダの直前までである．

▷ leader

分岐ダイバージェンスの再収束点（第 6 章）は常にリーダである．それは合流ブロックの最初の命令であり，分岐先だからである．

例 10.18 (PTX のループの基本ブロック). 次のループは $0 + 1 + \cdots + (n-1)$ を求める．i は `%r1`，和 s は `%r2`，n は `%r3` にある．

```
      mov.u32  %r1, 0;             // (1) i = 0
      mov.u32  %r2, 0;             //     s = 0
loop: setp.hs.u32 %p1, %r1, %r3;   // (2) i >= n (符号なし)
@%p1  bra      done;
      add.u32  %r2, %r2, %r1;      // (3) s = s + i
      add.u32  %r1, %r1, 1;        //     i = i + 1
      bra      loop;
done: st.global.u32 [%rd1], %r2;   // (2),(3)
```

コメントは，命令をリーダにする規則を示している．最初の `mov` は規則 1 によりリーダである．`loop` の `setp` は `bra loop` の分岐先である（規則 2）．最初の `add` は条件分岐 `@%p1 bra done` の直後にある（規則 3）．`done` のストアは，分岐先であり，かつ `bra loop` の直後の命令でもある．4 つのリーダから 4 つのブロックが得られる．B1（2 つの `mov`），B2（`setp` と条件分岐），B3（2 つの `add` と `bra loop`），B4（ストア）である．図 10.5 が CFG を示す．B1→B2，フォールスルーによる B2→B3 と分岐成立時の B2→B4，そしてループを閉じる辺 B3→B2 である．

注意 ガード付きの分岐 `@%p1 bra` は条件ジャンプである．その次の命令は，単にフォールスルーでも到達するにもかかわらずリーダである．

B3→B2 のように，ループを開始するブロックへ戻る辺を**後退辺**（back edge）と呼ぶ．

ループ本体の終わりからループの判定への辺は後退辺である．10.5 節のデータフローの反復が 2 回以上のパスを必要とするのはこの辺のためである．

10.4 データフロー解析入門

多くの最適化は，複数の基本ブロックにまたがる事実を必要とする．

出典 M10，データフロー解析入門；Aho ら §9.1–9.2 [70]．

定義 10.19 (大域最適化). **大域最適化**は，関数全体の基本ブロックにわたって集めた情報を使い，その CFG のすべての経路を考慮する．

▷ global optimization

注意 浮動小数点は既定でこの規則を曲げる．nvcc は乗算とそれに続く加算を 1 つの融合積和に縮約し，丸めが 2 回から 1 回になるため，結果が最後のビットで異なりうる．`--fmad=false` でこの縮約を無効にできる（NVCC §4.2.7.11 [65]）．

局所的であれ大域的であれ，どの最適化も元のプログラムの意味を保存しなければならない．すなわち，どの入力に対しても最適化後のプログラムは同じ観測可能な結果を生まなければならない．代表的な大域最適化は，いずれも第 11 章で

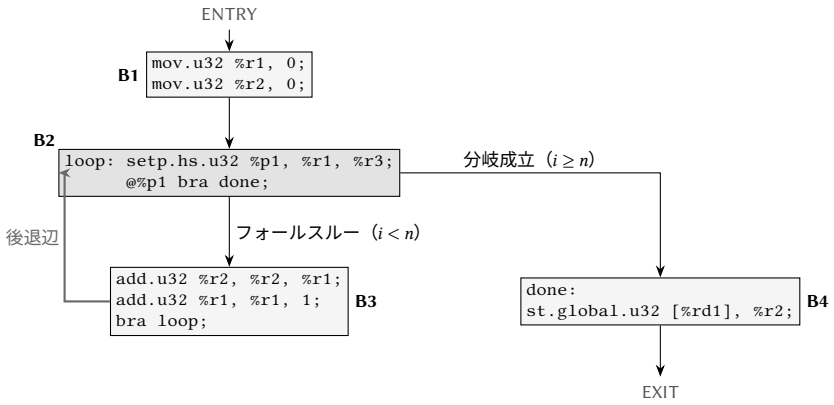


図 10.5: 例 10.18 のループの基本ブロックと CFG．辺 B3→B2 は前のブロックへ戻り，ループを閉じる．

扱うが，冗長な命令の除去（どの経路でも同じオペランドで 2 回計算される式），定数伝播と**コピー伝播**（変数を，それに代入された定数や別の変数で置き換えること．これによりさらなる冗長性が現れることが多い），不要コード除去（結果が一度も使われない命令の除去），コード移動（命令をより実行回数の少ない場所，たとえばループの外へ移すこと），誘導変数の検出と強度削減である．これらはどれも 1 つのブロックの中では判断できない．あるブロックの x が 3 ブロック前に代入された定数のままかどうかは，その間のすべての経路に依存する．

▷ copy propagation

コピー伝播を 1 行で示す．`y = x` の後，後続の `z = y + 1` は `z = x + 1` になる．y の他の使用が残っていなければ，`y = x` は不要である．

定義 10.20 (データフロー解析). **データフロー解析**と呼ばれる一群の手法は，プログラムの各点について，その点に到達しうるすべての実行に関する事実を計算する．各基本ブロックの効果を解析し，その結果を CFG の辺に沿って伝播させることによる．解析ごとに調べる性質が異なる．

▷ data-flow analysis

10.4.1 抽象化：点，状態，伝達関数

データフロー解析は，プログラムをその文の間の状態によって記述する．

定義 10.21 (プログラム点，伝達関数). **プログラム点**は，文の前または後に位置する．解析は各点に状態，すなわちそこで成り立つ事実の集合を対応付ける．ブロック B について，$\mathrm{IN}[B]$ はその最初の文の前の点の状態，$\mathrm{OUT}[B]$ は最後の文の後の状態である．文 s の**伝達関数** f_s は，s の前の状態を s の後の状態に写す．

▷ program point

▷ transfer function

ブロックの内部では，ある文の後の点は次の文の前の点である（図 10.6）．したがって文 $s_1, \ldots, s_n$ からなるブロックの伝達関数は合成

$$\mathrm{OUT}[B] = f_B(\mathrm{IN}[B]), \qquad f_B = f_{s_n} \circ \cdots \circ f_{s_2} \circ f_{s_1} \tag{10.1}$$

である．ブロックの間では，制御はどの先行ブロックからも到着しうるので，ブロックの入口の状態はすべての先行ブロックの状態を組み合わせたものでなければならない．

定義 10.22 (合流演算子，前向き解析). **合流演算子**は，複数の辺から合流点に到着する状態を 1 つの状態に組み合わせる．**前向き解析**では，状態は制御と同じ向きに流れる．方程式は

$$\mathrm{IN}[B] = \bigcup_{P \in \mathrm{pred}(B)} \mathrm{OUT}[P], \qquad \mathrm{OUT}[B] = f_B(\mathrm{IN}[B]) \tag{10.2}$$

である．

▷ meet operator

▷ forward analysis

後ろ向きの問題は逆向きに，値の使用からそれが計算された場所へと進む．$\mathrm{IN}[B] = f_B(\mathrm{OUT}[B])$，$\mathrm{OUT}[B] = \bigcup_{S \in \mathrm{succ}(B)} \mathrm{IN}[S]$ である．後に示す図 10.7 の CFG では，B5 の先行ブロックは B3 と B4 なので，前向き解析では $\mathrm{IN}[\mathrm{B5}] = \mathrm{OUT}[\mathrm{B3}] \cup \mathrm{OUT}[\mathrm{B4}]$ となる．B5 の後続ブロックは B2 と EXIT なので，後ろ向き解析では $\mathrm{OUT}[\mathrm{B5}] = \mathrm{IN}[\mathrm{B2}] \cup \mathrm{IN}[\mathrm{EXIT}]$ となる．

レジスタ割り付けの背後にある後ろ向き解析である生存変数解析は第 11 章の主題である．本書で扱う 2 つの解析はどちらも和集合を合流演算とする．

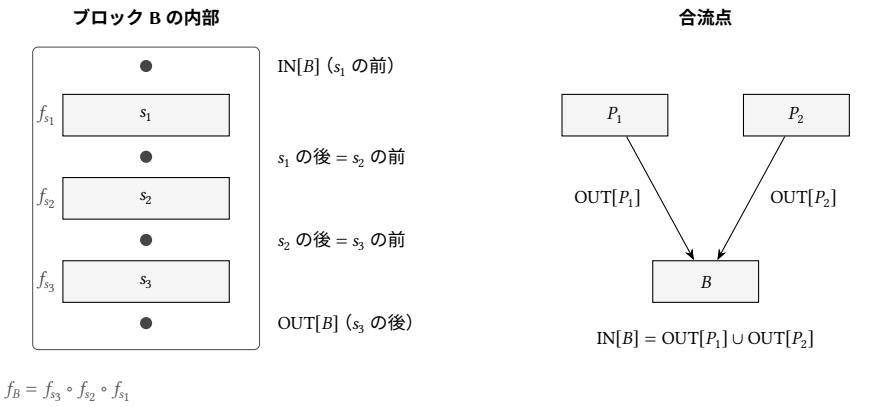


図 10.6: 左：3 つの文からなるブロックのプログラム点．各文の伝達関数は，その前の状態をその後の状態に写す．右：合流点では，合流演算（ここでは和集合）が先行ブロックの状態を組み合わせる．

10.4.2 到達定義

最初の解析は，値がどこから来るかに関するものである．

▷ definition
▷ use

定義 10.23 (定義，使用)．変数の**定義**とは，その変数に値を代入する文であり，**使用**とは，その変数を読む文である．`a = x + y` という文は `a` を定義し，`x` と `y` を使用する．

▷ reaching definitions

定義 10.24 (到達定義)．定義 d が点 p に**到達する**とは，d の直後の点から p への経路で，その途中で d が殺されない，すなわち同じ変数の他の定義に上書きされないものが存在することである．**到達定義**の解析は，各プログラム点について，そこに到達する定義の集合を計算する．

注意 経路は 1 つで足りる．定義が p に到達するのは，p へ至るある経路で生き残る場合であり，すべての経路で生き残る必要はない．解析はこの方向に保守的であり，実行時には決して通らない経路の定義を含めることがある．

`x` の定義がちょうど 1 つ，たとえば `x = 4` だけが `x` の使用に到達するなら，その使用は定数 4 で置き換えられる．2 つ目の定義も到達するなら置き換えられない．定数伝播はこのようにしてこの解析を使う．

▷ gen set
▷ kill set

定義 10.25 (gen 集合と kill 集合)．定義 d: `u = v + w` について，**gen 集合**は $\mathrm{gen}_d = \{d\}$，すなわち文が生成する定義であり，**kill 集合** kill_d は，プログラム中の `u` の他のすべての定義，すなわちこの文が上書きする定義の集合である．d の伝達関数は

$$f_d(x) = \mathrm{gen}_d \cup (x - \mathrm{kill}_d) \tag{10.3}$$

である．

記法：$x - y$ は差集合，すなわち y に含まれない x の要素である．x は文の前の点に到達する定義の集合を表す．

d に到着した定義は，d がそれを殺さない限り生き残り，d 自身が追加される．ブロックでは，これらの関数は式 (10.1) のように合成される．2 つの文では

$$\begin{aligned} f_2(f_1(x)) &= \mathrm{gen}_2 \cup \big((\mathrm{gen}_1 \cup (x - \mathrm{kill}_1)) - \mathrm{kill}_2\big) \\ &= \big(\mathrm{gen}_2 \cup (\mathrm{gen}_1 - \mathrm{kill}_2)\big) \cup \big(x - (\mathrm{kill}_1 \cup \mathrm{kill}_2)\big) \end{aligned}$$

となり，これは式 (10.3) と同じ形である．帰納法により，n 個の文からなるブロックの伝達関数は $f_B(x) = \mathrm{gen}_B \cup (x - \mathrm{kill}_B)$ であり，

$$\begin{aligned} \mathrm{kill}_B &= \mathrm{kill}_1 \cup \mathrm{kill}_2 \cup \cdots \cup \mathrm{kill}_n, \\ \mathrm{gen}_B &= \mathrm{gen}_n \cup (\mathrm{gen}_{n-1} - \mathrm{kill}_n) \cup (\mathrm{gen}_{n-2} - \mathrm{kill}_{n-1} - \mathrm{kill}_n) \\ &\quad \cup \cdots \cup (\mathrm{gen}_1 - \mathrm{kill}_2 - \cdots - \mathrm{kill}_n) \end{aligned} \tag{10.4}$$

である．ブロックは，そのいずれかの文が殺す定義をすべて殺し，自分の定義のうちブロック内の後の文が上書きしないものをすべて生成する．

ヒント gen_B はブロックを下から上へ読むと求まる．ある定義は，ブロック内の後の文が同じ変数を再び定義しない限り gen_B に属する．

例 10.26 (CFG の gen 集合と kill 集合)．図 10.7 の CFG は，`x[0]` から `x[n-1]` の絶対値の和を `s` に求める（$n \geq 1$ とする．判定はループの末尾にある）．定義は 7 つある．`i` は d_1 と d_7，`s` は d_2，d_4，d_6，`t` は d_3 と d_5 で定義される．ブロック B4 は 2 つの文を持つ．d_5: `t = -t` は $\mathrm{gen} = \{d_5\}$，$\mathrm{kill} = \{d_3\}$，d_6: `s =`

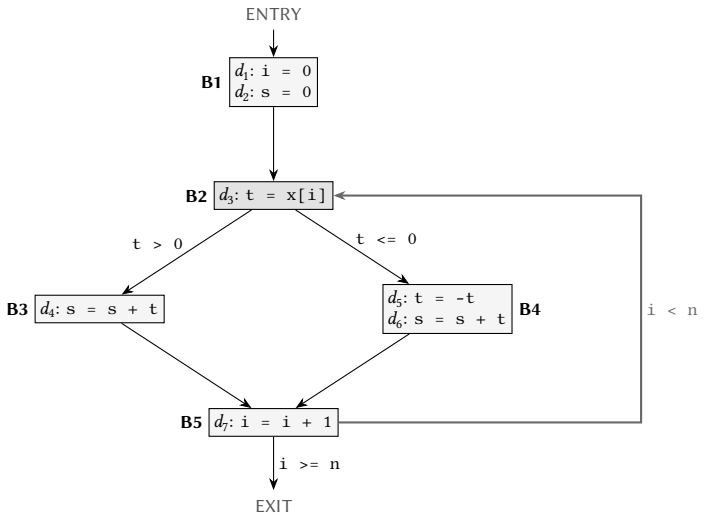


図 10.7: 到達定義の例の CFG．t が正なら t を，そうでなければ −t を加えるループである．辺 B5→B2 が後退辺である．

表 10.1: 図 10.7 の各ブロックの gen 集合と kill 集合．

ブロック	gen_B	kill_B
B1	$\{d_1, d_2\}$	$\{d_4, d_6, d_7\}$
B2	$\{d_3\}$	$\{d_5\}$
B3	$\{d_4\}$	$\{d_2, d_6\}$
B4	$\{d_5, d_6\}$	$\{d_2, d_3, d_4\}$
B5	$\{d_7\}$	$\{d_1\}$

s + t は $\mathrm{gen} = \{d_6\}$，$\mathrm{kill} = \{d_2, d_4\}$ である．式 (10.4) により，$\mathrm{kill}_{\mathrm{B4}} = \{d_2, d_3, d_4\}$，$\mathrm{gen}_{\mathrm{B4}} = \{d_6\} \cup (\{d_5\} - \{d_2, d_4\}) = \{d_5, d_6\}$ となる．表 10.1 に 5 つのブロックすべてを示す．

10.5 到達定義の例

すべてのブロックの gen 集合と kill 集合がわかれば，到達定義は CFG 全体にわたる連立方程式から求まる．

出典 M10，到達定義の例；Aho ら §9.2.4 [70]．

10.5.1 方程式と反復アルゴリズム

ENTRY 以外のすべてのブロック B について，到達定義は，和集合を合流演算とする式 (10.2) の前向き問題である．

$$\mathrm{IN}[B] = \bigcup_{P \in \mathrm{pred}(B)} \mathrm{OUT}[P], \qquad \mathrm{OUT}[B] = \mathrm{gen}_B \cup (\mathrm{IN}[B] - \mathrm{kill}_B) \tag{10.5}$$

これに境界条件 $\mathrm{OUT}[\mathrm{ENTRY}] = \varnothing$ が加わる．関数の始まりにはどの定義も到達しない．CFG にループがあると方程式は循環する．IN[B2] は OUT[B5] に依存し，それは IN[B2] に依存する．方程式は反復によって解く．

出口ノードは文を持たないので OUT[EXIT] = IN[EXIT] であり，これは関数の終わりに到達する定義である．

定義 10.27 (反復アルゴリズム，不動点)．データフロー問題の**反復アルゴリズム**は，すべての状態に初期値を与えて始め，すべてのブロックについて方程式を順に評価するパスを，1 回のパス全体でどの状態も変化しなくなるまで繰り返す．そのとき状態は方程式の解であり，パスの**不動点**である．すなわち方程式をもう一度適用しても状態は変わらない．

▷ iterative algorithm

▷ fixed point

到達定義の場合，アルゴリズムは次のとおりである．

```
OUT[ENTRY] = ∅
for each block B ≠ ENTRY:   OUT[B] = ∅
```

表 10.2: 図 10.7 の到達定義のパスごとの推移．ビット k は d_k を表し，上付き添字はパスの番号である．3 回目のパスはパス 2 を再現する．

ブロック	OUT^0	IN^1	OUT^1	IN^2	OUT^2
B1	00 0 0 00 0	00 0 0 00 0	11 0 0 00 0	00 0 0 00 0	11 0 0 00 0
B2	00 0 0 00 0	11 0 0 00 0	11 1 0 00 0	11 1 1 11 1	11 1 1 01 1
B3	00 0 0 00 0	11 1 0 00 0	10 1 1 00 0	11 1 1 01 1	10 1 1 00 1
B4	00 0 0 00 0	11 1 0 00 0	10 0 0 11 0	11 1 1 01 1	10 0 0 11 1
B5	00 0 0 00 0	10 1 1 11 0	00 1 1 11 1	10 1 1 11 1	00 1 1 11 1
EXIT	00 0 0 00 0	00 1 1 11 1	00 1 1 11 1	00 1 1 11 1	00 1 1 11 1

```
while some OUT changed in the last pass:
    for each block B ≠ ENTRY:
        IN[B] = ⋃_{P∈pred(B)} OUT[P]
        OUT[B] = gen_B ∪ (IN[B] − kill_B)
```

このアルゴリズムは停止する．どの OUT 集合も空から始まり，式 (10.5) は入力が増えたときにしか定義を追加しないので，どの集合も縮むことはない．集合は有限な定義の集合の部分集合なので，増えるのは有限回に限られる．空集合から始めると最小の解が得られ，それはその点への経路が実際に存在する定義だけを含む．

定義 10.28 (ビットベクタ). **ビットベクタ**は，番号付けされた N 個の要素からなる固定の集合の部分集合を N ビットで表す．要素 k が部分集合に含まれるときビット k が 1 である．和集合はビットごとの OR，積集合はビットごとの AND になり，$x - y$ は x AND NOT y になる．

定義に $d_1, \ldots, d_N$ と番号を付けると，アルゴリズムの各集合は 1 つのビットベクタとなり，伝達関数全体は機械語ワード当たり 2 回のビット演算になる．

注意 パスがブロックを訪れる順序は，パスの回数を変えるが結果は変えない．次の例のように先行ブロックを後続ブロックより先に訪れると，1 回のパスで情報が遠くまで伝わり，さらにパスを要するのは後退辺だけになる．

▷ bit vector

例の 7 つの定義は 1 つの機械語ワードに収まる．1000 個の定義を持つ関数では，集合 1 つに 64 ビットのワードが 16 個必要である．

10.5.2 例の計算

表 10.2 は，図 10.7 の CFG にアルゴリズムを適用したものであり，B1 から B5，EXIT の順に訪れる．各ビットベクタは $d_1, \ldots, d_7$ を，その定義を含むブロックごとに区切って並べる（B1：$d_1 d_2$；B2：d_3；B3：d_4；B4：$d_5 d_6$；B5：d_7）．

例 10.29 (1 回目のパス). B1 の先行ブロックは ENTRY だけなので，IN[B1] = ∅，OUT[B1] = $\text{gen}_{B1} = \{d_1, d_2\}$ である．B2 の先行ブロックは B1 と B5 であり，OUT[B5] はまだ空なので，IN[B2] = $\{d_1, d_2\}$，OUT[B2] = $\{d_3\} \cup (\{d_1, d_2\} - \{d_5\}) = \{d_1, d_2, d_3\}$ となる．B3 は d_2 を殺して d_4 を加え，$\{d_1, d_3, d_4\}$ となる．B4 は d_2 と d_3 を殺して d_5, d_6 を加え，$\{d_1, d_5, d_6\}$ となる．合流点 B5 では両者の和集合は $\{d_1, d_3, d_4, d_5, d_6\}$ であり，B5 は d_1 を d_7 で置き換える．OUT[B5] = $\{d_3, d_4, d_5, d_6, d_7\}$ であり，これは EXIT にも到達する．

例 10.30 (2 回目と 3 回目のパス). パス 2 では OUT[B5] はもはや空ではなく，後退辺がその定義を B2 に運ぶ．IN[B2] = $\{d_1, d_2\} \cup \{d_3, \ldots, d_7\}$，すなわち 7 つすべてである．B2 が殺すのは d_5 だけなので，OUT[B2] には d_4，d_6，d_7 が加わり，B3 と B4 の IN 集合にも同じく加わる．このうち B3 と B4 が先へ渡すのは d_7 だけである．どちらも残りの 2 つを殺すか再生成するからである．B5 で新しいのは d_7 だけであり，これは B5 がいずれにせよ生成する．したがって OUT[B5] は変わらない．B2，B3，B4 が変化したので 3 回目のパスを実行する．その IN[B2] はパス 2 と同じなので，どの集合も変化せず，パス 2 の状態が不動点である．

この解はオプティマイザが問う質問に答える．`i` の 2 つの定義 d_1 と d_7 はどちらも B2 の使用 `x[i]` に到達するので，そこで `i` を定数 0 で置き換えることはできない．後退辺がなければ d_1 だけが到達するはずである．初期化 d_2（`s = 0`）は B2 には到達するが B5 には到達しない．ループ本体を通るどちらの経路も `s` を再定義するからである．EXIT に到達する定義 d_3 から d_7 は，関数がその終わりでなお観測できる値の定義である．

`t` の定義のうち B3 と B4 に到達するのは d_3 だけである．d_5 は後退辺を通って B2 に到着するが，そこで殺される．したがって B3 と B4 での `t` の各使用は，同じ反復でロードされた値を読む．

要点 — 章のまとめ

コンパイラは，AST と IR を構築するフロントエンド，IR を書き換えるオプティマイザ，コードを生成するバックエンドからなる．CUDA では nvcc がホストコードとデバイスコードを分ける．デバイスコードは cicc（または Clang と LLVM の NVPTX バックエンド）によって PTX になり，PTX は ptxas によって SASS になる．nvcc が作るファットバイナリは両方を持つので，ビルド時に存在しなかった GPU 向けにドライバが PTX を JIT コンパイルできる．PTX は仮想 ISA であり，0 個から 4 個のオペランドを持つ型付きの命令，無制限の仮想レジスタ，`setp` で設定するガードプレディケートを持つ．オプティマイザは，3 つのリーダ規則で求めた基本ブロックに分割され，制御フローグラフで結ばれた 3 番地コードを対象とする．大域最適化はデータフロー解析に基づく．ブロックごとの伝達関数，合流点での合流演算，そして不動点に達する反復アルゴリズムである．到達定義は $\mathrm{OUT}[B] = \mathrm{gen}_B \cup (\mathrm{IN}[B] - \mathrm{kill}_B)$ と先行ブロックにわたる和集合としての $\mathrm{IN}[B]$ による前向き解析であり，ビットベクタで効率よく計算される．この解析は，定義域，方向，伝達関数，合流演算，境界条件，初期値によって定まる．第 11 章は生存変数解析についてこれら 6 項目を埋める．

第 11 章

コンパイラの基礎 II

講義動画	M11，レッスン 1–4（Live-Variable Analysis；SSA；Examples of Compiler Optimizations；Divergence Analysis）
資料	Cytron ら [71]（支配辺境による SSA の構築）；Chaitin [72]（グラフ彩色によるレジスタ割り付け）；LLVM Language Reference [69]（`phi`，`select`）；LLVM のパス [73]（`mem2reg`）；PTX ISA [19]
教科書	Aho ら [70] §6.2 Three-Address Code，§8.8 Register Allocation and Assignment，§9.1 The Principal Sources of Optimization，§9.2 Introduction to Data-Flow Analysis，§9.6 Loops in Flow Graphs；CPG [5] §5.2.3 Multiprocessor Level，§5.3.2 Device Memory Accesses，§5.4.2 Control Flow Instructions，§7.1.5 __noinline__ and __forceinline__，§7.39 #pragma unroll；PMPP [6] ch. 10
キーワード	生存変数，生存変数解析，後ろ向き解析，def 集合と use 集合，レジスタ割り付け，干渉グラフ，レジスタスピル，静的単一代入，φ 関数，経路収束条件，支配辺境，ループ展開，関数のインライン展開，不要コード除去，定数伝播，強さの低減，ループ不変式の移動，誘導変数，ダイバージェンス解析，スレッド依存値と一様値

第 10 章では CUDA プログラムをコンパイラに通して PTX までたどり，PTX を基本ブロックと制御フローグラフに切り分け，前向き問題である到達定義を例にデータフロー解析を組み立てた．本章はその道具立てを，後ろ向きの対となる生存変数解析と，その最も重要な利用先であるレジスタ割り付けによって完成させる．レジスタ割り付けは，GPU では 1 つの SM に何個のワープが載るかを決める．続いて，LLVM が，したがって NVIDIA の LLVM ベースの NVVM コンパイラもプログラムを保持する表現である静的単一代入形式を示し，これらの解析が可能にする古典的な最適化，すなわちループ展開，インライン展開，不要コード除去，定数伝播，強さの低減，コード移動を取り上げる．最後の節は，並列コードだけが提起する問い，すなわちどの分岐がダイバージェントになりうるかを扱う．これは第 6 章のハードウェア，すなわち SIMT スタックとその再収束点を，コンパイラがスレッドインデックスの定義・使用連鎖の上で行う解析に結びつける．

11.1 生存変数解析

到達定義は，各プログラム点について，そこで見える値をどの代入が生成しえたかを問う．生存変数解析はその逆の問いを立てる．ある点に存在する値のうち，どれが後でまだ必要とされるか，である．

出典 M11，生存変数解析；Aho ら §9.2 Introduction to Data-Flow Analysis [70]．

定義 11.1 (生存変数)．変数 x がプログラム点 p で**生存変数**であるとは，p から始まる制御フローグラフ上のある経路に沿って，その経路上で x が再定義される前に，p における x の値が使用されうることをいう．そうでなければ x は p で**死んでいる**（dead）．各点について生存変数の集合を計算するデータフロー解析が**生存変数解析**である．

▷ live variable

▷ liveness analysis

死んだ値は無駄にレジスタを占有する．したがって生存変数解析の主な用途はレジスタ割り付け（11.1.2 節）である．レジスタが必要なのは生存変数だけであり，死んだ変数のレジスタは別の変数に渡すことができる．同じ情報から，どの代入が無用かもわかる．これが 11.3 節の不要コード除去の基礎となる．

第 6 章はハードウェアについて同じ概念を用いた．生存レジスタとは，後でまだ読まれる値を保持するレジスタである．生存変数解析はそれをコンパイル時に，すべての変数とすべての点について計算する．

11.1.1 データフロー方程式

1 つの基本ブロック B が生存性に及ぼす効果は，2 つの集合で要約される．

定義 11.2 (def 集合と use 集合)．ブロック B の **def 集合** def_B は，B がその変数を B 内で使用する前に定義する変数の集合である．**use 集合** use_B は，B がその変数を B 内で定義する前に使用しうる変数の集合である．

▷ def set
▷ use set

注意 `k = k - 1` のような代入は `k` を書く前に読むので，`k` は use_B に属し，定義により def_B には属さない．ときに行われるように両方の集合に入れても IN 集合は同じになる．use_B に属する変数はいずれにせよ入口で生存しているからである．

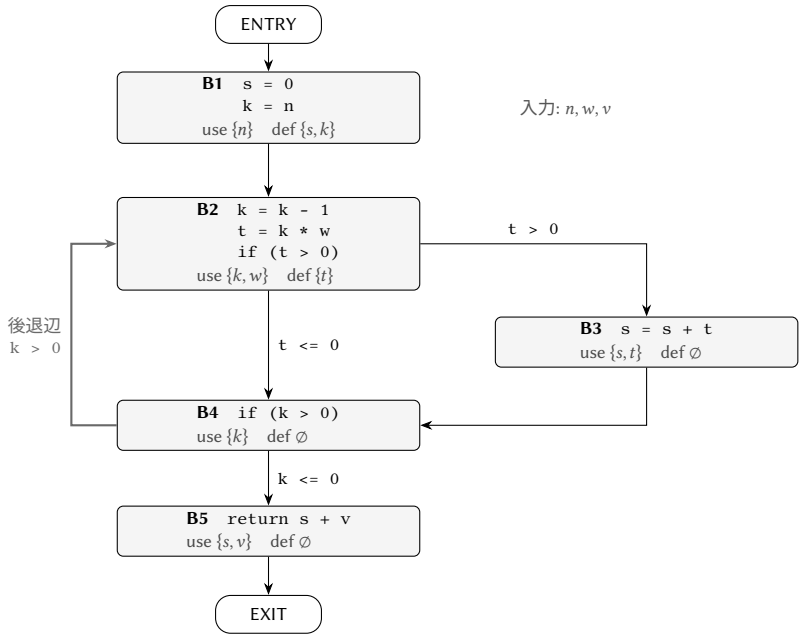


図 11.1: 本節の通しの例．t で分岐する k のループである．各ブロックにはコードと use 集合，def 集合を示す．n，w，v は入力である．

use_B に属する変数は，B の後で何が起ころうと B の入口で生存している．B 自身がそれを読むからである．def_B に属する変数は，B がまずそれを読むのでない限り B の入口で死んでいる．B がそれを上書きするからである．それ以外の変数は B を変化なく通過し，出口で生存しているときに限り入口でも生存している．IN[B] と OUT[B] を B の直前と直後の生存変数の集合とすると，次の方程式が得られる．

$$\mathrm{IN}[\mathrm{EXIT}] = \emptyset, \tag{11.1}$$

$$\mathrm{IN}[B] = use_B \cup \big(\mathrm{OUT}[B] - def_B\big), \tag{11.2}$$

$$\mathrm{OUT}[B] = \bigcup_{S \in \mathrm{succ}(B)} \mathrm{IN}[S]. \tag{11.3}$$

式 (11.1) は境界条件である．プログラムの終了時に生存している変数はない．式 (11.2) はブロックの伝達関数，式 (11.3) は合流である．変数が B の出口で生存しているのは，少なくとも 1 つの後続の入口で生存しているとき，かつそのときに限る．その後続を通って続く経路があるからである．

記法は第 10 章と同じである．succ(B) は B の後続の集合，IN と OUT はブロックの前後のプログラム点における値である．

定義 11.3 (後ろ向き解析). 情報が制御フローグラフの辺に逆らって流れ，ブロックの IN 集合がその OUT 集合から，OUT 集合が後続の IN 集合から計算されるデータフロー問題を**後ろ向き解析**という．生存性が後ろ向きなのは，値が必要かどうかがプログラムが**後で**何をするかに依存するからである．到達定義が前向きなのは，定義が到達するかどうかが**それ以前に**何が起こったかに依存するからである．

▷ backward analysis

方程式は到達定義と同じ反復法を逆方向に走らせて解く．すべての IN 集合を空から始め（式 (11.1) と，その他のブロックの初期値），ブロックを繰り返し訪れて，そのたびに後続の現在の IN 集合から OUT[B] を，OUT[B] から IN[B] を計算し，1 回の走査全体でどの IN 集合も変わらなくなるまで続ける．集合は増えるだけであり，変数は有限個しかないので，この方法は不動点に到達する．それは方程式の最小解であり，最も精密な解である．

ヒント 後ろ向き問題ではブロックを逆順（出口から）に訪れるとよい．そうすれば読まれる値の多くがすでに最新になっており，走査の回数が減る．結果は順序に依存せず，依存するのは作業量だけである．

例 11.4. 図 11.1 は，k を n から数え下げ，$t = k \cdot w$ が正であるたびにそれを s に加えるループである．結果は $s + v$ である．ブロックは B5，B4，B3，B2，B1 の順に訪れ，表 11.1 に 1 回目と 2 回目の走査の後の集合を示す．

- **走査 *1*.** OUT[B5] = IN[EXIT] = ∅ なので IN[B5] = $\{s, v\}$ である．B4 の後続は B2 と B5 である．IN[B2] はまだ空なので OUT[B4] = $\{s, v\}$，IN[B4] = $\{k\} \cup \{s, v\} = \{k, s, v\}$ となる．次に IN[B3] = $\{s, t\} \cup \{k, s, v\} = \{k, s, t, v\}$ であ

ブロック	use_B	def_B	OUT[1]	IN[1]	OUT[2]	IN[2]
B5	$\{s, v\}$	$\varnothing$	$\varnothing$	$\{s, v\}$	$\varnothing$	$\{s, v\}$
B4	$\{k\}$	$\varnothing$	$\{s, v\}$	$\{k, s, v\}$	$\{k, s, v, w\}$	$\{k, s, v, w\}$
B3	$\{s, t\}$	$\varnothing$	$\{k, s, v\}$	$\{k, s, t, v\}$	$\{k, s, v, w\}$	$\{k, s, t, v, w\}$
B2	$\{k, w\}$	$\{t\}$	$\{k, s, t, v\}$	$\{k, s, v, w\}$	$\{k, s, t, v, w\}$	$\{k, s, v, w\}$
B1	$\{n\}$	$\{s, k\}$	$\{k, s, v, w\}$	$\{n, v, w\}$	$\{k, s, v, w\}$	$\{n, v, w\}$

表 11.1: 図 11.1 の生存変数．ブロックは訪れた順に並べる．上付き添字は走査の番号であり，3 回目の走査は 2 回目を繰り返す．

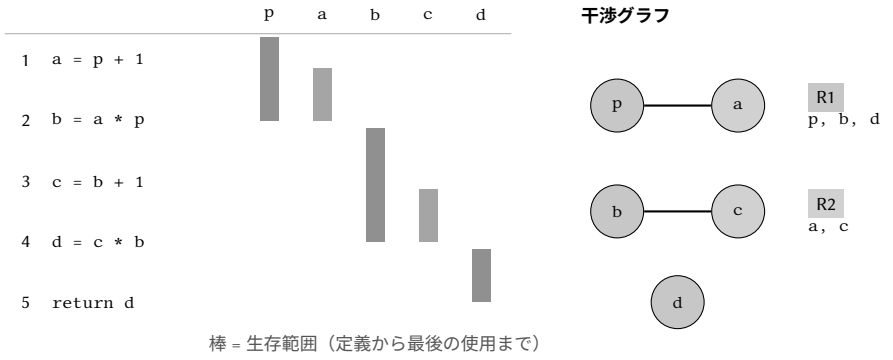


図 11.2: 生存変数解析がレジスタ割り付けを駆動する．左：5 命令の列の各値の生存範囲．右：干渉グラフ．2 色で足りるので，5 個の値は 2 個のレジスタに収まる．

る．OUT[B2] は IN[B3] と IN[B4] の和集合 $\{k, s, t, v\}$ であり，$def = \{t\}$ を除いて $use = \{k, w\}$ を加えると IN[B2] $= \{k, s, v, w\}$ となる．最後に IN[B1] $= \{n\} \cup (\{k, s, v, w\} - \{s, k\}) = \{n, v, w\}$ である．

- **走査 2**．今度は後退辺が情報を運ぶ．IN[B2] はもう空でないので OUT[B4] $= \{k, s, v, w\}$ となり，w が IN[B4] と IN[B3] に入る．OUT[B2] には w が加わるが，w はすでに IN[B2] にあるので，IN[B2] とそれより上はすべて変わらない．
- **走査 3** はすべての集合を計算し直すが何も変わらない．不動点に到達した．

結果はコードから予想されるとおりである．入力 w と v は毎回の反復と最後に読まれるのでループ全体で生存している．n は B1 の入口でのみ生存している．t は B2 から B3 への辺でのみ生存しているので，B4 ではそのレジスタは再び空いている．

本書の数値：ループ本体で同時に生存する変数は最大 5 個（B2→B3 の辺での k, s, t, v, w）なので，このループはスピルなしで 5 個のレジスタを必要とする．n は B1 が k にコピーした後は死んでいるので，そのうちの 1 個を共有できる．

11.1.2 レジスタ割り付け

定義 11.5 (レジスタ割り付け). プログラムの値を有限個の機械レジスタに対応づけるコンパイラのパスが**レジスタ割り付け**である．2 つの値は，同じ点で同時に生存することがなければ 1 つのレジスタを共有できる．**干渉グラフ**は値ごとに 1 つの節点を持ち，同時に生存する 2 つの値の間に辺を持つ．レジスタの割り当ては，隣接する節点に異なる色を与えるこのグラフの彩色である．

▷ register allocation
▷ interference graph

図 11.2 はこれを直線的な命令列に適用したものである．命令 2 の後では p も a も再び読まれないので，どちらのレジスタも b を受け取れる．命令 4 の後では b も c も再び読まれないので，どちらのレジスタも d を受け取れる．同時に生存する値の組は p と a，b と c の 2 組だけなので，2 個のレジスタで 5 個の値をまかなえる．

分岐を含むプログラムについても，ブロックの IN 集合と OUT 集合を用いて同じ推論を行う．図 11.1 では t は OUT[B4] に属さない．B4 の 2 つの後続のうち，B2 は t を読む前に再定義し，B5 は t を読まないので，t は後退辺上で死んでいる．t のレジスタは，t の最後の読み出しから B2 が再び書くまで，ほかの値のために空いている．逆に B2 と B4 は s を読まないが，s はそれらの IN 集合と OUT 集合に属する（表 11.1）．B3 と B5 が後でそれを読むので，s はそれらの読み出しに至るすべての経路で生存し続け，ループ全体を通じてレジスタを保持する．

ある点で生存している値がレジスタの数より多いと彩色は失敗し，いくつかの

スピルを伴うグラフ彩色によるレジスタ割り付け（彩色に失敗したときスピルする）は Chaitin [72] にさかのぼる．Aho ら §8.8 [70] はこれを標準的な大域的手法として示している．

	到達定義（第 10 章）	生存変数
定義域	定義の集合	変数の集合
方向	前向き	後ろ向き
伝達関数	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$
合流	先行の $\cup$	後続の $\cup$
方程式	$\mathrm{OUT}[B] = f_B(\mathrm{IN}[B])$, $\mathrm{IN}[B] = \bigcup_P \mathrm{OUT}[P]$	$\mathrm{IN}[B] = f_B(\mathrm{OUT}[B])$, $\mathrm{OUT}[B] = \bigcup_S \mathrm{IN}[S]$
境界条件	$\mathrm{OUT}[\mathrm{ENTRY}] = \varnothing$	$\mathrm{IN}[\mathrm{EXIT}] = \varnothing$
初期化	$\mathrm{OUT}[B] = \varnothing$	$\mathrm{IN}[B] = \varnothing$

表 11.2: 到達定義と生存変数解析の対比．

値はレジスタファイルから出なければならない．

定義 11.6 (スピルとフィル). **レジスタスピル**は，生存している値をレジスタからメモリ（通常はスタックスロット）に格納してレジスタを空ける操作である．その値を次の使用の前にレジスタへ戻す後のロードが**フィル（再ロード）**である．

▷ register spill

▷ fill (reload)

PTX にはレジスタ割り付けがない．そのレジスタは仮想的で個数に制限がなく，フロントエンドが出力する PTX にはスピルが含まれない．レジスタは `ptxas` が PTX を実際のアーキテクチャ向けの SASS に変換するときに割り付けられ，それが挿入するスピルはスレッドのローカルメモリ（第 3 章）に置かれる．CPG §5.3.2 は，ローカルメモリに置かれる変数の 1 つとして “any variable if the kernel uses more registers than available (this is also known as register spilling)” を挙げている [5]．ローカルメモリはスレッドに私的だがデバイスメモリ上にあるので，L1 キャッシュでミスしたフィルはグローバルメモリアクセスと同じだけのコストがかかる．

PTX カーネルは必要なだけ仮想レジスタを宣言する．たとえば `.reg .b32 %r<100>;` は `%r0` から `%r99` を宣言する（PTX ISA，Parameterized Variable Names [19]）．

GPU ではアロケータが 2 つのコストを比較衡量する．スレッドあたりのレジスタが少なければ常駐ワープが増え，したがってレイテンシ隠蔽が進む（第 3 章）．レジスタが多ければスピルが減る．1 個のレジスタが効くことがある．65,536 個のレジスタを持つ SM では，512 スレッドのブロックは，スレッドあたり 64 レジスタなら 2 個載るが，65 レジスタでは 1 個しか載らない（CPG §5.2.3 Multiprocessor Level [5]）．上限はスレッドあたり 255 レジスタであり（CPG Table 21 [5]），プログラマは `-maxrregcount` オプションまたは `__launch_bounds__()` 修飾子でこれを下げ，占有率のためにスピルを受け入れることができる．

ヒント `--ptxas-options=-v` を付けてコンパイルすると，コンパイラはカーネルごとにレジスタ数とローカルメモリ使用量（`lmem`，CPG §5.3.2 [5]）を報告する．`ptxas` はスピルのストアとロードのバイト数も表示し，それが 0 でなければアロケータのレジスタが足りなかったことを意味する．

11.1.3 生存変数解析と到達定義の比較

2 つの解析はデータフロー問題の標準的な対である．反復解法は同じで，集合が異なり，方向が逆である（表 11.2）．

> **要点**
> 生存変数解析は $\mathrm{IN}[\mathrm{EXIT}] = \varnothing$ から後ろ向きに，$\mathrm{IN}[B] = use_B \cup (\mathrm{OUT}[B] - def_B)$，$\mathrm{OUT}[B]$ を後続の和集合として計算する．同時に生存することのない値はレジスタを共有できる．同時に生存する値が多すぎると，`ptxas` はその一部をローカルメモリにスピルする．

11.2 静的単一代入

通常の 3 番地コードでは変数が何度も代入されうるので，どの解析もまず，ある使用がどの代入を指すかを突き止めなければならない．これは到達定義が答える問いである．定義からそれが到達する使用への結びつきが**定義・使用連鎖**である．静的単一代入形式は，この連鎖をプログラムの名前そのものに組み込む．

出典 M11，SSA；Aho ら §6.2 Three-Address Code [70]；Cytron ら [71]．

▷ def-use chain

定義 11.7 (静的単一代入). すべての変数がプログラムテキストのちょうど 1 か所で代入されているとき，プログラムは **SSA 形式**である．この性質を**静的単一代入**（SSA）と呼ぶ．**静的**というのは，その 1 つの代入がループの中で何度も実行されうるからである．プログラムは，変数の各定義に新しい版 a1，a2，…を与え，各使用をそれに到達する版に名前を付け替えることで変換される．

▷ SSA form

▷ static single assignment (SSA)

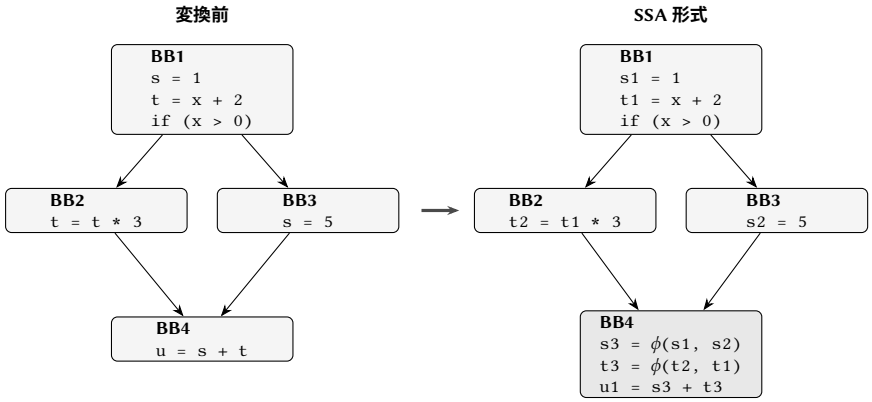


図 11.3: SSA 形式への変換．各定義は新しい版を得る．合流点 BB4 では，ϕ 関数が BB2（第 1 引数）と BB3（第 2 引数）から到着する s と t の版を合流させる．

SSA 形式では使用がその定義を名指すので，定義・使用連鎖は直接の結びつきであり，別の解析を必要としない．定数伝播，不要コード除去をはじめ多くの最適化が単純になる．SSA がほとんどのコンパイラの中間表現の基礎となっているのはそのためである．[1]

LLVM IR はすべての仮想レジスタについて SSA 形式である（LLVM Language Reference [69]）．第 10 章の NVPTX バックエンドはプログラムをその形式で受け取る．

[1] LLVM のフロントエンドは自分では SSA を構築しない．Clang はすべての局所変数をロードとストアを伴うスタックスロット（`alloca`）として出力し，`mem2reg` パスがそうしたスロットを SSA レジスタに昇格させる．その際，以下で導入する支配辺境に基づく標準的なアルゴリズムで ϕ 関数を挿入し，枝刈りされた SSA（pruned SSA）を得る（LLVM のパス [73]）．

11.2.1 合流と φ 関数

直線的なコードでは名前の付け替えは簡単である．複数の先行を持つブロックではそうはいかない．図 11.3 では，BB4 における s の使用に BB2 経由では s1 が，BB3 経由では s2 が到達し，両方の経路について正しい単一の名前は存在しない．

定義 11.8 (合流点と φ 関数). **合流点**とは，2 つ以上の先行を持つブロックである．合流点の先頭に置かれる **φ 関数** $x_3 = \phi(x_1, x_2)$ は先行ごとに 1 つの引数を持ち，制御が到着した辺に対応する引数を値とする新しい版 x_3 を定義する．ϕ の後では，以降のすべての使用が x_3 を指す．

▷ join point

▷ phi function

図 11.3 の `s3 = φ(s1, s2)` は，制御が BB2 から来たときは s1，BB3 から来たときは s2 である．t3 は t2（BB2）と t1（BB3）を合流させる．すると `u1 = s3 + t3` の使用は，名前ごとにちょうど 1 つの到達定義を持つ．ϕ はどの機械の命令でもない．プログラムが SSA 形式を抜けるとき，それは各入力辺上の移動命令になるか，2 つの経路が短ければ，プレディケートによって 2 つのレジスタの一方を選ぶ選択命令，すなわち**条件付き移動**になる．

LLVM は ϕ を，各値の横に先行のラベルを添えて書く．`mem2reg` が残したままの，それ以上最適化する前の SSA 形式では，関数 `min(a, b)` は分岐を保ち，2 つの結果をブロック end で合流させる．

▷ conditional move

条件付き移動の GPU における対応物は PTX の `selp` 命令であり，プレディケートオペランドによって 2 つのソースオペランドの一方を選ぶ（PTX ISA [19]）．第 6 章のガード付き `@p` 実行と異なり，この命令自体はプレディケートされない．

```
define i32 @min(i32 %a, i32 %b) {
entry:
  %lt = icmp slt i32 %a, %b
  br i1 %lt, label %ta, label %tb
ta:
  br label %end
tb:
  br label %end
end:
  %r = phi i32 [ %a, %ta ], [ %b, %tb ]
  ret i32 %r
}
```

ターゲット向けに下げられるとき，`%r` は両方の入力辺で書かれるレジスタになるか，1 つの条件付き移動になる．

SimplifyCFG はこのダイヤモンドを 1 つの `select i1 %lt, i32 %a, i32 %b`，すなわち IR の条件付き移動に置き換える（LLVM Language Reference [69]）．続いて InstCombine がこの min のパターンを `@llvm.smin.i32` に正規化する．

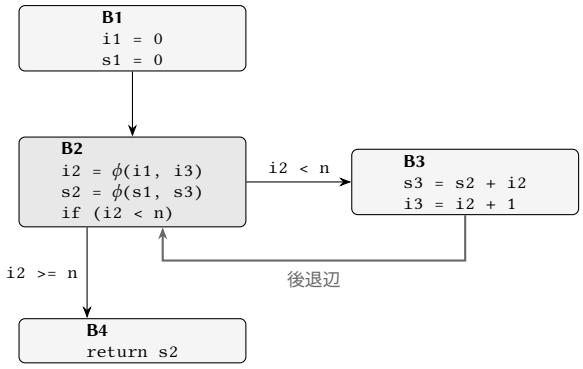


図 11.4: 数え上げループの SSA 形式．ループヘッダ B2 は入口からの経路と後退辺を合流させるので，i と s はそこで，そしてそこでのみ ϕ 関数を得る．

11.2.2 φ 関数をどこに置くか

最も単純な配置は，すべての合流点にすべての変数の ϕ を挿入する．これは正しいが無駄が多い．図 11.3 では変数 u は BB4 でしか定義されず，x は再定義されないので，どちらの ϕ も 1 つの値をそれ自身と合流させるだけである．ϕ が必要なのは，異なる定義が実際に出会う場所だけである．

定義 11.9 (経路収束条件). **経路収束条件**は，次のすべてが成り立つとき，ブロック z に変数 a の ϕ を要求する．

▷ path-convergence criterion

1. あるブロック x が a の定義を含む．
2. あるブロック $y \neq x$ が a の定義を含む．
3. x から z への空でない経路 P_{xz} がある．
4. y から z への空でない経路 P_{yz} がある．
5. P_{xz} と P_{yz} は z 以外に共通の節点を持たない．
6. z は，P_{xz} と P_{yz} の両方で終端より前に現れることはない．ただし一方に現れることはありうる．

開始節点はすべての変数の定義を含むものとみなす．挿入した ϕ はそれ自体が a の新しい定義なので，それ以上 ϕ が必要なくなるまでこの条件を繰り返し適用する．

ϕ を節約するのは条件 5 である．経路がそれより前の合流点ですでに出会っている 2 つの定義は，そこで合流済みである．その前の合流点の ϕ が先へ流れる唯一の定義であり，さらに下流の 2 つ目の ϕ は冗長になる．条件 6 はループを許す．経路は，後退辺に沿って 2 度目に z に到着する前に，一度 z を通過してよい．開始節点の規則は，一方の分岐でのみ代入される変数で効いてくる．図 11.3 の BB3 が w = 7 も設定し，それ以前に w への代入がないとすると，開始節点から BB2 を通る経路と BB3 からの経路は BB4 で初めて出会うので，BB4 は w2 = ϕ(w0, w1) を受け取る．その第 1 引数は開始節点の初期版 w0 である．

注意 講義 vs 論文：実用のコンパイラは経路を探索しない．Cytron らは定義ブロックの反復支配辺境に ϕ 関数を置き，その時間は実際上は線形（最悪の場合は 2 乗）である [71]．mem2reg が構築する枝刈りされた（pruned）SSA は，結果が使われない ϕ も取り除く（LLVM のパス [73]）．

例 11.10. 図 11.4 は n 未満の整数を合計する．名前の付け替え前には，i と s は B1 と B3 で定義される．経路 B1→B2 と B3→B2 は B2 だけを共有するので，B2 で条件 5 と 6 がともに成り立ち，各変数はそこで ϕ を得る．その第 2 引数は後退辺から来る．B4 では，B1 からのすべての経路と B3 からのすべての経路が B2 を通るので条件 5 が成り立たず，B4 は s の 2 つの定義が到達するにもかかわらず s2 を直接使う．n は開始時にしか定義されないので条件 2 が成り立たず，どこにも ϕ は要らない．反復によっても何も加わらない．B2 の新しい定義がほかの定義と出会うのは B2 自身だけであり，そこにはすでに ϕ がある．

より高速なアルゴリズムは支配に基づく．これは第 6 章の後支配の前向き版である．ブロック d がブロック n の**支配節点**であるとは，入口から n へのすべての経路が d を通ることをいう．d の**支配辺境**は，d が z のある先行を支配するが z 自身は狭義には支配しないようなブロック z の集合，すなわち d からの経路が d を迂回する経路と最初に出会うブロックである [71]．d にある定義は，ちょうど d の支配辺境のブロックに，次にそれらのブロックの支配辺境に，と順に ϕ 関数を必要とする．これは経路を列挙することなく経路収束条件の結果を与える．

▷ dominator

▷ dominance frontier

図 11.4 では B2 が B3 と B4 を支配する．B3 の支配辺境は {B2} であり，ちょうどその定義に ϕ が必要な場所である．支配節点：Aho ら §9.6 [70]．

要点
SSA はすべての代入に固有の名前を与えるので，各使用はただ 1 つの定義を持つ．合流点では，ϕ 関数が異なる辺から到着する版を合流させる．ϕ が必要なのは異なる定義が最初に出会う場所だけであり，それは経路収束条件（または支配辺境）が決める．ϕ は移動命令か条件付き移動に下げられる．

11.3 コンパイラ最適化の例

以上の解析は変換を正当化するためにある．以下の各最適化はプログラムの意味を保たなければならず，それぞれがデータフロー解析または SSA 形式の与えるプログラムについての事実を必要とする．例は CUDA C++ または PTX で示し，表 11.3 にまとめる．

出典 M11，コンパイラ最適化；Aho ら §9.1 The Principal Sources of Optimization [70]；CPG §7.39 #pragma unroll，§7.1.5 [5]．

11.3.1 ループ展開

定義 11.11 (ループ展開). **ループ展開**と呼ばれる変換は，ループ本体を u 回（**展開係数**）複製し，新しいループの 1 回の反復が元の u 回の反復の仕事をするようにする．反復回数が未知であるか u の倍数でないときは，展開したループの後の**エピローグループ**が残りの反復を 1 回ずつ実行する．

▷ loop unrolling

▷ epilogue loop

```
// 変換前
for (int i = 0; i < n; i++)
  y[i] = c * x[i];

// 4倍に展開し，エピローグループを付けたもの
int i = 0;
for (; i + 3 < n; i += 4) {
  y[i]     = c * x[i];
  y[i + 1] = c * x[i + 1];
  y[i + 2] = c * x[i + 2];
  y[i + 3] = c * x[i + 3];
}
for (; i < n; i++)          // 0--3回の反復
  y[i] = c * x[i];
```

展開により，比較と分岐の列，およびインクリメント命令の 4 つに 3 つが消える．さらに重要なのは，4 つの独立な乗算とメモリアクセスが 1 つの基本ブロックに入ることである．そこではスケジューラがそれらを交互に並べてレイテンシを隠し，ベクトル化コンパイラはそれらを結合できる．代償はコードサイズと，GPU ではレジスタである．4 つのロードされた値は同時に生存する．

“By default, the compiler unrolls small loops with a known trip count”．`#pragma unroll` は省略可能な係数とともに任意のループを制御し，`#pragma unroll 1` は展開を禁止する（CPG §7.39 [5]）．

注意 カーネルをレジスタ予算の上限を超えさせる展開係数は，分岐で節約する以上のコストを，占有率の低下やスピルによって招くことがある．

11.3.2 関数のインライン展開

定義 11.12 (関数のインライン展開). **関数のインライン展開**と呼ばれる変換は，呼び出しを，呼ばれる関数の本体のコピーで置き換え，仮引数に実引数を代入する．その判断には呼び出し側と呼ばれる側の両方についての事実が必要なので，これは前節までの手続き内解析とは対照的な**手続き間解析** による判断である．

▷ function inlining

▷ inter-procedural analysis

```
__device__ float sq(float v) { return v * v; }
__device__ float norm2(float a, float b) {
  return sq(a) + sq(b);
}
// 呼び出し norm2(x, y) で両方の段をインライン展開した後:
float r = x * x + y * y;
```

インライン展開は，呼び出しと復帰，引数の受け渡し，スタックへのレジスタの退避を取り除く．さらに本体を呼び出し側の最適化にさらす．呼び出し地点の定数が本体に伝播し，これらの引数では不要となるコードが消える．代償はコードサイズである．各呼び出し地点が自分のコピーを受け取るからである．

CUDA コンパイラは `__device__` 関数を “when deemed appropriate” にインライン展開する．`__noinline__` は展開しないよう求め，`__forceinline__` は展開を強制する（CPG §7.1.5 [5]）．

11.3.3 不要コード除去

定義 11.13 (不要コード除去). **不要コード除去**と呼ばれる最適化は，結果がプログラムの出力にまったく影響しない命令を取り除く．直後に死んでいる変数への代入と，決して実行されないコードである．

▷ dead code elimination

```
ld.param.f32  %f1, [x];
mul.f32       %f2, %f1, %f1;    // 不要: %f2 は
add.f32       %f2, %f1, 1.0;    //   ここで上書きされる
st.global.f32 [%rd1], %f2;
```

`mul.f32` が書いた積は，どの命令にも読まれる前に上書きされる．`%f2` は `mul` の後で定義 11.1 の意味で死んでいるので，この命令は取り除かれる．この議論は生存変数解析による議論である．SSA 形式ではさらに単純になる．使用のない定義は見ただけで不要とわかるからである．ある命令を取り除くと，それが読んでいた定義が今度は不要になることがあるので，除去は繰り返される．

注意 副作用を持つ命令（ストア，アトミック，バリア）は，結果が使われないというだけでは決して不要にならない．取り除かれるのは純粋な計算だけである．

11.3.4 定数伝播と定数畳み込み

定義 11.14 (定数伝播と定数畳み込み). **定数伝播**と呼ばれる最適化は，ある使用に到達するすべての定義が同じ定数を代入するとき，その変数の使用を定数で置き換える．**定数畳み込み**と呼ばれる最適化は，オペランドがすべて定数である演算をコンパイル時に評価する．

▷ constant propagation

▷ constant folding

```
// 変換前
mov.f32  %f1, 2.0;
mov.f32  %f2, 4.0;
add.f32  %f3, %f1, %f2;
mul.f32  %f4, %f3, %f5;
// 伝播，畳み込み，伝播の後
mul.f32  %f4, %f5, 6.0;
```

2 つの定数を `add` に伝播すると $2.0 + 4.0$ となり，畳み込みがこれを定数 6.0 に変える．その定数を `mul` に伝播すると，2 つの `mov` 命令と `add` は使用を持たなくなり，不要コード除去がそれらを取り除く．こうして 1 つの最適化が次の最適化を可能にする．

定数伝播は到達定義（第 10 章）の利用者である．ある使用に到達するすべての定義が同じ定数を代入するなら，その使用は定数である．同じく第 10 章のコピー伝播は，コピーについて同じように働く．

11.3.5 強さの低減

定義 11.15 (強さの低減). **強さの低減**と呼ばれる最適化は，演算をそれと等価でより安価な演算で置き換える．除算を乗算に，乗算をシフトまたは加算に，べき乗関数の呼び出しを乗算に，である．

▷ strength reduction

2 つの古典的な例は，右シフトになる 2 による除算と，値とそれ自身の乗算になる 2 乗である．PTX では，2 乗を `ex2.approx.f32` の形にして，次の組として示される．

```
div.rn.f32     %f2, %f1, 2.0;    // 次の行になる
mul.f32        %f2, %f1, 0.5;
ex2.approx.f32 %f3, %f1;         // 次の行になる
mul.f32        %f3, %f1, %f1;
```

浮動小数点の 2 による除算は，0.5 による乗算と厳密に等しい．どちらも 2 のべき乗による拡大縮小だからである．シフトは整数のものである．`div.u32 %r2, %r1, 8` のような 2^k による符号なし除算は右シフト `shr.u32 %r2, %r1, 3` になるが，符号付きの除算は負の値に対する補正を必要とする．2 乗は呼び出しから低減される．`pow(a, 2)` は `a * a` になる．2 のべき乗でない定数による除算 $x/3$ は，浮動小数点では一部の結果を変えずに $x \cdot (1/3)$ に置き換えることはできない．したがってコンパイラがそれを行うのは，プログラムが近似除算を許すときだけである．単精度については `nvcc --prec-div=false`（`-use_fast_math` が含意する）がこれを許す．実行時にのみ 2 のべき乗となる除数はコンパイラの手に負えない．第 5 章はその書き換えを手作業で示している．

注意 講義 vs ISA：どちらの組もラベルが誤っている．1 つ目は浮動小数点の除算を乗算に変えたものでありシフトではない．右シフトが除算の代わりになるのは符号なし整数か非負の整数だけである．負の符号付きの値では，右シフトは $-\infty$ 方向に丸めるが，除算は 0 方向に丸めるからである．また `ex2` は x^2 ではなく 2^x を計算する（PTX ISA [19]）ので，2 つ目の組は等価ではない．正しい 2 乗の例は `pow(a, 2)` → `a * a` である．

最適化	節約するもの	知る必要があること
ループ展開	分岐，カウンタの更新．並列性を露出する	反復回数，レジスタ予算
関数のインライン展開	呼び出しのオーバーヘッド．他の最適化を可能にする	呼ばれる関数の本体（手続き間）
不要コード除去	結果が使われない命令	生存変数解析または SSA の使用
定数伝播と定数畳み込み	定数の実行時の計算	到達定義
強さの低減	高価な演算	より安価な形との厳密な等価性
ループ不変式の移動	ループ内の繰り返しの仕事	到達定義，ループ構造

表 11.3: 本節の最適化とその背後にある解析．

11.3.6 ループ不変式の移動と誘導変数

定義 11.16 (コード移動). **コード移動**と呼ばれる変換は，計算をプログラムの別の場所，すなわち同じ結果でより少ない回数しか実行されない場所へ移す．その最も一般的な形である**ループ不変式の移動**は，オペランドがループ内で変化しない計算をループの外に引き上げ，毎回の反復ではなく 1 回だけ実行されるようにする．

▷ code motion

▷ loop-invariant code motion

```
// 変換前: scale は毎回の反復で計算し直される
for (int i = 0; i < n; i++) {
  float scale = alpha / beta;
  y[i] += scale * x[i];
}
// 変換後: ループの前で1回だけ計算する
float scale = alpha / beta;
for (int i = 0; i < n; i++)
  y[i] += scale * x[i];
```

動的命令数は反復ごとに除算 1 回分だけ減る．これが許されるには，コンパイラは `alpha` も `beta` もループ内で変更されないこと（到達定義の問題である）と，ループが 0 回実行されるときに除算を実行しても害がないことを知っていなければならない．

命令がループ不変であるのは，その各オペランドが定数であるか，ループの外で定義されるか，別のループ不変な命令によって定義されるときである．この判定は繰り返し適用される（Aho ら §9.1 [70]）．

定義 11.17 (誘導変数). **誘導変数**とは，ループを制御するカウンタ `i` のように，ループの毎回の反復で同じ量だけ変化する変数である．PTX ではその更新は `add.s32 %r4, %r4, 1;` のような単一の命令である．

▷ induction variable

誘導変数を認識すると，さらなる最適化が可能になる．`x[i]` のアドレスは $base + 4i$ であり，それ自身が反復ごとに 4 ずつ増える誘導変数なので，強さの低減によって乗算をポインタへの 4 の加算に置き換えられる．展開に必要なループの反復回数は，カウンタの初期値，増分，上限から求まる．

11.4 ダイバージェンス解析

第 6 章では，ハードウェアがダイバージェント分岐をどう実行するかを示した．SIMT スタックは経路を部分的なマスクのもとで順に実行し，再収束点，通常は分岐の直接後支配節点でそれらを合流させる．再収束の情報を与えるのはコンパイラであり，ワープ内で一様であると証明できた分岐についてはスタック操作を省くか，より単純なコードを出力できる．したがってコンパイラは，どの分岐がダイバージェントになりうるかを静的に知る必要がある．

出典 M11，ダイバージェンス解析；CPG §5.4.2 Control Flow Instructions，§4.1 SIMT Architecture [5]；PMPP ch. 10 [6]．

定義 11.18 (ダイバージェンス解析). **ダイバージェンス解析**と呼ばれる静的解析は，カーネルの各分岐について，その条件がワープ内の異なるスレッドで異なる値に評価されうるかどうかを判定する．その出力は，ダイバージェントになりえて再収束の情報を必要とする分岐の集合である．それ以外の分岐はすべて一様である．

PTX はその結果を記録できる．`bra.uni` は，ワープのすべてのアクティブなスレッドが同じプレディケートと分岐先を持ち，ダイバージェントにならないことが保証された分岐を示す（PTX ISA，`bra` [19]）．

▷ divergence analysis

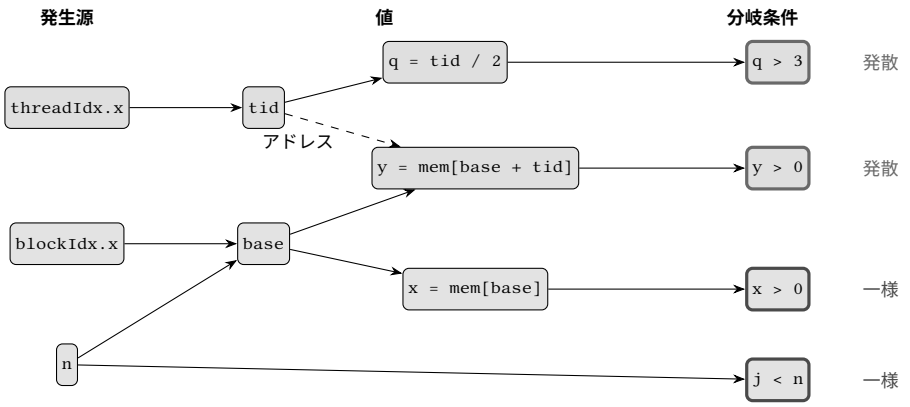


図 11.5: 上のカーネルの定義・使用の辺（発生源から分岐条件まで）．スレッド依存（赤）は `threadIdx.x` から広がる．分岐がダイバージェントになりうるかどうかは，その条件が決める．

この問いは GPU より古い．ループをベクトル命令に変換する**ベクトル化**を行うコンパイラは，まとめて詰め込むすべての反復が，ループ本体の分岐の内部も含めて，同じ命令を足並みをそろえて実行することを確かめなければならない．GPU のコンパイラはこれらの判定を受け継いだ．単純な if-else なら判定は容易だが，複雑な制御フローと同期があると難しくなる．

▷ vectorization

2 つの設定は同じ問題である．ベクトル化されたループの反復がワープのスレッドの役割を果たし，ループインデックスによる分岐がスレッドインデックスによる分岐の役割を果たす．

11.4.1 スレッド依存値と一様値

定義 11.19 (スレッド依存値と一様値)．式がプログラム点 p で**スレッド依存値**であるとは，p において異なるスレッドで異なる値に評価されうることをいう．ワープ（より強くはカーネル）のすべてのスレッドで同じ値を持つとき，それは p で**一様値**である．分岐がダイバージェントになるのは，その条件がスレッド依存であるときに限る．

▷ thread-dependent value

▷ uniform value

スレッド依存の源はスレッドインデックスである．`threadIdx` はレーンごとに異なる．カーネルの引数，`blockIdx`，`blockDim` と定数はブロック内で一様であり，それらだけから計算される値も一様である．依存はそこから定義・使用連鎖に沿って広がる．スレッド依存のオペランドから計算される値はスレッド依存である．次のカーネルは 4 つの分岐を持つ．

```
__global__ void k(const int *mem, int *out, int n) {
  int tid  = threadIdx.x;
  int q    = tid / 2;
  int base = blockIdx.x * n;
  int x    = mem[base];         // 全員が同じアドレス
  int y    = mem[base + tid];   // スレッドごとのアドレス
  int acc  = 0;
  if (q > 3)  acc += 1;         // ダイバージェント
  if (y > 0)  acc += 2;         // ダイバージェント
  if (x > 0)  acc += 4;         // 一様
  for (int j = 0; j < n; j++)   // 一様
    acc += j;
  out[base + tid] = acc;
}
```

図 11.5 に定義・使用の辺を示す．条件 `q > 3` は `tid` と `q` を介して `threadIdx.x` に依存する．間接的な依存もやはり依存である．2 つのロードはアドレスが異なる．メモリからロードされた値は，アドレスがスレッド依存ならスレッド依存である．異なるスレッドが異なるワードを読むからである．`mem[base]` はすべてのスレッドが 1 つのアドレスで読むので，`x` は一様であり，それによる分岐も一様である．ループ条件 `j < n` は引数 `n` と一様なカウンタ `j` にだけ依存する．すべてのスレッドが同じ回数だけ反復する．

したがって分岐の判定は，定義・使用連鎖上の到達可能性の問いになる．ス

ダイバージェンスに必要なのはワープ内で一様であることだけである．`blockIdx.x` と `base` はブロック間では異なるが，ワープが 2 つのブロックにまたがることはないので，1 つのワープのスレッド間で異なることはない．

レッドインデックスに依存する定義が条件に到達するか，である．到達定義と同様に，依存の事実は何も変わらなくなるまで制御フローグラフを通じて伝播される．ループがあるため反復が必要になる．値は自分自身の後退辺を通じてスレッド依存になりうるからである．

11.4.2 保守的解析

定義 11.20 (保守的解析). 静的解析は，その近似が安全側にのみ誤るとき**保守的解析**である．ここでは，一様な分岐をダイバージェントと報告することはあっても，ダイバージェントな分岐を一様と報告することは決してない．

一様な分岐をダイバージェントとして扱っても，失うのは性能だけである．不要な再収束点とそのためのスタック操作である．ダイバージェントな分岐を一様として扱うのは誤りなので，実用的な解析はすべて保守的である．難しいのは精度である．`threadIdx` から導かれる値をすべてスレッド依存とみなす解析は，すぐにすべての分岐とすべてのループをダイバージェントの可能性ありとみなし，役に立たなくなる．そこでコンパイラはそうした偽陽性を刈り込む規則を加える．たとえば，ブロック内のワープ間では異なるがワープ内では一様な値（`threadIdx.x / warpSize` など）を認識する．そして制御フローグラフが大きく不規則になるほど，安全かつ精密であり続けることは難しくなる．

> **要点 — 章のまとめ**
> 生存変数解析は後ろ向きのデータフロー問題である．$\mathrm{IN}[B] = use_B \cup (\mathrm{OUT}[B] - def_B)$，$\mathrm{OUT}[B]$ は後続の IN 集合の和集合，$\mathrm{IN}[\mathrm{EXIT}] = \varnothing$ とし，反復によって不動点まで解く．レジスタ割り付けは生存範囲の干渉グラフを彩色する．レジスタが足りなくなると `ptxas` はローカルメモリにスピルし，占有率とメモリトラフィックを引き換えにする．SSA 形式はすべての定義に固有の名前を与え，異なる定義が最初に出会う合流点で ϕ 関数によって版を合流させる．ループ展開，インライン展開，不要コード除去，定数伝播と定数畳み込み，強さの低減，ループ不変式の移動は，それぞれそうした解析に基づく．ダイバージェンス解析は，スレッド依存を `threadIdx` から定義・使用連鎖に沿って（健全であるためには制御依存にも沿って）分岐条件まで伝播させる．解析は保守的でなければならず，その価値は一様な分岐を刈り込むことにある．

注意 講義はデータ依存だけを追うが，それでは健全な解析には足りない．`b = 0; if (q > 3) b = 1;` の後，合流点における `b` の値は，両方の代入が定数であるにもかかわらずスレッド間で異なる．その 2 つを選んだ分岐がダイバージェントだったからである．健全な解析は，ダイバージェント分岐のもとで定義され，その合流点で合流される値（SSA 形式では ϕ）もスレッド依存とみなす．これが制御依存である．

▷ conservative analysis

CPG §5.4.2 は，素朴な解析がダイバージェントとみなす標準的な一様分岐を挙げている．`threadIdx / warpSize` だけに依存する条件はダイバージェントになりえない．ワープは連続したスレッド ID からなるからである（CPG §4.1 [5]）．しかしこの条件は `threadIdx` から導かれる．

ヒント 分岐条件を `threadIdx.x / 32`（1 次元のブロックの場合），`blockIdx`，またはカーネルの引数によって書けば，ワープ内で一様に保たれる．これが解析に対するプログラマの側の取り組みである（第 5 章）．

第 12 章

GPU による機械学習の高速化

講義動画	M12，レッスン 1–3（GPU and ML；Floating Point Formats；Tensor Core）
資料	NVIDIA A100 [1] および H100 [2] ホワイトペーパー（指定，テンソルコアの資料として）；NVIDIA GEMM ガイド [54]（任意）；PTX ISA [19]；IEEE 754-2019 [74]；Cambier ら [75]（S2FP8）；Micikevicius ら [76]（FP8）；Kung [77]；Jouppi ら [78]；Dao ら [79, 80]（FlashAttention，背景）
教科書	K12 [3] §1.3.9；PMPP [6] ch. 14，ch. 16，App. A；CPG [5] §7.24 Warp Matrix Functions，§7.26 Asynchronous Barrier，§7.27 Asynchronous Data Copies；BPG [8] §7.3 Numerical Accuracy and Precision
キーワード	機械学習，GEMM，演算強度，演算／バイト比，テンソルコア，行列積和演算，シストリックアレイ，Transformer Engine，浮動小数点形式，IEEE 754，非正規化数，bfloat16，TF32，FP8，量子化，混合精度，構造化スパース性，SpMV，非同期コピー

機械学習は，この 10 年で GPU を最も大きく形作ってきたワークロードである．第 1 章のテンソルコア，FP16，FP8，Transformer Engine はいずれもそのために追加された．最後の章となる本章は，GPU がなぜ機械学習に適しているのか，そしてそのためにどのような専用ハードウェアを加えるのかを問う．まずニューラルネットワークを支配する演算，とりわけ行列積を特定し，第 8 章の演算強度とルーフラインの考え方でそれを評価する．次に，IEEE 単精度から 8 ビットまで数値形式をたどる．各演算が何バイトを移動するかは形式が決めるからである．最後の部分では行列ユニットそのものの設計に移る．どの演算を高速化するか，ユニットをどう構成するか，メモリからどうデータを供給するか，疎行列のゼロをどう活用するかである．前の 2 章のコンパイラの視点は背景にとどまる．新しいユニットはソフトウェアからは ISA の新しい命令として見える．

12.1 GPU と機械学習

機械学習の分野は，パラメータをデータに合わせて調整したモデルを構築する．今日その支配的なモデルは**深層ニューラルネットワーク**（DNN）であり，それぞれが大きな数値配列を変換する層を積み重ねたものである．本節では，GPU がなぜこのワークロードに適しているのか，それがどのような演算から成るのか，行列積をどう評価するのか，そしてそれを高速化するハードウェアの 2 つの道，すなわち SIMD レーンと専用の行列ユニットを説明する．

▷ machine learning (ML)

▷ deep neural network (DNN)

出典 M12，GPU と ML；PMPP ch. 16；[54]．

12.1.1 GPU が機械学習に適している理由

GPU の 4 つの性質が，ML の学習と推論の要求に合っている．

浮動小数点スループット． ML は高いデータ並列性をもつ非常に多数の浮動小数点演算を行い，GPU のトランジスタの大部分は浮動小数点スループットに充てられている（第 1 章）．

メモリ帯域幅． GPU とそのメモリはパッケージ上で直接接続されているので，積層された広帯域メモリ（HBM）を採用しやすい．CPU ソケットは標準のメモリインタフェースを維持しなければならない．

GPU 当たりの HBM 帯域：900 GB/s（V100），1555 GB/s（A100 40 GB），3000 GB/s（H100 SXM5，暫定値）（AWP Table 4 [1]；HWP Table 3 [2]）．

柔軟な形式． GPU は新しい数値形式を素早く追加する．テンソル形式，bfloat16，8 ビット浮動小数点，INT8，INT4 である（12.2 節）．

緩和された意味論． ML は統計的な計算である．最後の数ビットが異なる結果や，スレッド間で順序が入れ替わった演算は，モデルが学習する内容を変えない（浮動小数点加算は結合的でないので，順序の入れ替えが変えるのは最後の数ビットだけである：BPG §7.3.2 Floating Point Math Is Not Associative）．これは GPU の弱いメモリ一貫性と精密例外の欠如に合っている．どちらも提供するにはハードウェアのコストがかかる．

第 1 章は CPU の精密例外と，それを保証しない GPU とを対比している．

12.1.2 ニューラルネットワークの演算

DNN の層は 3 種類の演算から構成され，いずれも高いデータ並列性をもつ．

定義 12.1 (DNN の演算の種類)．DNN の演算は 3 つの種類に分かれる．

- **要素ごとの演算**は，配列のすべての要素に同じ関数を独立に適用する．典型例は**活性化関数**であり，層のすべての出力に適用される非線形関数である．たとえば $\mathrm{ReLU}(x) = \max(0, x)$ がそうである．
- リダクション（第 1 章）は多数の要素を 1 つにまとめる．DNN での例は**プーリング**であり，配列の各ウィンドウを 1 つの値，たとえばその最大値（最大プーリング）で置き換える．
- **内積** $\sum_k a_k b_k$ は，2 つのベクトルを要素ごとに掛け，その積を合計する．内積は行列積と畳み込みを構成する．

▷ element-wise operation
▷ activation function
▷ pooling
▷ dot product

演算量を支配するのは内積の種類である．**全結合層**はすべての入力をそれぞれ固有の重みですべての出力に結ぶので，入力のバッチと重み行列の積は 1 つの**行列積**になる．**畳み込み層**は小さなフィルタを入力の上で滑らせるが，これも同様に行列積として計算される（PMPP ch. 16）．

▷ fully connected layer
▷ matrix multiplication
▷ convolution layer

12.1.3 GEMM とその演算強度

ライブラリは ML の行列積を 1 つの標準的な形で表す．

定義 12.2 (GEMM)．**GEMM（一般行列積）**という演算は

$$C \leftarrow \alpha AB + \beta C, \tag{12.1}$$

を計算する．ここで A は $M \times K$ 行列，B は $K \times N$，C は $M \times N$ であり，α，β はスカラである．$\alpha = 1$，$\beta = 0$ のとき，これは単純な積 $C = AB$ である．

要素ごとの演算とリダクションはすべての要素を 1 回ずつ読み書きするので，ルーフラインモデル（第 8 章）の意味でメモリ律速となる．オペランドを再利用するのは内積の種類だけである．

▷ GEMM (general matrix multiplication)

C の MN 個の要素はそれぞれ長さ K の内積なので，GEMM は MNK 回の融合積和演算（第 8 章），すなわち $2MNK$ 回の浮動小数点演算を行う．FP16 の入力（要素当たり 2 バイト）では，A と B を読み C を書くので，各行列がメモリインタフェースをちょうど 1 回通るなら $2(MK + NK + MN)$ バイトとなる．その比が演算強度（第 8 章）である．

GEMM ガイドは “a total of 2 * M * N * K FLOPS” と数えている [54, §2]．

$$I_{\mathrm{GEMM}} = \frac{2MNK}{2(MK + NK + MN)} = \frac{MNK}{MK + NK + MN}\ \mathrm{FLOP/B}. \tag{12.2}$$

マシンにもそれ自身の強度があり，ピーク演算性能をメモリ帯域幅で割った値である．両者の比較が，どの資源がカーネルを制限するかを決める．

注意 式 (12.2) の演算強度は 1 次の見積もりである．各行列がメモリからちょうど 1 回読み書きされると仮定し，キャッシュとオンチップでの再利用を無視している．実際のカーネルはより多くのバイト（A と B のタイルは何度も読まれる）か，より少ないバイト（L2 に留まるオペランド）を移動する．

定義 12.3 (演算／バイト比；演算律速とメモリ律速)．プロセッサの**演算／バイト比**は，そのピーク演算性能をピークメモリ帯域幅で割ったもので，単位は FLOP/バイトである．これは第 8 章のルーフラインモデルのリッジポイントにあたる．カーネルの演算強度がこの比のどちら側にあるかで，カーネルは**演算律速／メモリ律速**となる．比より上なら演算ユニットがボトルネック（演算律速）であり，下ならメモリ帯域幅がボトルネック（メモリ律速）である．

▷ ops:byte ratio

例 12.4．FP16 で $M = N = K = 1024$ の正方 GEMM は $I = 1024^3/(3 \cdot 1024^2) = 1024/3 \approx 341\,\mathrm{FLOP/B}$ をもち，V100 の 138.9 FLOP/B を大きく上回るので，演算律速である．$M = K = 4096$，$N = 16$ の細長い GEMM，すなわち重み行列を 16 個の入力のバッチに適用する場合は

FP16 のテンソルコア入力と FP32 の累積を用いる V100：“the FLOPS:B ratio is 138.9 if data is loaded from the GPU’s memory” [54, §2]．

$$I = \frac{4096 \cdot 16 \cdot 4096}{4096 \cdot 4096 + 16 \cdot 4096 + 4096 \cdot 16} = \frac{16 \cdot 4096}{4096 + 32} \approx 15.9\,\mathrm{FLOP/B}$$

となり，比を大きく下回る．これはメモリ律速であり，演算ユニットを速くしても速くならない．小さい次元 N が演算強度を抑える．$N \le M, K$ なら常に $I < N$ だからである．

ヒント $N \ll M, K$ の細長い GEMM では，FP16 で $I \approx N\,\mathrm{FLOP/B}$（FP8 では $2N$）である．演算強度は最も小さい次元から読み取れる．

GEMM ガイドは，式 (12.2) が捉えない 2 つの効果を加えている．GPU は C をタイル単位で計算し，各タイルを 1 つのスレッドブロックに割り当てるからである．第 1 の効果である**タイル量子化**は “occurs when matrix dimensions are not divisible by the thread block tile size”[54, §3.1] というものである．行や列の最後のタイルは

▷ tile quantization

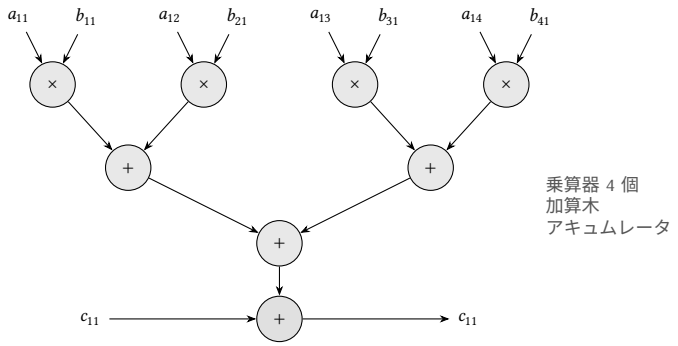


図 12.1: SIMD の内積 1 ステップ．4 個の乗算器，加算木，累積が $c_{11} \mathrel{+}= \sum_k a_{1k} b_{k1}$ を計算する．4×4 の積にはこのステップが 16 回必要である．

一部しか役に立たないが，完全なタイルと同じコストがかかる．第 2 の**ウェーブ量子化**は，“the total number of tiles is quantized to the number of multiprocessors on the GPU”[54, §3.2] であるために生じる．タイルは SM 当たり 1 タイルのウェーブとして実行され，タイルの少ない最後のウェーブはほとんどの SM を遊ばせる．テンソルコアはまた，GEMM の次元が FP16 で 8 要素の倍数（A100 では 64 の倍数）のとき最も効率がよい [54, §2.2]．

▷ wave quantization

ガイドの例：108 個の SM をもつ A100 では，$9 \times 12 = 108$ タイルがちょうど 1 ウェーブになるが，$9 \times 13 = 117$ タイルではタイルが 9 個しかない第 2 のウェーブが加わる [54, §3.2]．

12.1.4 SIMD レーンと行列ユニット

通常の SIMT レーン上では，行列積はスカラの積和演算のループである．Listing 12.1 のカーネルは，C の要素ごとに 1 スレッドを用いて 4×4 の積を計算する．各スレッドは k についてのループを逐次実行し，累積中の和をプライベートなレジスタに保持する．

Listing 12.1: SIMT レーン上の 4×4 行列積．出力要素ごとに 1 スレッド．

```
__global__ void matmul4(const float *A,
                        const float *B, float *C)
{
  const int N = 4;
  int row = threadIdx.y, col = threadIdx.x;
  float sum = 0.0f;           // プライベートなレジスタ
  for (int i = 0; i < N; ++i)
    sum += A[row * N + i] * B[i * N + col];
  C[row * N + col] = sum;
}
// 起動：matmul4<<<1, dim3(4, 4)>>>(dA, dB, dC);
```

CUDA スレッドのレジスタはプライベートなので，1 つの出力要素の累積全体を 1 つのスレッドが行わなければならない．スレッドを減らせば（たとえば行ごとに 1 つの 4 スレッド），単にループが長くなるだけである．逆に 1 つの内積を複数のスレッドに分散させる場合，たとえば 16 スレッドがそれぞれ 1 つの積 $a_{ik} b_{kj}$ を計算する場合は，部分和を共有メモリかワープシャッフル（第 5 章）でリダクションする必要があり，命令と同期が増える．

SIMD データパスは，1 回の演算でせいぜい 1 つの要素しか計算できない．4 個の乗算器が A の 1 行と B の 1 列を受け取り，加算木が 4 つの積を合計し，結果が累積される（図 12.1）．C の第 1 行だけでも，その演算が 4 回必要である．

第 5 章のリダクションのパターン（共有メモリでの木構造リダクション，`__shfl_xor_sync`）がまさにこのオーバーヘッドである．

$$c_{1j} = \sum_{k=1}^{4} a_{1k} b_{kj}, \qquad j = 1, \ldots, 4,$$

▷ multiply-accumulate (MAC)

それぞれが A の第 1 行と B の 1 列との内積であり，2 つの 4×4 行列の積には，出力要素ごとに 1 回，計 16 回のこうした演算が必要である．

定義 12.5 (MAC，MMA，テンソルコア命令). **積和演算（MAC）**は，積を累積中の和に加える演算 $c \leftarrow c + ab$ である．これを 1 回の丸めで行うものが FMA である．**行列積和演算（MMA）**は同じ考えを行列全体に適用し，$D \leftarrow AB + C$ を 1 つ

注意 講義 vs ISA：`HMMA.1688` はワープレベルの形状 $m16n8k8$（16×8 の結果，$K = 8$）を表し，PTX の `mma` の形状 `.m16n8k8` [19] にあたる．4×4 の積ではない．$4 \times 4 \times 4$ という数字は Volta のテンソルコア 1 個のクロック当たりのデータパスである [1]．

▷ matrix multiply-accumulate (MMA)

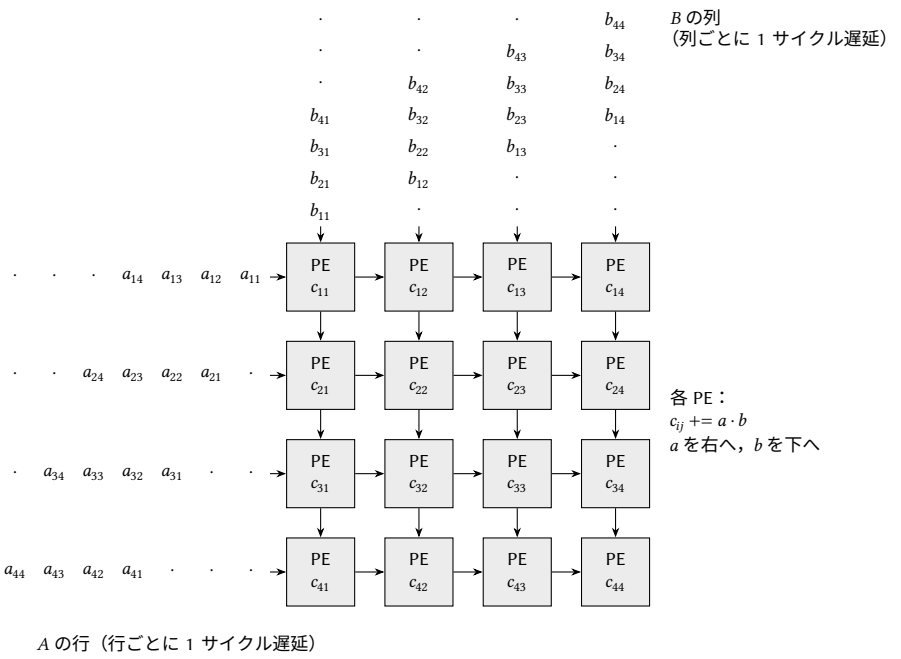


図 12.2: $C = AB$ の開始時における 4×4 の出力定常型シストリックアレイ．スキューにより，a_{ik} と b_{kj} はサイクル $i+j+k-2$ に PE(i, j) で出会う．点は空のスロットを表す．

の演算として行う．テンソルコア（第 1 章）は NVIDIA の MMA ユニットであり，GPU はそれを数百個もつ．NVIDIA の機械語（SASS）では，半精度の MMA 命令は **HMMA** であり，たとえば `HMMA.1688.F16` である．1 つの命令が，SIMT のループでは多数の命令を要する仕事を行う．

▷ HMMA

ワープの 32 スレッドは HMMA を一緒に発行し，それぞれが自分のレジスタからオペランドの断片を供給する．こうしてテンソルコアは上で述べたプライベートレジスタの問題を回避する．テンソルコアの世代は，主としてクロック当たりに完了する積の大きさと，受け付ける形式とで異なる（表 12.1）．A100 のテンソルコアは “can execute 256 FP16 FMA operations per clock”，すなわち $8 \times 4 \times 8$ の混合精度の積を実行し，TF32，BF16，FP64 の入力に加えて INT8，INT4，バイナリ（1 ビット）を受け付ける [1]．整数とバイナリのモードは，両者の間の世代である Turing に由来する．スパース性のサポートはピークをさらに 2 倍にする（12.3 節）．

CUDA C++ はワープレベルの行列関数（`wmma::load_matrix_sync`，`wmma::mma_sync`）を通じてテンソルコアに到達する：CPG §7.24 Warp Matrix Functions．PTX と SASS は第 10 章の主題である．

表 12.1: NVIDIA の 3 世代のテンソルコア（AWP [1]；HWP V1.02，SXM5，暫定値 [2]）．ピーク性能は密／ 2:4 スパース性ありの値．

	V100	A100	H100
SM 当たりのテンソルコア数	8	4	4
コア当たりクロック当たりの FP16 FMA	64	256	512
コア当たりクロック当たりの積	$4 \times 4 \times 4$	$8 \times 4 \times 8$	—
FP16 テンソル TFLOP/s	125	312 / 624	1000 / 2000
TF32 テンソル TFLOP/s	—	156 / 312	500 / 1000
FP8 テンソル TFLOP/s	—	—	2000 / 4000
FP32（非テンソル）TFLOP/s	15.7	19.5	60

12.1.5 シストリックアレイ

行列積のための伝統的なハードウェアはシストリックアレイである．

定義 12.6 (シストリックアレイ). **シストリックアレイ**は，それぞれが MAC ユニッ

▷ systolic array

トである同一の処理要素（PE）を格子状に並べ，隣接する PE どうしだけを接続したものである．データは端から入り，1 サイクルに 1 PE ずつ移動する．各 PE は通過する値で計算し，それを次へ渡す．[1]

図 12.2 の**出力定常型**の配置では，PE(i,j) が出力要素 c_{ij} を受け持つ．A の各行は左から，B の各列は上から入り，各行と各列は前のものに対して 1 サイクルずつ遅れる（**スキュー**）．各サイクルで各 PE は到着した a と b を掛け，その積を定常的に保持する部分和に加え，a を右へ，b を下へ渡す．サイクル 1 で PE$(1,1)$ は $a_{11}b_{11}$ を計算する．サイクル 2 では a_{11} は右へ移って PE$(1,2)$ で b_{12} と出会い，その間に PE$(1,1)$ は $a_{12}b_{21}$ を加える．以下同様である．

このスキューのもとでは，PE(i,j) はステップ k をサイクル $t = i + j + k - 2$ に実行する．$n \times n$ のアレイでは最初の PE がサイクル 1 に開始し，最後の PE(n,n) はサイクル $2n-1$ に開始してサイクル $3n-2$ に終了する．表 12.2 は $n = 4$ を追跡したものである．64 回の MAC に 10 サイクルかかり，16 個の PE すべてが動作するサイクルは 1 つもない．

[1] Kung は，アレイの中をデータが律動的に，脈打つように流れることにちなんでこの構造を名付けた．メモリから取り出された各値は出ていくまでに多くの PE で使われるので，メモリ帯域幅が制約となる場合にこの構造は魅力的である [77]．Google の最初の TPU は 256×256 のアレイ，すなわち “a 65,536 8-bit MAC matrix multiply unit” を中心に構成され，ピークは 92 TOPS である [78]．

表 12.2: 1 回の $4 \times 4 \times 4$ の積における図 12.2 の 4×4 アレイの動作（本章のモデル：PE(i,j) はステップ k をサイクル $i + j + k - 2$ に実行する）．

サイクル	1	2	3	4	5	6	7	8	9	10
動作中の PE	1	3	6	10	12	12	10	6	3	1
完了した出力	–	–	–	1	2	3	4	3	2	1

定義 12.7 (充填と排出). シストリックアレイの**充填と排出**とは，計算の開始時，スキューされた入力がまだ入りつつあり一部の PE だけが動作しているサイクルと，終了時，最後のオペランドが通過して PE が再び遊休になっていくサイクルのことである．

▷ fill and drain

この追跡では，アレイは 16 個の PE で 10 サイクルの間に 64 回の MAC を行い，利用率は $64/160 = 40\,\%$ である．残りは充填と排出である．多数のタイルを連続して流し込めば，前のタイルが排出される間に次のタイルがアレイを充填し，利用率は 100 % に近づく．

注意 講義はアレイがサイクル 4 前後で完全に動作し，約 7 サイクルで終了すると説明しているが，これは上のモデルとは数え方が異なる．本章のモデルでは，サイクル 4 で動作する PE は 16 個中 10 個であり，PE$(4,4)$ はサイクル $2n-1 = 7$ に開始し，積はサイクル $3n-2 = 10$ に完了する．
狭い形式にこのスケーリングが必要な理由は 12.2 節で説明する．FP8 E4M3 が扱える絶対値は 2^{-9} から 448 までしかない．

12.1.6 Transformer Engine とデータ移動

Hopper の Transformer Engine（第 1 章）は，テンソルデータを狭い形式の表現可能な範囲へ動的にスケーリングすることで Transformer モデルの層を高速化し，FP8 演算を実用可能にする [2]．高速な演算は問題の半分にすぎず，ユニットにはデータを供給しなければならない．テンソルメモリアクセラレータ（第 1 章）は，テンソルのブロックをグローバルメモリと共有メモリの間でまとめて移動し，アドレスを自ら生成する．

定義 12.8 (コピー記述子). **コピー記述子**とは，多次元テンソルと，そこからコピーするブロック（ベースアドレス，テンソルのサイズとストライド，ブロックの形状）を記述するデータ構造であり，TMA はそこから転送のすべてのアドレスを生成する．ブロックの座標は各コピー命令とともに与えられるので，1 つの記述子でテンソルのすべてのタイルに対応できる．Hopper では，すべてのスレッドが自分のアドレスを計算する代わりに，転送を発行する前に “a single thread creates a copy descriptor” [2].

▷ copy descriptor

> **要点**
> SIMD/SIMT レーンは行列積を高速化でき，専用の行列ユニットはそれをはるかに高速化する．どちらの場合も，メモリ帯域幅を使い切ることが決定的に重要である．演算強度が演算／バイト比を下回る GEMM は，ユニットを速くしても何も得られない．

12.2 浮動小数点形式

科学技術計算は IEEE の単精度（32 ビット）と倍精度（64 ビット）に依存しているが，ML は精度を下げる．本節では，狭い形式がなぜ得になるのか，浮動小数点数がどう符号化されるのか，そして ML 向けの形式が精度と範囲をどう交換するのかを説明する．

出典 M12，浮動小数点形式；PMPP App. A；BPG §7.3 Numerical Accuracy and Precision；[74, 1, 2]．

12.2.1 低精度が得になる理由

GEMM の浮動小数点演算の回数は形状で決まり，形式には依存しない．移動するバイト数は形式に依存する．入力の幅を半分にするとバイト数が半分になり，したがって演算強度は 2 倍になる．FP8 の入力（要素当たり 1 バイト）では，式 (12.2) は

$$I_{\text{GEMM, FP8}} = \frac{2MNK}{MK + NK + MN}, \tag{12.3}$$

となり，FP16 の値の 2 倍になる．例 12.4 の細長い GEMM は 15.9 から 31.8 FLOP/B に上がる．

式 (12.3) は C も入力の幅で数えている．FP32 の出力なら $2MN$ バイトが加わる．

定義 12.9 (量子化，混合精度，アキュムレータ). 値を，それが計算された形式より少ないビットで表現すること，たとえば FP32 の重みを FP8 で格納することを**量子化**という．**混合精度**演算では，演算の入力は狭い形式で，累積中の和を保持するレジスタまたは変数である**アキュムレータ**は広い形式のままである．典型的には FP16 または FP8 の入力と FP32 の累積である．

▷ quantization
▷ mixed precision
▷ accumulator

量子化には 3 つの利点がある．格納量の削減，演算強度の向上，そして最近のハードウェアでは演算スループットの向上である．同じ面積のユニットがサイクル当たりにより多くの狭い演算を完了するからである．古いユニットでは最初の 2 つだけが当てはまった．オペランドは縮んだが，サイクル当たりの演算数は変わらなかった．

Hopper：“FP8 halves data storage requirements and doubles throughput compared to FP16 or BF16” [2]．表 12.1 の FP16 と FP8 の行を比較せよ．

注意 アキュムレータが広いままなのには理由がある．狭い項の長い和は小さな加数を失う（第 5 章，低精度）．Volta のテンソルコアはすでに FP16 の入力を “FP32 accumulation” で扱っていた [1]．

12.2.2 符号，指数部，仮数部

浮動小数点数は科学的記数法の 2 進版である．10 進では $0.0012 = 1.2 \times 10^{-3}$ と $0.0000012 = 1.2 \times 10^{-6}$ は仮数 1.2 を共有し，指数だけが異なる．2 進の形式も同じように基数 2 を用いる．

定義 12.10 (浮動小数点形式). 2 進の**浮動小数点形式**は，数を 3 つのフィールドに格納する．**符号ビット** s，w ビットの**指数部**フィールド e，p ビットの**仮数部**フィールド f である．全体の幅は $1 + w + p$ ビットである．指数は負と正の両方のべき（1 より小さい値と大きい値）を表さなければならず，符号ビットは値のほうに属するので，指数はオフセット付きで格納される．このオフセットが**指数バイアス** $b = 2^{w-1} - 1$ である．**正規化数**は $0 < e < 2^w - 1$ をもち，その値は

$$x = (-1)^s \times \left(1 + \frac{f}{2^p}\right) \times 2^{e-b}, \tag{12.4}$$

である．先頭の 1 は暗黙であり，格納されない．

▷ floating-point format
▷ sign bit
▷ exponent
▷ mantissa (fraction)

“mantissa”，“fraction”，“significand” は実際には同じフィールドを指す．IEEE 754 は格納されるフィールドを trailing significand と呼ぶ [74]．

▷ exponent bias

指数部のビットが多いほど範囲が広くなり，仮数部のビットが多いほど精度が高くなる．図 12.3 は同じ値を本章の 6 つの形式で符号化し，表 12.3 はそれらのパラメータを示す．形式の幅は 2 のべきである必要はない．TF32 の幅は $1+8+10 = 19$ ビットである．

定義 12.11 (ML 向けの形式). **bfloat16**（BF16）という形式は “includes an 8-bit exponent, 7-bit mantissa, and 1 sign-bit”[1] である．FP32 の範囲を FP16 より低い精度でもつ．**TF32**（TensorFloat-32）という形式は “8-bit exponent (same as FP32), 10-bit mantissa (same precision as FP16) and 1 sign-bit”[1] をもつ．A100 のテンソルコアは FP32 の入力にこれを用いる．8 ビットの形式 **FP8** には 2 つの符号化がある．“E4M3 with 4 exponent bits, 3 mantissa bits, and 1 sign bit” と “E5M2, with 5 exponent bits, 2 mantissa bits, and 1 sign bit” である [2]．

▷ bfloat16 (BF16)

実際には，E4M3 は重みと活性化に，E5M2 はより広い範囲にわたる勾配に使われる傾向がある [76]．

▷ TensorFloat-32 (TF32)
▷ FP8

2 つの FP8 の符号化は学習の仕事を分担する．“E4M3 supports computations requiring less dynamic range with more precision, while E5M2 provides a wider

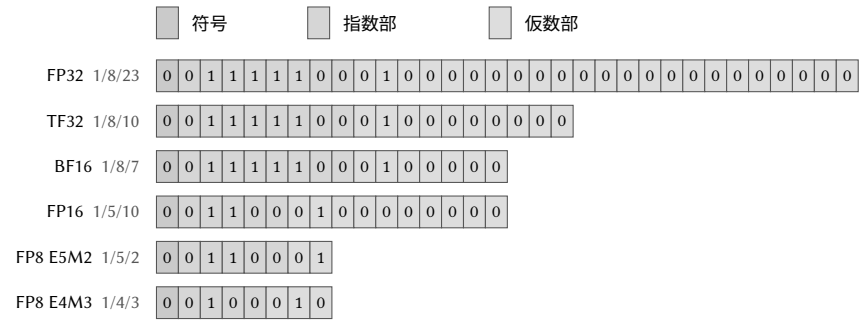


図 12.3: 6 つの形式における値 $0.15625 = 1.01_2 \times 2^{-3}$（左は符号／指数部／仮数部のビット数）．バイアス付き指数は FP32，TF32，BF16 では $-3+127=124$，FP16 と E5M2 では $-3+15=12$，E4M3 では $-3+7=4$ である．仮数部は 01 の後にゼロが続く．

dynamic range and less precision” [2]．形式の**ダイナミックレンジ**とは，表現可能な正の絶対値の最大値と最小値の比であり，しばしば 2 のべきの数（バイナード）で示される．

表 12.3: 図 12.3 の形式のパラメータ．最大値と最小値は式 (12.4) から導いたものである．ただし E4M3 の最大値は指数フィールド 1111 を用いる（無限大なし．本文の注意を参照）．$\varepsilon = 2^{-p}$ は 1 のすぐ上の値の間隔である．

形式	$s/w/p$	バイアス	正規化数の最大	正規化数の最小	ε
FP64	1/11/52	1023	$\approx 1.80 \times 10^{308}$	2^{-1022}	2^{-52}
FP32	1/8/23	127	$\approx 3.40 \times 10^{38}$	2^{-126}	2^{-23}
TF32	1/8/10	127	$\approx 3.40 \times 10^{38}$	2^{-126}	2^{-10}
BF16	1/8/7	127	$\approx 3.39 \times 10^{38}$	2^{-126}	2^{-7}
FP16	1/5/10	15	65 504	2^{-14}	2^{-10}
FP8 E5M2	1/5/2	15	57 344	2^{-14}	2^{-2}
FP8 E4M3	1/4/3	7	448	2^{-6}	2^{-3}

注意 E4M3 は上端では IEEE 風ではない．無限大をもたず，全ビット 1 のパターン `S.1111.111` だけが NaN である [76]．PTX ISA はその NaN を `0x7f` と `0xff` としている [19, §5.2.3 Alternate Floating-Point Data Formats]．したがって最大値は $1.110_2 \times 2^{15-7} = 1.75 \times 256 = 448$ であり，IEEE の規則から得られる 240 ではない．

▷ dynamic range

例 12.12. FP16 と BF16 における範囲と精度の対比．値 70 000 は FP16 の最大値 65 504 を超えてオーバーフローする．BF16 では範囲内であり，2^{16} と 2^{17} の間の BF16 の値の間隔は $2^{16-7} = 512$ なので，最も近い倍数 $137 \times 512 = 70\,144$ に丸められる．逆に，$1 + 2^{-8}$ は FP16（$\varepsilon = 2^{-10}$）では正確に表せるが，BF16 では 1 と $1 + 2^{-7}$ のちょうど中間にあり，仮数部が偶数になる隣の値 1 に丸められる．増分は失われる．

ヒント BF16 は FP32 の上位半分である．FP32 から BF16 への変換は，仮数部の下位 16 ビットを切り捨てか丸めで落とす．

12.2.3 非正規化数，特殊値，IEEE 754

式 (12.4) はゼロを表せず，その最小の絶対値 2^{1-b} とゼロの間には隙間が残る．指数フィールドの値 0 と $2^w - 1$ は，これらの穴を埋めるために予約されている．

定義 12.13 (非正規化数，アンダーフロー，オーバーフロー，NaN). **非正規化数**は指数フィールド $e = 0$ をもち，暗黙の先頭の 1 なしに値 $(-1)^s \times (f/2^p) \times 2^{1-b}$ をとる．$f = 0$ なら ±0 である．非正規化数は，ゼロと最小の正規化数の間の隙間を等間隔の値で埋める．絶対値が最小の正規化数 2^{1-b} を下回る非ゼロの結果は**アンダーフロー**を起こす（非正規化数かゼロに丸められる）．最大の有限値を超える結果は**オーバーフロー**を起こす（既定の丸めでは ±∞ になる．E4M3 のように無限大をもたない形式では飽和するか NaN になる）．指数フィールド $e = 2^w - 1$ は ±∞（$f = 0$）と**非数（NaN）**（$f \neq 0$）を符号化する．NaN は $0/0$ や $\infty - \infty$ のような無効な演算の結果である．

▷ subnormal number

▷ underflow

▷ overflow

▷ NaN (not a number)

図 12.4(a) は，指数部 2 ビット（バイアス 2）と仮数部 2 ビットのおもちゃの形式の非負の値をすべて列挙したもので，すべての指数コードを数に用いている．$[2^k, 2^{k+1})$ の正規化数は等間隔であり，その間隔は 2 のべきごとに 2 倍になる．3 つの非正規化数は最も低いバイナードの間隔をゼロまで延長する．したがって表現可能な値はゼロ付近で密，大きな絶対値で疎である．精度，すなわち値に対する

おもちゃの形式：$s/w/p = 1/2/2$，$b = 2$．正規化数は $e = 1, 2, 3$ に対して $1.f \times 2^{e-2}$ で，0.5 から 3.5 まで．非正規化数 $0.f \times 2^{-1}$ は 1/8，2/8，3/8．

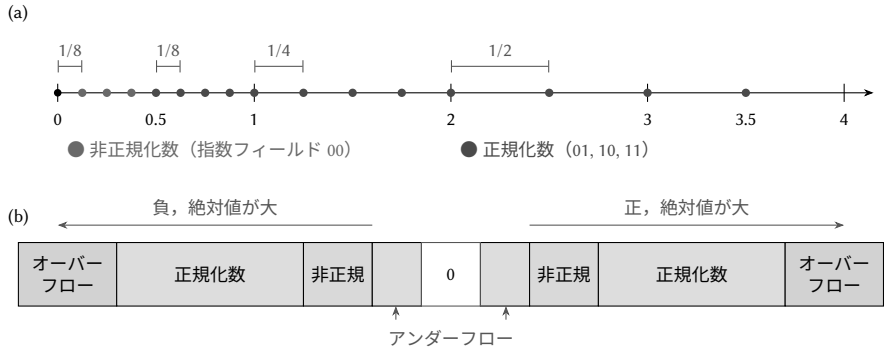


図 12.4: (a) おもちゃの形式（1/2/2 ビット，バイアス 2，無限大なし）の非負の値．3 つの非正規化数と，その後に 2 のべきごとに間隔が 2 倍になる正規化数が続く．(b) 対数目盛の絶対値軸上での形式の領域．

相対的な間隔はほぼ一定だが，絶対誤差は絶対値とともに大きくなる．図 12.4(b) は実際の形式の領域を対数目盛で示す．

定義 12.14 (IEEE 754；丸めモード). **IEEE 754** という規格は，2 進浮動小数点演算を定義する [74]．この規格は，2 進形式（binary16，binary32，binary64，...）の符号，指数，仮数の配置，特殊値（±0，±∞，NaN），非正規化数，基本演算の正確な結果，そして正確な結果が表現できないときに表現可能な結果を選ぶ**丸めモード**を規定する．丸めモードには +∞ 方向（切り上げ），−∞ 方向（切り下げ），ゼロ方向，最近接への丸めがある．

▷ IEEE 754

IEEE 754 の binary16，binary32，binary64 が本章の FP16，FP32，FP64 である（第 1 章で 3 つの精度を導入した）．

▷ rounding mode

注意 既定の丸めモードは，“切り上げ，切り下げ” という列挙が省いているもので，最近接偶数への丸め（ties to even）である [74]．例 12.12 がその例である．

BF16，TF32，FP8 は IEEE 754 の交換形式ではない．その符号化規則を借用しており，E4M3 は範囲の上端でそれから外れる．

12.2.4 実際の量子化

ビットが少ないほど演算は少なく安価になるので，ML では通常，演算の入力を低精度に量子化し，アキュムレータは高精度に保つ．テンソルの値が均等に分布することはまれである．ほとんどの重みと勾配はゼロ付近に集まり，長い裾をもつ．対数領域で等間隔に値が並ぶ形式ではコードが無駄になるので，ML の実践では量子化の前に値を変換する．これが非一様量子化である．最も重要なのはゼロ付近の分解能である．

12.1.6 節の Transformer Engine は最も単純なそうした変換を適用する．テンソルの値の統計からそれをスケーリングし，その後で FP8 にキャストする [2]．

▷ shifted and squeezed FP8 (S2FP8)

定義 12.15 (S2FP8). **S2FP8** という形式 [75] は，テンソル X を，テンソルごとの 2 つのパラメータ α（スクイーズ）と β（シフト）によるべき変換の後に FP8 で格納する．

$$X_{\text{truncated}} = \left[2^{-\beta} \cdot \text{truncate}_{\text{FP8}}\left(2^{\beta} \cdot |X|^{\alpha}\right) \right]^{1/\alpha}. \tag{12.5}$$

変換は絶対値に作用し，各要素は X の符号を保つ．対数領域では $\log_2(2^{\beta}|X|^{\alpha}) = \alpha \log_2 |X| + \beta$ である．α は指数の広がりを FP8 の範囲に合うように拡大縮小し，β はその中心をずらす．こうして形式のコードがテンソルの値のある場所に来る．どちらもテンソルごとに，非ゼロ要素にわたる $\log_2 |X|$ の平均 μ と最大値 m から計算される．$\alpha = 15/(m-\mu)$，$\beta = -\alpha\mu$ であり，変換後の値の指数の平均が 0，最大値が 15 になる [75, §3.2]．

要点
浮動小数点形式は，与えられた幅の中での範囲（指数部のビット）と精度（仮数部のビット）の選択である．ML は入力の幅を下げ，アキュムレータは広く保つ．入力の幅を半分にするごとに演算強度は 2 倍になり，最近のテンソルコアではピーク性能も 2 倍になる．

12.3 テンソルコア

テンソルコアは，一般的な課題，すなわち GPU に ML 用のアクセラレータを追加するという課題の 1 つの事例である．本節では設計上の問いを順にたどる．どの機能を作るか，演算ユニットをどう作るか，そのデータをどう格納し移動するか，スパース性をどう活用するかである．

出典 M12，テンソルコアの設計；CPG §7.24，§7.26，§7.27；PMPP ch. 14；[1, 2]．

12.3.1 設計上の問いと手順

ML アクセラレータの構成要素の設計は 4 つの問いを提起する．どの機能を追加するか，既存の GPU に比べてどのような利益をもたらすか，演算ユニットをどう作り，いくつ置くか，そしてそれらが必要とするデータをどう格納し移動するかである．その答えは共通の手順に従う．

1. 頻繁に実行される演算を，アルゴリズムの知識またはプロファイリングから特定する．
2. 既存のデータ並列ユニットに対する利益を見積もる．その演算が，既存の ISA によるソフトウェアに比べてハードウェアでどれだけ速くなるかである．
3. インタフェースを設計する．新しい命令，ユニットへのデータの供給方法，格納形式，オペランドがレジスタに置かれるかメモリ（プライベートまたは共有）に置かれるかである．
4. アクセラレータの構成要素を 1 つのユニットに統合するかどうかを，プログラム可能性と使いやすさを，ハードウェアのコストおよびタイミングのオーバーヘッドと比較して決める．

V100 では，同じ精度で約 4 倍である．FP16 テンソルコアの 125 TFLOP/s に対し，SIMT レーン上の FP16 は 31.4 TFLOP/s（AWP Table 4 [1]）．FP32 の 15.7 TFLOP/s に対する 8 倍という比には，形式が狭くなった分が含まれる．

第 10 章と第 11 章のコンパイラが関わるのは手順 3 である．新しいユニットには新しい命令を通じてしか到達できず，それをコンパイラかライブラリが生成しなければならない．

12.3.2 行列積和演算ユニット

GPU が MMA ユニットを選んだのは，GEMM が ML を支配し（12.1 節），SIMD レーンでは依然として行と列の組を 1 つずつ処理するからである．設計空間は，多数の小さな MMA ユニットと，シストリックアレイのような 1 つの大きなユニットとの間にあり，どのユニットも GPU の他の機能からチップ面積を奪う．生の性能と同じくらいプログラム可能性が重要である．NVIDIA は 4×4 のユニット（Volta のクロック当たり 4×4×4 の積）から始め，他のベンダも同様のユニットをもつ．Intel の **AMX**（Advanced Matrix Extensions）は，x86 CPU に 16×16 の行列タイルとタイル命令を追加する．

▷ Advanced Matrix Extensions (AMX)

注意 講義 vs ドキュメント：Intel は，最大 16 行 ×64 バイトの AMX タイルレジスタを 8 個と記述している．その INT8 の例は 16×64 の入力を掛けて 16×16 の INT32 の結果を得る [81]．したがって 16×16 はその例の出力タイルであり，固定の行列サイズではない．

MMA ユニットの内部には 3 つの構成がありうる．

シストリックアレイ． 大きなタイルには効率的だが，小さなタイルでは充填と排出の時間が無駄になる．4×4 のアレイは 1 回の積では 40 % しか動作していなかった（表 12.2）．

方式 1：並列ユニット． 出力要素ごとに 1 つの内積ユニット（図 12.1）を置く．4 FMA のユニット 16 個が 4×4×4 の積全体を一度に計算する（図 12.5，左）．

方式 2：パイプライン化． より少ないユニットを時間的に共有する．第 1 行と第 1 列，次に第 1 行と第 2 列というように進み，連続する組のロード，演算，ストアを重ね合わせる（図 12.5，右）．

並列ユニットは最大の面積で最高のスループットを与え，パイプライン化したユニットはスループットを面積と交換する．

方式 1 はサイクル当たり 16×4 = 64 回の FMA を行い，これは Volta のテンソルコア 1 個の性能に等しい（表 12.1）．

12.3.3 オペランドの格納と移動

MMA ユニットのオペランドはメモリから来るが，その量は大きい．4×4×4 の演算には，レジスタに少なくとも 3×16 個の値（A，B，C）が必要である．NVIDIA のレジスタはスレッドにプライベートなので，そこから 1 つのユニットにデータを供給するのは複雑になる．12.1 節のワープレベルのインタフェースは，オペランドを 32 スレッドのレジスタに分散させる．代替策はユニットに近いメモリ，すなわち共有メモリ（第 3 章）であり，その代償はまずグローバルメモリからそこへデータを移動することである．

その移動のために，NVIDIA は GPU 内部に非同期コピーを追加した．A100 の命令は “loads data directly from global memory into shared memory, optionally bypassing L1 cache, and eliminating the need for intermediate register file (RF) usage” [1]．スレッドはもはやデータを待たないので，データが到着したことを知る手

レジスタファイルのサイズとバンク構成，そして幅の広いオペランドがそれらに与える負荷は，第 4 章と第 6 章の主題である．

注意 非同期と呼ばれるコピーは 2 種類ある．第 5 章の `cudaMemcpyAsync` はホストが発行するホストとデバイス間の転送である．A100 のコピーはグローバルメモリと共有メモリの間のデバイス命令（PTX の `cp.async`，CUDA C++ の `memcpy_async`，CPG §7.27）である．

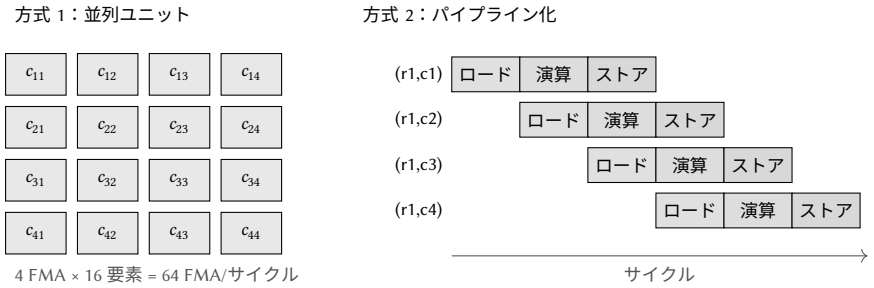


図 12.5: 4×4×4 の MMA ユニットを作る 2 つの方法．方式 1：C の要素ごとに 1 つ，計 16 個の内積ユニット．方式 2：行と列の組が 1 サイクルずつ間隔をあけて通過するパイプライン化したユニット．

段が必要になる．

定義 12.16 (非同期バリア). **非同期バリア**は，到着（arrive）と待機（wait）の操作に分割されたバリアである．スレッドは到着を通知し，独立した仕事を続け，結果が必要になったときにだけ待つ（CPG §7.26 Asynchronous Barrier）．非同期コピーと組み合わせると，ブロックは次のタイルのコピーを開始し，現在のタイルで計算し，コピーが使われる直前にだけその完了を待つことができる．

▷ asynchronous barrier

`__syncthreads()`（第 3 章）は同期的な場合であり，到着と待機が 1 つの操作になっている．Hopper はバリアを拡張し，TMA 転送のバイト数を数えるようにした（非同期トランザクションバリア [2]）．

12.3.4 アテンション：GEMM を超えるデータ移動

高速なユニットにデータを供給し続けるという同じ関心が，Transformer モデルの主要なカーネルを動かしている．**アテンション** と呼ばれる演算は，N 行のクエリ，キー，バリューの行列 Q，K，V に対して $\mathrm{softmax}(QK^{\mathsf{T}}/\sqrt{d})V$ を計算する．ここで d は行の長さである．中間の $N \times N$ のスコア行列は大きく，素直な実装はそれを HBM に書き出して読み戻す．**FlashAttention** は “an IO-aware exact attention algorithm that uses tiling to reduce the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM”[79] である．スコア行列全体を HBM に格納することなく同じ結果を計算する．その論拠は式 (12.2) と同じである．アテンションは演算ではなくメモリトラフィックに制限されるので，利得はバイト数の削減から得られる．

▷ attention

▷ FlashAttention

FlashAttention-2 はスレッドブロックとワープへの仕事の分割を改善し，“reaching 50-73 % of the theoretical maximum FLOPs/s on A100”[80]．

12.3.5 スパースな演算

HPC やグラフのワークロードの多くの行列はほとんどがゼロであり，ML は意図的にさらにそれを作り出す．

定義 12.17 (スパース性，疎行列，圧縮形式). 行列やテンソルの**スパース性**とは，その要素のうちゼロであるものの割合である．**疎行列**とは，非ゼロだけを格納し計算することが得になるほど多くのゼロをもつ行列である．**疎行列圧縮形式**は非ゼロだけをその位置とともに格納する．最も単純なのは座標形式（COO）で，非ゼロごとに（行，列，値）の 3 つ組を 1 つもつ．

▷ sparsity

▷ sparse matrix

▷ compressed sparse format

SpMV で通常用いられる圧縮行格納（CSR）形式は，COO の行インデックスを行ごとに 1 つのオフセットで置き換える（PMPP ch. 14）．

図 12.6(a) と (b) は，非ゼロを 6 個もつ 4×4 行列を密な形式と COO 形式で示す．密な形式が 16 個の数を必要とするのに対し，COO は 18 個を必要とする．行列がずっと疎でない限り，インデックスのオーバーヘッドが節約したゼロを上回り，不規則なインデックスはハードウェアによるサポートを難しくする．そこで ML は非ゼロを置ける場所を制限する．

定義 12.18 (枝刈り，構造化スパース性). **枝刈り**と呼ばれる操作は，学習済みネットワークの寄与の小さい重みをゼロにし，そのスパース性を高める．**構造化スパース性**と呼ばれる制約はそのパターンを制限する．小さなグループごとに要素の一定の割合だけが非ゼロであってよいとすることで，どのグループも同じサイズに圧縮され，インデックスは短く規則的になる．

▷ pruning

▷ structured sparsity

注意 ここでの 1 ビットの左／右インデックスは，2 つに 1 つの非ゼロのためのおもちゃである．2:4 スパース性では残す値はそれぞれ 4 つの位置のどこにでも置けるので，FP16 と BF16 ではグループの非ゼロは “are indicated by two 2-bit indices in the metadata operand”[19]（`mma.sp`）．

50 % の構造化スパース性で要素の組ごとに 1 つの非ゼロとすると，16 個の要素は，8 個の値と，残した各値がその組の左右どちらの要素だったかを示す 8 個の 1 ビットインデックスとして格納される（図 12.6(c)）．固定された配置はハー



図 12.6: 1 つの疎行列の 3 つの形式．(a) 密な形式，16 個の値．(b) COO の 3 つ組，18 個の数．(c) 列の組（赤線の左側と右側）ごとに 1 つの値を，その位置が左（L）か右（R）かを示すビットとともに保持する構造化形式．

ドウェアを単純にし，格納量をほぼ半分にする．さらにユニットはゼロによる乗算を省き，スループットを高める．

定義 12.19 (2:4 構造化スパース性). A100 とともに導入された **2:4 構造化スパース性** という方式は，“a new 2:4 sparse matrix definition that allows two non-zero values in every four-entry vector” [1] である．A の 1 行は，値の半分と，4 つのグループごとに残した 2 つの値の位置を示すメタデータとして格納される．

▷ 2:4 structured sparsity

図 12.6 の行列の第 1 行 $[0, 4, 0, 2]$ はすでに 2:4 である．インデックス 1 と 3（0 から数える）とともに 4 と 2 を残す．

A100 のスパース形式は “can be compressed efficiently and reduce memory storage and bandwidth by almost 2x” [1]，そのスパースなテンソルコアの性能は密な場合の 2 倍である（表 12.1）．行列と行列の積を超えて，**疎行列ベクトル積**（SpMV），すなわち疎な A による $y = Ax$ は，多くの HPC とグラフのアルゴリズムの中核であり，その高速化は活発な研究課題である．演算強度が低くアクセスが不規則なので，密な行列ユニットからはほとんど恩恵を受けない．

ヒント FP16 の 2:4 では，4 つのグループを 2 つの値と 2 つの 2 ビットインデックスとして格納する．64 ビット中 $32+4=36$ ビット，約 56 % である．

▷ sparse matrix-vector multiplication (SpMV)

要点 — 章のまとめ

GPU は ML に適している．ML は浮動小数点演算が多く，データ並列で，緩和された意味論を許容し，GPU は HBM をもち新しい形式を素早く採用する．DNN は GEMM に支配される．その演算強度 $MNK/(MK + NK + MN)$（FP16）を演算／バイト比と比べれば，演算律速かメモリ律速かがわかる．MMA ユニット（テンソルコア）は，4×4 の積の 16 回の SIMD 内積を 1 つの命令で置き換える．シストリックアレイは充填と排出の代償を払う．浮動小数点形式は指数部のビット（範囲）と仮数部のビット（精度）を交換する．ML は入力を狭め（BF16，TF32，FP8），アキュムレータを広く保ち，値を変換する（S2FP8）．テンソルコアの設計では，並列またはパイプライン化したユニット，非同期コピーとバリアで供給されるオペランドの格納，そして疎な形式を選ぶ．2:4 構造化スパース性は格納量をほぼ半分にし，テンソルコアのピーク性能を 2 倍にする．

参考文献

[1] NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Architecture*. Whitepaper. Optional reading, Module 12. NVIDIA, 2020. URL: https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf.

[2] NVIDIA Corporation. *NVIDIA H100 Tensor Core GPU Architecture*. Whitepaper V1.02. H100 figures in this version are preliminary. Fetched from mirror https://www.ece.lsu.edu/koppel/gp/refs/gtc22-whitepaper-hopper.pdf. NVIDIA, 2022. URL: https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper.

[3] Hyesoon Kim et al. *Performance Analysis and Tuning for General Purpose Graphics Processing Units (GPGPU)*. Synthesis Lectures on Computer Architecture 20. Reissued by Springer, doi:10.1007/978-3-031-01737-7. Chapters: 1 GPU Design, Programming, and Trends (pp. 1–24); 2 Performance Principles (p. 25); 3 From Principles to Practice: Analysis and Tuning (p. 39); 4 Using Detailed Performance Analysis to Guide Optimization (p. 49). Full section TOC verified from the PDF on co-author Jee Choi's page (https://jeewhanchoi.github.io/publication/pdf/perf_analysis_book.pdf). Morgan & Claypool, 2012. ISBN: 978-1-60845-954-4. DOI: 10.2200/S00451ED1V01Y201209CAC020.

[4] Tor M. Aamodt, Wilson Wai Lun Fung, and Timothy G. Rogers. *General-Purpose Graphics Processor Architectures*. Synthesis Lectures on Computer Architecture 44. Reissued by Springer, doi:10.1007/978-3-031-01759-9. Chapters (start pages per Google Books): 1 Introduction (1); 2 Programming Model (9); 3 The SIMT Core: Instruction and Register Data Flow (21); 4 Memory System (67); 5 Crosscutting Research on GPU Computing Architectures (85). Section headings not verified. Morgan & Claypool, 2018. ISBN: 978-1-62705-923-7. DOI: 10.2200/S00848ED1V01Y201804CAC044.

[5] NVIDIA Corporation. *CUDA C++ Programming Guide*. Release 12.4. Assigned: sections 1–2 and 4 (Module 1), section 3 (Module 3). Numbering refers to the 12.x guide; from CUDA 13 the guide was reorganised into five parts at https://docs.nvidia.com/cuda/cuda-programming-guide/. 2024. URL: https://docs.nvidia.com/cuda/archive/12.4.0/cuda-c-programming-guide/index.html.

[6] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. 4th ed. Not on the course reading lists; used here as a secondary textbook. Cambridge, MA: Morgan Kaufmann (Elsevier), 2022. ISBN: 978-0-323-91231-0.

[7] Timothy G. Mattson, Beverly A. Sanders, and Berna L. Massingill. *Patterns for Parallel Programming*. Copyright 2005 printing. Optional reading: ch. 2 (Module 1); ch. 3, appendices A and B (Module 2). Boston: Addison-Wesley, 2004. ISBN: 0-321-22811-1.

[8] NVIDIA Corporation. *CUDA C++ Best Practices Guide*. Assigned: sections 9 (Performance Metrics), 10 (Memory Optimizations), 11 (Execution Configuration Optimizations), Module 5. Accessed 2026-10-29. 2026. URL: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html.

[9] Intel Corporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual: Volume 1*. 248966-051US. Section 2.4.1 Golden Cove Microarchitecture Overview: six decoders, up to six instructions decoded per cycle. Intel Corporation. Oct. 2026. URL: https://cdrdv2.intel.com/v1/dl/getContent/671488 (visited on 10/30/2026).

[10] Glenn Hinton et al. "The Microarchitecture of the Pentium 4 Processor". In: *Intel Technology Journal* (2001). Q1 2001 issue. Willamette: 20-stage branch-misprediction pipeline.

[11] Darrell Boggs et al. "The Microarchitecture of the Intel Pentium 4 Processor on 90nm Technology". In: *Intel Technology Journal* 8.1 (Feb. 2004). Prescott: 31-stage branch-misprediction pipeline, pp. 1–17.

[12] Advanced Micro Devices. *AMD EPYC 9654*. Product page: 96 cores, 192 threads; launched 2022. 2026. URL: https://www.amd.com/en/products/processors/server/epyc/4th-generation-9004-and-8004-series/amd-epyc-9654.html (visited on 10/30/2026).

[13] Blaise Barney and Donald Frederick. *Introduction to Parallel Computing Tutorial*. Lawrence Livermore National Laboratory. Required reading, Module 2. Page version dated June 16, 2026; accessed 2026-10-29. 2026. URL: https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial.

[14] OpenMP Architecture Review Board. *OpenMP Application Programming Interface*. Version 5.2. OpenMP Architecture Review Board. Nov. 2021. URL: https://www.openmp.org/specifications/ (visited on 10/30/2026).

[15] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard*. Version 4.1. Message Passing Interface Forum. Nov. 2023. URL: https://www.mpi-forum.org/docs/ (visited on 10/30/2026).

[16] Linux man-pages project. *pthread_create(3)*. Feb. 2026. URL: https://man7.org/linux/man-pages/man3/pthread_create.3.html (visited on 10/30/2026).

[17] Free Software Foundation. *GNU Offloading and Multi-Processing Runtime Library (GNU libgomp): OMP_NUM_THREADS*. "If undefined one thread per CPU is used." 2026. URL: https://gcc.gnu.org/onlinedocs/libgomp/OMP_005fNUM_005fTHREADS.html (visited on 10/30/2026).

[18] Mark Harris. *Using Shared Memory in CUDA C/C++*. NVIDIA Technical Blog. Required reading, Module 5. 2013. URL: https://developer.nvidia.com/blog/using-shared-memory-cuda-cc/.

[19] NVIDIA Corporation. *Parallel Thread Execution ISA*. Version 9.4. Accessed 2026-10-29. 2026. URL: https://docs.nvidia.com/cuda/parallel-thread-execution/index.html.

[20] Peter Kipfer and Rüdiger Westermann. "Improved GPU Sorting". In: *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*. Ed. by Matt Pharr and Randima Fernando. Assigned in Module 5 as Project 2 preparation. Addison-Wesley, 2005. Chap. 46. URL: https://developer.nvidia.com/gpugems/gpugems2/part-vi-simulation-and-numerical-algorithms/chapter-46-improved-gpu-sorting.

[21] Mark Harris. *How to Optimize Data Transfers in CUDA C/C++*. NVIDIA Technical Blog. Dec. 2012. URL: https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/ (visited on 10/30/2026).

[22] Mark Harris. *GPU Pro Tip: CUDA 7 Streams Simplify Concurrency*. NVIDIA Technical Blog. Jan. 2015. URL: https://developer.nvidia.com/blog/gpu-pro-tip-cuda-7-streams-simplify-concurrency/ (visited on 10/30/2026).

[23] NVIDIA Corporation. *Nsight Systems User Guide*. 2026. URL: https://docs.nvidia.com/nsight-systems/UserGuide/index.html (visited on 10/30/2026).

[24] NVIDIA Corporation. *Nsight Compute Documentation*. 2026. URL: https://docs.nvidia.com/nsight-compute/NsightCompute/index.html (visited on 10/30/2026).

[25] NVIDIA Corporation. *CUDA Profiler User's Guide*. 2026. URL: https://docs.nvidia.com/cuda/profiler-users-guide/index.html (visited on 10/30/2026).

[26] NVIDIA Corporation. *Maxwell Tuning Guide*. Oct. 2026. URL: https://docs.nvidia.com/cuda/maxwell-tuning-guide/index.html (visited on 10/30/2026).

[27] Kenneth E. Batcher. "Sorting Networks and Their Applications". In: *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference (AFIPS '68 (Spring))*. ACM, 1968, pp. 307–314. DOI: 10.1145/1468075.1468121.

[28] Wilson W. L. Fung et al. "Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow". In: *Proceedings of the 40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2007, pp. 407–420. DOI: 10.1109/MICRO.2007.30.

[29] Veynu Narasiman et al. "Improving GPU Performance via Large Warps and Two-Level Warp Scheduling". In: *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-44)*. ACM, 2011, pp. 308–317. DOI: 10.1145/2155620.2155656.

[30] Mark Gebhart et al. "Energy-efficient Mechanisms for Managing Thread Context in Throughput Processors". In: *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2011, pp. 235–246. DOI: 10.1145/2000064.2000093.

[31] Mark Gebhart, Stephen W. Keckler, and William J. Dally. "A Compile-Time Managed Multi-Level Register File Hierarchy". In: *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-44)*. ACM, 2011, pp. 465–476. DOI: 10.1145/2155620.2155675.

[32] Mark Gebhart et al. "Unifying Primary Cache, Scratch, and Register File Memories in a Throughput Processor". In: *Proceedings of the 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2012, pp. 96–106. DOI: 10.1109/MICRO.2012.18.

[33] Hyeran Jeon et al. "GPU Register File Virtualization". In: *Proceedings of the 48th International Symposium on Microarchitecture (MICRO-48)*. Full text not obtained (ACM DL blocked automated access); metadata verified. ACM, 2015, pp. 420–432. DOI: 10.1145/2830772.2830784.

[34] NVIDIA Corporation. *NVIDIA's Next Generation CUDA Compute Architecture: Fermi*. Whitepaper V1.1. NVIDIA, 2009. URL: https://www.nvidia.com/content/PDF/fermi_white_papers/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf (visited on 11/03/2026).

[35] Jason Power, Mark D. Hill, and David A. Wood. "Supporting x86-64 Address Translation for 100s of GPU Lanes". In: *Proceedings of the 20th IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2014, pp. 568–578. DOI: 10.1109/HPCA.2014.6835965.

[36] Rachata Ausavarungnirun et al. "Mosaic: A GPU Memory Manager with Application-Transparent Support for Multiple Page Sizes". In: *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-50)*. ACM, 2017, pp. 136–150. DOI: 10.1145/3123939.3123975.

[37] Bingyao Li et al. “Improving Address Translation in Multi-GPUs via Sharing and Spilling Aware TLB Design”. In: *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-54)*. ACM, 2021, pp. 1154–1168. DOI: 10.1145/3466752.3480083.

[38] Debashis Ganguly et al. “Interplay between Hardware Prefetcher and Page Eviction Policy in CPU-GPU Unified Virtual Memory”. In: *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)*. ACM, 2019, pp. 224–235. DOI: 10.1145/3307650.3322224.

[39] Hyojong Kim et al. “Batch-Aware Unified Memory Management in GPUs for Irregular Workloads”. In: *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2020, pp. 1357–1370. DOI: 10.1145/3373376.3378529.

[40] Timothy G. Rogers, Mike O’Connor, and Tor M. Aamodt. “Cache-Conscious Wavefront Scheduling”. In: *Proceedings of the 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2012, pp. 72–83. DOI: 10.1109/MICRO.2012.16.

[41] Adwait Jog et al. “Orchestrated Scheduling and Prefetching for GPGPUs”. In: *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*. The course list cites the SIGARCH Computer Architecture News version, doi:10.1145/2508148.2485951. ACM, 2013, pp. 332–343. DOI: 10.1145/2485922.2485951.

[42] Adwait Jog et al. “OWL: Cooperative Thread Array Aware Scheduling Techniques for Improving GPGPU Performance”. In: *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2013, pp. 395–406. DOI: 10.1145/2451116.2451158.

[43] Jen-Cheng Huang et al. “GPUMech: GPU Performance Modeling Technique Based on Interval Analysis”. In: *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2014, pp. 268–279. DOI: 10.1109/MICRO.2014.59.

[44] Changxi Liu, Yifan Sun, and Trevor E. Carlson. “Photon: A Fine-grained Sampled Simulation Methodology for GPU Workloads”. In: *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. ACM, 2023, pp. 1227–1241. DOI: 10.1145/3613424.3623773.

[45] Samuel Williams, Andrew Waterman, and David Patterson. “Roofline: An Insightful Visual Performance Model for Multicore Architectures”. In: *Communications of the ACM* 52.4 (2009), pp. 65–76. DOI: 10.1145/1498765.1498785.

[46] Sunpyo Hong and Hyesoon Kim. “An Analytical Model for a GPU Architecture with Memory-level and Thread-level Parallelism Awareness”. In: *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2009, pp. 152–163. DOI: 10.1145/1555754.1555775.

[47] Tejas S. Karkhanis and James E. Smith. “A First-Order Superscalar Processor Model”. In: *Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2004, pp. 338–349. DOI: 10.1109/ISCA.2004.1310786.

[48] Ali Bakhoda et al. “Analyzing CUDA Workloads Using a Detailed GPU Simulator”. In: *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2009, pp. 163–174. DOI: 10.1109/ISPASS.2009.4919648.

[49] Mahmoud Khairy et al. “Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling”. In: *Proceedings of the 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 473–486. DOI: 10.1109/ISCA45697.2020.00047.

[50] Yifan Sun et al. “MGPUSim: Enabling Multi-GPU Performance Modeling and Optimization”. In: *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)*. ACM, 2019, pp. 197–209. DOI: 10.1145/3307650.3322230.

[51] HPArch, Georgia Institute of Technology. *MacSim: A Heterogeneous Architecture Timing Model Simulator*. URL: https://github.com/gthparch/macsim (visited on 11/03/2026).

[52] Jason Power et al. “gem5-gpu: A Heterogeneous CPU-GPU Simulator”. In: *IEEE Computer Architecture Letters* 14.1 (2015), pp. 34–36. DOI: 10.1109/LCA.2014.2299539.

[53] Timothy Sherwood et al. “Automatically Characterizing Large Scale Program Behavior”. In: *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2002, pp. 45–57. DOI: 10.1145/605397.605403.

[54] NVIDIA Corporation. *Matrix Multiplication Background User’s Guide*. Optional reading, Module 12. Accessed 2026-10-29. NVIDIA Deep Learning Performance Documentation. 2026. URL: https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html.

[55] Lieven Eeckhout. *Computer Architecture Performance Evaluation Methods*. Synthesis Lectures on Computer Architecture 10. Reissued by Springer, doi:10.1007/978-3-031-01727-8. Optional reading, Module 8. Morgan & Claypool, 2010. DOI: 10.2200/S00273ED1V01Y201006CAC010.

[56] Daniel Sanchez and Christos Kozyrakis. “ZSim: Fast and Accurate Microarchitectural Simulation of Thousand-Core Systems”. In: *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2013, pp. 475–486. DOI: 10.1145/2485922.2485963.

[57] Akhil Arunkumar et al. “MCM-GPU: Multi-Chip-Module GPUs for Continued Performance Scalability”. In: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2017, pp. 320–332. DOI: 10.1145/3079856.3080231.

[58] NVIDIA Corporation. *NVLink and NVLink Switch*. Generation table: 900 GB/s and 18 links per GPU (4th gen, Hopper), 1,800 GB/s and 18 links (5th gen, Blackwell), 3,000 GB/s and 36 links (6th gen, Vera Rubin, preliminary). 2026. URL: https://www.nvidia.com/en-us/data-center/nvlink/ (visited on 11/03/2026).

[59] NVIDIA Corporation. *GPUDirect RDMA*. 2026. URL: https://docs.nvidia.com/cuda/gpudirect-rdma/index.html (visited on 11/03/2026).

[60] NVIDIA Corporation. *Multi-Process Service*. Release 615. Up to 60 client CUDA contexts per device per MPS server (default CUDA_DEVICE_MAX_CONNECTIONS of 2). NVIDIA Corporation. Sept. 2026. URL: https://docs.nvidia.com/deploy/pdf/CUDA_Multi_Process_Service_Overview.pdf (visited on 11/03/2026).

[61] Fuxun Yu et al. *A Survey of Multi-Tenant Deep Learning Inference on GPU*. 2022. arXiv: 2203.09040 [cs.DC].

[62] Zhisheng Ye et al. “Deep Learning Workload Scheduling in GPU Datacenters: A Survey”. In: *ACM Computing Surveys* 56.6 (2024), 146:1–146:38. DOI: 10.1145/3638757.

[63] NVIDIA Corporation. *NVIDIA Multi-Instance GPU User Guide*. Release 615. §6.1 Terminology (eight memory slices), §6.2 Partitioning (Table 2, A100 profiles), §7.2 Application Considerations (CUDA MPS on MIG). NVIDIA Corporation. Oct. 2026. URL: https://docs.nvidia.com/datacenter/tesla/mig-user-guide/pdf/MIG_User_Guide.pdf (visited on 11/03/2026).

[64] NVIDIA Corporation. *nvidia-cuda-mps-control(1): NVIDIA CUDA Multi Process Service management program*. driver 580.65.06. Manual page, Ubuntu package nvidia-compute-utils-580-server; client limit 16 for SM 3.5 to 6.0, 48 for SM 7.0. NVIDIA Corporation. 2025. URL: https://manpages.ubuntu.com/manpages/noble/man1/nvidia-cuda-mps-control.1.html (visited on 11/03/2026).

[65] NVIDIA Corporation. *CUDA Compiler Driver NVCC*. Version 13.4. Sections 3 The CUDA Compilation Trajectory, 5 GPU Compilation. Accessed 2026-11-03. 2026. URL: https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/index.html.

[66] LLVM Project. *Compiling CUDA with clang*. 2026. URL: https://llvm.org/docs/CompileCudaWithLLVM.html (visited on 11/03/2026).

[67] NVIDIA Corporation. *NVVM IR Specification*. Accessed 2026-11-03. 2026. URL: https://docs.nvidia.com/cuda/nvvm-ir-spec/index.html.

[68] Chris Lattner and Vikram Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation”. In: *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2004, pp. 75–86. DOI: 10.1109/CGO.2004.1281665.

[69] LLVM Project. *LLVM Language Reference Manual*. Sections “’phi’ Instruction” and “’select’ Instruction”. 2026. URL: https://llvm.org/docs/LangRef.html (visited on 11/03/2026).

[70] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd ed. The “Dragon book”; copyright 2007. Sections cited: 6.2 Three-Address Code (SSA), 8.8 Register Allocation and Assignment, 9.1 The Principal Sources of Optimization, 9.2 Introduction to Data-Flow Analysis, 9.6 Loops in Flow Graphs. Pearson Addison-Wesley, 2006. ISBN: 978-0-321-48681-3.

[71] Ron Cytron et al. “Efficiently Computing Static Single Assignment Form and the Control Dependence Graph”. In: *ACM Transactions on Programming Languages and Systems* 13.4 (Oct. 1991), pp. 451–490. DOI: 10.1145/115372.115320.

[72] Gregory J. Chaitin. “Register Allocation & Spilling via Graph Coloring”. In: *Proceedings of the 1982 SIGPLAN Symposium on Compiler Construction*. ACM, 1982, pp. 98–101. DOI: 10.1145/800230.806984.

[73] LLVM Project. *LLVM’s Analysis and Transform Passes*. Section “mem2reg: Promote Memory to Register”. 2026. URL: https://llvm.org/docs/Passes.html (visited on 11/03/2026).

[74] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Standard IEEE Std 754-2019. Revision of IEEE Std 754-2008. DOI verified to resolve 2026-10-30. IEEE, 2019. DOI: 10.1109/IEEESTD.2019.8766229.

[75] Léopold Cambier et al. *Shifted and Squeezed 8-bit Floating Point format for Low-Precision Training of Deep Neural Networks*. S2FP8, named on the M12 quantization slide. Venue reported as ICLR 2020 (not verified in this pass). 2020. arXiv: 2001.05674 [cs.LG].

[76] Paulius Micikevicius et al. *FP8 Formats for Deep Learning*. E4M3 and E5M2 encodings; E4M3 has no infinities and a single NaN bit pattern (abstract, checked 2026-11-03). 2022. arXiv: 2209.05433 [cs.LG].

[77] H. T. Kung. "Why Systolic Architectures?" In: *Computer* 15.1 (1982), pp. 37–46. DOI: 10.1109/MC.1982.1653825.

[78] Norman P. Jouppi et al. "In-Datacenter Performance Analysis of a Tensor Processing Unit". In: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. Abstract: a 65,536 8-bit MAC matrix multiply unit with a peak of 92 TOPS. 2017, pp. 1–12. DOI: 10.1145/3079856.3080246.

[79] Tri Dao et al. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. Not on the lowyx reading lists; background for Project 4 (from Spring 2026). 2022. DOI: 10.52202/068431-1189. arXiv: 2205.14135.

[80] Tri Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. Published at ICLR 2024 (venue not re-verified in this pass). Not on the lowyx reading lists. 2023. arXiv: 2307.08691 [cs.LG].

[81] Alberto Villarreal. *Code Sample: Intel Advanced Matrix Extensions (Intel AMX) – Intrinsics Functions*. Palette 1: 8 tile registers, each at most 16 rows of 64 bytes; INT8 example multiplies 16 x 64 inputs into a 16 x 16 INT32 result. Intel Corporation. 2024. URL: https://www.intel.com/content/www/us/en/developer/articles/code-sample/advanced-matrix-extensions-intrinsics-functions.html (visited on 11/03/2026).

索引

A
Advanced Matrix Extensions, → AMX
AMX, 136
AST, → 抽象構文木

B
BF16, → bfloat16
bfloat16, 133
BSP, → バルク同期並列（BSP）

C
CCWS, → キャッシュ考慮型ウェーブフロントスケジューリング
CFG, → 制御フローグラフ
CPI, → 命令当たりサイクル数（CPI）
CPU, 1
CTA, → 協調スレッド配列（CTA）
CWP, → 計算ワープ並列度

D
direct memory access, → DMA
DNN, → 深層ニューラルネットワーク
DWF, → 動的ワープ形成

F
FIFO, 82
FlashAttention, 137
FMA, → 融合積和演算（FMA）
FP16, → 半精度
FP32, → 単精度
FP64, → 倍精度
FP8, 133

G
GEMM, → GEMM（一般行列積）
GMMU, → GPU メモリ管理ユニット
GPGPU, 8
GPM, → GPU モジュール
GPU, 1
GTO, → GTO スケジューリング

H
HBM, → 広帯域メモリ（HBM）
HMMA, 131

I
IEEE 754, 135
ILP, → 命令レベル並列性
IPC, → サイクル当たり命令数
IPDOM, → 直接後支配節点
IR, → 中間表現
ISA, → 命令セットアーキテクチャ

L
least recently used, → LRU
LLD, → ウェーブフロント内局所性損失検出器
LRF, → ラストリザルトファイル
LRR, → ルーズラウンドロビン

M
MAC, → 積和演算（MAC）
MCM, → マルチチップモジュール
Message Passing Interface, → MPI
MIG, → マルチインスタンス GPU（MIG）
MIMD, 5
miss status holding register, → MSHR
ML, → 機械学習
MLP, → メモリレベル並列性
MMA, → 行列積和演算（MMA）
MPS, → マルチプロセスサービス（MPS）
MRF, → 主レジスタファイル
MSHR, 90
MWP, → メモリワープ並列度

N
NaN, → 非数（NaN）
NIC, → ネットワークインタフェースカード
NUMA, → 不均一メモリアクセス（NUMA）
NVM, → 不揮発性メモリ

O
ORF, → オペランドレジスタファイル

P
Parallel Thread Execution, → PTX
PC, → プログラムカウンタ
PCIe, → PCI Express
PFN, → 物理フレーム番号
PPN, → 物理フレーム番号
PTE, → ページテーブルエントリ
pthreads, → POSIX スレッド
PTW, → ページテーブルウォーカ

Q
QoS, → サービス品質（QoS）

R
RDMA, → リモートダイレクトメモリアクセス（RDMA）
RFC, → レジスタファイルキャッシュ
RR, → ラウンドロビン

S
S2FP8, 135
SFU, → 特殊関数ユニット
shifted and squeezed FP8, → S2FP8
SIMD, 2
SimPoint, 93
SIMT, 5
SISD, 2
SM, → ストリーミング・マルチプロセッサ
SPMD, 2
SpMV, → 疎行列ベクトル積
SSA, → 静的単一代入

SVM, → 共有仮想メモリ

T
TBNp, → ツリーベース近傍プリフェッチ
TensorFloat-32, → TF32
TF32, 133
TMA, → テンソルメモリアクセラレータ
Transformer Engine, 9
translation lookaside buffer, → TLB

U
UVA, → 統合仮想アドレッシング（UVA）

V
VPN, → 仮想ページ番号

ア
アーキテクチャレジスタ, 66
IOMMU, 76
アウトオブオーダ実行, 2
アキュムレータ, 133
アクセラレータ, 2
アクティブマスク, 42
アクティブワープ, 26
アクティブワープ集合／待機ワープ集合, 79
値の生存期間, 64
アテンション, 137
アトミック操作, 16
アドレス変換, 70
アムダールの法則, 4
アライメント, 44
アンダーフロー, 134
一次近似モデル, 87
一様値, 126
1 対 1 通信, 18
if 変換, 60
イベント駆動シミュレーション, 92
入れ子の分岐, 61
インオーダ実行, 2
インターコネクト, 97
インターバル解析, 91
ウェーブフロント, 7
ウェーブフロント内局所性損失検出器, 79
ウェーブ量子化, 130
ウォームアップ, 93
後ろ向き解析, 118
SIMT スタック, 61
SIMT レーン, 6, 42
SSA 形式, 120
SRAM, 43
枝刈り, 137
Nsight Compute, 47
Nsight Systems, 47
nvcc, 106
NVSwitch, 98
NVPTX, 107
NVLink, 97
エピローグループ, 123
FPGA エミュレーション, 82
MPI, 18
LRU, 68
LLVM, 106
L1 データキャッシュ, 68
演算強度, 89
演算強度（DRAM バイト当たり）, 89
演算／バイト比, 129
演算律速, 89, 129
オーバーフロー, 134
OpenMP, 12
OpenCL, 56
オペコード, 108
オペランド, 108
オペランドコレクタ, 38
オペランドバッファ, 38
オペランドレジスタファイル, 66
オンデマンドページング, 74

カ
ガードプレディケート, 109
カーネル, 20
カーネル起動, 21
カーネル単位サンプリング, 93
カーネルの並行実行, 102
解析モデル, 87
階層型レジスタファイル, 65
ガイド付きスケジューリング, 15
書き込みポート, 36
仮数部, 133
仮想アーキテクチャ, 106
仮想 ISA, 2
仮想アドレス, 70
仮想ページ番号, 70
仮想メモリ, 70
活性化関数, 129
環境変数, 12
干渉グラフ, 119
関数のインライン展開, 123
機械学習, 128
基本ブロック, 60, 110
キャッシュ考慮型ウェーブフロントスケジューリング, 79
キャッシュスラッシング, 79
キャッシュミス, 33
キャッシュライン, 44
cuDNN, 56
cuBLAS, 56
キューベースモデル, 82
境界交換, 19
境界条件, 30
協調スレッド配列（CTA）, 21
共有仮想メモリ, 74
共有変数, 15
共有メモリ, 23
共有メモリのカーブアウト, 68
共有メモリのバンク競合, 39
共有メモリプログラミング, 12
行列積, 129
行列積和演算（MMA）, 130

局所最適化, 111
kill 集合, 113
近接メモリ／遠隔メモリ, 96
CUDA グラフ, 102
CUDA コア, → ストリーミングプロセッサ（CUDA コア）
CUDA ストリーム, 49
空間分割, 101
組み込み関数, 52
グラフィックスパイプライン, 8
Clang, 106
グリッド, 21
クリティカルセクション, 13
グローバルメモリ, 24
クロックドメイン, 82
計算と転送のオーバーラップ, 49
計算ワープ並列度, 91
経路収束条件, 122
ゴーストセル, → ハロー領域
コード移動, 125
コード生成, 105
Cooperative Groups, 54
コアレスアクセス, 44
後支配節点, 60
構造化スパース性, 137
後続ブロック, 111
広帯域メモリ（HBM）, 9
合流演算子, 112
合流点, 121
コピー記述子, 132
コピー後実行モデル, 74
コピー伝播, 112
コピーと実行の並行処理, 49
混合精度, 133
コンスタントメモリ, 24
コンテキストスイッチ, 34
コンパイラ, 105
コンパイラ指示文, 12
Compute Capability, 27

サ

サービス品質（QoS）, 101
サイクル当たり命令数, 3
サイクルレベルシミュレーション, 82
最古優先, 78
再収束点, 60
細粒度マルチスレッディング, 34
再ロード, → フィル（再ロード）
SASS, 108
サブコア, 88
3 次元積層メモリ, 43
3 番地コード, 110
サンプリングシミュレーション, 93
CTA 考慮スケジューリング, 80
GTO スケジューリング, 78
CPI スタック, 90
GPU インスタンス, 101
GPUDirect RDMA, 99
GPU メモリ管理ユニット, 75
GPU モジュール, 96
GEMM（一般行列積）, 129
gen 集合, 113
SYCL, 56
資源制約, 25
指示節, 14
指数バイアス, 133
指数部, 133
シストリックアレイ, 131
事前追い出し, 77
実行駆動シミュレーション, 82
実行構成, 21
実効帯域幅, 51
JIT コンパイル, 108
支配節点, 122
支配辺境, 122
シミュレータ, 81
SIMD 効率, 63
集団通信, 18
充填と排出, 132
主レジスタファイル, 65
使用, 113
条件付き移動, 121
小数部, → 仮数部
初期参照ミス, 79
シリコンインターポーザ, 43
深層ニューラルネットワーク, 128
スーパースカラ, 2
数学ライブラリ, 56
スクラッチパッドメモリ, 23
スコアボード, 38
ステンシル, 18
ストライド, 39
ストリーミングプロセッサ（CUDA コア）, 6
ストリーミング・マルチプロセッサ, 6
スパース性, 137
Thrust, 56
スループット, 1
スレッド, 12
スレッド依存値, 126
スレッドインデックス, 21
スレッド階層, 21
スレッドブロック, 21
スレッドブロッククラスタ, 24
スレッドブロックスケジューリング, 97
制御フローグラフ, 111
生存変数, 117
生存変数解析, 117
生存レジスタ, 64
静的, 38
静的スケジューリング, 15
静的単一代入, 120
静的命令数, 38
性能モデリング, 81
精密例外, 1
積和演算（MAC）, 130
セクタ, 85
セクタキャッシュ, 85

セグメントマッピング, 73
ゼロコピー, 50
ゼロパディング, 30
全結合層, 129
先行ブロック, 111
占有率, 26
ソーティングネットワーク, 57
相互排除, 13
疎行列, 137
疎行列圧縮形式, 137
疎行列ベクトル積, 138
ソフトウェア管理キャッシュ, 68

タ

大域最適化, 111
ダイナミックレンジ, 134
ダイバージェンス解析, 125
ダイバージェント分岐, 60
ダイバージェントワープ, 60
代表ワークロード, 94
タイムスライシング, 101
タイリング, 30
タイル, 30
タイル量子化, 129
タグ, 67
タスクキュー, 11
タスク分割, 4
畳み込み, 30
畳み込み層, 129
多段ページテーブル, 71
単精度, 8
チェックポイント, 93
置換ポリシ, 68
逐次部分の割合, 4
チップレット, 96
チャンクサイズ, 15
中間表現, 105
抽象構文木, 107
頂点シェーダ, 8
直接後支配節点, 60
強さの低減, 124
ツリーベース近傍プリフェッチ, 76
データ競合, 13
データ駆動モデル, 94
データ再利用, 30
データセンタ, 100
データ転送, 47
データフロー解析, 112
データ分割, 5
DMA, 75
TLB, 71
TLB ミス, 71
DRAM, 43
定義, 113
定義・使用連鎖, 120
定数畳み込み, 124
定数伝播, 124
ディレクティブ, 109
テクスチャメモリ, 24
デコード, 40
手続き間解析, 123
デノーマル数, → 非正規化数
デバイス, 20
デバイスコード, 20
デバイスメモリ, 28
def 集合, 117
テンソルコア, 9
テンソルメモリアクセラレータ, 9
伝達関数, 112
統合仮想アドレッシング（UVA）, 50
統合型シェーダ, 8
統合ローカルメモリ, 68
到達定義, 113
動的, 38
動的スケジューリング, 15
動的命令数, 38
動的ワープ形成, 63
特殊関数ユニット, 40
特殊レジスタ, 40
独立スレッドスケジューリング, 55
トレース, 82
トレース駆動シミュレーション, 82

ナ

内積, 129
内部断片化, 73
2:4 構造化スパース性, 138
NCCL, 56
2 レベルワープスケジューリング, 79
ネットワークインタフェースカード, 99

ハ

ハードウェア管理キャッシュ, 67
倍精度, 8
バイトニックソート, 57
パイプラインパターン, 11
パイプライン幅, 82
バックエンド, 105
パディング, 40
バリア, 16
バリア同期, 23
バルク同期並列（BSP）, 23
ハロー領域, 19
バンク, 36
バンク競合, 36
バンクレベル並列性, 80
半精度, 9
反復アルゴリズム, 114
PCI Express, 47
PTX, 108
ピクセルシェーダ, 8
非コアレスアクセス, 44
非数（NaN）, 134
非正規化数, 134
非生存レジスタ, 64
ビットベクタ, 115
HIP, 56
非同期コピー, 49

非同期バリア, 137
ピン留めメモリ, 48
プーリング, 129
ファーストタッチ配置, 97
ファーフォールト, 75
φ 関数, 121
ファットバイナリ, 108
フィル（再ロード）, 120
フィルタ, 30
フェッチグループ, 79
フォーク／ジョインパターン, 11
負荷分散, 15
不揮発性メモリ, 65
不均一メモリアクセス（NUMA）, 96
符号ビット, 133
物理アドレス, 70
物理フレーム番号, 70
物理レジスタ, 66
浮動小数点形式, 133
不動点, 114
不要コード除去, 124
プライベート変数, 15
プリフェッチ, 76
プリプロセッサ, 107
フリンの分類, 5
プレディケート, 60
プレディケート実行, 52
プレディケートレジスタ, 109
ブロードキャスト, 18
プログラムカウンタ, 34
プログラム点, 112
プログラムフェーズ, 93
プロセス, 18
ブロックインデックス, 21
プロファイラ, 46
プロファイリング, 46
フロントエンド, 105
分割型到着／待機バリア, 55
分岐予測, 3
分散メモリプログラミング, 12
ページ, 70
ページウォークキャッシュ, 73
ページ追い出し, 77
ページオフセット, 70
ページテーブル, 71
ページテーブルウォーカ, 71
ページテーブルウォーク, 71
ページテーブルエントリ, 71
ページフォールト, 71
ページプリフェッチ, 76
ページマイグレーション, 97
ページロックメモリ, → ピン留めメモリ
並行性, 10
並列セクション, 17
並列プログラミングパターン, 10
ベクトル化, 126
ポート, 36
POSIX スレッド, 14
保守的解析, 127
ホスト, 2
ホストコード, 20
ホストバイパス, 99
ホストメモリ, 28
ホットスポット, 46

マ

マージソート, 57
マイクロベンチマーク, 94
前向き解析, 112
マスタ／ワーカパターン, 11
マルチインスタンス GPU（MIG）, 101
マルチ GPU, 95
マルチスレッディング, 3
マルチチップモジュール, 96
マルチテナンシ, 100
マルチプロセスサービス（MPS）, 101
マルチプロセッサ, 3
丸めモード, 135
ミスイベント, 91
ミューテックス, 13
命令当たりサイクル数（CPI）, 90
命令最適化, 52
命令セットアーキテクチャ, 2
命令発行, 40
命令フェッチ, 40
命令レベル並列性, 3
メッセージパッシング, 18
メモリ確保, 28
メモリコアレッシング, 44
メモリ帯域幅, 42
メモリトランザクション, 44
メモリのオーバーサブスクリプション, 77
メモリフェンス, 110
メモリ律速, 89, 129
メモリレベル並列性, 34
メモリワープ並列度, 91
モノリシック GPU, 95

ヤ

use 集合, 117
融合積和演算（FMA）, 87
誘導変数, 125
ユニファイドメモリ, 29
要素ごとの演算, 129
読み出しポート, 36

ラ

ラージページ, 73
ラージワープ, 63
ライトバック, 40
ラウンドロビン, 78
ラストリザルトファイル, 66
ラベル, 109
ランク, 18
リーダ（先頭命令）, 111
リダクション, 5
リッジポイント, 89

リモートダイレクトメモリアクセス（RDMA）, 99
量子化, 133
利用率, 100
理論帯域幅, 51
ルーズラウンドロビン, 78
ループ展開, 123
ループ不変式の移動, 125
ループ並列性, 11
ルーフラインモデル, 89
レーン, → SIMT レーン
レイテンシ, 1
レイテンシ隠蔽, 34
レジスタ, 25
レジスタ圧力, 25
レジスタスピル, 120
レジスタバンク競合, 36
レジスタファイル, 34
レジスタファイル仮想化, 66
レジスタファイルキャッシュ, 65
レジスタリネーミング, 66
レジスタ割り付け, 119
ローカルメモリ, 24
ロック, 13

ワ

ワーキングセット, 79
ワープ, 7
ワープ間局所性, 79
ワープサイズ, 7
ワープシャッフル, 53
ワープスケジューラ, 77
ワープダイバージェンス, 60
ワープ投票関数, 54
ワープ内局所性, 79

English index

Symbols
2:4 structured sparsity, 138
3D-stacked memory, 43

A
abstract syntax tree (AST), 107
accelerator, 2
accumulator, 133
activation function, 129
active mask, 42
active warp, 26
active warp set / pending warp set, 79
address translation, 70
Advanced Matrix Extensions (AMX), 136
alignment, 44
Amdahl's law, 4
analytical model, 87
architectural register, 66
arithmetic intensity, 89
asynchronous barrier, 137
asynchronous copy, 49
atomic operation, 16
attention, 137

B
back end, 105
backward analysis, 118
bank, 36
bank conflict, 36
bank-level parallelism, 80
barrier, 16
barrier synchronization, 23
basic block, 60
bfloat16 (BF16), 133
bit vector, 115
bitonic sort, 57
block index, 21
boundary condition, 30
boundary exchange, 19
branch prediction, 3
broadcast, 18
bulk synchronous parallel (BSP), 23

C
cache line, 44
cache miss, 33
cache thrashing, 79
cache-conscious wavefront scheduling (CCWS), 79
checkpoint, 93
chiplet, 96
chunk size, 15
Clang, 106
clause, 14
clock domain, 82
coalesced access, 44
code generation, 105
code motion, 125
collective communication, 18
compiler, 105
compiler directive, 12
compressed sparse format, 137
compulsory miss, 79
computation warp parallelism (CWP), 91
compute capability, 27
compute-bound, 89
concurrency, 10
concurrent copy and execute, 49
concurrent kernel execution, 102
conditional move, 121
conservative analysis, 127
constant folding, 124
constant memory, 24
constant propagation, 124
context switch, 34
control flow graph (CFG), 111
convolution, 30
convolution layer, 129
cooperative groups, 54
cooperative thread array (CTA), 21
copy descriptor, 132
copy propagation, 112
copy-then-execute model, 74
CPI stack, 90
CPU, 1
critical section, 13
CTA-aware scheduling, 80
cuBLAS, 56
CUDA Graph, 102
CUDA stream, 49
cuDNN, 56
cycle-level simulation, 82
cycles per instruction (CPI), 90

D
data decomposition, 5
data race, 13
data reuse, 30
data transfer, 47
data-driven model, 94
data-flow analysis, 112
datacenter, 100
dead code elimination, 124
dead register, 64
decode, 40
deep neural network (DNN), 128
def set, 117
def-use chain, 120
definition, 113
device, 20

device code, 20
device memory, 28
directive, 109
distributed-memory programming, 12
divergence analysis, 125
divergent branch, 60
divergent warp, 60
DMA (direct memory access), 75
dominance frontier, 122
dominator, 122
dot product, 129
double precision (FP64), 8
DRAM, 43
dynamic, 38
dynamic instruction count, 38
dynamic range, 134
dynamic scheduling, 15
dynamic warp formation (DWF), 63

E
effective bandwidth, 51
element-wise operation, 129
environment variable, 12
epilogue loop, 123
event-driven simulation, 92
execution configuration, 21
execution-driven simulation, 82
exponent, 133
exponent bias, 133

F
far fault, 75
fat binary, 108
fetch group, 79
FIFO, 82
fill (reload), 120
fill and drain, 132
filter, 30
fine-grained multithreading, 34
first-order model, 87
first-touch page placement, 97
fixed point, 114
FlashAttention, 137
floating-point format, 133
Flynn's taxonomy, 5
fork/join pattern, 11
forward analysis, 112
FP8, 133
FPGA emulation, 82
front end, 105
fully connected layer, 129
function inlining, 123
fused multiply-add (FMA), 87

G
GEMM (general matrix multiplication), 129
gen set, 113
global memory, 24
global optimization, 111
GPGPU, 8
GPU, 1
GPU instance, 101
GPU memory management unit (GMMU), 75
GPU module (GPM), 96
GPUDirect RDMA, 99
graphics pipeline, 8
greedy-then-oldest (GTO), 78
grid, 21
guard predicate, 109
guided scheduling, 15

H
half precision (FP16), 9
halo (ghost cells), 19
hardware-managed cache, 67
hierarchical register file, 65
high bandwidth memory (HBM), 9
HIP, 56
HMMA, 131
host, 2
host bypass, 99
host code, 20
host memory, 28
hotspot, 46

I
IEEE 754, 135
if-conversion, 60
immediate post-dominator (IPDOM), 60
in-order execution, 2
independent thread scheduling, 55
induction variable, 125
instruction fetch, 40
instruction optimization, 52
instruction set architecture (ISA), 2
instruction-level parallelism (ILP), 3
instructions per cycle (IPC), 3
inter-procedural analysis, 123
inter-warp locality, 79
interconnect, 97
interference graph, 119
intermediate representation (IR), 105
internal fragmentation, 73
interval analysis, 91
intra-warp locality, 79
intrinsic function, 52
IOMMU, 76
issue, 40
iterative algorithm, 114

J
JIT compilation, 108
join point, 121

K
kernel, 20
kernel launch, 21
kernel-level sampling, 93
kill set, 113

L
L1 data cache, 68
label, 109
large page, 73
large warp, 63
last result file (LRF), 66
latency, 1
latency hiding, 34
leader, 111
live register, 64
live variable, 117
liveness analysis, 117
LLVM, 106
load balancing, 15
local memory, 24
local optimization, 111
lock, 13
loop parallelism, 11
loop unrolling, 123
loop-invariant code motion, 125
loose round-robin (LRR), 78
lost intra-wavefront locality detector (LLD), 79
LRU (least recently used), 68

M
machine learning (ML), 128
main register file (MRF), 65
mantissa (fraction), 133
master/worker pattern, 11
math library, 56
matrix multiplication, 129
matrix multiply-accumulate (MMA), 130
meet operator, 112
memory allocation, 28
memory bandwidth, 42
memory coalescing, 44
memory fence, 110
memory oversubscription, 77
memory transaction, 44
memory warp parallelism (MWP), 91
memory-bound, 89
memory-level parallelism (MLP), 34
merge sort, 57
message passing, 18
microbenchmark, 94
MIMD, 5
miss event, 91
mixed precision, 133
monolithic GPU, 95
MPI (Message Passing Interface), 18
MSHR (miss status holding register), 90
multi-chip module (MCM), 96
multi-GPU, 95
Multi-Instance GPU (MIG), 101
multi-level page table, 71
Multi-Process Service (MPS), 101
multi-tenancy, 100
multiply-accumulate (MAC), 130
multiprocessor, 3
multithreading, 3
mutex, 13
mutual exclusion, 13

N
NaN (not a number), 134
NCCL, 56
near memory / far memory, 96
nested branch, 61
network interface card (NIC), 99
non-uniform memory access (NUMA), 96
non-volatile memory (NVM), 65
Nsight Compute, 47
Nsight Systems, 47
nvcc, 106
NVLink, 97
NVPTX, 107
NVSwitch, 98

O
occupancy, 26
oldest-first, 78
on-demand paging, 74
opcode, 108
OpenCL, 56
OpenMP, 12
operand, 108
operand buffer, 38
operand collector, 38
operand register file (ORF), 66
operational intensity, 89
ops:byte ratio, 129
out-of-order execution, 2
overflow, 134
overlap of computation and transfer, 49

P
padding, 40
page, 70
page eviction, 77
page fault, 71
page migration, 97
page offset, 70
page prefetching, 76
page table, 71
page table entry (PTE), 71
page table walk, 71
page table walker (PTW), 71
page walk cache, 73
parallel programming pattern, 10
parallel sections, 17
path-convergence criterion, 122
PCI Express (PCIe), 47
performance modeling, 81
phi function, 121
physical address, 70
physical frame number (PFN), 70
physical register, 66
pinned memory, 48

pipeline pattern, 11
pipeline width, 82
pixel shader, 8
point-to-point communication, 18
pooling, 129
port, 36
POSIX threads (pthreads), 14
post-dominator, 60
pre-eviction, 77
precise exception, 1
predecessor, 111
predicate, 60
predicate register, 109
predicated execution, 52
prefetching, 76
preprocessor, 107
private variable, 15
process, 18
profiler, 46
profiling, 46
program counter (PC), 34
program phase, 93
program point, 112
pruning, 137
PTX (Parallel Thread Execution), 108

Q
quality of service (QoS), 101
quantization, 133
queue-based model, 82

R
rank, 18
reaching definitions, 113
read port, 36
reconvergence point, 60
reduction, 5
register, 25
register allocation, 119
register bank conflict, 36
register file, 34
register file cache (RFC), 65
register file virtualization, 66
register pressure, 25
register renaming, 66
register spill, 120
remote direct memory access (RDMA), 99
replacement policy, 68
representative workload, 94
resource constraint, 25
ridge point, 89
Roofline model, 89
round-robin (RR), 78
rounding mode, 135

S
sampled simulation, 93
SASS, 108
scoreboard, 38
scratchpad memory, 23
sector, 85
sectored cache, 85
segment mapping, 73
serial fraction, 4
shared memory, 23
shared memory bank conflict, 39
shared memory carve-out, 68
shared variable, 15
shared virtual memory (SVM), 74
shared-memory programming, 12
shifted and squeezed FP8 (S2FP8), 135
sign bit, 133
silicon interposer, 43
SIMD, 2
SIMD efficiency, 63
SimPoint, 93
SIMT, 5
SIMT lane, 6
SIMT stack, 61
simulator, 81
single precision (FP32), 8
SISD, 2
software-managed cache, 68
sorting network, 57
sparse matrix, 137
sparse matrix-vector multiplication (SpMV), 138
sparsity, 137
spatial partitioning, 101
special function unit (SFU), 40
special register, 40
split arrive/wait barrier, 55
SPMD, 2
SRAM, 43
SSA form, 120
static, 38
static instruction count, 38
static scheduling, 15
static single assignment (SSA), 120
stencil, 18
stream processor (CUDA core), 6
streaming multiprocessor (SM), 6
strength reduction, 124
stride, 39
structured sparsity, 137
sub-core, 88
subnormal number, 134
successor, 111
superscalar, 2
SYCL, 56
systolic array, 131

T
tag, 67
task decomposition, 4
task queue, 11
tensor core, 9
Tensor Memory Accelerator (TMA), 9
TensorFloat-32 (TF32), 133

texture memory, 24
theoretical bandwidth, 51
thread, 12
thread block, 21
thread block cluster, 24
thread block scheduling, 97
thread hierarchy, 21
thread index, 21
thread-dependent value, 126
three-address code, 110
throughput, 1
Thrust, 56
tile, 30
tile quantization, 129
tiling, 30
time slicing, 101
TLB (translation lookaside buffer), 71
TLB miss, 71
trace, 82
trace-driven simulation, 82
transfer function, 112
Transformer Engine, 9
tree-based neighborhood prefetching, 76
two-level warp scheduling, 79

U
uncoalesced access, 44
underflow, 134
unified local memory, 68
unified memory, 29
unified shader, 8
unified virtual addressing (UVA), 50
uniform value, 126
use, 113
use set, 117
utilization, 100

V
value lifetime, 64
vectorization, 126
vertex shader, 8
virtual address, 70
virtual architecture, 106
virtual ISA, 2
virtual memory, 70
virtual page number (VPN), 70

W
warm-up, 93
warp, 7
warp divergence, 60
warp scheduler, 77
warp shuffle, 53
warp size, 7
warp vote function, 54
wave quantization, 130
wavefront, 7
working set, 79
write port, 36
write-back, 40

Z
zero copy, 50
zero padding, 30